



DesignWare® DW_apb_uart

Databook

*DW_apb_uart – **Product Codes***

Copyright Notice and Proprietary Information

© 2023 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History	7
Preface	13
Product Codes	13
Databook Organization	15
Related Documentation	15
Web Resources	16
Reference Documentation	16
Synopsys Statement on Inclusivity and Diversity	16
Replacement of Noninclusive Terminology	16
Customer Support	17
Chapter 1	
Product Overview	19
1.1 DesignWare System Overview	19
1.2 General Product Description	21
1.2.1 DW_apb_uart Block Diagram	23
1.3 Standards Compliance	24
1.4 Features	25
1.5 Area and Power Number	26
1.6 Deliverables	26
1.7 Verification Environment Overview	26
1.8 Licenses	26
1.9 Speed and Clock Requirements	27
1.10 Where To Go From Here	27
Chapter 2	
Functional Description	29
2.1 UART (RS232) Serial Protocol	29
2.2 9-bit Data Transfer	31
2.2.1 Transmit Mode	32
2.2.2 Receive Mode	34
2.3 RS485 Serial Protocol	36
2.3.1 DE Assertion and De-assertion Timing	36
2.3.2 RS485 Modes	37
2.3.3 Sample Scenarios	40
2.4 Fractional Baud Rate Support	43
2.4.1 Fractional Division Used to Generate Baud Clock	45
2.4.2 Calculating the Fractional Value Error	45
2.5 IrDA 1.0 SIR Protocol	47

2.6	FIFO Support	49
2.7	Clocks and Resets	51
2.7.1	Clocks	51
2.7.2	Resets	51
2.8	Synchronization	52
2.9	Back-to-Back Character Stream Transmission	54
2.9.1	Dual Clock Mode	54
2.9.2	Single Clock Mode	56
2.10	Interrupts	56
2.11	Auto Flow Control	58
2.12	Programmable THRE Interrupt	61
2.13	Clock Gate Enable	63
2.14	DMA Support	66
2.14.1	DMA Modes	67
2.14.2	Transmit Watermark Level and Transmit FIFO Underflow	70
2.14.3	Choosing Transmit Watermark Level	70
2.14.4	Selecting DEST_MSIZ and Transmit FIFO Overflow	72
2.14.5	Receive Watermark Level and Receive FIFO Overflow	73
2.14.6	Choosing the Receive Watermark Level	73
2.14.7	Selecting SRC_MSIZ and Receive FIFO Underflow	73
2.14.8	Handshaking Interface Operation	74
2.14.9	Potential Deadlock Conditions in DW_apb_uart/DW_ahb_dmac Systems	77
2.15	APB Interface	81
2.15.1	APB 3.0 Support	82
2.15.2	APB 4.0 Support	83
Chapter 3		
	Parameter Descriptions	85
3.1	System Configuration Parameters	86
3.2	UART Configuration Parameters	88
3.3	Storage Configuration Parameters	92
3.4	Synchronization Configuration Parameters	93
3.5	Additional Features Parameters	94
Chapter 4		
	Signal Descriptions	97
4.1	APB Completer Interface Signals	99
4.2	Application Interface Signals	102
4.3	Miscellaneous Signals	103
4.4	FIFO Interface Signals	104
4.5	Modem Interface Signals	107
4.6	DMA Interface Signals	109
4.7	Serial Interface Signals	113
4.8	Infrared Interface Signals	114
4.9	Clock Control Interface Signals	115
4.10	Debug Interface Signals	116
4.11	RS485 Interface Signals	117
4.12	Interrupt Interface Signals	119
Chapter 5		

Register Descriptions	121
5.1 uart_memory_map/uart_address_block Registers	124
5.1.1 RBR	126
5.1.2 THR	128
5.1.3 DLL	130
5.1.4 IER	132
5.1.5 DLH	135
5.1.6 IIR	136
5.1.7 FCR	138
5.1.8 LCR	141
5.1.9 MCR	145
5.1.10 LSR	149
5.1.11 MSR	156
5.1.12 SCR	161
5.1.13 LPDLL	162
5.1.14 LPDLH	164
5.1.15 SRBRn (for n = 0; n <= 15)	166
5.1.16 STHRn (for n = 0; n <= 15)	168
5.1.17 FAR	170
5.1.18 TFR	172
5.1.19 RFW	173
5.1.20 USR	175
5.1.21 TFL	178
5.1.22 RFL	179
5.1.23 SRR	180
5.1.24 SRTS	182
5.1.25 SBCR	184
5.1.26 SDMAM	186
5.1.27 SFE	188
5.1.28 SRT	189
5.1.29 STET	191
5.1.30 HTX	193
5.1.31 DMSA	194
5.1.32 TCR	195
5.1.33 DE_EN	198
5.1.34 RE_EN	199
5.1.35 DET	200
5.1.36 TAT	202
5.1.37 DLF	204
5.1.38 RAR	205
5.1.39 TAR	207
5.1.40 LCR_EXT	208
5.1.41 UART_PROT_LEVEL	212
5.1.42 REG_TIMEOUT_RST	213
5.1.43 CPR	215
5.1.44 UCV	219
5.1.45 CTR	220

Chapter 6

Programming the DW_apb_uart	221
6.1 Programming Examples	222
6.2 Programming Flow in RS485 Mode	223
6.2.1 Full Duplex Mode (XFER_MODE=0)	223
6.2.2 Software-Enabled Half Duplex Mode (XFER_MODE=1)	224
6.2.3 Hardware enabled Half Duplex mode (XFER_MODE=2)	224
6.3 Programming Flow in 9-bit Data Mode	226
6.3.1 Transmit Mode 0	226
6.3.2 Transmit Mode 1	227
6.3.3 Hardware Address Match Receive mode	228
6.3.4 Software Address Match Receive Mode	229
6.4 Programming Flow for Fractional Baud Rate	229
Chapter 7	
Verification	231
7.1 Verification Environment Overview	232
7.2 Testbench Directories and Files	233
7.3 Packaged Testcases	233
Chapter 8	
Integration Considerations	235
8.1 Accessing Top-level Constraints	235
8.2 Coherency	235
8.2.1 Writing Coherently	236
8.2.2 Reading Coherently	243
8.3 Timing Exceptions	246
8.4 Performance	247
8.4.1 Power Consumption, Frequency, Area and DFT Coverage	247
Appendix A	
Basic Core Module (BCM) Library	251
A.1 BCM Library Components	251
A.2 Synchronizer Methods	252
A.2.1 Synchronizers Used in DW_apb_uart	252
A.2.2 Synchronizer 1: Simple Double Register Synchronizer	252
A.2.3 Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset	253
A.2.4 Synchronizer 3: Simple Double Register Synchronizer with Acknowledge	253
Appendix B	
Internal Parameter Descriptions	255
Appendix C	
Application Notes	257
Appendix D	
Glossary	261

Revision History

This table shows the revision history for the databook from release to release. This is being tracked from version 3.06b onward.



Note

- A change-bar version of this databook is available on the following IP web page under the "Documentation" section: https://www.synopsys.com/dw/ipdir.php?c=DW_apb_uart
- Links and references to any chapter, section, table, figure, and page numbers in the Revision History table are assured to be valid only for the current version in the table.

Version	Date	Description
4.04a	April 2023	Note: Replaced noninclusive and insensitive terms with globally accepted inclusive terminology. See "Replacement of Noninclusive Terminology" on page 16. Added: ■ "Reference Documentation" on page 16 ■ "Synopsys Statement on Inclusivity and Diversity" on page 16 ■ "Deliverables" on page 26 ■ "Where To Go From Here" on page 27 Updated: ■ Version number changed for 2023.04a release ■ "APB Interface" on page 81 ■ "Verification Environment Overview" on page 232 ■ "Synchronizers Used in DW_apb_uart" on page 252 ■ Chapter 3, "Parameter Descriptions", Chapter 4, "Signal Descriptions", Chapter 5, "Register Descriptions" and Appendix B, "Internal Parameter Descriptions" auto-extracted with change bars from the RTL

Version	Date	Description
4.03a	December 2020	Added: ■ “Clocks and Resets” on page 51 ■ “Timing Exceptions” on page 246 Updated: ■ Version number changed for 2020.12a release ■ “Performance” on page 247 ■ “BCM Library Components” on page 251 ■ “Parameter Descriptions” on page 85, “Signal Descriptions” on page 97, “Register Descriptions” on page 121, and “Internal Parameter Descriptions” on page 255 are auto extracted with change bars from the RTL. Renamed: ■ Synchronizer Methods to Appendix A, “Basic Core Module (BCM) Library” ■ Clock Support to “Synchronization” on page 52 Removed: ■ Reset Signal ■ Index chapter
4.02a	July 2018	Added: ■ APB Interface ■ Support for configurable Synchronization Depth through parameter SYNC_DEPTH Updated: ■ Version number changed for 2018.07a release ■ Performance ■ Parameter Descriptions, Signal Descriptions, Register Descriptions, and Internal Parameter Descriptions are auto extracted with change bars from the RTL. Removed: ■ Chapter 2, “Building and Verifying a Component or Subsystem” and added the contents in the newly created user guide.

Version	Date	Description
4.01a	October 2016	- Version number changed for 2016.10a release - Parameter Descriptions and Register Descriptions auto-extracted from the RTL - Added the DMA_HS_REQ_ON_RESET and LSR_STATUS_CLEAR parameters in Parameter Descriptions - Added xprop directory in Table 2-1 and Table 2-4 - Added “Running VCS XPROP Analyzer” - Added a note in DMA Support - Deleted the “Running Leda on Generated Code with coreConsultant”, and reference to Leda directory in Table 2-1 - Deleted the “Running Leda on Generated Code with coreAssembler” section, and reference to Leda directory in Table 2-4 - Moved Internal Parameter Descriptions to Appendix - Moved to Fractional Baud Rate Support - Moved Table 2-3 to Interrupts and modified the Interrupt Reset Control field for Interrupt ID 0110.
4.00a	June 2015	- Added following section in Functional Description: 9-bit Data Transfer - RS485 Serial Protocol - Fractional Baud Rate Support - Updated Register Descriptions, Signal Descriptions, Parameter Descriptions, and Programming the DW_apb_uart chapters - Added “Running SpyGlass® Lint and SpyGlass® CDC” - Added “Running Spyglass on Generated Code with coreAssembler” - Added Chapter B, Internal Parameter Descriptions - Signal Descriptions auto-extracted from the RTL - Added Appendix A, Basic Core Module (BCM) Library - Updated area and performance number in sections “Area” and “Power Consumption”
3.15a	June 2014	- Version change for 2014.06a release - Added “Performance” section in the “Integration Considerations” chapter - Corrected Default Input/Output Delay in Signals chapter
3.14c	May 2013	- Corrected the de-assert sequence in “Reset Signals” on page 67 - Corrected the label of the UART_ADD_ENCODED_PARAMS parameter - Updated the template
3.14b	September 2012	Added the product code on the cover and in Table 1-1.
3.14b	June 2012	Added new RTC_FCT coreConsultant parameter.

Version	Date	Description
3.13a	March 2012	- Enhanced timing information for serial clock modules - Corrected reset values for MSR[3:0] bits - Added note to write MCR before LCR for SIR mode - Updated CPR register description
3.12c	November 2011	Version change for 2011.11a release.
3.12b	October 2011	- Updated DLAB bit description of the LCR register - Added flow charts in programming chapter - Edited “Product Overview” material and “Functional Description” material for better flow in reading - Enhanced SIRE bit description of MCR register
3.12a	June 2011	- Updated material for Stick Parity bit of Line Control Register - Updated system diagram in Figure 1-1 - Enhanced “Related Documents” section in Preface - Corrected address offset and R/W for LPDLL, LPDLH, and DMASA registers
3.11a	April 2011	Added note for break condition in BI bit of LSR register.
3.11a	April 2011	- Corrected description of APB_DATA_WIDTH parameter - Added sections for “Potential Deadlock Conditions in DW_apb_uart/ DW_ahb_dmac Systems” and “Reset Signals” - Edited descriptions for Parity Error and Framing Error bits in LSR register - Corrected dma* signals in Figure 3-25 - Added register in acknowledge page in “RTL Diagram of Data Synchronization Module” diagram
3.10a	January 2011	Corrected description of reset operation in Answer 7 of “Application Notes”
3.10a	January 2011	Corrected “DW_apb_uart Testbench” illustration
3.10a	November 2010	- Corrected DW_ahb_dmac response in “Receive Watermark Level and Receive FIFO Overflow” section - Modified values to which APB_DATA_WIDTH parameter can be set
3.10a	September 2010	Corrected LCR link to LSR link in RBR register description.
3.10a	September 2010	- Enhanced USR[0] busy description to explain non-busy conditions - Corrected names of include files and vcs command used for simulation - Added information regarding false start bit detection - Updated the ADDITIONAL_PARAMETERS description

Version	Date	Description
3.08a	June 2010	■ Corrected synchronous description from “pclk” to “N/A” for cts_n, dsr_n, dcd_n and ri_n signals ■ Added: - Syntax for include files in database tables - +v2k option in vcs command syntax - Information for back-to-back character stream transmission
3.08a	January 2010	Revised FCR register description for condition in which FIFOs are not implemented
3.08a	December 2009	Updated databook to new template for consistency with other IIP/VIP/PHY databooks
3.08a	July 2009	Corrected equations for avoiding underflow when programming a source burst transaction
3.08a	May 2009	Removed references to QuickStarts, as they are no longer supported
3.08a	April 2009	Enhanced “Clock Support” section
3.08a	October 2008	Version change for 2008.10a release
3.07b	June 2008	Version change for 2008.06a release
3.07a	January 2008	■ Updated for revised installation guide and consolidated release notes titles ■ Changed references of “Designware AMBA” to simply “DesignWare” ■ Performance information temporarily removed ■ Corrections made to Figures 7 and 8
3.07a	December 2007	■ Correction of Figure 8 ■ Removed area tables pending more current data
3.06b	June 2007	Version change for 2007.06a release.

Preface

This databook provides information that you need to interface the DW_apb_uart component to the Advanced Peripheral Bus (APB). This component conforms to the [AMBA Specification, Revision 2.0](#) from Arm®.

The information in this databook includes a functional description, pin and parameter descriptions, and a memory map. Also provided are an overview of the component testbench, a description of the tests that are run to verify the component, and synthesis information.

Product Codes

The following table lists all the components associated with the product license for DesignWare AMBA Fabric.

Table 1 Product Codes and Licenses for DesignWare AMBA Fabric

Component	Description	Product Licenses
DW_ahb	High Performance, Low Latency Interconnect Fabric for AMBA 2 AHB/AHB5	DesignWare AMBA Fabric Source License: DWC-AMBA-Fabric-Source (Product Code: 3768-0) Add-on - DesignWare AMBA Fabric Source AHB5 License: DWC-AMBA-AHB5-Fabric-Source (Product code: F944-0)
DW_ahb_ah2h	High Performance, High Bandwidth AMBA 2/AHB5 - AHB to AHB bridge	
DW_ahb_h2h	Area Efficient, Low Bandwidth AMBA 2/AHB5 - AHB to AHB Bridge	
DW_ahb_icm	Configurable Multi-Layer Interconnection Matrix AMBA 2 AHB/AHB5	
DW_ahb_ictl	Configurable Vectored Interrupt Controllers for AHB bus systems	
DW_apb	High Performance, Low Latency Interconnect Fabric & Bridge for AMBA 2 APB(APB2)/APB3/APB4 for direct connect to AMBA 2 AHB Fabric	
DW_apb_ictl	Configurable Vectored Interrupt Controllers for AMBA 2 APB bus systems	

Component	Description	Product Licenses
DW_axi	High Performance, Hybrid Architecture, Low Latency Interconnect Fabric for AMBA 3 AXI/ AMBA 4 AXI	DesignWare AMBA Fabric Source License: DWC-AMBA-Fabric-Source (Product Code: 3768-0) Add-on - DesignWare AMBA Fabric Source AHB5 License: DWC-AMBA-AHB5-Fabric-Source (Product Code: F944-0) Add-on - DesignWare AXI Mission Critical License: DWC-AXI-SAFETY (Product Code: H615-0) Note: This license is available only with the Product Code H678-0. This is applicable only for DW_axi.
DW_axi_a2x	High Performance, Configurable Bridge Between AHB and AXI Components or AXI and AXI Components. With support for AMBA 3 AXI and AMBA 4 AXI	
DW_axi_gm	Simplify the connection of third party/custom Manager Controllers to any AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_gs	Simplify the connection of third party/custom Subordinate Controllers to any AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_hmx	Configurable, High Performance Interface from AHB Manager to an AXI Fabric or AXI Subordinate	
DW_axi_rs	Configurable Standalone Pipelining stage for AMBA 3 AXI/AMBA 4 AXI subsystems for Timing Management	
DW_axi_x2h	Bridge from AMBA 3 AXI/AMBA 4 AXI to AMBA 2 AHB/AHB5, enabling easy Integration of Legacy AHB Designs with newer AXI Systems	
DW_axi_x2p	High performance, Low Latency Interconnect Fabric and Bridge for AMBA 2 APB (APB2)/APB3/APB4 for direct connect to AMBA 3 AXI/AMBA 4 AXI Fabric	
DW_axi_x2x	Flexible Bridge between Multiple AMBA 3 AXI Components or Buses	
DW_apb_gpio	General Purpose I/O Pad Control Peripheral for the AMBA APB Bus	DesignWare APB Peripheral Source License: DWC-APB-Periph-Source (Product Code: 3771-0) Add-on - DesignWare APB GPIO Mission Critical License: DWC-APB-GPIO-SAFETY (Product Code: H107-0) Note: This license is available only with the Product Code 3771-0. This is applicable only for DW_apb_gpio. Add-on - DesignWare APB WDT Mission Critical License: DWC-APB-WDT-SAFETY (Product Code: H108-0) Note: This license is available only with the Product Code 3771-0. This is applicable only for DW_apb_wdt.
DW_apb_rap	Programmable controller for the Remap and Pause features of the DW_ahb AHB Interconnect	
DW_apb_rtc	Configurable High Range Counter with an AMBA APB Interface	
DW_apb_timers	Configurable System Counters Controlled through an AMBA APB interface	
DW_apb_wdt	A Programmable Watchdog Timer Peripheral for the AMBA APB bus	

Component	Description	Product Licenses
DW_apb_i2c	A Highly Configurable, Programmable Controller or Target I2C Device with an APB Interface	DesignWare APB Advanced Peripheral Source License: DWC-APB-Advanced-Source (Product Code: 3772-0)
DW_apb_i2s	A Configurable Controller or Target device for the three-wire Interface (I2S) for streaming stereo audio between devices	
DW_apb_ssi	A Configurable, Programmable, Full-duplex, Controller or Target Synchronous Serial Interface	
DW_apb_uart	A Configurable and Programmable Universal Asynchronous Receiver/Transmitter (UART) for the AMBA APB Bus	

Databook Organization

The chapters of this databook are organized as follows:

- Chapter 1, “[Product Overview](#)” provides a system overview, a component block diagram, basic features, and an overview of the verification environment.
- Chapter 2, “[Functional Description](#)” describes the functional operation of the DW_apb_uart.
- Chapter 3, “[Parameter Descriptions](#)” identifies the configurable parameters supported by the DW_apb_uart.
- Chapter 4, “[Signal Descriptions](#)” provides a list and description of the DW_apb_uart signals.
- Chapter 5, “[Register Descriptions](#)” describes the programmable registers of the DW_apb_uart.
- Chapter 6, “[Programming the DW_apb_uart](#)” provides information needed to program the configured DW_apb_uart.
- Chapter 7, “[Verification](#)” provides information on verifying the configured DW_apb_uart.
- Chapter 8, “[Integration Considerations](#)” includes information you need to integrate the configured DW_apb_uart into your design.
- Appendix A “[Basic Core Module \(BCM\) Library](#)” documents the synchronizer methods (blocks of synchronizer functionality) used in DW_apb_uart to cross clock boundaries.
- Appendix B, “[Internal Parameter Descriptions](#)” provides a list of internal parameter descriptions that might be indirectly referenced in expressions in the Signals, Parameters, or Registers chapters.
- Appendix C, “[Application Notes](#)” includes information you need to integrate the configured DW_apb_uart into your design.
- Appendix D, “[Glossary](#)” provides a glossary of general terms.

Related Documentation

- Using DesignWare Library IP in coreAssembler – Contains information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools
- coreAssembler User Guide – Contains information on using coreAssembler

- coreConsultant User Guide – Contains information on using coreConsultant

To see a complete listing of documentation within the DesignWare Synthesizable Components for AMBA 2, see the *DesignWare Synthesizable Components for AMBA 2, AMBA 3 AXI, and AMBA 4 AXI Installation Guide (Documentation Overview)*.



Note

Information on the DW_apb_uart component in this databook assumes that the reader is fully familiar with the National Semiconductor 16550 (UART) component specification.

Information provided on IrDA SIR mode assumes that the reader is fully familiar with the IrDa Serial Infrared Physical Layer Specification. This specification can be obtained from the following website:

<http://www.irda.org>

Web Resources

- DesignWare IP product information: <https://www.synopsys.com/designware-ip.html>
- Your custom DesignWare IP page: <https://www.synopsys.com/dw/mydesignware.php>
- Documentation through SolvNetPlus: <https://solvnetplus.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL): <https://www.synopsys.com/keys>

Reference Documentation

The following references contain useful information concerning the protocols addressed by this DesignWare Library IP:

- AMBA Specification, Revision 2.0, Arm® for AHB and APB interface
- AMBA3 APB Protocol Specification, v1.0, Arm® for APB3 interface
- AMBA4 APB Protocol Specification v2.0, Arm® for APB4 interface

For additional references, see “[Standards Compliance](#)” on page 24.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Replacement of Noninclusive Terminology

The following table has the list and definition of the inclusive terminology used in this document:

Table 2 Replacement of Noninclusive Terminology

Noninclusive Terms	Replacement Terms	Definition
Master	Manager, Controller, Requester, Primary	Device or model that initiates and controls another device or peripheral.
Slave	Subordinate, Target, Completer, Secondary	Device or model that is controlled by and responds to a manager.

**Note**

To comply to the AMBA Specifications and to aid in seamless integration, Synopsys will not change parameter, signal, and register names.

Customer Support

To obtain support for your product, prepare the required files and contact the support center using one of the methods described:

- Prepare the following debug information, if applicable:
 - For environment set-up problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, select the following menu:
File > Build Debug Tar-file
 Check all the boxes in the dialog box that apply to your issue. This option gathers all the Synopsys product data needed to begin debugging an issue and writes it to the `<core tool startup directory>/debug.tar.gz` file.
 - For simulation issues outside of coreConsultant or coreAssembler:
 - Create a waveforms file (such as VPD or VCD, or FSDB).
 - Identify the hierarchy path to the DesignWare instance.
 - Identify the timestamp of any signals or locations in the waveforms that are not understood.
- *For the fastest response*, enter a case through SolvNetPlus:
 - a. <https://solvnetplus.synopsys.com>

**Note**

SolvNetPlus does not support Internet Explorer.

- b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).
- c. Complete the mandatory fields that are marked with an asterisk and click **Save**.
 Make sure to include the following:
 - **Product L1:** DesignWare Cores
 - **Product L2:** AMBA

- d. After creating the case, attach any debug files you created.

For more information about general usage information, see the following article in SolvNetPlus:

<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>

- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
 - Include the Product L1 and Product L2 names, and Version number in your e-mail so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood.
 - Attach any debug files you created.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

1

Product Overview

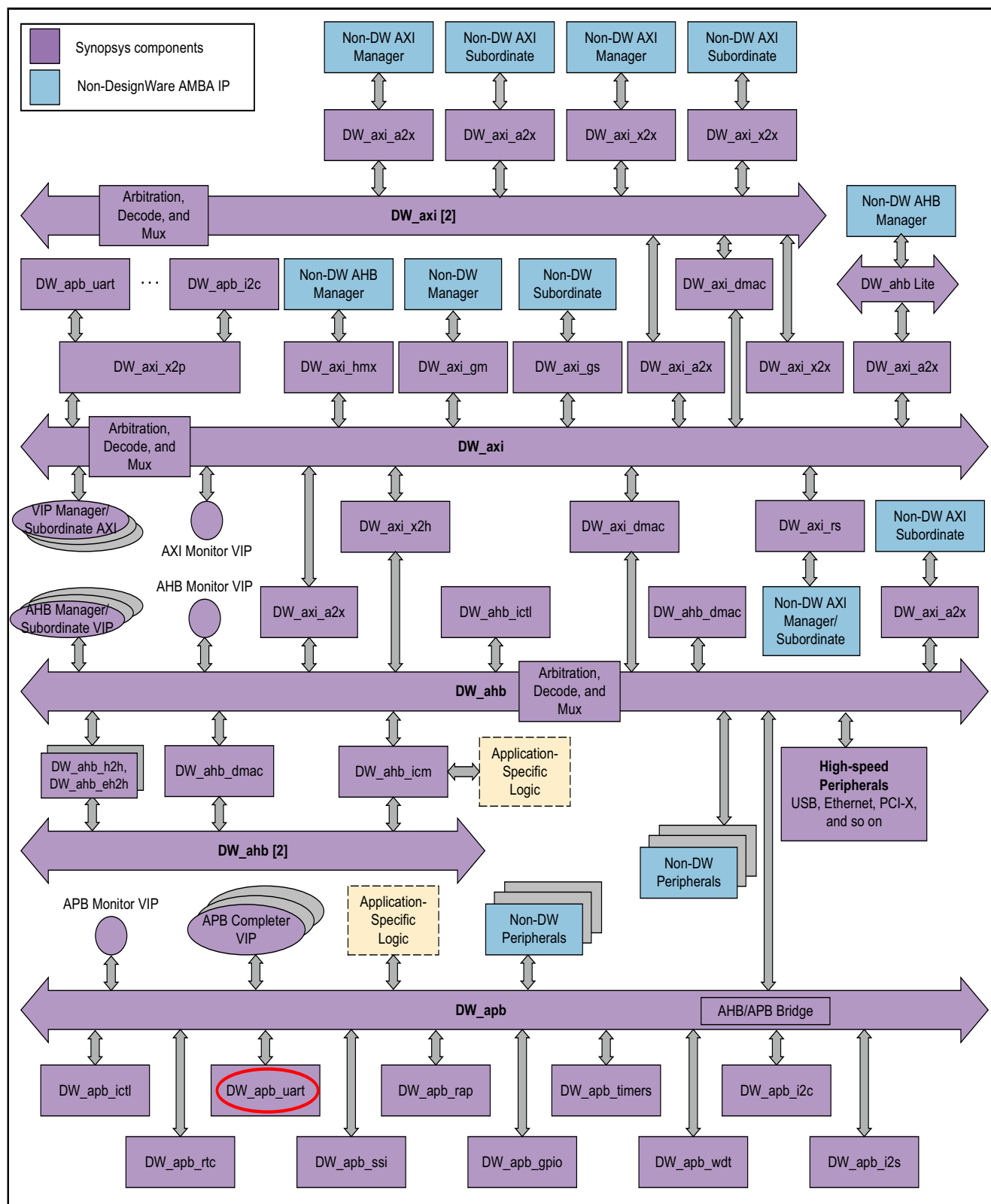
The DW_apb_uart is a programmable Universal Asynchronous Receiver/Transmitter (UART). This component is an AMBA 2.0-compliant Advanced Peripheral Bus (APB) completer device and is part of the family of DesignWare Synthesizable Components.

1.1 DesignWare System Overview

The Synopsys DesignWare Synthesizable Components environment is a parameterizable bus system containing AMBA version 2.0-compliant AHB (Advanced High-performance Bus) and APB (Advanced Peripheral Bus) components, and AMBA version 3.0-compliant AXI (Advanced eXtensible Interface) components.

[Figure 1-1](#) illustrates one example of this environment, including the AXI bus, the AHB bus, and the APB bus. Included in this subsystem are synthesizable IP for AXI/AHB/APB peripherals, bus bridges, and an AXI interconnect and AHB bus fabric. Also included are verification IP for AXI/AHB/APB requester/completer models and bus monitors. To access the product page and documentation for AMBA components, see the [DesignWare IP Solutions for AMBA Interconnect](#) page. (SolvNetPlus ID required)

Figure 1-1 Example of DW_apb_uart in a Complete System



You can connect, configure, synthesize, and verify the DW_apb_uart within a DesignWare subsystem using coreAssembler, documentation for which is available on the web in the *coreAssembler User Guide*.

If you want to configure, synthesize, and verify a single component such as the DW_apb_uart component, you might prefer using coreConsultant, documentation for which is available in the *coreConsultant User Guide*.

1.2 General Product Description

The DW_apb_uart is modeled after the industry-standard 16550. However, the register address space is relocated to 32-bit data boundaries for APB bus implementation. The DW_apb_uart can be configured, synthesized, and verified using the Synopsys coreConsultant GUI.

The DW_apb_uart is used for serial communication with:

- Peripherals
- Modems (data carrier equipment, DCE)
- Data sets

Data is written from a requester (CPU) over the APB bus to the UART, and it is converted to serial form and transmitted to the destination device. Serial data is also received by the UART and stored for the requester (CPU) to read back.

The DW_apb_uart contains registers that control:

- Character length
- Baud rate
- Parity generation/checking
- Interrupt generation

Although there is only one interrupt output signal (intr) from the DW_apb_uart, there are several prioritized interrupt types that can be responsible for its assertion. Each of the interrupt types can be separately enabled or disabled by the control registers.

The following describe various functionalities that you can configure into the DW_apb_uart:

- **Transmit and receive data FIFOs** - To reduce the time demand placed on the requester by the DW_apb_uart, optional FIFOs are available to buffer transmit and receive data. The requester does not have to access the DW_apb_uart each time a single byte of data is received. The optional FIFOs can be selected at configuration time.

The FIFOs can be configured as external customer-supplied FIFO RAMs or as internal DesignWare D-flip-flop-based RAMs (DW_ram_r_w_s_dff).

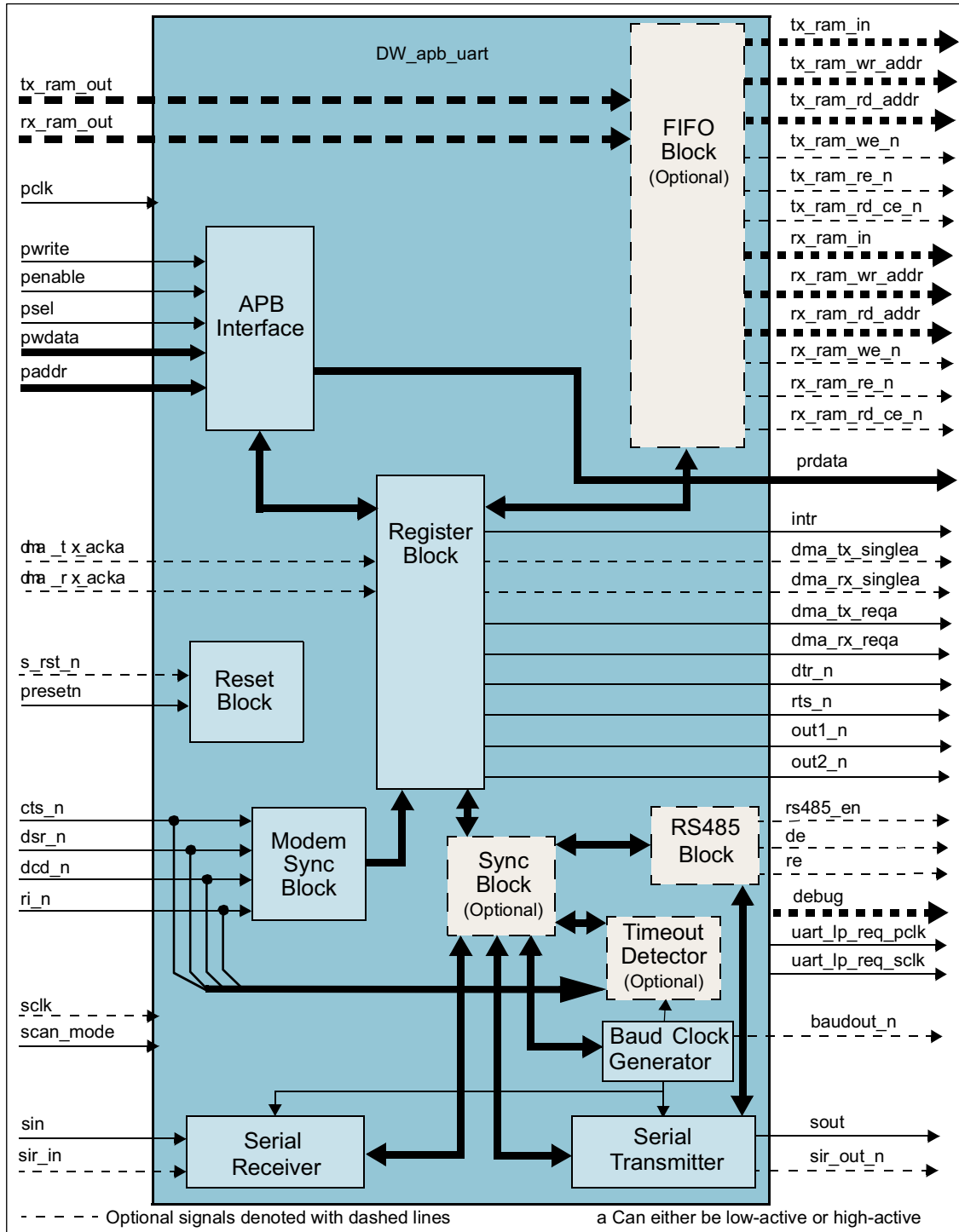
- When external RAM support is chosen, both synchronous or asynchronous read-port memories are supported.
- When FIFO support is selected, an optional test/debug mode is available to allow the receive FIFO to be written by the requester and the transmit FIFO to be read by the requester.

- **DMA controller interface** – The DW_apb_uart can interface with a DMA controller through external signals (dma_tx_req_n and dma_rx_req_n) in order to indicate when data is ready to be read or when the transmit FIFO is empty. Additional optional DMA signals are available for compatibility with a DesignWare DMA controller interface, such as the DW_ahb_dmac.
- **Asynchronous clock support** – To solve problems surrounding CPU data synchronization in relation to the required serial baud clock requirements, an optional separate serial data clock can be selected. Full handshaking and level-synchronization guarantees all data crossing between the two clock domains.
- **Auto flow control** – The DW_apb_uart uses a D16750-compatible Auto Flow Control Mode to increase system efficiency and decrease software load. When FIFOs and the Auto Flow Control are selected and enabled, the request-to-send (rts_n) output and clear-to-send (cts_n) input automatically control serial data flow.
- **RS485 interface Support** – For integration into systems for which an RS485 interface is required, the DW_apb_uart can be configured for a software-programmable RS485 mode. If this mode is not selected, only the UART (RS232 standard) serial data format is available.
- **Programmable Transmit Holding Register Empty (THRE) interrupt** – The DW_apb_uart uses a Programmable Transmitter Holding Register Empty (THRE) Interrupt Mode to increase system performance. When FIFOs and the THRE Mode are selected and enabled, THRE Interrupts are active at or below a programmed TX FIFO threshold level. Additionally, the Line Status THRE switches from indicating TX FIFO empty to TX FIFO full, which allows software to set a threshold that keeps the transmitter FIFO from running empty whenever there is data to transmit.
- **Serial infrared support** – For integration in systems where Infrared SIR serial data format is required, the DW_apb_uart can be configured for a software-programmable IrDA SIR Mode. If this mode is not selected, only the UART (RS232 standard) serial data format is available.
- **Increase built-in diagnostic capabilities** – To increase the built-in diagnostic capabilities of the DW_apb_uart, the Modem Control Loopback Mode has been extended. Modem Status bits actually reflect Modem Control Register deltas, as well as the bits themselves. Additionally, when FIFOs and Auto Flow Control Mode are selected and enabled, the Modem Control RTS is internally looped back to the CTS in order to control the transmitter, which allows local testing of the Auto CTS mode. Furthermore, the controllability of rts_n through the receiver FIFO threshold can be observed using the RTS Modem Status bit, which allows local verification of the Auto RTS mode.
- **Level 1 and Level 2 debug support** – To help with debug issues, optional debug signals are available on the DW_apb_uart. To comply with level 1 and level 2 debug support requirements, many internal points of interest to the debugger are available as outputs.

1.2.1 DW_apb_uart Block Diagram

Figure 1-2 illustrates the DW_apb_uart block diagram.

Figure 1-2 DW_apb_uart Functional Block Diagram



The following list describes each of the major blocks as shown in [Figure 1-2](#):

- **Reset block** – resets clock domains.
- **APB Completer interface** – connects to APB bus.
- **Register block** – responsible for the main UART functionality including control, status and interrupt generation.
- **Modem Synchronization block** – synchronizes the modem input signal.
- **FIFO block** (optional) – responsible for FIFO control and storage – when using internal RAM – or optionally signaling to control external RAM.
- **Synchronization block** (optional) – implemented when the peripheral is configured to have a separate serial data clock (i.e. two clock implementation).
- **Timeout Detector block** (optional) – indicates the absence of character data movement in the receiver FIFO within a given time period; this is used to generate character timeout interrupts when enabled.
 This block can also have optional clock gate enable outputs – `uart_lp_req_pclk` for single clock implementations or `uart_lp_req_pclk` and `uart_lp_req_sclk` for two clock implementations – in order to indicate:
 - TX and RX pipeline is clear; that is, there is no data
 - No activity has occurred
 - Modem control input signals have not changed within a given time period
- **Baud Clock Generator** – produces the transmitter and receiver baud clock along with the output reference clock signal (`baudout_n`).
- **Serial Transmitter** – converts the parallel data – written to the UART – into serial form and adds all additional bits, as specified by the control register, for transmission. These serial data, referred to as a character, can exit the block in two formats:
 - Serial UART
 - IrDA 1.0 SIR
- **Serial Receiver** – converts the serial data character – specified by the control register – received in either the UART or IrDA 1.0 SIR format to parallel form. This block controls:
 - Parity error detection
 - Framing error detection
 - Line break detection
- **RS485 block** (optional) – implemented when the peripheral is configured to have an RS485 interface, responsible for the generation of driver enable (`de`) and receiver enable (`re`) signals required by the RS485 Transceiver.

1.3 Standards Compliance

The DW_apb_uart component conforms to the AMBA Specification, AMBA 2 APB(APB2)/APB3/APB4. Readers are assumed to be familiar with this specification.

1.4 Features

The DW_apb_uart supports the following features:

- AMBA APB interface allows integration into AMBA SoC implementations
- 9-bit serial data support
- False start bit detection
- Programmable fractional baud rate support
- Multi-drop RS485 interface support
- Configurable parameters for the following:
 - APB data bus widths of 8, 16 and 32
 - Additional DMA interface signals for compatibility with DesignWare DMA interface
 - DMA interface signal polarity
 - Transmit and receive FIFO depths of 0, 16, 32, 64, 128, 256, 512, 1024, 2048
 - Internal or external FIFO (RAM) selection
 - Use of two clocks — pclk and sclk — instead of just pclk
 - IrDA 1.0 SIR mode support with up to 115.2 Kbaud data rate and a pulse duration (width) as specified in the IrDA physical layer specification:
$$\text{width} = 3/16 \times \text{bit period}$$
 - IrDA 1.0 SIR low-power reception capabilities
 - Baud clock reference output signal
 - Clock gate enable outputs used to indicate that the TX and RX pipeline is clear (no data) and no activity has occurred for more than one character time, so that clocks can be gated
 - FIFO access mode — for FIFO testing — enabling the requester to write to the receive FIFO and read from the transmit FIFO
 - Additional FIFO status registers
 - Shadow registers to reduce software overhead and also include a software programmable reset
 - Auto Flow Control mode, as specified in the D16750 standard
 - Loopback mode that enables greater testing of Modem Control and Auto Flow Control features (Loopback support in IrDA SIR mode is available)
 - Transmitter Holding Register Empty (THRE) interrupt mode
 - Busy functionality
- Ability to set some configuration parameters during instantiation
- Configuration identification registers present
- Functionality based on the 16550 industry standard
 - Programmable character properties, such as:
 - Number of data bits per character (5-8)

- Optional parity bit (with odd, even select or Stick Parity)
- Number of stop bits (1, 1.5 or 2)
- Line break generation and detection
- DMA signaling with two programmable modes
- Prioritized interrupt identification
- Programmable FIFO enable/disable
- Programmable serial data baud rate as calculated by the following:

$$\text{baud rate} = (\text{serial clock frequency}) / (16 \times \text{divisor})$$
- External read enable signal for RAM wake-up when using external RAMs
- Modem and status lines are independently controlled
- Separate system resets for each clock domain to prevent metastability
- Complete RTL version

1.5 Area and Power Number

For more information on power consumption, frequency, area and DFT coverage, see [“Performance”](#) on page 247.

1.6 Deliverables

DW_apb_uart is packaged as a .run file. The license file required to use this Synopsys DesignWare IP is delivered separately. Use the Synopsys coreConsultant tool to configure, synthesize, and simulate the IP.

The DW_apb_uart image contains the following:

- System Verilog RTL source code
- System Verilog based test suite that integrates the DUT and AMBA SVT APB2/APB3/APB4 VIP
- Synthesis scripts for Synopsys Design Compiler and Synopsys Fusion Compiler; Synplify Pro Simulation regression scripts for Synopsys VCS tool
- Spyglass and VC SpyGlass Lint, CDC, RDC checker rules used for linting, Clock Domain Crossing checks and Reset Domain crossing checks respectively

1.7 Verification Environment Overview

The DW_apb_uart includes an extensive verification environment, which sets up and invokes your selected simulation tool to execute tests that verify the functionality of the configured component. You can then analyze the results of the simulation.

The “Verification Environment” chapter discusses the specific procedures for verifying the DW_apb_uart.

1.8 Licenses

Before you begin using the IP, you must have a valid license. For more information, see *Licenses* in the *DesignWare Synthesizable Components for AMBA 2 Installation Guide*.

1.9 Speed and Clock Requirements

The DW_apb_uart has been synthesized and simulated with a pclk of 166 MHz in 28nm technology. It meets timing requirements at these speeds. The sclk signal is set to 25 MHz with a baud divisor of 1 to give a max baud rate of just over 1.5 M. This is the baud rate referred to in the National 16550 specification.

1.10 Where To Go From Here

At this point, you may want to get started working with the DW_apb_uart component within a subsystem or by itself. Synopsys provides several tools within its coreTools suite of products for the purposes of configuration, synthesis, and verification of single or multiple synthesizable IP components — coreConsultant and coreAssembler. For information on the different coreTools, see *Guide to coreTools Documentation*.

For more information about configuring, synthesizing, and verifying just your DW_apb_uart component, see “Overview of the coreConsultant Configuration and Integration Process” in *DesignWare Synthesizable Components for AMBA 2 User Guide*.

For more information about implementing your DW_apb_uart component within a DesignWare subsystem using coreAssembler, see “Overview of the coreAssembler Configuration and Integration Process” in *DesignWare Synthesizable Components for AMBA 2 User Guide*.

2

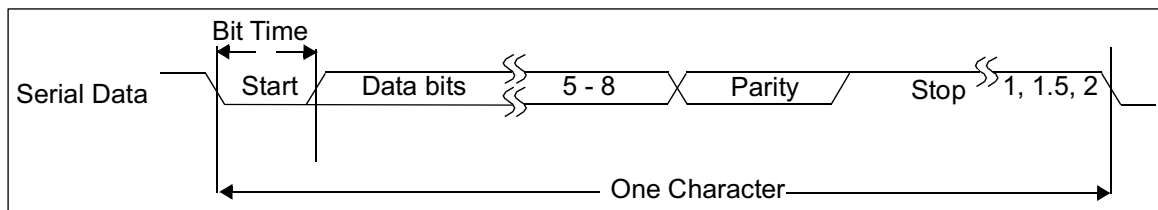
Functional Description

This chapter describes the functional operation of the DW_apb_uart. This chapter includes the following topics:

- [“UART \(RS232\) Serial Protocol”](#) on page 29
- [“9-bit Data Transfer”](#) on page 31
- [“RS485 Serial Protocol”](#) on page 36
- [“Fractional Baud Rate Support”](#) on page 43
- [“IrDA 1.0 SIR Protocol”](#) on page 47
- [“FIFO Support”](#) on page 49
- [“Clocks and Resets”](#) on page 51
- [“Synchronization”](#) on page 52
- [“Back-to-Back Character Stream Transmission”](#) on page 54
- [“Interrupts”](#) on page 56
- [“Auto Flow Control”](#) on page 58
- [“Programmable THRE Interrupt”](#) on page 61
- [“Clock Gate Enable”](#) on page 63
- [“DMA Support”](#) on page 66
- [“APB Interface”](#) on page 81

2.1 UART (RS232) Serial Protocol

Because the serial communication between the DW_apb_uart and a selected device is asynchronous, additional bits (start and stop) are added to the serial data to indicate the beginning and end. Utilizing these bits allows two devices to be synchronized. This structure of serial data – accompanied by start and stop bits – is referred to as a character, as shown in [Figure 2-1](#).

Figure 2-1 Serial Data Format

An additional parity bit can be added to the serial character. This bit appears after the last data bit and before the stop bits in the character structure in order to provide the DW_apb_uart with the ability to perform simple error checking on the received data.

The DW_apb_uart Line Control Register (section “LCR” in [“Register Descriptions”](#) on page 121) is used to control the serial character characteristics. The individual bits of the data word are sent after the start bit, starting with the Least-Significant Bit (LSB). These are followed by the optional parity bit, followed by the stop bits, which can be 1, 1.5, or 2.

**Note**

The STOP bit duration implemented by DW_apb_uart can appear longer due to:

- Idle time inserted between characters for some configurations
- Baud clock divisor values in the transmit direction

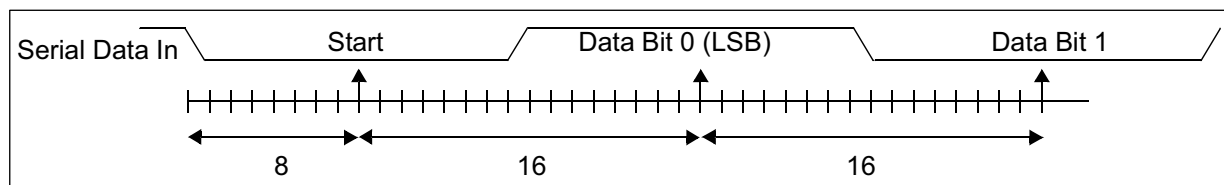
For details on idle time between transmitted transfers, see [“Back-to-Back Character Stream Transmission”](#) on page 54.

All the bits in the transmission are transmitted for exactly the same time duration; the exception to this is the half-stop bit when 1.5 stop bits are used. This duration is referred to as a Bit Period or Bit Time; one Bit Time equals sixteen baud clocks.

To ensure stability on the line, the receiver samples the serial input data at approximately the midpoint of the Bit Time once the start bit has been detected. Because the exact number of baud clocks is known for which each bit is transmitted, calculating the midpoint for sampling is not difficult; that is, every sixteen baud clocks after the midpoint sample of the start bit.

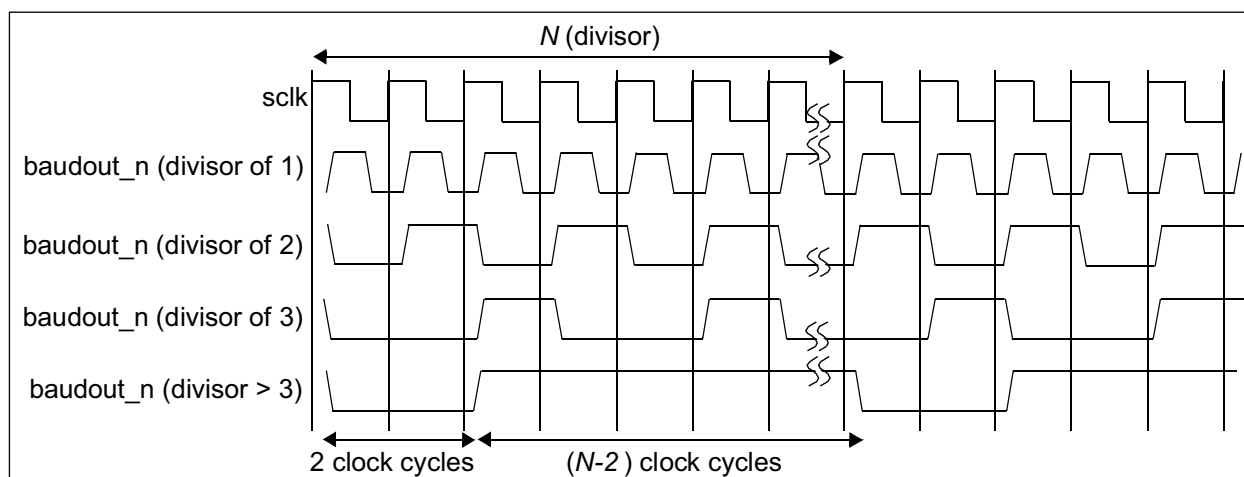
Together with serial input debouncing, this sampling helps to avoid the detection of false start bits. Short glitches are filtered out by debouncing, and no transition is detected on the line. If a glitch is wide enough to avoid filtering by debouncing, a falling edge is detected. However, a start bit is detected only if the line is again sampled low after half a bit time has elapsed.

[Figure 2-2](#) shows the sampling points of the first two bits in a serial character.

Figure 2-2 Receiver Serial Data Sample Points

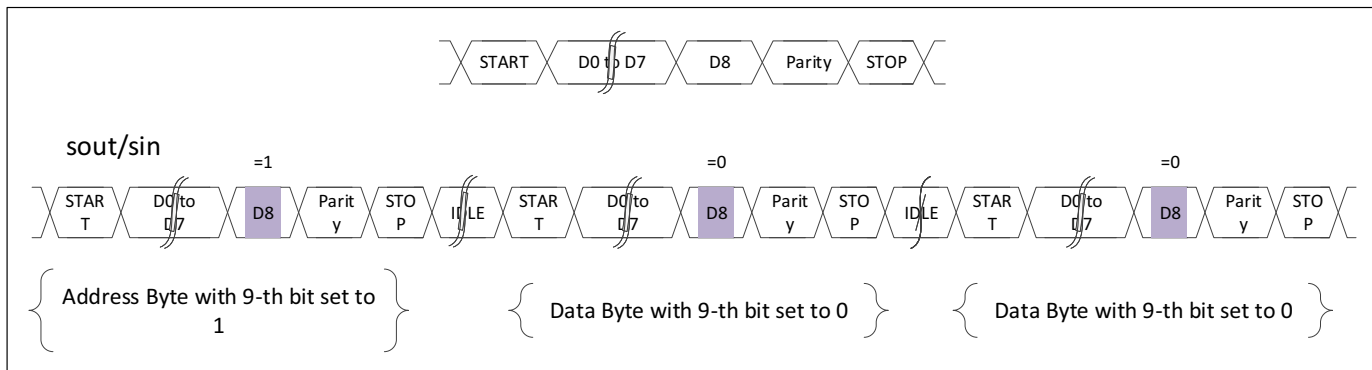
As part of the 16550 standard, an optional baud clock reference output signal (`baudout_n`) provides timing information to receiving devices that require it. The baud rate of the DW_apb_uart is controlled by the serial clock — `skclk` or `pclk` in a single clock implementation — and the Divisor Latch Register (DLH and DLL).

Figure 2-3 shows the timing diagram for the `baudout_n` output for different divisor values.

Figure 2-3 Baud Clock Reference Timing Diagram

2.2 9-bit Data Transfer

The DW_apb_uart can be configured to have 9-bit data transfer in both transmit and receive mode. The 9th bit in the character appears after the 8th bit and before the parity bit in the character. Figure 2-4 shows the serial transmission for a character in which D8 represents the 9th bit and also shows general serial transmission in the 9-bit mode.

Figure 2-4 9-Bit Character

By enabling 9-bit data transfer mode, you can use DW_apb_uart in multi-drop systems where one requester is connected to multiple completers in a system. The requester communicates with one of the completers. When the requester wants to transfer a block of data to a completer, it first sends an address byte to identify the target completer.

The differentiation between the address/data byte is done based on the 9th bit in the incoming character. If the 9th bit is set to 0, then the character represents a data byte. If the 9th bit is set to 1, then the character represents address byte. All the completer systems compare the address byte with their own address and only the target completer (in which the address has matched) is enabled to receive data from the requester. The requester then starts transmitting data bytes to the target completer. The non-addressed completer systems ignore the incoming data until a new address byte is received.

In [Figure 2-4](#), note that one address is followed by 2 data bytes. The address byte goes out with the 9th bit (D8) set to 1 and the data bytes go out with 9th bit (D8) set to 0. The parity bit is an optional field.

Configuration of the DW_apb_uart for 9-bit data transfer does the following:

- LCR_EXT[0] bit is used to enable or disable the 9-bit data transfer.
- LCR_EXT[1] bit is used to choose between hardware and software based address match in the case of receive.
- LCR_EXT[2] bit is used to enable to send the address in the case of transmit.
- LCR_EXT[3] bit is used to choose between hardware and software based address transmission.
- TAR and RAR registers are used to transmit address and to match the received address, respectively.
- THR, RBR, STHR and SRBR registers are of 9-bit which is used to do the data transfers in 9-bit mode.
- LSR[8] bit is used to indicate the address received interrupt.

**Note**

The 9-bit data mode is supported only when the DWC-APB-Advanced-Source source license exists.

2.2.1 Transmit Mode

DW_apb_uart supports two types of transmit modes:

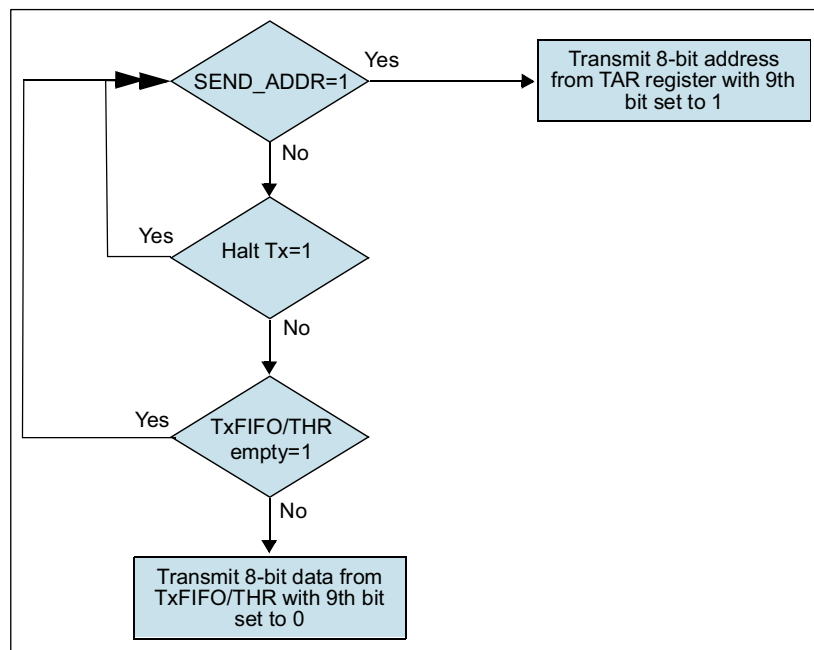
- Transmit Mode 0 (when (LCR_EXT[3]) is set to 0)
- Transmit Mode 1 (when (LCR_EXT[3]) is set to 1)

2.2.1.1 Transmit Mode 0

In transmit mode 0, the address is programmed in the Transmit Address Register (TAR) and data is written into the Transmit Holding Register (THR) or the Shadow Transmit Holding Register (STHR). The 9th bit of the THR and STHR register is not applicable in this mode.

Figure 2-5 illustrates the transmission of address and data based on SEND_ADDR (LCR_EXT[2]), Halt Tx, and TxFIFO/THR empty conditions.

Figure 2-5 Auto Address Transmit Flow Chart



The address of the target completer to which the data is to be transmitted is programmed in the TAR register. You must enable the SEND_ADDR (LCR_EXT[2]) bit to transmit the target completer address present in the TAR register on the serial UART line with 9th data bit set to 1 to indicate that the address is being sent to the completer. The DW_apb_uart clears the SEND_ADDR bit after the address character starts transmitting on the UART line.

The data required to transmit to the target completer is programmed through Transmit Holding Register (THR). The data is transmitted on the UART line with 9th data bit set to 0 to indicate data is being sent to the completer.

If the application is required to fill the data bytes in the TxFIFO before sending the address on the UART line (before setting LCR_EXT[2]=1), then it is recommended to set the "Halt Tx" to 1 such that DW_apb_uart does not start sending out the data in the TxFIFO as data byte. Once the TxFIFO is filled, then program SEND_ADDR (LCR_EXT[2]) to 1 and then set "Halt Tx" to 0.

2.2.1.2 Transmit Mode 1

In transmit mode 1, THR and STHR registers are of 9-bit wide and both address and data are programmed through the THR and STHR registers. The DW_apb_uart does not differentiate between address and data, and both are taken from the TxFIFO. The SEND_ADDR (LCR_EXT[2]) bit and Transmit Address Register (TAR) are not applicable in this mode. The software must pack the 9th bit with 1/0 depending on whether address/data has to be sent.

2.2.2 Receive Mode

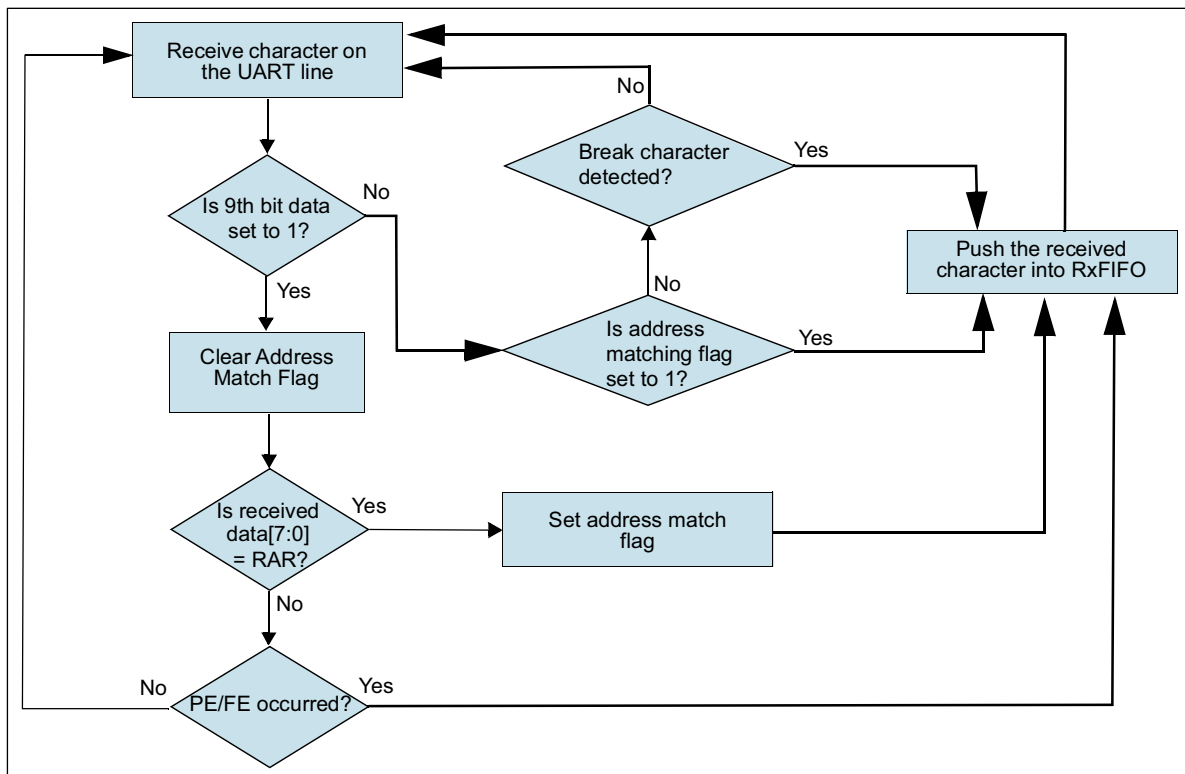
The DW_apb_uart supports two receive modes:

- Hardware Address Match Receive Mode (when ADDR_MATCH (LCR_EXT[1]) is set to 1)
- Software Address Match Receive Mode (when ADDR_MATCH (LCR_EXT[1]) is set to 0)

2.2.2.1 Hardware Address Match Receive Mode

In the hardware address match receive mode, the DW_apb_uart matches the received character with the address programmed in the Receive Address register (RAR), if the 9th bit of the received character is set to 1. If the received address is matched with the programmed address in RAR register, then subsequent data bytes (with 9th bit set to 0) are pushed into the RxFIFO. If the address matching fails, then DW_apb_uart controller discards further data characters until a matching address is received.

[Figure 2-6](#) illustrates the flow chart for the reception of data bytes based on the address matching feature.

Figure 2-6 Hardware Address Match Receive Mode

DW_apb_uart receives the character irrespective of whether the 9th bit data is set to 1. If 9th bit of the received character is set to 1, then it clears internal address match flag and then compares the received 8-bit character information with the address programmed in the RAR register.

If the received address character matches with the address programmed in the RAR register, then the address match flag is set to 1 and the received character is pushed to the RxFIFO in FIFO-mode or to RBR register in non-FIFO mode and the ADDR_RCVD bit in LSR register is set to indicate that the address has been received.

In case of parity or if a framing error is found in the received address character and if the address is not matched with the RAR register, then the received address character is still pushed to RxFIFO or RBR register with ADDR_RCVD and PE/FE error bit set to 1.

The subsequent data bytes (9th bit of received character is set to 0) are pushed to the Rx_FIFO in FIFO mode or to the RBR register in non-FIFO mode until the new address character is received.

If any break character is received, DW_apb_uart treats it as a special character and pushes to the RxFIFO or RBR register based on the FIFO_MODE irrespective of address match flag.

**Note**

The break character can be used to alert the complete system in case all completers are in sleep mode (entered in to the low power mode). Therefore, the break character is treated as special character.

2.2.2.2 Software Address Match Receive Mode

In this mode of operation, the DW_apb_uart does not perform the address matching for the received address character (9th bit data set to 1) with the RAR register. The DW_apb_uart always receives the 9-bit data and pushes in to RxFIFO in FIFO mode or to the RBR register in non-FIFO mode. The user must compare the address whenever address byte is received and indicated through ADDR_RCVD bit in the Line Status register. The user can flush/reset the RxFIFO in case of address not matched through 'RCVR FIFO Reset' bit in FIFO Control Register (FCR).

2.3 RS485 Serial Protocol

The RS485 standard supports serial communication over a twisted pair configuration, such as RS232. The difference between the RS232 and RS485 standards is its use of a balanced line for transmission. This usage is also known as the differential format that sends the same signal on two separate lines with phase delay and then compares the signals at the end, subtracts any noise, and adds them to regain signal strength. This process allows the RS485 standard to be viable over significantly longer distances than its short range RS232 counterpart.

DW_apb_uart supports the RS485 serial protocol that enables transfer of serial data using the RS485 interface. The Driver Enable (DE) and Receiver Enable (RE) signals are generated for enabling the RS485 interface support. The 'de' and 're' signals are hardware generated and the assertion/de-assertion times for these signals are programmable. The active level of these signals are configurable.

Configuration of the DW_apb_uart for RS485 interface does the following:

1. Bit 0 of the Transceiver Control Register (TCR) enables or disables the RS485 mode.
2. Bit 1 and bit 2 of TCR are used to select the polarity of RE and DE signals.
3. Bit [4:3] of the TCR selects the type of transfer in RS485 mode.
4. Driver output enable (DE_EN) and Receiver output enable (RE_EN) registers are used for software control of DE and RE signals.
5. Driver output Enable Timing (DET) register is used to program the assertion and de-assertion timings of DE signal.
6. Turn Around Timing (TAT) register is used to program the turnaround time from DE to RE and RE to DE.



Note

RS485 interface mode is supported only when the source license DWC-APB-Advanced-Source exists.

2.3.1 DE Assertion and De-assertion Timing

The assertion and de-assertion timings of the DE signal are controlled through the DET register:

- DE assertion time (DET[7:0]): The assertion time is the time between the activation of the DE signal and the beginning of the START bit. The value represented is in terms of serial clock cycles.
- DE de-assertion time (DET[15:8]): The de-assertion time is the time between the end of the last stop bit, in a transmitted character, and the de-activation of the DE signal. The value represented is in terms of serial clock cycles.

Hardware ensures that these values are met for DE assertion and DE de-assertion before/after active data transmission.

In [Figure 2-7](#), t1 represents DE assertion time and t2 represents DE de-assertion time. Note that for simplicity only one data is illustrated in [Figure 2-7](#); however, DE does not get de-asserted if there are more data characters in transmit FIFO. DE gets de-asserted only after all the data characters are transmitted.

Figure 2-7 DE Assertion and De-Assertion



2.3.2 RS485 Modes

DW_apb_uart consists of the following RS485 modes based on the XFER_MODE field in the Transceiver Control Register (TCR):

- Full Duplex Mode – In this mode, XFER_MODE of TCR is set to 0.
- Software-Controlled Half Duplex Mode – In this mode, XFER_MODE of TCR is set to 1.
- Hardware-Controlled Half Duplex Mode – In this mode, XFER_MODE of TCR is set to 2

2.3.2.1 Full Duplex Mode

The full duplex mode supports both transmit and receive transfers simultaneously.

In Full Duplex mode, the 'de' signal:

- Goes active if both these conditions are satisfied:
 - When the DE Enable (DE_EN[0]) field of Driver Output Enable Register is set to 1.
 - Transmitter Holding Register is not empty in non-FIFO mode or transmitter FIFO is not empty in FIFO mode.
- Goes inactive if both these conditions are satisfied
 - When the current ongoing transmitting serial transfer is completed.
 - Either DE Enable (DE_EN[0]) of Driver Output Enable Register is set to 0, transmitter FIFO is empty in FIFO mode or Transmitter Holding Register is empty in non-FIFO mode.

In Full Duplex mode, the 're' signal:

- Goes active when RE Enable (RE_EN[0]) of Receiver Output Enable Register is set to 1.
- Goes inactive when RE Enable (RE_EN[0]) of Receiver Output Enable Register is set to 0.

The user can choose when to transmit or when to receive. Both 're' and 'de' can be simultaneously asserted or de-asserted at any time. DW_apb_uart does not impose any turnaround time between transmit and receive ('de' to 're') or receive to transmit ('re' to 'de') in this mode. This mode can directly be used in full duplex operation where separate differential pair of wires is present for transmit and receive.

2.3.2.2 Software-Controlled Half Duplex Mode

The software-controlled half duplex mode supports either transmit or receive transfers at a time but not both simultaneously. The switching between transmit to receive or receive to transmit is through programming the Driver output enable (DE_EN) and Receiver output enable (RE_EN) registers.

In software-controlled Half Duplex mode, the 'de' signal:

- Goes active if the following conditions are satisfied:
 - The DE Enable (DE_EN[0]) field of the Driver Output Enable Register is set to 1.
 - Transmitter Holding Register is not empty in non-FIFO mode or transmitter FIFO is not empty in FIFO mode.
 - If any receive transfer is ongoing, then the signal waits until receive has finished, and after the turnaround time counter ('re to de') has elapsed.
- Goes inactive if the following conditions are satisfied:
 - The current ongoing transmitting serial transfer is completed.
 - The DE Enable (DE_EN[0]) field of Driver Output Enable Register is set to 0.
 - Either transmitter FIFO is empty in FIFO mode or Transmitter Holding Register is empty in non-FIFO mode.

In software-controlled half duplex mode, the 're' signal:

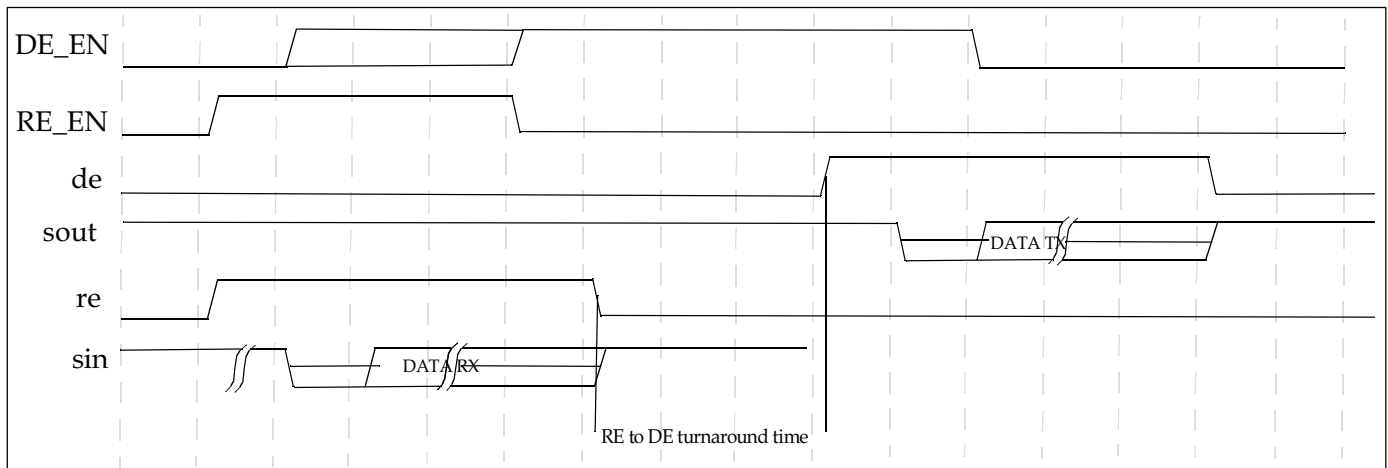
- Goes active if the following conditions are satisfied:
 - When RE Enable (RE_EN[0]) field of Receiver Output Enable Register is set to 1.
 - If any transmit transfer is ongoing, then the signal waits until transmit has finished and after the turnaround time counter ('de to re') has elapsed.
- Goes in-active under the following conditions:
 - The current ongoing receive serial transfer is completed.
 - When RE Enable (RE_EN[0]) of Receiver Output Enable Register is set to 0.

The user must enable either DE or RE but not both at any point of time. As 're' and 'de' signals are mutually exclusive, the user must ensure that both of them are not programmed to be active at any point of time.

In this mode, the hardware ensures that a proper turnaround time is maintained while switching from 're' to 'de' or from 'de' to 're' (value of turnaround is obtained from the TAT register, in terms of serial clock cycles) as shown in [Figure 2-8](#) and [Figure 2-9](#).

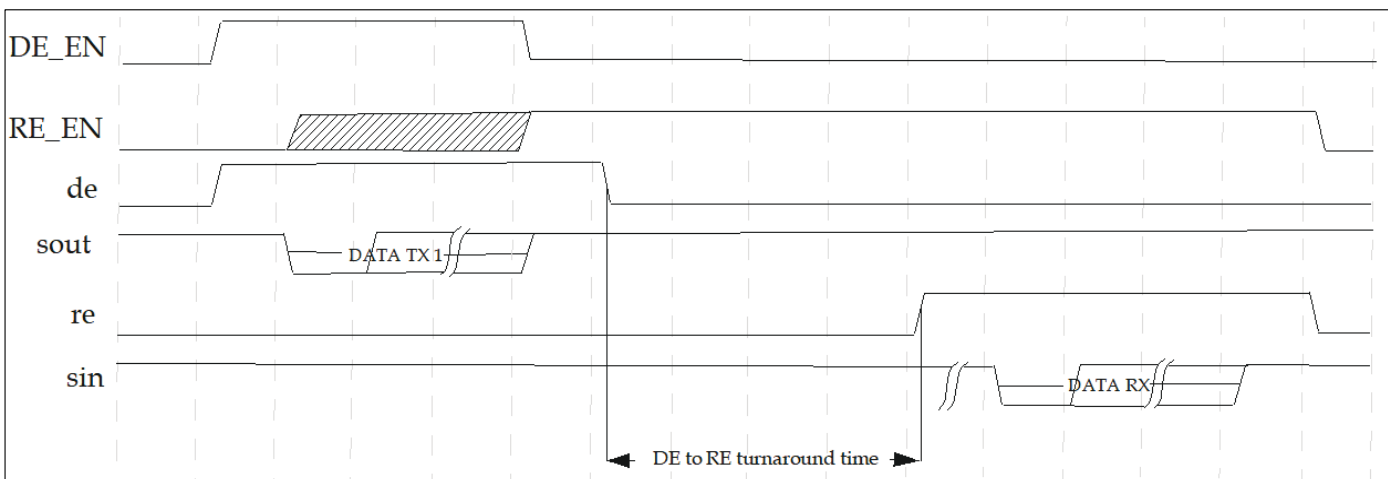
2.3.2.2.1 RE to DE Turnaround Time

DW_apb_uart inserts the wait state (as programmed in TAT[31:16] times serial clock) before switching to transmit mode from receive mode as shown in the [Figure 2-8](#) (applicable only when TCR[4:3] = 1 or 2 (XFER_MODE)).

Figure 2-8 RE to DE Turnaround Time

2.3.2.3 DE to RE Turnaround Time

DW_apb_uart inserts the wait state (as programmed in TAT[15:0] times serial clock) before switching to receive mode from transmit mode as shown in the [Figure 2-9](#) (applicable only when TCR[4:3] = 1 or 2 (XFER_MODE).)

Figure 2-9 DE to RE Turnaround Time

2.3.2.4 Hardware-Controlled Half Duplex Mode

The hardware-controlled half duplex mode supports either transmit or receive transfers at a time but not both simultaneously. If both 'DE Enable' and 'RE Enable' bits of Driver output enable (DE_EN) and Receiver output enable (RE_EN) registers are enabled, the switching between transmit to receive or receive to transmit is automatically done by the hardware based on the empty condition of Tx-FIFO.

In hardware-controlled half duplex mode, the 'de' signal:

- Goes active if the following conditions are satisfied:

- The DE Enable (DE_EN[0]) field of Driver Output Enable Register is set to 1.
- Transmitter Holding Register is not empty in non-FIFO mode or transmitter FIFO is not empty in FIFO mode.
- If any receive transfer is ongoing, then the signal waits until receive is finished and after the turnaround time counter ('re to de') has elapsed.
- Goes inactive if the following conditions are satisfied
 - The current ongoing transmitting serial transfer is completed.
 - Either transmitter FIFO is empty in FIFO mode or Transmitter Holding Register is empty in non-FIFO mode or the DE Enable (DE_EN[0]) of Driver output Enable Register is set to 0.

In hardware-controlled half duplex mode, the 're' signal:

- Goes active if the following conditions are satisfied:
 - When RE Enable (RE_EN[0]) field of Receiver Output Enable Register is set to 1.
 - Either transmitter FIFO is empty in FIFO mode or Transmitter Holding Register is empty in non-FIFO mode.
 - If any transmit transfer is ongoing, then the signal waits until transmit is finished and after the turnaround time counter ('de to re') has elapsed.
- Goes inactive under the following conditions:
 - The current ongoing receive serial transfer has completed.
 - Either transmitter FIFO is non-empty in FIFO mode or Transmitter Holding Register is non-empty in non-FIFO mode or the RE Enable (RE_EN[0]) of Receiver output Enable Register is set to 0.

In this mode, the hardware ensures that a proper turnaround time is maintained while switching from 're' to 'de' or from 'de' to 're' (value of turnaround is obtained from the TAT register, in terms of serial clock cycles) as shown in [Figure 2-8](#) and [Figure 2-9](#).

2.3.3 Sample Scenarios

Consider a scenario in which the DW_apb_uart controller is receiving 3 characters and another UART device is sending those characters. While the 1st character is being received by the DW_apb_uart controller, if the software writes into the TX FIFO of the DW_apb_uart controller, then at the end of the first character DW_apb_uart controller switches the mode from receive to transmit. DW_apb_uart de-asserts 're' and assert 'de' signal. This makes the DW_apb_uart controller not to receive the subsequent characters. Hence, in hardware switching half duplex mode, the user has to ensure that complete receive data has been received before writing in to the Tx-FIFO to avoid missing of receive characters.

Following sections explain the behavior of DW_apb_uart in XFER_MODE=2 for different scenarios.

2.3.3.1 Normal Scenario of Transmission

[Figure 2-10](#) is a sample scenario for normal transmission.

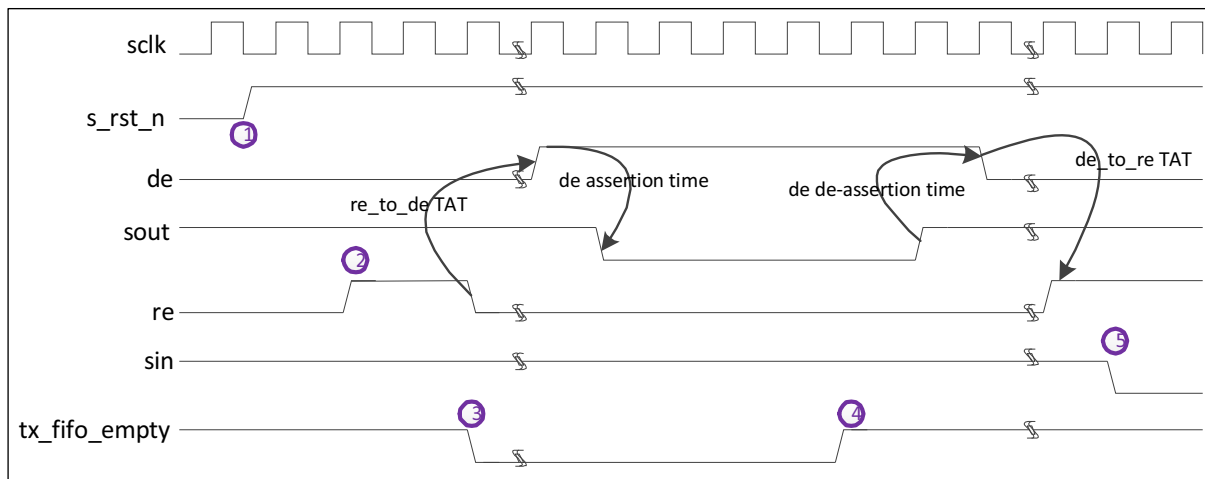
Figure 2-10 Scenario When XFER_MODE=2

Figure 2-10 shows the following activities at various points in this scenario:

1. At this point, reset is removed, and 'de' and 're' signals are driven to their configured reset values (UART_DE_POL/UART_RE_POL).
2. At this point, the software programs DE_EN and RE_EN register to 1. At this point in time, tx_fifo_empty * is 1 indicating that there is no data in TX FIFO. Hence, the 'de' signal remains deasserted and 're' gets asserted.

* tx_fifo_empty is internal signal of DW_apb_uart

3. At this point, the software fills the TX FIFO and there is no ongoing Receive transfer. Therefore, the 're' signal goes low. However, the DW_apb_uart controller waits until 're_to_de' TAT value before asserting 'de' signal. After the 'de' gets asserted, the transmission of character starts considering the 'de-asserting timing'.
4. At this point, TX FIFO becomes empty. After transmitting the current character, DW_apb_uart deasserts the 'de' signal (after de-assertion time). DW_apb_uart controller waits until 'de_to_re' TAT values before asserting 're' signal back.
5. At this point, DW_apb_uart controller starts receiving the character.

2.3.3.2 Scenario When Receive is in Progress While TX FIFO is Being Filled

In this scenario, TX FIFO is filled when a character is being received. In this case, DW_apb_uart is expected to wait till the current character is finished before changing the role and start transmitting.

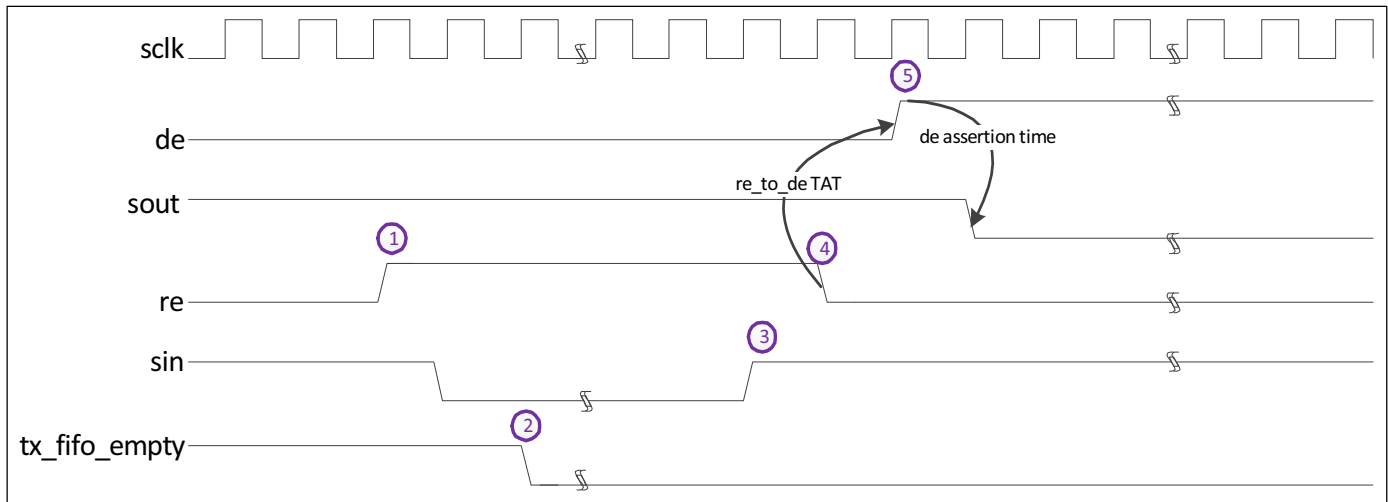
Figure 2-11 Receive in Progress, When TX FIFO is Filled

Figure 2-11 shows the following activities at various points in this scenario:

1. The software programs DE_EN and RE_EN to 1, thereby asserting the 're' signal. After this, the DW_apb_uart controller starts receiving the character.
2. The software programs TX FIFO thereby making 'tx_fifo_empty' to go low. However, the DW_apb_uart controller waits until the current character is received before asserting the 'de' signal.
3. The incoming character is fully received.
4. The 're' signal gets de-asserted, after the STOP bit is fully received.
5. After the 're_to_de' TAT, the 'de' signal gets asserted and the DW_apb_uart controller starts transmitting after DET timings.

2.3.3.3 TX FIFO Filled Before Enabling DE_EN and RE_EN Registers

In this case, TX FIFO is filled prior to enabling DE_EN or RE_EN. The DW_apb_uart controller enables the 'de' instead of 're' in this case because TX FIFO already has the data to transmit.

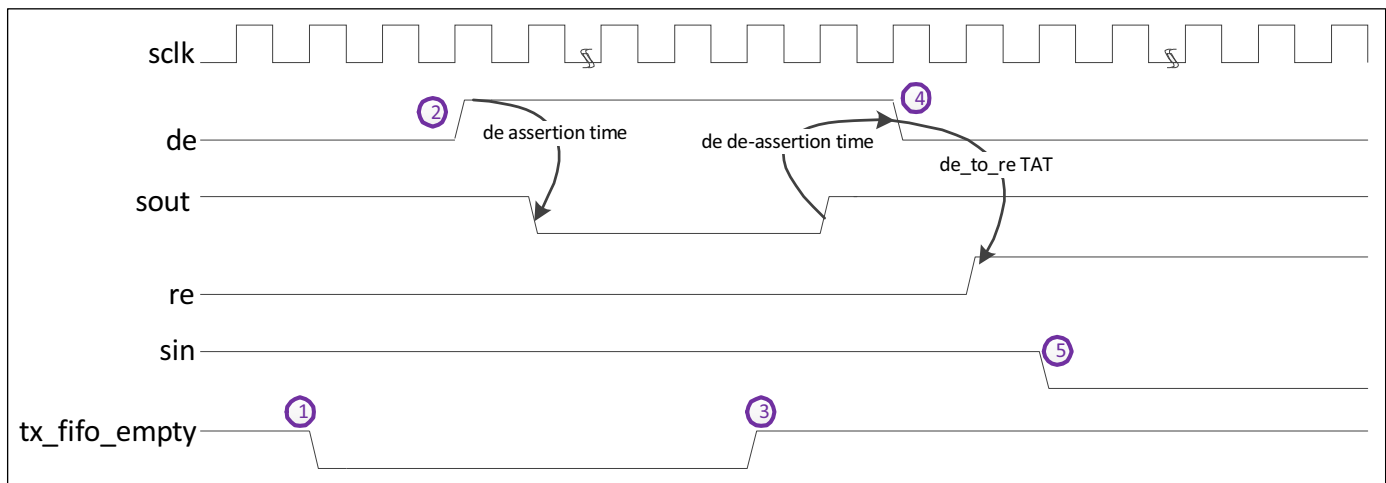
Figure 2-12 TX FIFO is Filled Before Enabling DE/RE

Figure 2-12 shows the following activities at various points in this scenario:

1. The software programs the TX FIFO thereby making 'tx_fifo_empty' signal to go low.
2. The software programs 'DE_EN' and 'RE_EN' to 1. As the data is already present in the TX FIFO, DW_apb_uart controller asserts the 'de' signal. DW_apb_uart starts sending the character after the DET timings.
3. TX FIFO becomes empty.
4. The 'de' signal gets de-asserted after the DET timing. After 'de_to_re' TAT, 're' signal gets asserted.
5. DW_apb_uart controller starts receiving the incoming character.

2.4 Fractional Baud Rate Support

DW_apb_uart supports fractional baud rate that enables a user to program the fractional part of the divisor value to generate fractional baud rate that results in reduced frequency error. The UART interface usage has been evolving to include ever increasing baud rate speeds. The DW_apb_uart needs to be software configurable to handle the baud rates within 2% frequency error.

The Baud rate of DW_apb_uart is controlled by sclk in asynchronous serial clock (CLOCK_MODE=2) implementation or pclk in single clock implementation (CLOCK_MODE=1) and the Divisor Latch Register (DLH and DLL).

The baud rate is determined by the following factors:

- Serial clock operating frequency (sclk in Asynchronous serial clock implementation or pclk in single clock implementation)
- The desired baud rate
- The baud rate generator divisor value, DIVISOR (composed of DLH & DLL registers)
- The acceptable Baud-rate error, %ERROR

The equation to calculate the baud rate is as follows:

$$\text{Baud Rate} = \frac{\text{Serial Clock Operating Frequency}}{(16 \times \text{DIVISOR})} \quad 1$$

Where,

DIVISOR – Number (in hexadecimal) to program the DLL and DLH.

Serial clock frequency – Frequency at sclk or pclk pin of DW_apb_uart.

From Equation (1), DIVISOR can be calculated as:

$$\text{DIVISOR} = \frac{\text{Serial Clock Operating Frequency}}{(16 \times \text{Baud Rate})} \quad 2$$

Also from Equation (1), it can also be shown that:

$$\text{Serial clock frequency} = \text{Baud Rate} \times 16 \times \text{DIVISOR} \quad 3$$

The Error between the Baud rate and Baud rate (selected) is given as:

$$\text{Percentage ERROR} = \frac{|\text{Baud Rate} - \text{Baud Rate (Selected)}|}{\text{Baud Rate}} \times 100 \quad 4$$



Note

Fractional Baud rate is supported only when the source license DWC-APB-Advanced-Source exists.

Configuration of the DW_apb_uart for Fractional Baud Rate does the following:

- The configurable parameter DLF_SIZE is used to choose the width of the register that stores fractional part of the divisor.
- The fractional value of the divisor is programmed in the Divisor Latch Fraction Register (DLF). The fractional value is computed by using the $(\text{Divisor Fraction value}) / (2^{\text{DLF_SIZE}})$ formula. shows fractional values when the DLF_SIZE=4.

Table 2-1 Divisor Latch Fractional Values

DLF Value	Fraction	Fractional Value
0000	0/16	0.0000
0001	1/16	0.0625
0010	2/16	0.125
0011	3/16	0.1875

Table 2-1 Divisor Latch Fractional Values (Continued)

DLF Value	Fraction	Fractional Value
0100	4/16	0.25
0101	5/16	0.3125
0110	6/16	0.375
0111	7/16	0.4375
1000	8/16	0.5
1001	9/16	0.5625
1010	10/16	0.625
1011	11/16	0.6875
1100	12/16	0.75
1101	13/16	0.8125
1110	14/16	0.875
1111	15/16	0.9375

The programmable fractional baud rate divisor enables a finer resolution of baud clock than the conventional integer divider. The programmable fractional baud clock divider allows for the programmability of both an integer divisor as well as fractional component. The average frequency of the baud clock from the fractional baud rate divisor is dependent upon both the integer divisor and the fractional component, thereby providing a finer resolution to the average frequency of the baud clock.

$$\text{Baud Rate Divisor} = \frac{\text{Serial Clock Frequency}}{(16 \times \text{Required Baud Rate})} = \text{BRD}_I + \text{BRD}_F \quad (5)$$

Where,

BRD_I - Integer part of the divisor.

BRD_F - Fractional part of the divisor.

2.4.1 Fractional Division Used to Generate Baud Clock

Fractional division of clock is used by the N/N+1 divider, where N is the integer part of the divisor. N/N+1 division works on the basis of achieving the required average timing over a long period by alternating the division between two numbers. If N=1 and ratio of N/N+1 is same, which means equal number of divide by 1 and divide by 2 over a period of time, average time period would come out to be divided by 1.5. Varying the ratio of N/N+1 any value can be achieved above 1 and below 2.

2.4.2 Calculating the Fractional Value Error

Following is a sample for calculating the fractional value error.

Consider the following values:

- Required Baud Rate (RBR) = 4454400
- Serial Clock (SCLK) = 133 MHz
- DLF_SIZE = 4

Then, as per equation (5), Baud Rate Divisor (BRD) is as follows:

$$\text{BRD} = \frac{133}{16 \times 4454400} = 1.866132364 \quad (6)$$

In (6), the integer and fractional parts are as follows:

- Integer part (BRD_I) = 1
- Fractional part (BRD_F) = 0.866132364

Therefore, Baud Rate Divisor Latch Fractional Value (DLF) is as follows:

$$\text{DLF} = \text{BRD}_F \times 2^{\text{DLF_SIZE}} = 0.866132364 \times 16 = 13.858117824 = 14 \text{ (roundoff value)} \quad (7)$$

The Generated Baud Rate Divider (GD) is as follows:

$$\text{GD} = \text{BRD}_I + \frac{\text{DLF}}{2^{\text{DLF_SIZE}}} = 1 + \frac{14}{16} = 1.875 \quad (8)$$

Therefore, the Generated Baud Rate (GBR) is as follows:

$$\text{GBR} = \frac{\text{Serial Clock}}{(16 \times \text{GD})} = \frac{133}{16 \times 1.875} = 4433333.333 \quad (9)$$

Now the error is calculated as follows:

$$\text{Error} = \frac{\text{GBR} - \text{RBR}}{\text{RBR}} = 0.004729 \quad (11)$$

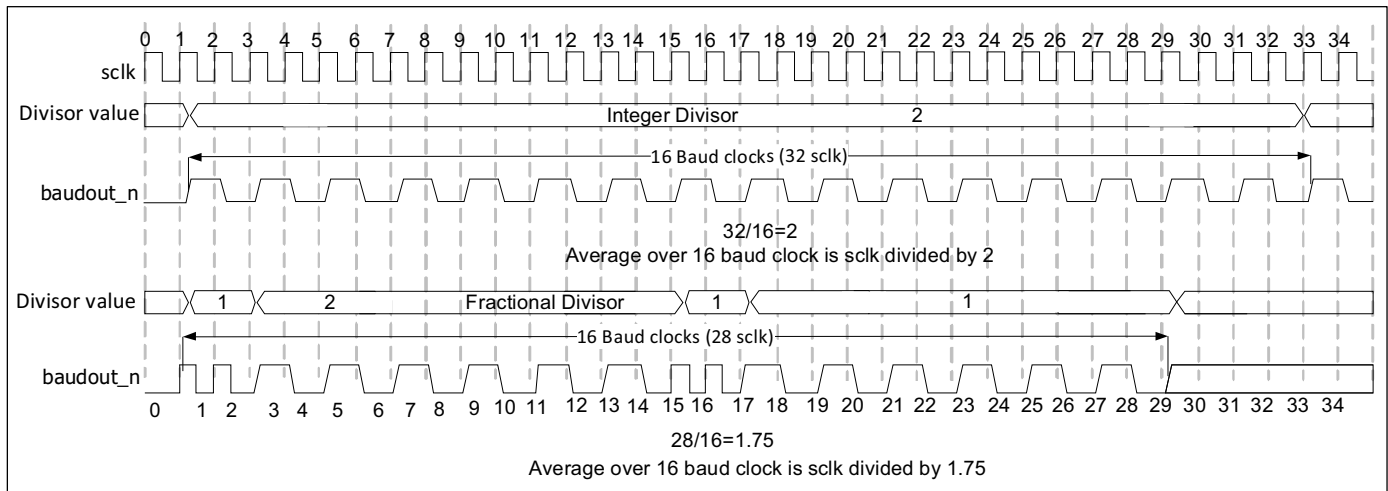
The error percentage is as follows:

$$\text{Error \%} = 0.004729 \times 100 = 0.473 \quad (12)$$

2.4.2.1 Timing Waveforms

If serial clock is 25 MHz and Baud rate required = 892857, divisor comes out to be 1.75. Without a fractional division a value of 1 or 2 results in baud rate of 1562500 or 781250 which is more than 2% frequency error. However, if you divide 12 clocks by 2 and then 4 clocks by 1, over an average period of 16 clocks you achieve division by 1.75. Using this divisor baud rate of 892857 can be achieved.

As shown in the [Figure 2-13](#), the fractional baud clock is generated between N(1) and N+1(2) values to generate the fractional baud rate of 1.75 to achieve the divisor baud rate of 892857 with 0% frequency error compared to 12.49% frequency error in integer baud clock generator.

Figure 2-13 Example of Integer and Fractional Division Over 16 Clock Periods

2.5 IrDA 1.0 SIR Protocol

The Infrared Data Association (IrDA) 1.0 Serial Infrared (SIR) mode supports bi-directional data communications with remote devices using infrared radiation as the transmission medium. IrDA 1.0 SIR mode specifies a maximum baud rate of 115.2 Kbaud.



Attention

Information provided on IrDA SIR mode in this section assumes that the reader is fully familiar with the IrDa Serial Infrared Physical Layer Specifications. This specification can be obtained from the following website:

<http://www.irda.org>

The data format is similar to the standard serial — sout and sin — data format. Each data character is sent serially in this order:

1. Begins with a start bit
2. Followed by 8 data bits
3. Ends with at least one stop bit

Thus, the number of data bits that can be sent is fixed. No parity information can be supplied, and only one stop bit is used in this mode. Trying to adjust the number of data bits sent or enable parity with the Line Control Register (LCR) has no effect.

Configuration of the DW_apb_uart for IrDA 1.0 SIR does the following:

- Bit 6 of the Mode Control Register (MCR) enables or disables the IrDA 1.0 SIR mode.
- Disabling IrDA SIR mode causes the logic to not be implemented; the mode cannot be activated, which reduces total gate counts.
- When IrDA SIR mode is enabled and active, serial data is transmitted and received on the sir_out_n and sir_in ports, respectively.

**Note**

To enable SIR mode, write the appropriate value to the MCR register before writing to the LCR register. For details of the recommended programming sequence, see [“Programming Examples”](#) on page 222.

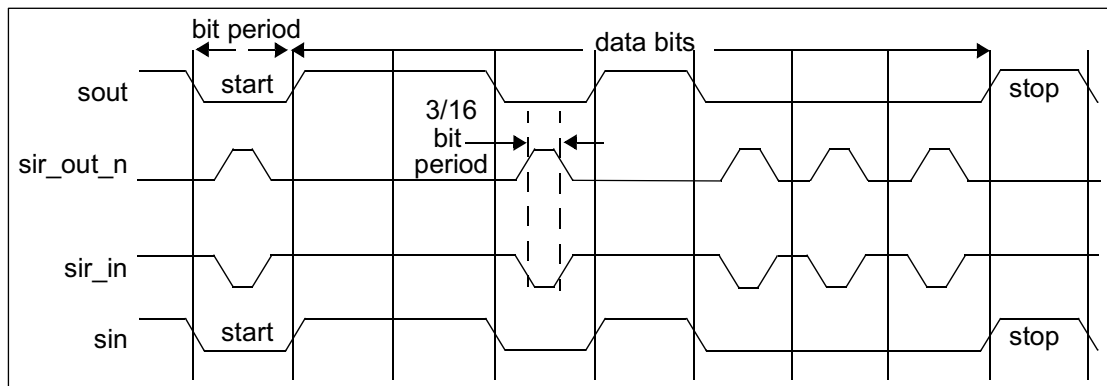
Transmission or non-transmission of a single infrared pulse indicates the following:

- Transmitting a single infrared pulse indicates logic 0
- Non-transmission of a pulse indicates logic 1

The width of each pulse is 3/16ths of a normal serial bit time. Thus, each new character begins with an infrared pulse for the start bit. However, received data is inverted from transmitted data due to infrared pulses energizing the photo transistor base of the IrDA receiver, which pulls its output low. This inverted transistor output is then fed to the DW_apb_uart `sir_in` port, which gives it the correct UART polarity.

[Figure 2-14](#) shows the timing diagram for the IrDA SIR data format in comparison to standard serial format.

Figure 2-14 IrDA SIR Data Format



As previously mentioned, the DW_apb_uart can be configured to support a low-power reception mode. When the DW_apb_uart is configured in this mode, it is possible to receive SIR pulses of 1.41 microseconds (minimum pulse duration), as well as nominal 3/16 of a normal serial bit time. In order to use this low-power reception mode, you must program the Low Power Divisor Latch (LPDLL/LPDLH) registers.

For all `sclk` frequencies greater than or equal to 7.37 MHz, pulses of 1.41 μ s are detectable; these pulses comply with the requirements of the Low Power Divisor Latch registers. However, there are several values of `sclk` that do not allow detection of such a narrow pulse, as indicated in [Table 2-2](#).

Table 2-2 Narrow Pulse Exceptions

SCLK	Low Power Divisor Latch Register Value	Min Pulse Width for Detection *
1.84MHz	1	3.77us
3.69MHz	2	2.086us
5.53MHz	3	1.584us
* 10% has been added to the internal pulse width signal to cushion the effect of pulse reduction due to the synchronization and data integrity logic so that a pulse slightly narrower than these may be detectable.		

When IrDA SIR mode is enabled, the DW_apb_uart operates in a manner similar to when the mode is disabled, with one exception: data transfers can only occur in half-duplex fashion when IrDA SIR mode is enabled. This is because the IrDA SIR physical layer specifies a minimum of 10ms delay between transmission and reception; this 10ms delay must be generated by software.

2.6 FIFO Support

You can configure the DW_apb_uart to implement FIFOs that buffer transmit and receive data; this is illustrated in [Figure 1-2](#). If FIFO support is not selected, then no FIFOs are implemented and only a single receive data byte and transmit data byte can be stored at a time in the RBR and THR registers; this implies a 16450-compatible mode of operation. However, in this mode of operation, most of the enhanced features are unavailable.

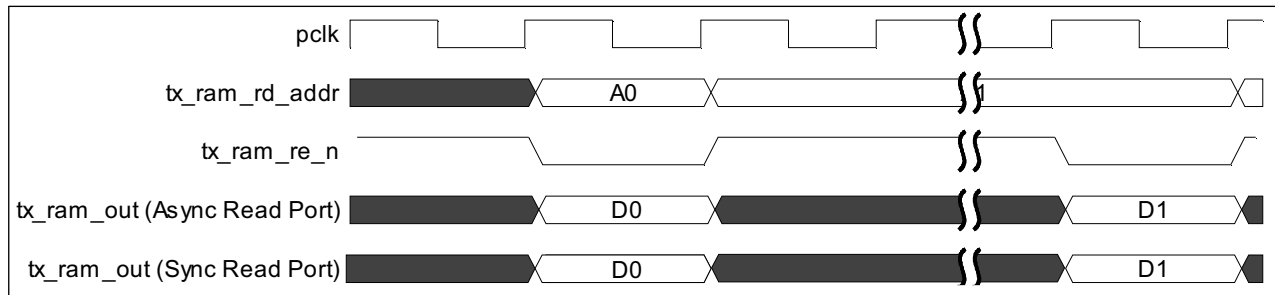
In FIFO mode, the FIFOs can be selected to be either of the following:

- External customer-supplied FIFO RAMs
- Internal DesignWare D-flip-flop-based RAMs (DW_ram_r_w_s_dff)

If the configured FIFO depth is greater than 256, the FIFO memory selection is restricted to be external. Additionally, selecting internal memory restricts the Memory Read Port Type to D-flip-flop-based Synchronous read port RAMs.

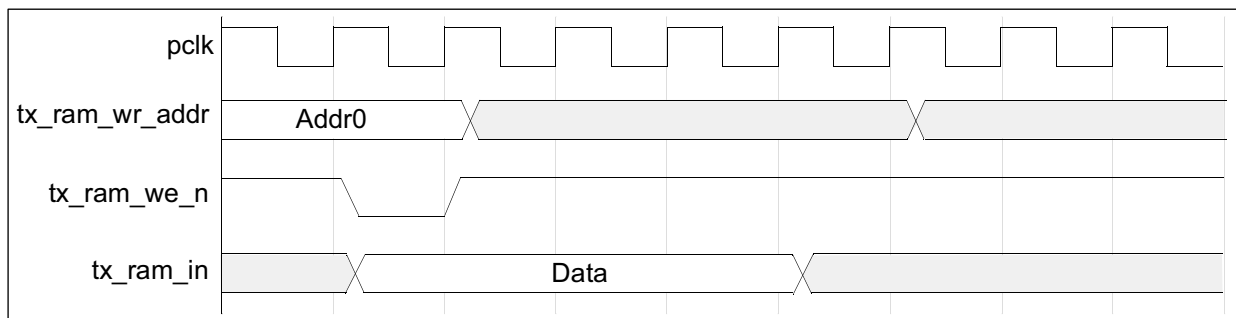
When external RAM support is chosen, either synchronous or asynchronous RAMs can be used. Asynchronous RAM provides read data during the clock cycle that has the memory address and read signals active, for sampling on the next rising clock edge. Synchronous single stage RAM registers the data at the current address out and is not available until the next clock cycle; that is, the second rising clock edge.

[Figure 2-15](#) shows the timing diagram for both asynchronous and synchronous RAMs.

Figure 2-15 Timing for RAM Reads**Note**

This timing diagram illustrated in [Figure 2-15](#) assumes the RAM has a chip select port that is tied to an active value; therefore, the chip is always enabled. This is why the second synchronous read data appears at the same cycle as the asynchronous read data; that is, the address for the second read has been sampled along with the chip select on an earlier edge. Once the tx_ram_re_n output enable asserts the data, the value on the register output is seen on that same cycle.

Similarly, you can use synchronous RAM for writes, which registers the data at the current address out. [Figure 2-16](#) shows the timing diagram for RAM writes.

Figure 2-16 Timing for RAM Writes

When FIFO support is selected, an optional programmable FIFO Access mode is available for test purposes, which allows:

- Receive FIFO to be written by requester
- Transmit FIFO to be read by requester

When FIFO Access mode is not selected, none of the corresponding logic is implemented and the mode cannot be enabled, reducing overall gate counts.

When FIFO Access mode has been selected it can be enabled with the FIFO Access Register (FAR[0]). Once enabled, the control portions of the transmit and receive FIFOs are reset and the FIFOs are treated as empty.

Data can be written to the transmit FIFO as normal; however no serial transmission occurs in this mode – normal operation halted – and thus no data leave the FIFO. The data that has been written to the transmit

FIFO can be read back with the Transmit FIFO Read (TFR) register, which when read gives the current data at the top of the transmit FIFO.

Similarly, data can be read from the receive FIFO as normal. Since the normal operation of the DW_apb_uart is halted in this mode, data must be written to the receive FIFO so the data can be read back.

Data is written to the receive FIFO using the Receive FIFO Write (RFW) register. The upper two bits of the 10-bit register are used to write framing error and parity error detection information to the receive FIFO, as follows:

- RFW[9] indicates framing error
- RFW[8] indicates parity error

Although these bits cannot be read back through the Receive Buffer Register, they can be checked by reading the Line Status Register and checking the corresponding bits when the data in question is at the top of the receive FIFO.

2.7 Clocks and Resets

2.7.1 Clocks

The DW_apb_uart can be configured to have either one clock (pclk) or two clocks (pclk and sclk).

- pclk - APB interface clock
- sclk - Serial interface clock.

sclk is asynchronous to pclk and it is present when the CLOCK_MODE parameter is set to 2.

The serial clock accommodates accurate serial baud rate settings, as well as APB bus interface requirements. When using a single-system clock, available system clock settings for accurate baud rates are greatly restricted.

2.7.2 Resets

When configured for asynchronous serial clock operation, the DW_apb_uart includes two separate reset signals:

- The presetn signal resets logic in pclk clock domain
- The s_rst_n signal resets logic in sclk clock domain

To avoid serious operational failures, both clock domains of the DW_apb_uart must be reset before any attempt is made to send or receive data on the serial line. It is an illegal operation to reset just one clock domain of the DW_apb_uart without resetting the other clock domain.

The resets are active-low and can be asynchronously asserted but the de-assertion must be synchronous to the respective clock. DW_apb_uart does not have logic to perform this synchronization, it must be taken care externally.

When asserting the reset signals, the s_rst_n signal should be asserted before or at the same time as the presetn signal; this prevents any unexpected activity on the serial line that might result from resetting the programming registers without resetting the serial logic. Similarly, when de-asserting the reset signals, the s_rst_n signal should be de-asserted before the presetn signal is de-asserted.

The safest procedure for resetting DW_apb_uart is as follows:

1. Assert the s_rst_n signal and the preseln signal.
2. De-assert the s_rst_n signal synchronously with sclk.
3. De-assert the preseln signal synchronously with pclk.

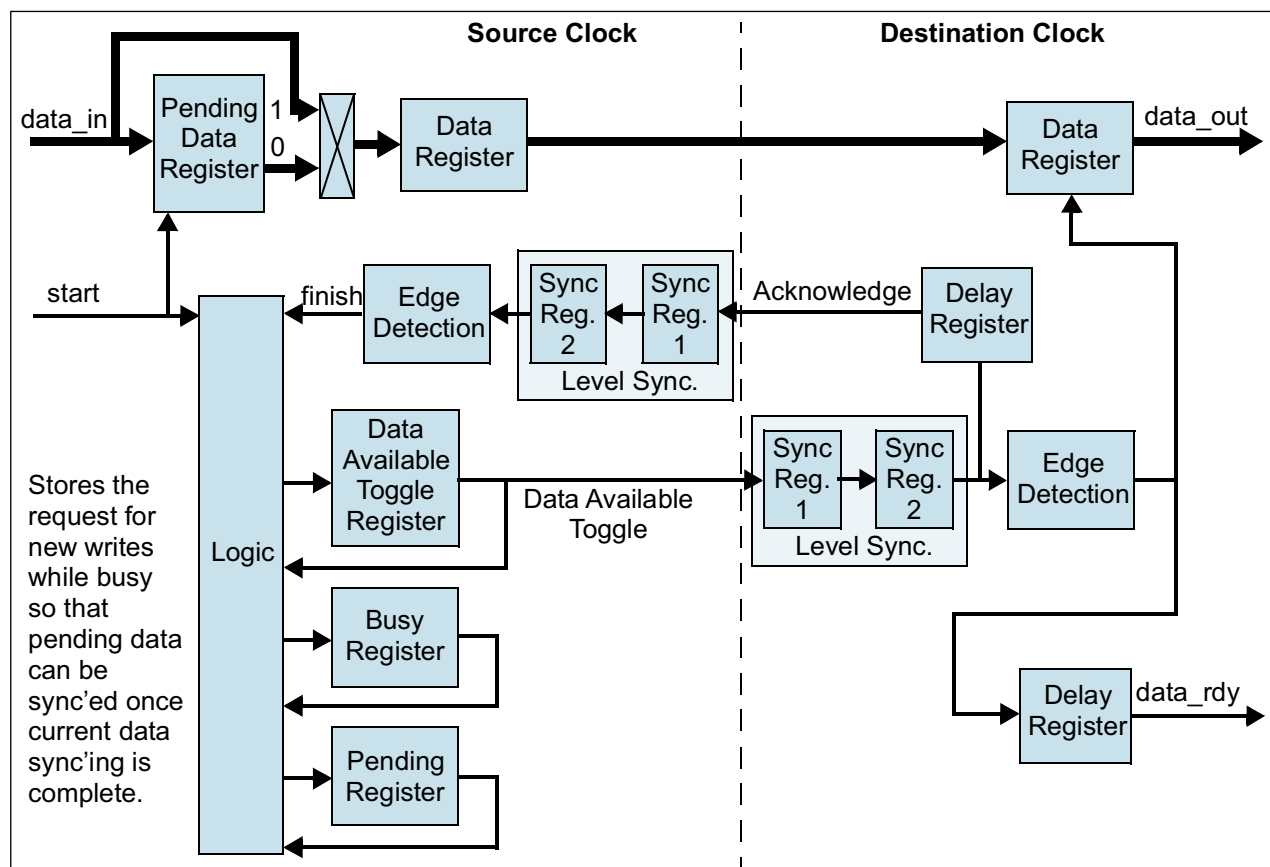
Both reset signals should be active for at least three cycles of the respective clock signal.

2.8 Synchronization

When a two-clock design is chosen, a synchronization module is implemented for synchronization of all control and data across the two-system clock boundaries; this is illustrated in [Figure 1-1](#) on page 20.

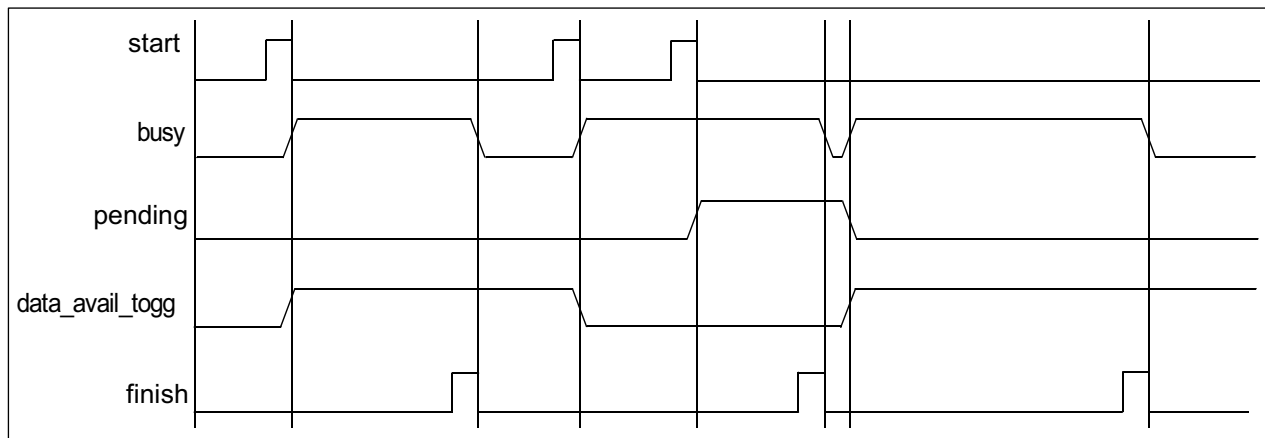
The RTL diagram for the data synchronization module is shown in [Figure 2-17](#); this module can have pending data capability.

Figure 2-17 RTL Diagram of Data Synchronization Module



The timing diagram shown in [Figure 2-18](#) shows the data synchronization process.

Figure 2-18 Timing Diagram for Data Synchronization Module



The arrival of new source domain data is indicated by the assertion of start. Since data is now available for synchronization, the process is started and busy status is set. If start is asserted while busy and pending data capability has been selected, the new data is stored.

When no longer busy, the synchronization process starts on the stored pending data. Otherwise the busy status is removed when the current data has been synchronized to the destination domain and the process continues. If only one clock is implemented, all synchronization logic is absent and signals are simply passed through this module.

There are two types of signal synchronization:

- Data-synchronized signals – full synchronization handshake takes place on signals
- Level-synchronized signals – signals are passed through two destination clock registers

Both synchronization types incur additional data path latencies. However, this additional latency has no negative affect on received or transmitted data, other than to limit how much faster `sclk` can be in relation to `pclk` for back-to-back serial communications with no idle assertion.

A serial clock that exceeds this limit does not leave enough time for a complete incoming character to be received and pushed into the receiver FIFO. To ensure that you do not exceed the limit, the following equation must hold true:

$$((2 * pclk_cycles) + 4) < (39 * (\text{Baud Divisor}))$$

Where:

pclk_cycles is expressed in sclk cycles

For example, if the Baud Divisor is programmed to 1 and a serial clock is 18 times faster than the pclk signal, the equation becomes:

$$((2 * 18) + 4) < (39 * 1) \longrightarrow 40 < 39$$

Thus the equation does not hold true, and the ratio 18:1 (sclk:pclk) exceeds the limit at this Baud rate.

Here are a few things to keep in mind:

- A divisor greater than 1 at a clock ratio of 18:1 (sclk:pclk) does not cause data corruption issues due to synchronization, as the synchronization process has more time to transfer the received data to the peripheral clock domain before the next character bit is received.

In most cases, however, the pclk signal is faster than sclk, so this should never be an issue.

- There is slightly more time required after initial serial control register programming before serial data can be transmitted or received.
- The serial clock modules must have time to see new register values and reset their respective state machines. This total time is guaranteed to be no more than eight clock cycles of the slower of the two system clocks. Therefore, no data should be transmitted or received before this maximum time expires, after initial configuration.

Each NOP usually takes one bus cycle to retire. However, the actual number of NOPs that need to be inserted in the assembly code is dependent on the maximum number of instructions that can be retired in a single cycle. So for example, if the processor uses a 4-dispatch pipe, then four NOPs could potentially retire in one bus cycle. Assuming that the next opcode (NOP) is fetched as per the slower clock – with eight clock cycles of the slower clock as the reference – a minimum of thirty-two NOPs need to be included in the assembly code after a software reset.

In systems where only one clock is implemented, there are no additional latencies.

2.9 Back-to-Back Character Stream Transmission

This section describes:

- Scenarios under which the DW_apb_uart is capable of transmitting back-to-back characters on the serial interface, with no idle time between them
- Worst-case idle time that exists between back-to-back characters

When the Transmit FIFO contains multiple data entries, the DW_apb_uart transmits the characters in the FIFO back-to-back on the serial bus. However, if the CLOCK_MODE configuration parameter equals 2, synchronization delays in the DW_apb_uart can cause an IDLE period between the end of the current STOP bit and the beginning of the next START bit; this appears as an extended STOP bit duration on the serial bus.

2.9.1 Dual Clock Mode

When the CLOCK_MODE parameter equals 2 – indicating an asynchronous relationship between pclk and sclk – the DW_apb_uart has a synchronization delay between the transmitter in the sclk domain and the TX FIFO in the pclk domain when querying if another character is ready for transmission. The transmitter begins the handshake one baud clock cycle before the end of the current STOP bit. The duration of the synchronization delay is given by the following equations:

$$\text{sync_delay} = (1\text{sclk} + 3\text{pclk}) + 1\text{pclk} + (1\text{pclk} + 3\text{sclk})$$

$$\text{sync_delay} = 4\text{sclk} + 5\text{pclk}$$

If the *sync_delay* duration is longer than one baud clock period, an IDLE period is inserted between the end of a STOP bit and the beginning of the next START bit.

To prevent insertion of the IDLE period, the following condition must be true:

$$\text{sync_delay} \leq \text{bclk_period}$$

The baud clock period is given by the following equation:

$$bclk_period = \{DLH, DLL\} * sclk$$

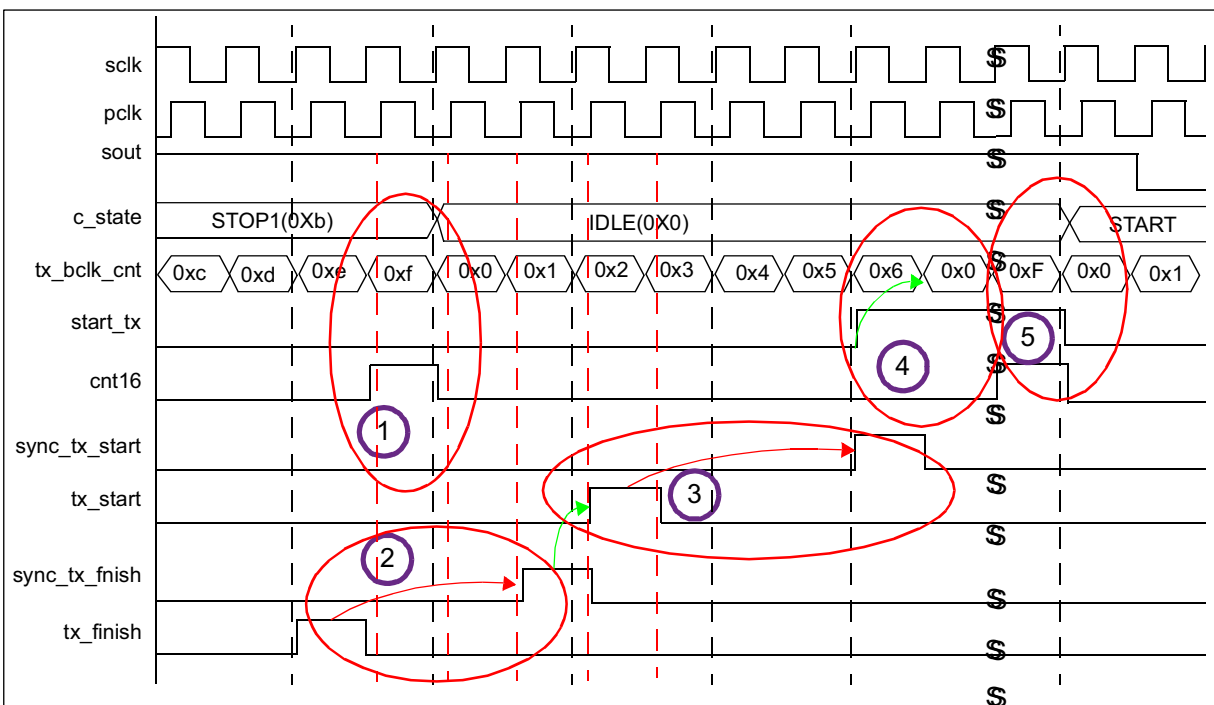
The worst case timing of the inserted IDLE period is given by:

$$worst_case_idle_duration = sync_delay + (15 * bclk_period)$$

The *worst_case_idle_duration* can be added to the programmed STOP bit duration to give the overall STOP bit period.

Figure 2-19 illustrates an example of character finish to character start delay.

Figure 2-19 Character Finish to Character Start Delay



1. The baud divisor is set to 1 ($\{DLH, DLL\} = 1$), so every *sclk* is a baud clock cycle. The transmit state machine changes state every sixteen baud clocks—eight in the case of a half STOP bit. At this point in Figure 2-19, after 16 baud clock cycles of the STOP1 state, the state machine enters the IDLE state on the next cycle because *start_tx* is not yet asserted.
2. One baud clock before the end of the STOP state, the transmit state machine decodes that the current character is complete and asserts *tx_finish*, which is synchronized to the *pclk* domain to become *sync_tx_finish*; this synchronization accounts for the “1sclk + 3pclk” term in *sync_delay*.
3. In the *pclk* domain, there is a one-*pclk* cycle delay — “1pclk” term in *sync_delay*—before the signal *tx_start* is asserted from the assertion of *sync_tx_finish*. *Tx_start* must then be synchronized to the *sclk* domain — “1pclk + 3sclk” term in *sync_delay*—to instruct the state machine to commence the START bit of the next character.
4. *Start_tx* asserts in the *sclk* domain, and causes the baud clock counter (*tx_bclk_cnt*) to go to 0.

5. Once sixteen baud clocks have been counted, the state machine can transition into the START state, and one cycle later *sout* is de-asserted.

2.9.2 Single Clock Mode

If `CLOCK_MODE` equals 1, there is no idle time between back-to-back characters if data is ready in the transmit FIFO. In this case, because *sync_delay* equals one `pclk` as described in “Dual Clock Mode”, the requirement to avoid idle time between consecutive characters is met for all {DLH,DLL} values.

$$\text{sync_delay} \leq \{\text{DLH,DLL}\} * \text{sclk}$$

For example, when {DLH, DLL} equals 1 (bearing in mind that when `CLOCK_MODE` = 1 : `pclk` = `sclk`), then

$$1 \text{ pclk} \leq 1 * \text{pclk}$$

2.10 Interrupts

Assertion of the DW_apb_uart interrupt output signal (*intr*)—a positive-level interrupt—occurs whenever one of the several prioritized interrupt types are enabled and active.

When an interrupt occurs, the requester accesses the IIR register.

The following interrupt types can be enabled with the IER register:

- Receiver Error
- Receiver Data Available
- Character Timeout (in FIFO mode only)
- Transmitter Holding Register Empty at/below threshold (in Programmable THRE interrupt mode)
- Modem Status
- Busy Detect Indication

These interrupt types are explained in detail in [Table 2-3](#). Also, see [Appendix 4, “Signal Descriptions”](#) for more information on interrupts.

Table 2-3 Interrupt Control Functions

Interrupt ID				Interrupt Set and Reset Functions			
Bit3	Bit2	Bit1	Bit0	Priority Level	Interrupt Type	Interrupt Source	Interrupt Reset Control
0	0	0	1	–	None	None	–
0	1	1	0	Highest	Receiver line status	Overrun/parity/ framing errors, break interrupt, or address received interrupt	For Overrun/parity/framing/break interrupt reset control, the behavior is as follows: ■ If LSR_STATUS_CLEAR=0 (RBR Read or LSR Read), then the status is cleared on: - Reading the line status register - Or - In addition to an LSR read, the Receiver line status is also cleared when RX_FIFO is read. ■ If LSR_STATUS_CLEAR=1 (LSR Read), the status is cleared only on: - Reading the line status register. ■ For address received interrupt, the status is cleared on: - Reading the line status register
0	1	0	0	Second	Received data available	Receiver data available (non-FIFO mode or FIFOs disabled) or RCVR FIFO trigger level reached (FIFO mode and FIFOs enabled)	Reading the receiver buffer register (non-FIFO mode or FIFOs disabled) or the FIFO drops below the trigger level (FIFO mode and FIFOs enabled)
1	1	0	0	Second	Character timeout indication	No characters in or out of the RCVR FIFO during the last 4 character times and there is at least 1 character in it during this time	Reading the receiver buffer register
0	0	1	0	Third	Transmit holding register empty	Transmitter holding register empty (Prog. THRE Mode disabled) or XMIT FIFO at or below threshold (Prog. THRE Mode enabled)	Reading the IIR register (if source of interrupt); or, writing into THR (FIFOs or THRE Mode not selected or disabled) or XMIT FIFO above threshold (FIFOs and THRE Mode selected and enabled).

Table 2-3 Interrupt Control Functions

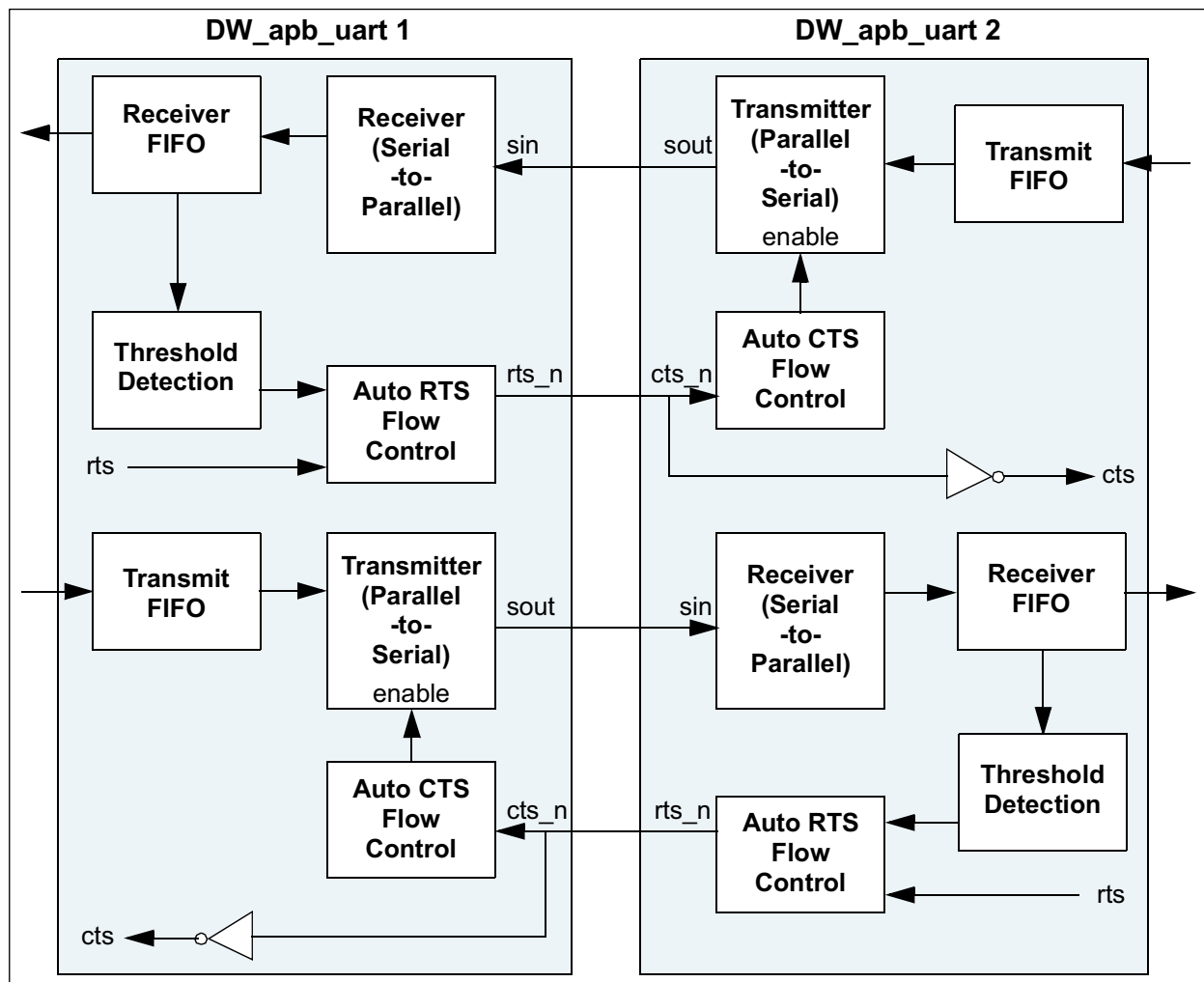
Interrupt ID				Interrupt Set and Reset Functions			
Bit3	Bit2	Bit1	Bit0	Priority Level	Interrupt Type	Interrupt Source	Interrupt Reset Control
0	0	0	0	Fourth	Modem status	Clear to send or data set ready or ring indicator or data carrier detect. Note that if auto flow control mode is enabled, a change in CTS (that is, DCTS set) does not cause an interrupt.	Reading the Modem status register
0	1	1	1	Fifth	Busy detect indication	UART_16550_COMPATIBLE = NO and requester has tried to write to the Line Control Register while the DW_apb_uart is busy (USR[0] is set to 1).	Reading the UART status register

2.11 Auto Flow Control

The DW_apb_uart can be configured to have a D16750-compatible Auto RTS and Auto CTS serial data flow control mode available; if FIFOs are not implemented, this mode cannot be selected. When Auto Flow Control is not selected, none of the corresponding logic is implemented and the mode cannot be enabled, reducing overall gate counts. When Auto Flow Control mode is selected, it can be enabled with the Modem Control Register (MCR[5]).

Figure 2-20 shows a block diagram of the Auto Flow Control functionality.

Figure 2-20 Auto Flow Control Block Diagram



Auto RTS and Auto CTS are described as follows:

- **Auto RTS** – Becomes active when the following occurs:
 - Auto Flow Control is selected during configuration
 - FIFOs are implemented
 - RTS (MCR[1] bit and MCR[5] bit are both set)
 - FIFOs are enabled (FCR[0] bit is set)
 - SIR mode is disabled (MCR[6] bit is not set)

When Auto RTS is enabled, the rts_n output is forced inactive (high) when the receiver FIFO level reaches the threshold set by FCR[7:6], but only if the RTC flow-control trigger is disabled. Otherwise, the rts_n output is forced inactive (high) when the FIFO is almost full, where “almost full” refers to two available slots in the FIFO. When rts_n is connected to the cts_n input of another UART device,

the other UART stops sending serial data until the receiver FIFO has available space; that is, until it is completely empty.

The selectable receiver FIFO threshold values are:

- ❑ 1
- ❑ $\frac{1}{4}$
- ❑ $\frac{1}{2}$
- ❑ 2 less than full

Since one additional character can be transmitted to the DW_apb_uart after rts_n has become inactive – due to data already having entered the transmitter block in the other UART – setting the threshold to “2 less than full” allows maximum use of the FIFO with a safety zone of one character.

Once the receiver FIFO becomes completely empty by reading the Receiver Buffer Register (RBR), rts_n again becomes active (low), signaling the other UART to continue sending data.

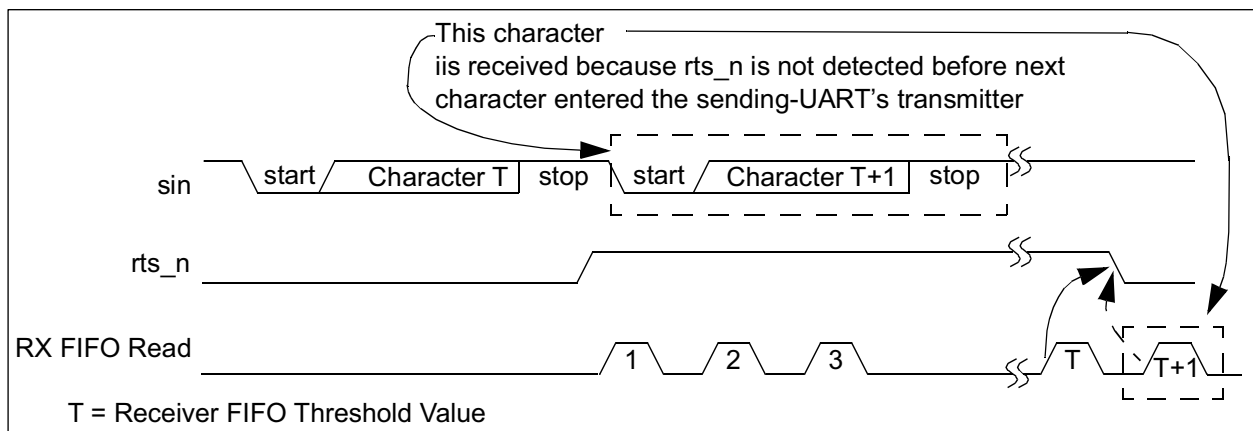


Note

Even if everything else is selected and the correct MCR bits are set, if the FIFOs are disabled through FCR[0] or the UART is in SIR mode (MCR[6] is set to 1), Auto Flow Control is also disabled. When Auto RTS is not implemented or disabled, rts_n is controlled solely by MCR[1].

Figure 2-21 shows a timing diagram of the Auto RTS operation.

Figure 2-21 Auto RTS Timing



- **Auto CTS** – becomes active when the following occurs:
 - ❑ Auto Flow Control is selected during configuration
 - ❑ FIFOs are implemented
 - ❑ AFCE (MCR[5] bit = 1)
 - ❑ FIFOs are enabled through FIFO Control Register FCR[0] bit
 - ❑ SIR mode is disabled (MCR[6] bit = 0)

When Auto CTS is enabled (active), the DW_apb_uart transmitter is disabled whenever the cts_n input becomes inactive (high); this prevents overflowing the FIFO of the receiving UART.

If the cts_n input is not inactivated before the middle of the last stop bit, another character is transmitted before the transmitter is disabled. While the transmitter is disabled, the transmitter FIFO can still be written to, and even overflowed.

Therefore, when using this mode, the following happens:

- UART status register can be read to check if transmit FIFO is full (USR[1] set to 0)
- Current FIFO level can be read using TFL register
- Programmable THRE Interrupt mode must be enabled to access “FIFO full” status using Line Status Register (LSR)

When using the “FIFO full” status, software can poll this before each write to the Transmitter FIFO; for details, see [“Programmable THRE Interrupt”](#) on page 61. When the cts_n input becomes active (low) again, transmission resumes.

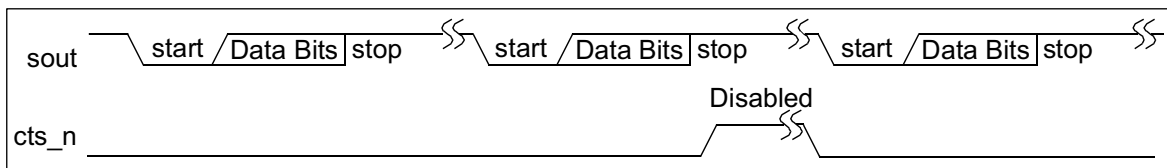


Note

When everything else is selected, if the FIFOs are disabled using FCR[0], Auto Flow Control is also disabled. When Auto CTS is not implemented or disabled, the transmitter is unaffected by cts_n.

Figure 2-22 illustrates a timing diagram that shows the Auto CTS operation.

Figure 2-22 Auto CTS Timing

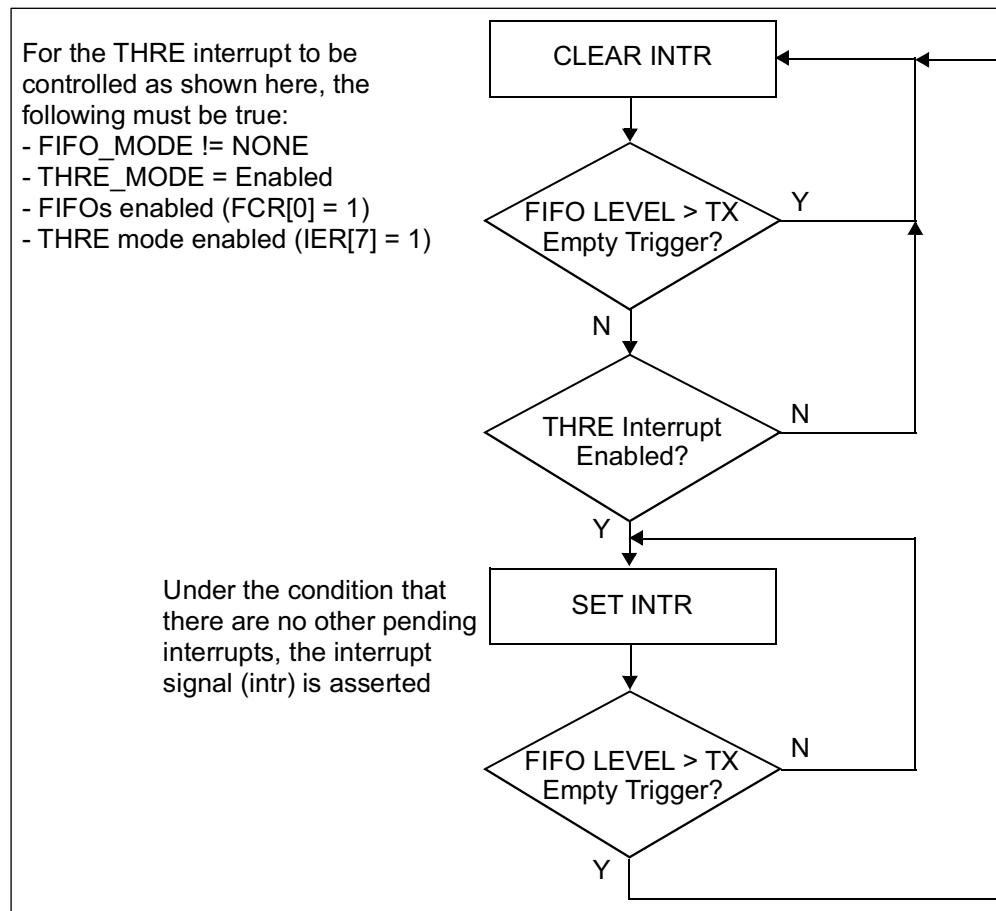


2.12 Programmable THRE Interrupt

The DW_apb_uart can be configured for a Programmable THRE Interrupt mode in order to increase system performance; if FIFOs are not implemented, then this mode cannot be selected.

- When Programmable THRE Interrupt mode is not selected, none of the logic is implemented and the mode cannot be enabled, reducing the overall gate counts.
- When Programmable THRE Interrupt mode is selected, it can be enabled using the Interrupt Enable Register (IER[7]).

When FIFOs and THRE mode are implemented and enabled, the THRE Interrupts and dma_tx_req_n are active at, and below, a programmed transmitter FIFO empty threshold level, as opposed to empty, as shown in the flowchart in [Figure 2-23](#).

Figure 2-23 Flowchart of Interrupt Generation for Programmable THRE Interrupt Mode

The threshold level is programmed into FCR[5:4]. Available empty thresholds are:

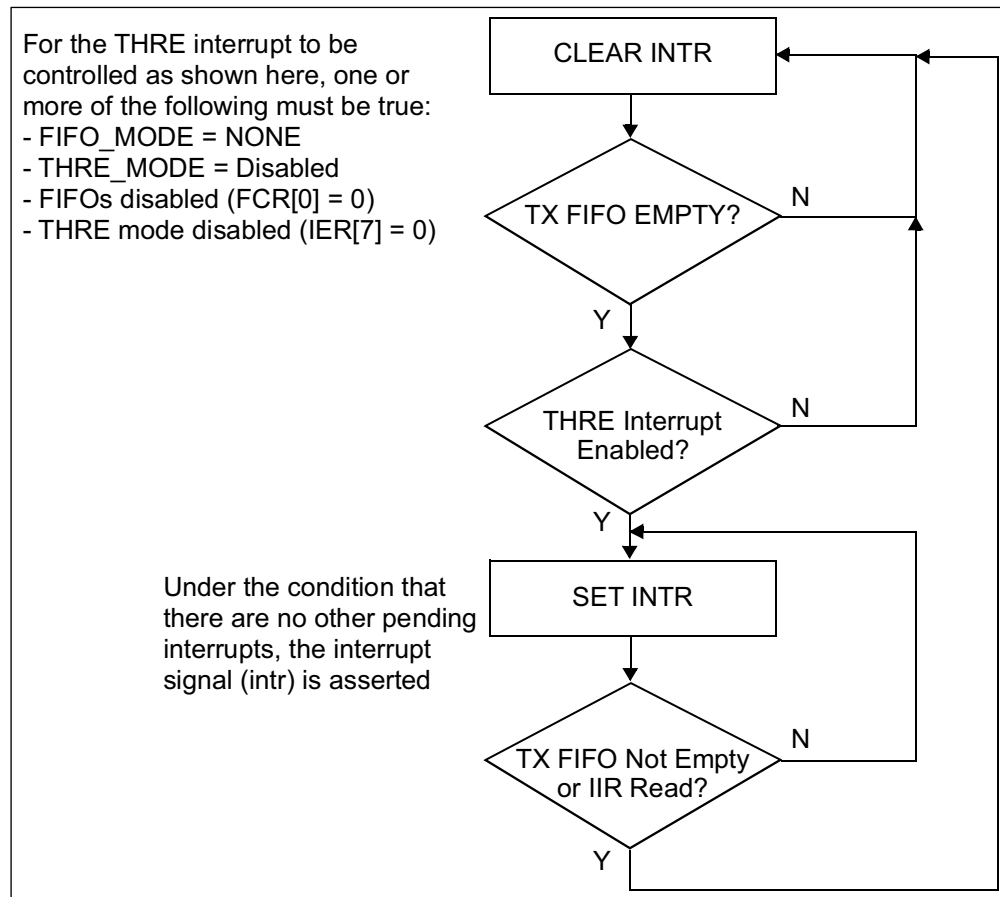
- empty
- 2
- $\frac{1}{4}$
- $\frac{1}{2}$

Selection of the best threshold value depends on the system's ability to begin a new transmission sequence in a timely manner. However, one of these thresholds should be optimal for increasing system performance by preventing the transmitter FIFO from running empty. For threshold setting details, see section “FCR” in [“Register Descriptions”](#) on page 121.

In addition to the interrupt change, the Line Status Register (LSR[5]) also switches from indicating that the transmitter FIFO is empty to the FIFO being full. This allows software to fill the FIFO for each transmit sequence by polling LSR[5] before writing another character. The flow then allows the transmitter FIFO to be filled whenever an interrupt occurs and there is data to transmit, rather than waiting until the FIFO is completely empty. Waiting until the FIFO is empty causes a reduction in performance whenever the system is too busy to respond immediately. Further system efficiency is achieved when this mode is enabled in combination with Auto Flow Control.

Even if everything else is selected and enabled, if the FIFOs are disabled using the FCR[0] bit, the Programmable THRE Interrupt mode is also disabled. When not selected or disabled, THRE interrupts and the LSR[5] bit function normally, signifying an empty THR or FIFO. Figure 2-24 illustrates the flowchart of THRE interrupt generation when not in programmable THRE interrupt mode.

Figure 2-24 Flowchart of Interrupt generation when not in Programmable THRE Interrupt Mode



2.13 Clock Gate Enable

The DW_apb_uart can be configured to have a clock gate enable output.

- You can enable the feature by setting the configuration parameter CLK_GATE_EN to 1. Furthermore, the feature is software controlled through the register bit LCR_EXT.CGEN. You can:
 - write a 1 into the register bit LCR_EXT.CGEN to enable it.
 - write a 0 into the register bit LCR_EXT.CGEN to disable it.
- When the clock gate enable option is not selected (CLK_GATE_EN=0), no logic is implemented, which reduces the overall gate count.
- When the clock gate enable option is selected, the clock gate enable signals – uart_lp_req_pclk for single clock implementations or uart_lp_req_pclk and uart_lp_req_sclk for two clock implementations which indicate the following:

- ❑ Transmit and receive pipeline is clear (no data).
- ❑ No activity has occurred.
- ❑ Modem control input signals have not changed in more than one character time – the time taken to TX/RX a character – so that clocks can be gated. A character is made up of:

start_bit + data_bits + parity (optional) + stop_bits

The assertion of clock gate enable signals is an indication that the UART is inactive, so clocks may be gated in order to put the device in a low-power mode.



Note

When low power mode is enabled in DW_apb_uart, it is expected that the clocks are gated when indicated by `uart_lp_req_sclk` or `pclk` ports.

If the clocks cannot be gated - you must disable the feature (`LCR_EXT.CGEN = 0`).

Therefore, the following must be true for at least one character time for the assertion of the clock gate enable signals to occur:

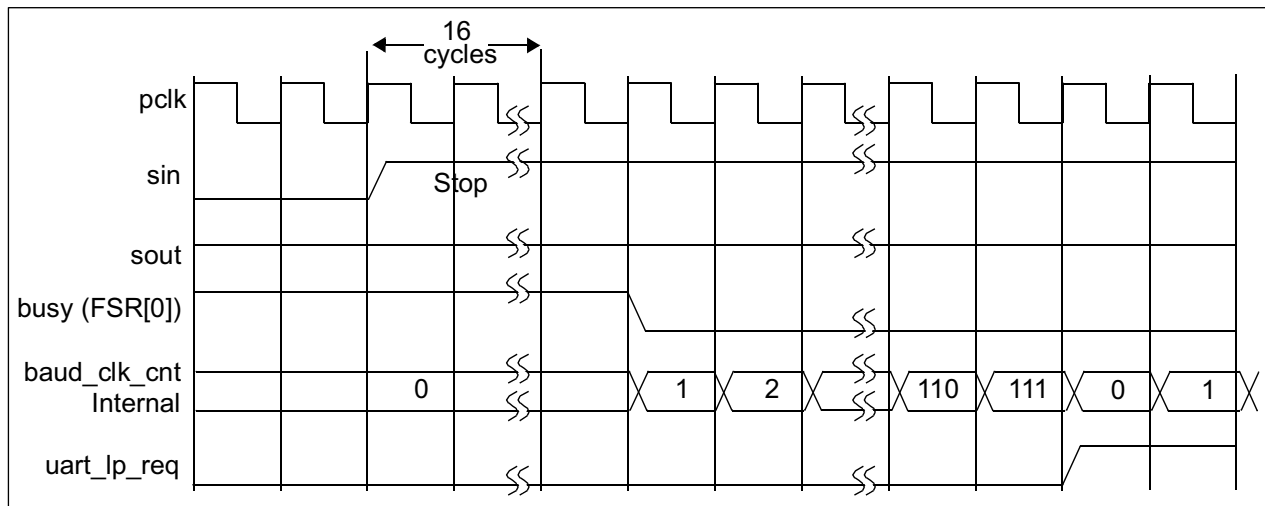
- No data in the RBR (in non-FIFO mode) or the RX FIFO is empty (in FIFO mode)
- No data in the THR (in non-FIFO mode) or the TX FIFO is empty (in FIFO mode)
- `sin/sir_in` and `sout/sir_out_n` are inactive (`sin/sir_in` are kept high and `sout` is high or `sir_out_n` is low) indicating no activity
- No change on the modem control input signals

Note, the clock gate enable assertion does not occur in the following modes of operation:

- Loopback mode
- FIFO access mode
- When transmitting a break

For example, assume a DW_apb_uart that is configured to have a single clock (`pclk`) and is programmed to transmit and receive characters of 7 bits (1 start bit, 5 data bits and 1 stop bit) and the baud clock divisor is set to 1. Therefore, the `uart_lp_req_pclk` signal is asserted (provided `LCR_EXT.CGEN=0x1`) if the transmit and receive pipeline is clear, no activity has occurred and the modem control input signals have not changed for 112 (7×16) `pclk` cycles.

Figure 2-25 illustrates this example.

Figure 2-25 Clock Gate Enable Timing

When the assertion criteria are no longer met, the clock gate enable signals are de-asserted and the clocks is resumed under any of these conditions:

- Either **sin** signal or **sir_in** signal goes low
- Write to any of registers is performed
- Modem control input signals have changed when DW_apb_uart is in low-power (sleep) mode

The clock gate enable signals are de-asserted asynchronously on arrival of earlier mentioned events because a clock is not available to synchronize the events. Therefore, you can decide to include 2-flop syncs externally before the clock gate cell to avoid metastability issues for clock-gate latch.

When the device wakes up from low power mode, there may be a delay in un-gating the SCLK or PCLK clocks due to an external synchronization delay of the clock gating enable "**uart_lp_req**" signal, which can result in data loss, such as the first write to a register in the PCLK domain failing or the first byte received in the SCLK domain being incorrect.

You can mitigate this issue at the software level as such:

- In the PCLK domain, during wake up from low power mode, it is recommended to implement a dummy write to a Scratchpad (SCR) register before writing to any other registers. This ensures that the PCLK is enabled and running before any other registers are written to.
- In the SCLK domain, you can include preamble bytes in the serial transmission to help prevent receive data loss during low power mode wake up. By including a specific number of preamble bytes (for example, 4 or any other suitable number of preamble bytes at the beginning of every transfer), the receiver hardware can wake up and prepare the firmware for the synchronization for the actual message.



Note A read to any register does not de-assert the clock gate enable signals. The **pclk** clock needs to be enabled to read any of the registers in low-power mode.

The time taken for the clocks to resume is important in preventing receive data synchronization problems, due to the DW_apb_uart RX block sampling:

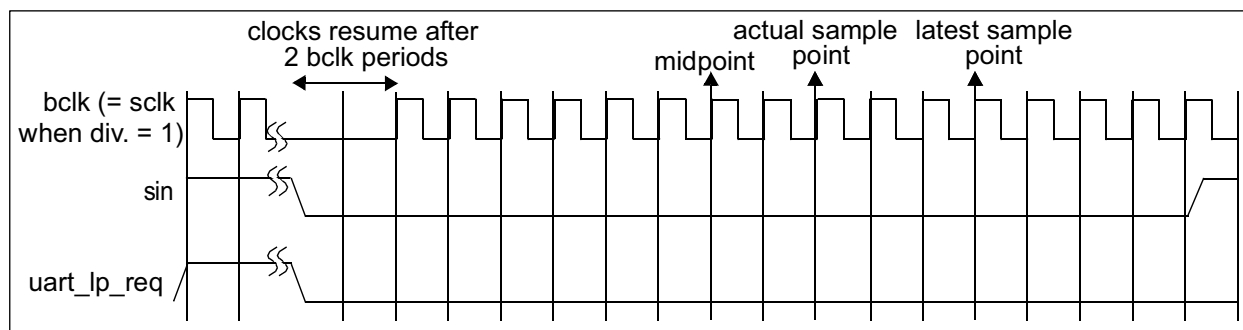
1. At mid-point of each bit period – after approximately 8 baud clocks – in UART (RS323) mode.
2. After that, every 16 baud clocks for a baud divisor of 1 that is 16 sclks; for a single clock implementation, this is 16 pclk.

Thus, if eight or more sclk periods pass before the serial clock starts up again, the DW_apb_uart can get out of synchronization with the serial data it is receiving; that is, the receiver can sample into the second bit period, and if it is still 0, the receiver uses this as the start bit, and so on.

In order to avoid this problem, the clock should be resumed within five clock periods of the baud clock, which is the same as sclk if the baud divisor is set to 1; this is worst-case. If the divisor is greater, it gives a greater number of sclk cycles available before the clock must resume. This means a sample point at the 13 baud clock (at the latest) out of the 16 that are transmitted for each bit period of the character in non-SIR mode.

Figure 2-26 shows the timing diagram that illustrates the previous scenario.

Figure 2-26 Resuming Clocks After Low Power Mode Timing



This synchronization problem is magnified in SIR mode because the pulse width is only 3/16 of a bit period – three baud clocks, which for a divisor of 1 is three sclks; thus, the pulse can be missed completely. The clocks must resume before three baud clock periods elapse. However, if the first character received while in sleep mode is used only for wake-up reasons and the actual character value is unimportant, this may not become a problem.

When DW_apb_uart is configured to have two clocks, if the timing of the received signal is not affected by the synchronization problem, then the minimum time to receive a character – if the baud divisor is 1 – is 112 sclks:

$$1 \text{ start_bit} + 5 \text{ data_bits} + 1 \text{ stop_bit} = 7 \times 16 = 112$$

Therefore, the pclk must be available before 112 sclk cycles pass in order for the received character to be synchronized to the pclk domain and stored in the RBR (in non-FIFO mode) or the RX FIFO (in FIFO mode).

2.14 DMA Support

The DW_apb_uart supports DMA signalling with the use of the dma_tx_req_n and dma_rx_req_n output signals to indicate:

- When data can be read

- When transmit FIFO is empty

For more information on the `dma_tx_req_n` and `dma_rx_req_n` signals, see “[Handshaking Interface Operation](#)” on page 74.

**Note**

The reset value of the `dma_tx_req_n` signal is based on the `DMA_HS_REQ_ON_RESET` parameter value.

- If `DMA_HS_REQ_ON_RESET=1`, `dma_tx_req_n` is asserted after a reset.
- If `DMA_HS_REQ_ON_RESET=0`, `dma_tx_req_n` is not asserted upon reset. It is asserted only after the LCR register is written.
- The reset under consideration is both a hardware reset (presetn) and a soft reset (by writing to the SRR register).

2.14.1 DMA Modes

The DW_apb_uart uses two DMA channels – one for transmit data and one for receive data. There are two DMA modes:

- mode 0 – bit 3 of FIFO Control Register set to 0
- mode 1 – bit 3 of FIFO Control Register set to 1

**Note**

Only DMA mode 0 is available when FIFOs are not implemented or disabled.

2.14.1.1 DMA Mode 0

DMA mode 0 supports single DMA data transfers at a time.

In mode 0, the `dma_tx_req_n` signal:

- Goes active-low under the following conditions:
 - When Transmitter Holding Register is empty in non-FIFO mode
 - When transmitter FIFO is empty in FIFO mode with Programmable THRE interrupt mode disabled
 - When transmitter FIFO is at or below programmed threshold with Programmable THRE interrupt mode enabled
- Goes inactive when:
 - Single character has been written into Transmitter Holding Register or transmitter FIFO with Programmable THRE interrupt mode disabled
 - Transmitter FIFO is above threshold with Programmable THRE interrupt mode enabled

In mode 0, the `dma_rx_req_n` signal:

- Goes active-low when single character is available in Receiver FIFO or Receive Buffer Register
- Goes inactive when Receive Buffer Register or Receiver FIFO are empty, depending on FIFO mode

2.14.1.2 DMA Mode 1

DMA mode 1 supports multi-DMA data transfers, where multiple transfers are made continuously until the receiver FIFO has been emptied or the transmit FIFO has been filled.

In mode 1, the `dma_tx_req_n` signal is asserted:

- When transmitter FIFO is empty with Programmable THRE interrupt mode disabled
- When transmitter FIFO is at or below programmed threshold with Programmable THRE interrupt mode enabled

In mode 1, the `dma_tx_req_n` signal is de-asserted when the transmitter FIFO is completely full.

In mode 1, the `dma_rx_req_n` signal is asserted:

- When Receiver FIFO is at or above programmed trigger level
- When character timeout has occurred; ERBFI does not need to be set

In mode 1, the `dma_rx_req_n` signal is de-asserted when the receiver FIFO becomes empty.

2.14.1.3 Additional DMA Interface

If required for a DMA controller – such as the `DW_ahb_dmac` – you can use the `DMA_EXTRA` parameter to configure the `DW_apb_uart` for additional DMA interface signals. In this case, asserting the fixed DMA signals – `dma_tx_req_n` and `dma_rx_req_n` – is similar to what is detailed in [DMA Mode 0](#) and [DMA Mode 1](#).

When configured for additional DMA signals, the `dma_tx_req_n` signal is asserted under the following conditions:

- When the Transmitter Holding Register is empty in non-FIFO mode
- When the transmitter FIFO is empty in FIFO mode with Programmable THRE interrupt mode disabled
- When the transmitter FIFO is at, or below the programmed threshold with Programmable THRE interrupt mode enabled

When configured for additional DMA signals, the `dma_rx_req_n` signal is asserted under the following conditions:

- When a single character is available in Receive Buffer Register in non-FIFO mode
- When Receiver FIFO is at or above programmed trigger level in FIFO mode

With the presence of the additional handshaking signals, the UART does not have to rely on internal status and level values to recognize the completion of a request and hence remove the request. Instead, the de-assertion of the DMA transmit and receive request is controlled by the assertion of the DMA transmit and receive acknowledge respectively.

When the UART is configured for additional DMA signals, responsibility of the data flow (transfer lengths) falls on the DMA (`DW_ahb_dmac`) and is controlled by the programmed burst transaction lengths. Thus, there is no need for DMA modes, and programming the `FCR[3]` has no effect.

2.14.1.4 Example DMA Flow

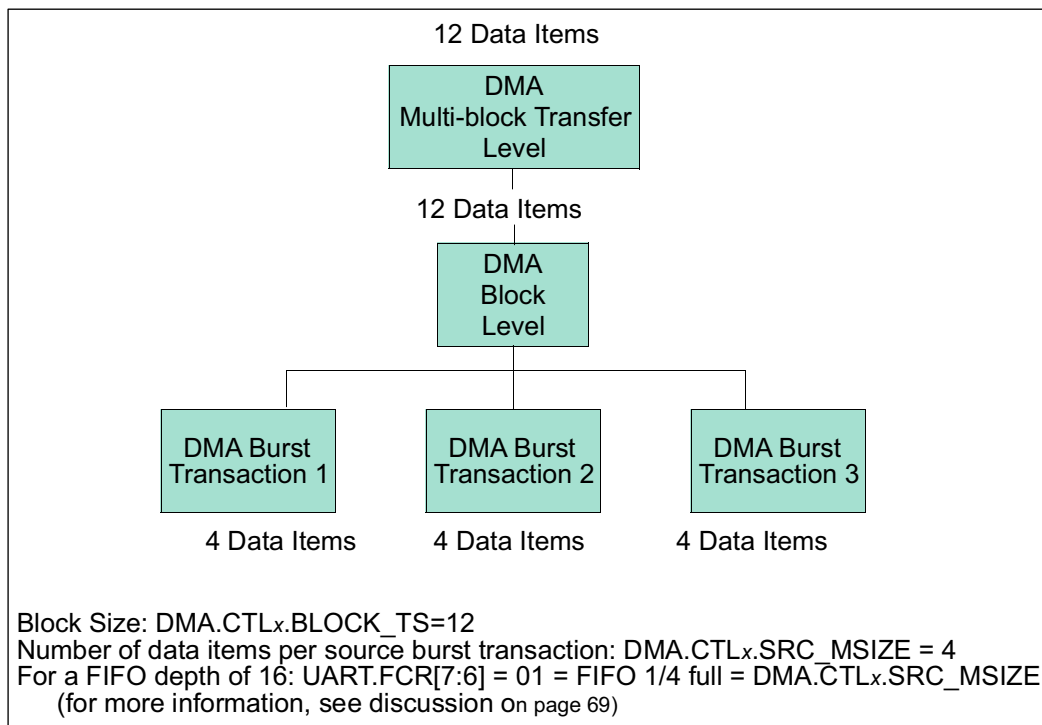
The extra handshaking signals are explained in the following DMA flow for a DW_apb_uart that is configured with FIFOs and Programmable THRE interrupt mode.

As a block flow control device, the DMA Controller is programmed by the processor with the number of data items (block size) that are to be transmitted or received by the DW_apb_uart; this is programmed into the BLOCK_TS field of the CTLx register.

The block is broken into a number of transactions, each initiated by a request from the DW_apb_uart. The DMA Controller must also be programmed with the number of data items (in this case, DW_apb_uart FIFO entries) to be transferred for each DMA request. This is also known as the burst transaction length, and is programmed into the SRC_MSIZE/DEST_MSIZE fields of the DW_ahb_dmac CTLx register for source and destination, respectively.

Figure 2-27 shows a single block transfer, where the block size programmed into the DMA Controller is 12 and the burst transaction length is set to 4.

Figure 2-27 Breakdown of DMA Transfer into Burst Transactions



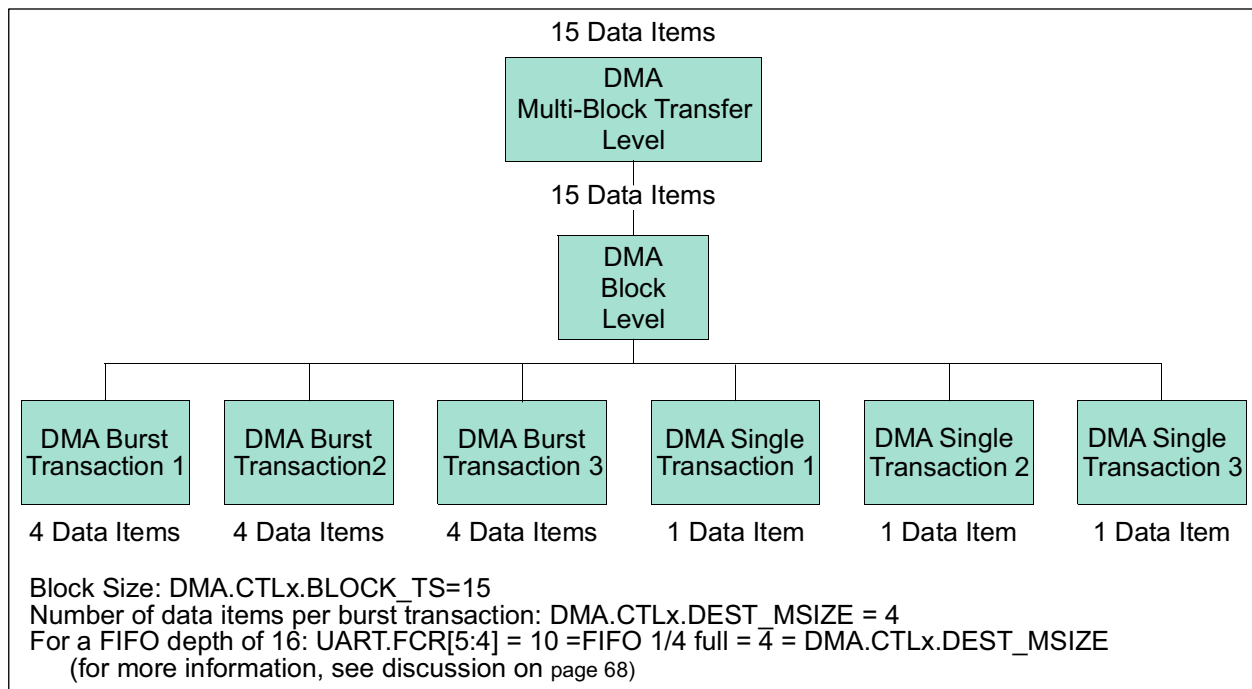
In this case, the block size is a multiple of the burst transaction length. Therefore, the DMA block transfer consists of a series of burst transactions. If the DW_apb_uart makes a transmit request to this channel, four data items are written to the DW_apb_uart transmit FIFO. Similarly, if the DW_apb_uart makes a receive request to this channel, four data items are read from the DW_apb_uart receive FIFO. Three separate requests must be made to this DMA channel before all twelve data items are written or read.



Note The source and destination transfer width settings in the DW_ahb_dmac – DMA.CTLx.SRC_TR_WIDTH and DMA.CTLx.DEST_TR_WIDTH – should be set to 3'b000 because the DW_apb_uart FIFOs are 8 bits wide.

When the block size programmed into the DMA Controller is not a multiple of the burst transaction length, as shown in [Figure 2-28](#), a series of burst transactions followed by single transactions are needed to complete the block transfer.

Figure 2-28 Breakdown of DMA Transfer into Single and Burst Transactions



2.14.2 Transmit Watermark Level and Transmit FIFO Underflow

During DW_apb_uart serial transfers, transmit FIFO requests are made to the DW_ahb_dmac whenever the number of entries in the transmit FIFO is less than or equal to the decoded level of the Transmit Empty Trigger (TET) of the FCR register (bits 5:4); this is known as the watermark level. The DW_ahb_dmac responds by writing a burst of data to the transmit FIFO buffer, of length CTLx.DEST_MSIZ.

Data should be fetched from the DMA often enough for the transmit FIFO to perform serial transfers continuously; that is, when the FIFO begins to empty, another DMA request should be triggered. Otherwise the FIFO runs out of data (underflow). To prevent this condition, you must set the watermark level correctly.

2.14.3 Choosing Transmit Watermark Level

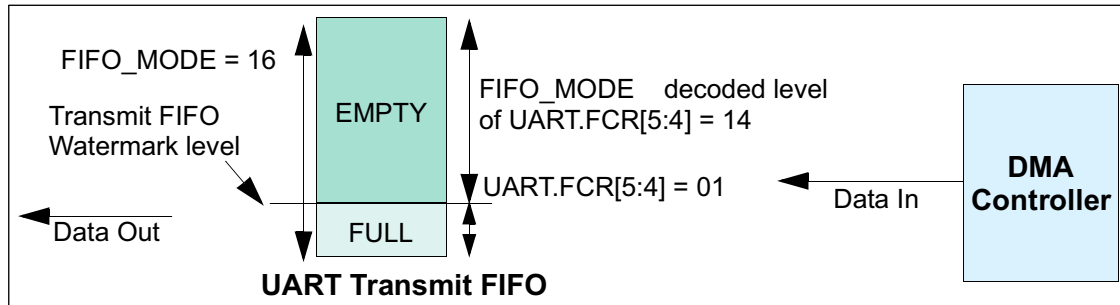
Consider the example where the following assumption is made:

$$\text{DMA.CTLx.DEST_MSIZ} = \text{FIFO_DEPTH} - \text{UART.FCR}[5:4]$$

The number of data items to be transferred in a DMA burst is equal to the empty space in the Transmit FIFO. Consider two different watermark level settings.

2.14.3.1 Case 1: FCR[5:4] = 01 — decodes to 2

Figure 2-29 Case 1 Watermark Levels



- Transmit FIFO watermark level = decoded level of UART.FCR[5:4] = 2
- $\text{DMA.CTLx.DEST_MSIZE} = \text{FIFO_MODE} - \text{UART.FCR[5:4]} = 14$
- UART transmit FIFO_MODE = 16
- $\text{DMA.CTLx.BLOCK_TS} = 56$

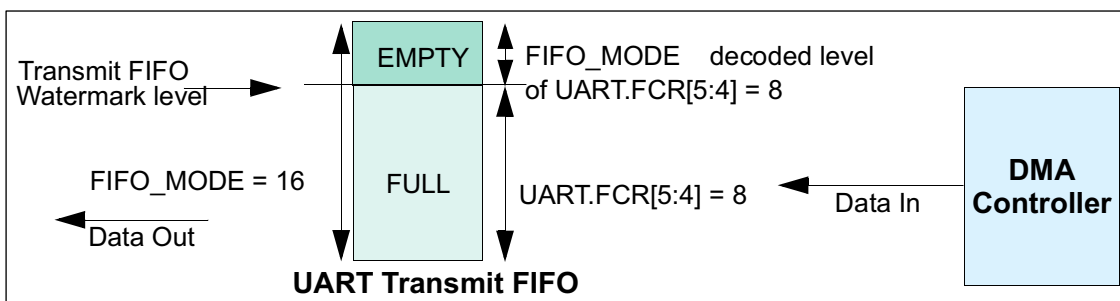
Therefore, the number of burst transactions needed equals the block size divided by the number of data items per burst:

$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 56 / 14 = 4$$

The number of burst transactions in the DMA block transfer is 4, but the watermark level — decoded level of UART.FCR[5:4] — is quite low. Therefore, the probability of a UART underflow is high where the UART serial transmit line needs to transmit data, but where there is no data left in the transmit FIFO. This occurs because the DMA has no time to service the DMA request before the transmit FIFO becomes empty.

2.14.3.2 Case 2: FCR[5:4] = 11 — FIFO 1/2 full (decodes to 8)

Figure 2-30 Case 2 Watermark Levels



- Transmit FIFO watermark level = decoded level of UART.FCR[5:4] = 8
- $\text{DMA.CTLx.DEST_MSIZE} = \text{FIFO_MODE} - \text{UART.FCR[5:4]} = 8$
- UART transmit FIFO_MODE = 16

- $\text{DMA.CTLx.BLOCK_TS} = 56$

Number of burst transactions in Block:

$$\text{DMA.CTLx.BLOCK_TS} / \text{DMA.CTLx.DEST_MSIZE} = 56 / 8 = 7$$

In this block transfer, there are seven destination burst transactions in a DMA block transfer, but the watermark level – decoded level of $\text{UART.FCR}[5:4]$ – is high. Therefore, the probability of a UART underflow is low because the DMA controller has enough time to service the destination burst transaction request before the UART transmit FIFO becomes empty.

Thus, the second case has a lower probability of underflow at the expense of more burst transactions per block. This provides a potentially greater amount of AMBA bursts per block and worse bus utilization than Case 1.

Therefore, the goal in choosing a watermark level is to minimize the number of transactions per block, while at the same time keeping the probability of an underflow condition to an acceptable level. In practice, this is a function of the ratio of:

$$\text{rate of UART data transmission: rate of DMA response to destination burst requests}$$

For example, both of the following increases the rate at which the DMA controller can respond to burst transaction requests:

- Promoting channel to highest priority channel in DMA
- Promoting DMA requester interface to highest priority requester in AMBA layer

This in turn enables the user to decrease the watermark level, which improves bus utilization without compromising the probability of an underflow occurring.

2.14.4 Selecting DEST_MSIZE and Transmit FIFO Overflow

As can be seen from [Figure 2-30](#), programming $\text{DMA.CTLx.DEST_MSIZE}$ to a value greater than the watermark level that triggers the DMA request can cause overflow when there is not enough space in the UART transmit FIFO to service the destination burst request. Therefore, use the following in order to avoid overflow:

$$\text{DMA.CTLx.DEST_MSIZE} \leq \text{UART.FIFO_DEPTH} - \text{decoded level of UART.FCR}[5:4] \quad (1)$$

In [Case 2: \$\text{FCR}\[5:4\] = 11\$ – FIFO 1/2 full \(decodes to 8\)](#), the amount of space in the transmit FIFO at the time the burst request is made is equal to the destination burst length, $\text{DMA.CTLx.DEST_MSIZE}$. Thus, the transmit FIFO can be full, but not overflowed, at the completion of the burst transaction.

Therefore, for optimal operation, $\text{DMA.CTLx.DEST_MSIZE}$ should be set at the FIFO level that triggers a transmit DMA request; that is:

$$\text{DMA.CTLx.DEST_MSIZE} = \text{UART.FIFO_DEPTH} - \text{decoded level of UART.FCR}[5:4] \quad (2)$$

This is the setting used in [Figure 2-28](#).

Adhering to equation (2) reduces the number of DMA bursts needed for a block transfer, which in turn improves AMBA bus utilization.



The transmit FIFO is not full at the end of a DMA burst transfer if the UART has successfully transmitted one data item or more on the UART serial transmit line during the transfer.

2.14.5 Receive Watermark Level and Receive FIFO Overflow

During DW_apb_uart serial transfers, receive FIFO requests are made to the DW_ahb_dmac whenever the number of entries in the receive FIFO is at or above the decoded level of Receiver Trigger (RT) of the FCR[7:6]. This is known as the watermark level. The DW_ahb_dmac responds by fetching a burst of data from the receive FIFO buffer of length CTLx.SRC_MSIZE.

Data should be fetched by the DMA often enough for the receive FIFO to accept serial transfers continuously; that is, when the FIFO begins to fill, another DMA transfer is requested. Otherwise, the FIFO fills with data (overflow). To prevent this condition, you must correctly set the watermark level.

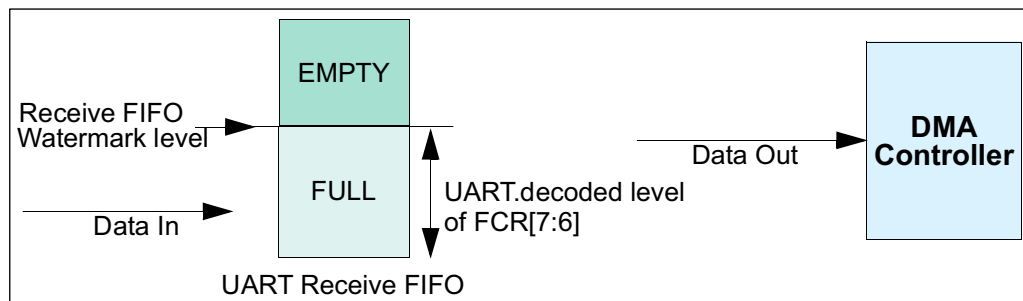
2.14.6 Choosing the Receive Watermark Level

Similar to choosing the transmit watermark level described earlier, the receive watermark level – decoded level of FCR[7:6] – should be set to minimize the probability of overflow. It is a trade-off between the number of DMA burst transactions required per block versus the probability of an overflow occurring.

2.14.7 Selecting SRC_MSIZE and Receive FIFO Underflow

As can be seen in [Figure 2-31](#), programming a source burst transaction length greater than the watermark level can cause underflow when there is not enough data to service the source burst request. Therefore, equation (3) below must be adhered to in order to avoid underflow.

Figure 2-31 UART Receive FIFO



If the number of data items in the receive FIFO is equal to the source burst length at the time the burst request is made – $\text{DMA.CTLx.SRC_MSIZE}$ – the receive FIFO can be emptied, but not underflowed, at the completion of the burst transaction. For optimal operation, $\text{DMA.CTLx.SRC_MSIZE}$ should be set at the watermark level; that is:

$$\text{DMA.CTLx.SRC_MSIZE} = \text{decoded level of FCR[7:6]} \quad (3)$$

Adhering to equation (3) reduces the number of DMA bursts in a block transfer, and this in turn can improve AMBA bus utilization.

**Note**

The receive FIFO is not empty at the end of the source burst transaction if the UART has successfully received one data item or more on the UART serial receive line during the burst.

2.14.8 Handshaking Interface Operation

- **dma_tx_req_n, dma_rx_req_n** – The request signals for source and destination—**dma_tx_req_n** and **dma_rx_req_n**—are activated when their corresponding FIFOs reach the watermark levels.

The DW_ahb_dmac uses edge detection of the **dma_tx_req_n** signal/**dma_rx_req_n** to identify a request on the channel. Upon reception of the **dma_tx_ack_n**/**dma_rx_ack_n** signal from the DW_ahb_dmac to indicate the burst transaction is complete, the DW_apb_uart de-asserts the burst request signals—**dma_tx_req_n**/**dma_rx_req_n**—until **dma_tx_ack_n**/**dma_rx_ack_n** is de-asserted by the DW_ahb_dmac.

When the DW_apb_uart samples that **dma_tx_ack_n**/**dma_rx_ack_n** is de-asserted, it can re-assert the **dma_tx_req_n**/**dma_rx_req_n** of the request line if their corresponding FIFOs exceed their watermark levels—back-to-back burst transaction. If this is not the case, the DMA request lines remain de-asserted.

Figure 2-32 shows a timing diagram of a burst transaction where $pclk = hclk$.

Figure 2-32 Burst Transaction – $pclk = hclk$

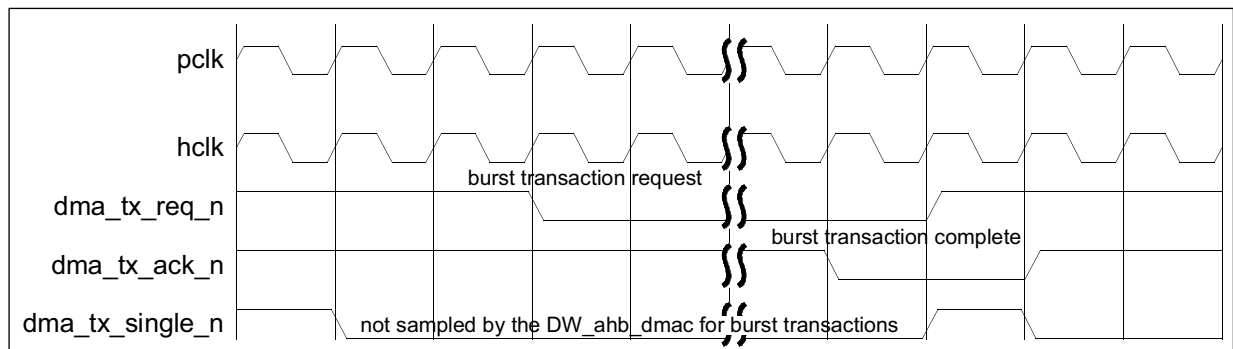
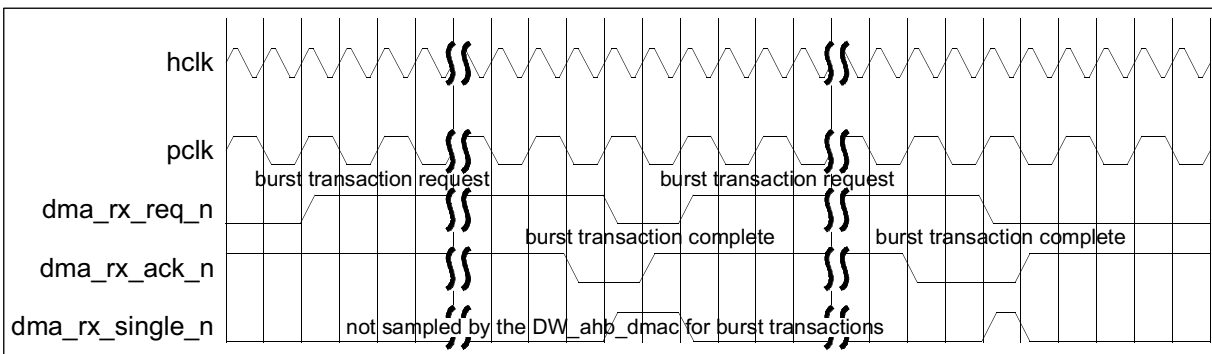


Figure 2-33 shows two back-to-back burst transactions where the **hclk** frequency is twice the **pclk** frequency.

Figure 2-33 Back-to-Back Burst Transactions – $hclk = 2 \cdot pclk$ 

The handshaking loop is as follows:

- `dma_tx_req_n/dma_rx_req_n` asserted by DW_apb_uart
- `dma_tx_ack_n/dma_rx_ack_n` asserted by DW_ahb_dmac
- `dma_tx_req_n/dma_rx_req_n` de-asserted by DW_apb_uart
- `dma_tx_ack_n/dma_rx_ack_n` de-asserted by DW_ahb_dmac
- `dma_tx_req_n/dma_rx_req_n` re-asserted by DW_apb_uart, if back-to-back transaction is required



Note

The burst transaction request signals, `dma_tx_req_n` and `dma_rx_req_n`, are generated in the DW_apb_uart off pclk and sampled in the DW_ahb_dmac by hclk. The acknowledge signals, `dma_tx_ack_n` and `dma_rx_ack_n`, are generated in the DW_ahb_dmac off hclk and sampled in the DW_apb_uart of pclk. The handshaking mechanism between the DW_ahb_dmac and the DW_apb_uart supports quasi-synchronous clocks; that is, hclk and pclk must be phase-aligned, and the hclk frequency must be a multiple of the pclk frequency.

- Note the following:
 - Once asserted, the burst request lines – `dma_tx_req_n/dma_rx_req_n` – remain asserted until their corresponding `dma_tx_ack_n/dma_rx_ack_n` signal is received, even if the respective FIFOs drop below their watermark levels during the burst transaction.
 - The `dma_tx_req_n/dma_rx_req_n` signals are de-asserted when their corresponding `dma_tx_ack_n/dma_rx_ack_n` signals are asserted, even if the respective FIFOs exceed their watermark levels.
- `dma_tx_single_n, dma_rx_single_n`
 - `dma_tx_single_n` – status signal that is asserted when there is at least one free entry in the transmit FIFO; it is cleared when the transmit FIFO is full.
 - `dma_rx_single_n` – status signal that is asserted when there is at least one valid data entry in the receive FIFO; it is cleared when the receive FIFO is empty.

These signals are needed by only the DW_ahb_dmac for the case where the block size – `CTLx.BLOCK_TS` – that is programmed into the DW_ahb_dmac is not a multiple of the burst transaction length – `CTLx.SRC_MSIZEx, CTLx.DEST_MSIZEx` – as shown in [Figure 2-28](#). In this case,

the DMA single outputs inform the DW_ahb_dmac that it is still possible to perform single data item transfers, so it can access all data items in the transmit/receive FIFO and complete the DMA block transfer. Otherwise, the DMA single outputs from the DW_apb_uart are not sampled by the DW_ahb_dmac.

This is illustrated in the following example.

Receive FIFO Channel of the DW_apb_uart:

DMA.CTLx.SRC_MSIZE = decoded level of UART.FCR[7:6] = 4

DMA.CTLx.BLOCK_TS = 12

Block transfer:

DMA.CTLx.SRC_MSIZE = decoded level of UART.FCR[7:6] = 4

DMA.CTLx.BLOCK_TS = 15

For the example in [Figure 2-27](#), with the block size set to 12, the `dma_rx_req_n` signal is asserted when four data items are present in the receive FIFO. The `dma_rx_req_n` signal is asserted three times during the DW_apb_uart serial transfer, ensuring that all 12 data items are read by the DW_ahb_dmac. All DMA requests read a block of data items and no single DMA transactions are required. The block transfer is made up of three burst transactions.

The first 12 data items are transferred using three burst transactions. But when the last three data frames enter the receive FIFO, the `dma_rx_req_n` signal is not activated because the FIFO level is below the watermark level. The DW_ahb_dmac samples `dma_rx_single_n` and completes the DMA block transfer using three single transactions. The block transfer is made up of three burst transactions, followed by three single transactions.

[Figure 2-34](#) shows a single transaction. The handshaking loop is as follows:

- a. `dma_tx_single_n/dma_rx_single_n` asserted by DW_apb_uart
- b. `dma_tx_ack_n/dma_rx_ack_n` asserted by DW_ahb_dmac
- c. `dma_tx_single_n/dma_rx_single_n` de-asserted by DW_apb_uart
- d. `dma_tx_ack_n/dma_rx_ack_n` de-asserted by DW_ahb_dmac

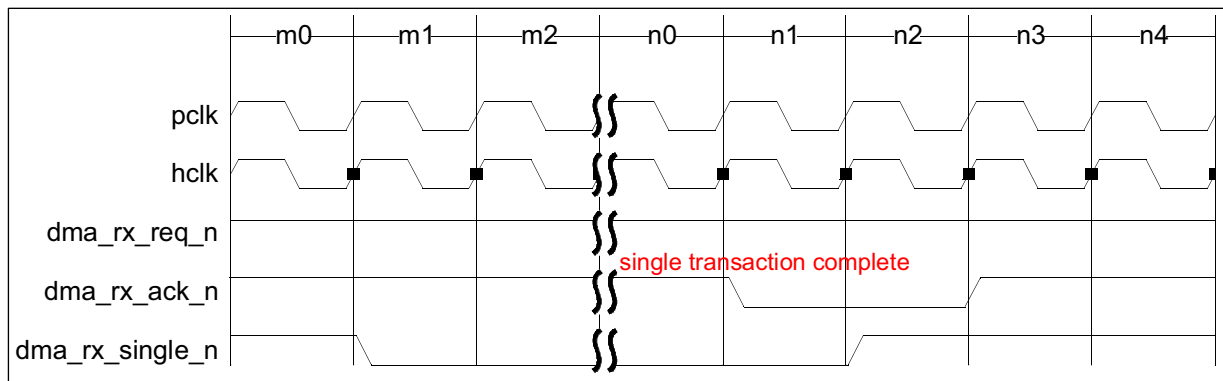
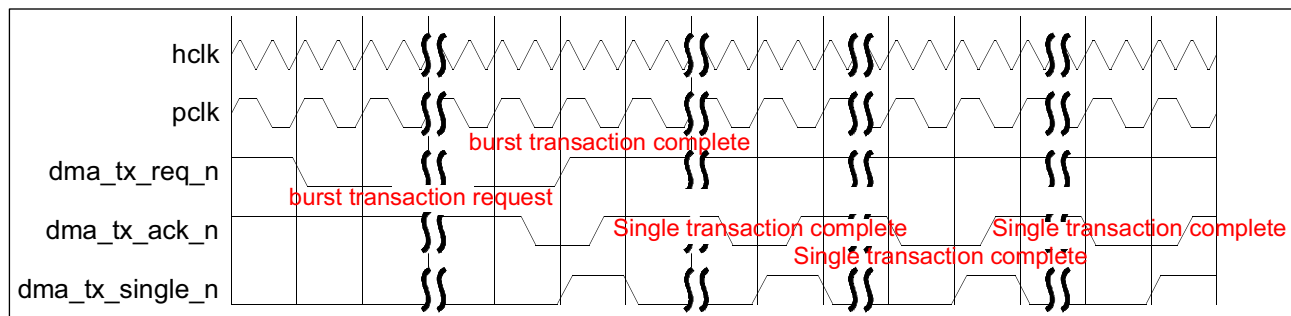
Figure 2-34 Single Transaction

Figure 2-35 shows a burst transaction, followed by three back-to-back single transactions, where the hclk frequency is twice the pclk frequency.

Figure 2-35 Burst Transaction + 3 Back-to-Back Singles – $hclk = 2 * pclk$ **Note**

The single transaction request signals, `dma_tx_single_n` and `dma_rx_single_n`, are generated in the DW_apb_uart on the pclk edge and sampled in DW_ahb_dmac on hclk. The acknowledge signals, `dma_tx_ack_n` and `dma_rx_ack_n`, are generated in the DW_ahb_dmac on the hclk edge hclk and sampled in the DW_apb_uart on pclk. The handshaking mechanism between the DW_ahb_dmac and the DW_apb_uart supports quasi-synchronous clocks; that is, hclk and pclk must be phase aligned and the hclk frequency must be a multiple of pclk frequency.

2.14.9 Potential Deadlock Conditions in DW_apb_uart/DW_ahb_dmac Systems

There is a risk of a deadlock occurring if all three of the following are true:

- DW_ahb_dmac is used to access UART FIFOs
- DMA burst transaction length is set to value smaller than or equal to DW_apb_uart Rx FIFO threshold
- When DW_apb_uart is used in DMA mode 1 with auto-flow control mode enabled

2.14.9.1 Deadlock When DMA Burst Transaction Length Smaller Than Rx FIFO Threshold

When operating in autoflow control mode with the RTC flow trigger threshold is disabled, the DW_apb_uart de-asserts `rts_n` when the Rx FIFO threshold is reached, and it asserts it again when the Rx FIFO is empty. At the same time, the DW_apb_uart asserts `dma_rx_req_n`, requesting a burst transaction from the DW_ahb_dmac.

If the DMA burst transaction length is equal to or greater than the Rx FIFO threshold, the DW_ahb_dmac reads from the Rx FIFO until it is empty, causing `rts_n` to be re-asserted. This in turn allows more data to be received by the DW_apb_uart and the Rx FIFO to fill again.

However, if the DW_ahb_dmac burst transaction length is smaller than the DW_apb_uart Rx FIFO threshold, some data is left in the DW_apb_uart Rx FIFO after completion of the burst transaction. This prevents the `rts_n` signal from being asserted.

Because the amount of data in the Rx FIFO is below the threshold, the DW_apb_uart asserts the `dma_rx_single_n` signal – instead of `dma_rx_req_n` – requesting a DMA single transaction from the DW_ahb_dmac. However, unless it is operating in the single transaction region, the DW_ahb_dmac ignores single transaction requests.

A deadlock condition is then reached:

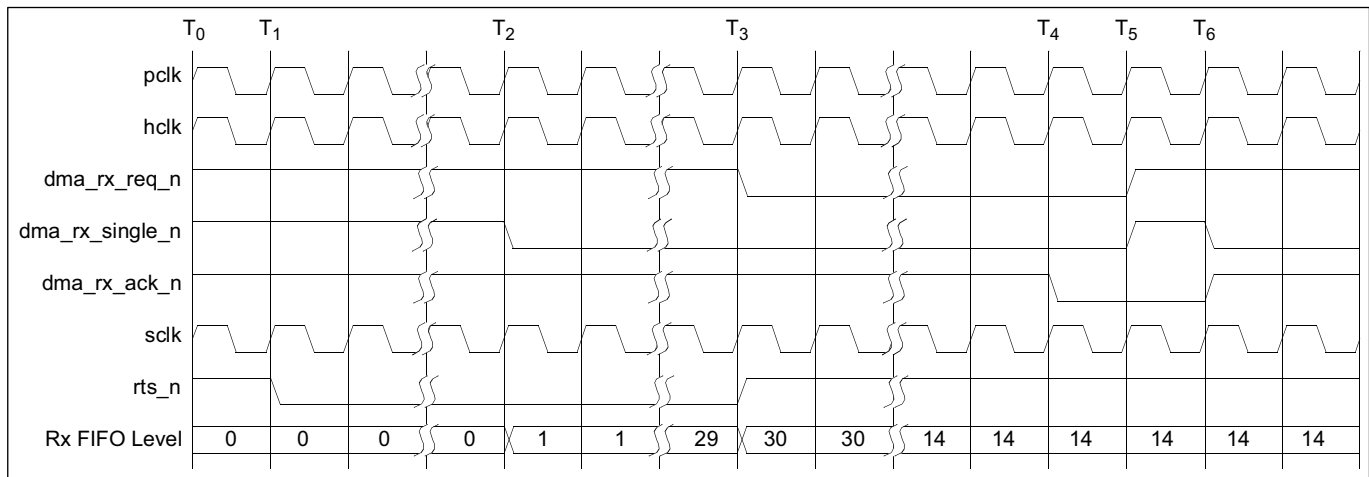
- DW_apb_uart does not receive any extra characters because the `rts_n` signal is de-asserted; no data can be pushed into the Rx FIFO to fill it up to the threshold level again and generate a new burst transaction request from the DW_ahb_dmac; only single transaction requests can be generated.
- Unless it has reached the single transaction region, the DW_ahb_dmac ignores single transaction requests and does not read from the Rx FIFO; the Rx FIFO cannot be emptied, which prevents the `rts_n` signal from being asserted again

Table 2-4 illustrates this condition.

Table 2-4 DW_apb_uart/DW_ahb_dmac Settings for Deadlock When Transaction Less Than Rx FIFO Threshold

DW_apb_uart Settings	DW_ahb_dmac Settings
■ Component configured for 32 byte deep Rx FIFO ■ Autoflow mode enabled (AFCE=1) ■ Rx FIFO threshold set to “2 less than full”; that is, 30 bytes (RCVR=11)	■ Block size set to 100 bytes (BLOCK_TS=100) ■ Source transaction width set to 1 byte (SRC_TR_WIDTH=1) ■ Source burst transaction length set to 16 (SRC_MSIZE=16)

The timing diagram in [Figure 2-36](#) illustrates the sequence of events that lead to this deadlock condition.

Figure 2-36 Example of DW_apb_uart and DW_ahb_dmac Deadlock Occurrence**Note**

For the sake of simplicity, pclk, hclk and sclk are shown to be identical; however, this is not a constraint for the occurrence of deadlock. Additionally, in the interest of simplicity, some events are represented as taking place simultaneously; however, in reality this might not be strictly the case and these events can be separated by a small number of clock cycles.

In [Figure 2-36](#), the following events are shown:

- T1 – The DW_apb_uart is programmed and enabled; rts_n is asserted to initiate the reception of characters.
- T2 – The first character is received by the DW_apb_uart and pushed into the Rx FIFO; dma_rx_single_n is asserted as a consequence, but because DW_ahb_dmac is not in the single transfer region, this request is ignored.
- T3 – The 30th character is received and pushed into the Rx FIFO. As a consequence:
 - rts_n is de-asserted, stopping any further characters from being received
 - dma_rx_req_n signal is asserted
 - DW_ahb_dmac attends this request and starts reading data from Rx FIFO
- T4 – The 16th character popped from the Rx FIFO is received by the DW_ahb_dmac, which asserts dma_rx_ack_n to signal the completion of the DMA burst transaction. Since the DMA burst transaction size is set to 16 and the Rx FIFO threshold is set to 30, there are fourteen characters left in the Rx FIFO after the DMA burst transaction completes.
- T5 – One cycle after dma_rx_ack_n is asserted, the DW_apb_uart de-asserts dma_rx_req_n and dma_rx_single_n as part of the DMA handshaking protocol.
- T6 – One cycle after dma_rx_req_n is de-asserted, the DW_ahb_dmac de-asserts dma_rx_ack_n to complete the DMA handshaking protocol. At the same time, the DW_apb_uart re-asserts dma_req_single_n because there are fourteen characters in the Rx FIFO. For the same reason, rts_n is kept de-asserted; since the DW_ahb_dmac is not in the single transfer region, it ignores the single transaction request and a deadlock is created.

This deadlock condition can be avoided if:

- The Rx FIFO threshold level is set to a value equal to or smaller than the DMA burst transaction size. This ensures the Rx FIFO is always empty after a DMA burst transaction completes and `rts_n` is asserted accordingly.
- The DMA block size is set to a value smaller than twice the DMA burst transaction length. This guarantees the DW_ahb_dmac enters the single transaction region after the DMA burst transaction completes. It then accepts single transaction requests from the DW_apb_uart, allowing the Rx FIFO to be emptied. In this case, the DMA burst size can be configured to be smaller than the Rx FIFO threshold level.

2.14.9.2 Deadlock When DMA Burst Transaction Length Equal To Rx FIFO Threshold

If the DMA burst transaction length is identical to the DW_apb_uart Rx FIFO threshold, there is risk of a deadlock condition occurring when a character is received after `rts_n` is de-asserted.

The DW_apb_uart de-asserts `rts_n` when the Rx FIFO threshold is reached. However, it is possible the component at the other end of the line starts transmitting a new character before it detects the de-assertion of its `cts_n` input. When this happens, the character transmission completes normally, which means an extra character is received and pushed into the Rx FIFO (unless it is already full).

At the same time that `rts_n` is de-asserted, the DW_apb_uart asserts `dma_rx_req`, requesting a DMA burst transaction from the DW_ahb_dmac. After the DW_ahb_dmac completes this burst transaction – with length equal to the Rx FIFO threshold – there is one character left in the Rx FIFO, preventing `rts_n` from being asserted again.

The DW_apb_uart asserts the `dma_rx_single_n` signal – instead of `dma_rx_req_n` – requesting a DMA single transaction from the DW_ahb_dmac. However, unless it is operating in the single-transaction region, the DW_ahb_dmac ignores single-transaction requests.

A deadlock condition is then reached:

- The DW_apb_uart does not receive any extra characters because the `rts_n` signal is de-asserted. No data can be pushed into the Rx FIFO to fill it up to the threshold level again and generate a new burst transaction request from the DW_ahb_dmac; only single-transaction requests can be generated.
- Unless it has reached the single-transaction region, the DW_ahb_dmac ignores single-transaction requests and does not read from the Rx FIFO. The Rx FIFO cannot be emptied, which prevents the `rts_n` signal from being asserted again.

This deadlock condition can be avoided if:

- The Rx FIFO threshold level is set to a value smaller than the DMA burst transaction size. This ensures that the Rx FIFO is always empty after a DMA burst transaction completes, regardless of whether or not one extra character is received and `rts_n` is asserted accordingly.
- The DMA block size is set to a value smaller than twice the DMA burst transaction length. This guarantees that the DW_ahb_dmac enters the single transaction region after the DMA burst transaction completes. It then accepts single transaction requests from the DW_apb_uart, allowing the Rx FIFO to be emptied.

**Note**

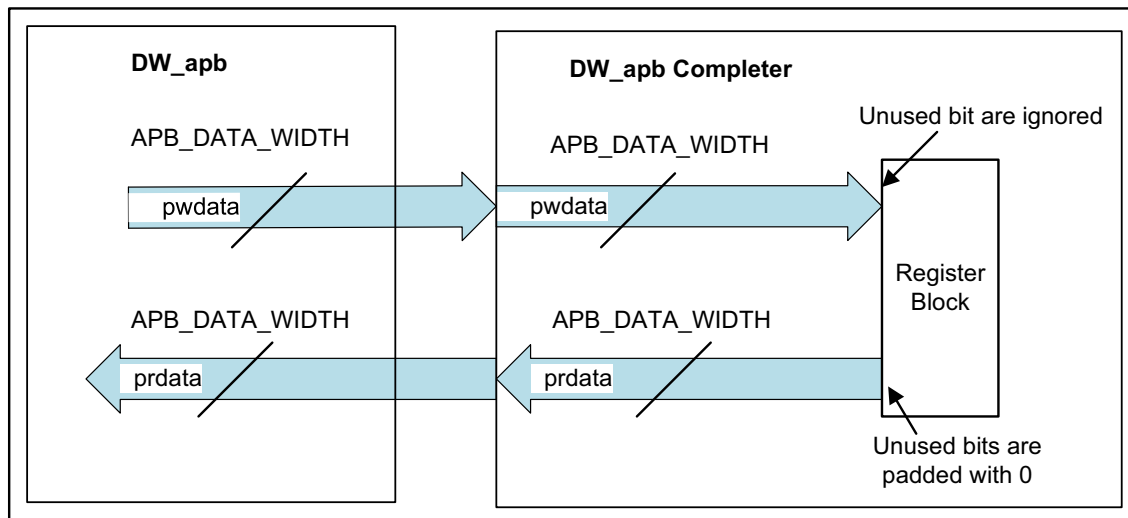
This deadlock condition is not expected to occur frequently under normal operating conditions. A timeout interrupt would be generated in this case, which can be used to detect the occurrence of this deadlock condition.

2.15 APB Interface

The DW_apb_uart peripheral has a standard AMBA 2.0 APB interface for reading and writing the internal registers. The host processor accesses data, control, and status information on the DW_apb_uart peripheral through the AMBA APB 2.0/3.0/4.0 interface. This peripheral supports APB data bus widths of 8, 16, or 32 bits, which is set with the APB_DATA_WIDTH parameter.

Figure 2-37 shows the read/write buses between the DW_apb and the APB completer.

Figure 2-37 Read/Write Buses Between the DW_apb and an APB Completer



The data, control and status registers within the DW_apb_uart are byte-addressable. The maximum width of the control or status register (except for registers mentioned in Table 2-5) in the DW_apb_uart is 8 bits. Therefore, if the APB data bus is 8, 16, or 32 bits wide, all read and write operations to the DW_apb_uart control and status registers require only one APB access.

The maximum width (excluding reserved bits) of registers mentioned in Table 2-5 can vary from 8 bits to 32 bits. Depending on these registers width and the APB data bus width (that is, the APB_DATA_WIDTH parameter), the APB interface may need to perform single or multiple accesses to registers mentioned in Table 2-5.

Table 2-5 Lists of Registers with Width (Excluding Reserved Bits) Greater than 8 Bits

Register	Name
RBR	Receive Buffer Register
THR	Transmit Holding Register
LSR	Line Status Register

Table 2-5 Lists of Registers with Width (Excluding Reserved Bits) Greater than 8 Bits

Register	Name
SRBRn (for n=0; n<=15)	Shadow Receive Buffer Register
STHRn (for n=0; n<=15)	Shadow Transmit Holding Register
RFW	Receive FIFO Write Register
TFL	Transmit FIFO Level Register
RFL	Receive FIFO Level Register
DET	Driver output Enable Timing Register
TAT	Turn Around Timing Register
DLF	Divisor Latch Fraction Register
CPR	Component Parameter Register
UCV	UART Component Version Register
CTR	Component Type Register

Chapter “[Integration Considerations](#)” on page 235 provides information about reading to and writing from the APB interface.

The APB3 and APB4 register accesses to the DW_apb_uart peripheral are discussed in the following sections:

- “[APB 3.0 Support](#)” on page 82
- “[APB 4.0 Support](#)” on page 83

2.15.1 APB 3.0 Support

The register interface of DW_apb_uart is compliant to APB 2.0, APB 3.0, and APB 4.0 specifications. To comply with the AMBA 3 APB Protocol Specification (v1.0) - APB3, DW_apb_uart supports the following signals:

- **PREADY** - The ready signal is used to extend the APB transfer and it is also used to indicate the end of the transaction when there is a high in the access phase of a transaction. The PREADY signal is always kept to its default value, that is high for all APB accesses except for the RBR and THR register access in FIFO mode (FIFO_MODE != NONE) when FIFOs are enabled (FCR[0] set to 1). During the RBR and THR access in FIFO mode, the PREADY signal is de-asserted to stall the APB transaction under following conditions:
 - **Receiver FIFO is empty:** The APB transaction completes and PREADY is asserted if the data is received in the Rx FIFO before the register read/write timeout happens. For more information on timeout, see Completer Error Response, PSLVERR on [page 83](#).

- **Transmitter FIFO is full:** The APB transaction completes and PREADY is asserted if the data is read out of the Tx FIFO before the register read/write timeout happens. For more information on timeout, see Completer Error Response, PSLVERR on [page 83](#).
- **PSLVERR** - The PSLVERR signal is enabled when the PSLVERR_RESP_EN parameter is set to 1, so that DW_apb_uart provides any completer error response from register interface (if required). The DW_apb_uart generates an error response under the following conditions:
 - Registers protected through PPROT (see [Table 2-6](#)) are accessed without relevant authorization levels. For more information, see “[APB 4.0 Support](#)” on [page 83](#).
 - The DW_apb_uart stalls the APB transaction by pulling PREADY low, as the Receiver FIFO is empty or Transmitter FIFO is full. To avoid locking of the bus for large number of clock cycle, a timeout option is provided through configuration parameter REG_TIMEOUT_VALUE. The timeout is triggered under following conditions:
 - Receiver FIFO remains empty or
 - Transmitter FIFO remains full

If the duration is equal to the timeout period that is REG_TIMEOUT_VALUE, then APB interface asserts PSLVERR signal to indicate the register read/write timeout.

2.15.2 APB 4.0 Support

The DW_apb_uart register interface is compliant to the APB 4.0 specification and to comply with the AMBA APB Protocol Specification (v2.0) - APB4, DW_apb_uart supports the following signals:

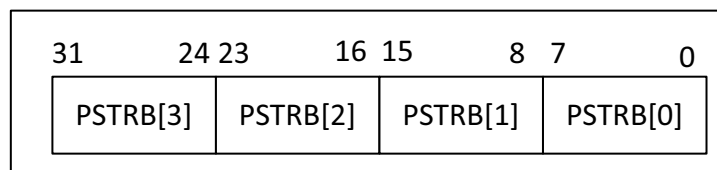
- **PSTRB** - This signal specifies the APB4 write strobe bus. In a write transaction, the PSTRB signal indicates validity of PWDATA bytes. DW_apb_uart component selectively writes to the bytes of the addressed register whose corresponding bit in the PSTRB signal is high. Bytes strobed low by the corresponding PSTRB bits are not modified. The incoming strobe bits for a read transaction is always zero as per the APB 4.0 protocol.

[Figure 2-38](#) on [page 84](#) shows the byte lane mapping of the PSTRB signal.



Note

- When FIFO access mode is enabled the data that is written to the RFWD field of RFW register is pushed into the RBR register.
- When APB_DATA_WIDTH=8, two APB writes are required to write the RFW register, since its width is 9 bits. In this scenario, RBR register is updated after the second APB write (with PSTRB[1]=1) into the RFW register.

Figure 2-38 PSTRB signal byte lane mapping

- **PPROT** - This signal supports the protection feature of the APB 4.0 protocol. The APB 4.0 protection feature is supported on the registers listed in [Table 2-6](#). The protection level register (UART_PROT_LEVEL) defines the APB4 protection level, that is the protected registers are updated only if the PPROT privilege is more than the protection privilege programmed in the protection level register. Otherwise, PSLVERR is asserted and the protected register is not updated, provided that PSLVERR_RESP_EN is set as high. If the PSLVERR_RESP_EN is low, then protection feature and PSLVERR generation logic is not implemented.

Table 2-6 List of Registers Protected Through PPROT

Register	Name
RBR	Receive Buffer Register
THR	Transmit Holding Register
FCR	FIFO Control Register
LCR	Line Control Register
MCR	Modem Control Register

Table 2-7 PPROT Level, Protection Level Programmed in UART_PROT_LEVEL, and Completer Error Response

PPROT			UART_PROT_LEVEL			PSLVERR
[2]	[1]	[0]	[2]	[1]	[0]	
X	X	0	X	X	1	HIGH
X	1	X	X	0	X	HIGH
0	X	1	1	X	X	HIGH

3

Parameter Descriptions

This chapter details all the configuration parameters. **You can use the coreConsultant GUI configuration reports to determine the actual configured state of the controller.** Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

The parameter descriptions in this chapter include the **Enabled:** attribute which indicates the values required to be set on other parameters before you can change the value of this parameter.

These tables define all of the configuration options for this component.

- System Configuration on [page 86](#)
- UART Configuration on [page 88](#)
- Storage Configuration on [page 92](#)
- Synchronization Configuration on [page 93](#)
- Additional Features on [page 94](#)

3.1 System Configuration Parameters

Table 3-1 System Configuration Parameters

Label	Description
APB System Configuration	
Register Interface Type	Selects Register Interface type as APB2, APB3 or APB4. By default, DW_apb_uart supports APB2 interface. Values: ■ APB2 (0) ■ APB3 (1) ■ APB4 (2) Default Value: APB2 Enabled: DWC-APB-Advanced-Source source license exists. Parameter Name: SLAVE_INTERFACE_TYPE
APB Data Bus Width	Width of APB data bus to which this component is attached. The data width can be set to 8, 16 or 32. Register access is on 32-bit boundaries, unused bits are held at static 0. Values: 8, 16, 32 Default Value: 32 Enabled: Always Parameter Name: APB_DATA_WIDTH
Completer Error Response Enable	Enable Completer Error response signaling. The component will refrain From signaling an error response if this parameter is disabled. This will result in disabling all features that require SLVERR functionality to be implemented. Values: ■ false (0) ■ true (1) Default Value: false Enabled: SLAVE_INTERFACE_TYPE>0 Parameter Name: PSLVERR_RESP_EN
UART Protection Level	Reset Value of UART_PROT_LEVEL register. A high on any bit of UART protection level requires a high on the corresponding pprot input bit to gain access to the protected registers. Else, SLVERR response is triggered. A zero on the protection bit will provide access to the register if other protection levels are satisfied. Values: 0x0, ..., 0x7 Default Value: 0x2 Enabled: SLAVE_INTERFACE_TYPE>1 && PSLVERR_RESP_EN==1 Parameter Name: PROT_LEVEL_RST

Table 3-1 System Configuration Parameters (Continued)

Label	Description
Hard-Code Protection Level?	Setting this parameter to 1 makes UART_PROT_LEVEL a read-only register. The register can be programmed at run-time by a user if this hard-code option is set to 0. Values: ■ false (0) ■ true (1) Default Value: false Enabled: SLAVE_INTERFACE_TYPE>1 && PSLVERR_RESP_EN==1 Parameter Name: HC_PROT_LEVEL
Width of Register Timeout Counter	Defines the width of Register timeout counter. If set to zero, the timeout counter register is disabled, and timeout is triggered as soon as the transaction tries to read an empty RX FIFO or write to a full TX FIFO. These are the only cases where PREADY signal goes low, in all other cases PREADY is tied high. Setting values from 4 to 8 for this parameter configures the timeout period from 2⁴ to 2⁸ pclk cycles. Values: 0, 4, 5, 6, 7, 8 Default Value: (SLAVE_INTERFACE_TYPE > 0 && PSLVERR_RESP_EN==1 && FIFO_MODE!=0) ? 4 : 0 Enabled: SLAVE_INTERFACE_TYPE>0 && PSLVERR_RESP_EN==1 && FIFO_MODE!=0 Parameter Name: REG_TIMEOUT_WIDTH
Hardcode Register Timeout Counter Value	Checking this parameter makes Register timeout counter a read-only register. The register can be programmed by user if the hardcode option is turned off. Values: ■ false (0) ■ true (1) Default Value: false Enabled: SLAVE_INTERFACE_TYPE>0 && PSLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0 && FIFO_MODE!=0 Parameter Name: HC_REG_TIMEOUT_VALUE
Register Timeout Counter Default Start Value	Defines the reset value of Register timeout counter. Values: 0, ..., POW_2_REG_TIMEOUT_WIDTH Default Value: 8 Enabled: SLAVE_INTERFACE_TYPE>0 && PSLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0 && FIFO_MODE!=0 Parameter Name: REG_TIMEOUT_VALUE

3.2 UART Configuration Parameters

Table 3-2 UART Configuration Parameters

Label	Description
UART Configuration	
RS485 Interface Support	Configures the peripheral for RS485 Interface support. If enabled, new signals 'de', 're' and 'rs485_en' are included in the interface to support RS485 transceiver. Values: ■ Disabled (0) ■ Enabled (1) Default Value: Disabled Enabled: This parameter is enabled if DWC-APB-Advanced-Source license is detected. Parameter Name: UART_RS485_INTERFACE_EN
Active High RS485 Driver Enable Signal?	Selects the polarity of the RS485 Driver Enable (de) signal. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: UART_RS485_INTERFACE_EN==1 Parameter Name: UART_DE_POL
Active High RS485 Receiver Enable Signal?	Selects the polarity of the RS485 Receiver Enable (re) signal. Values: ■ false (0x0) ■ true (0x1) Default Value: true Enabled: UART_RS485_INTERFACE_EN==1 Parameter Name: UART_RE_POL
Enable 9-bit Mode Support?	Configures the peripheral to have 9-bits of data per character. The 9th-bit of the data byte sent from the requester is set to 1 to indicate the address byte while cleared to 0 to indicate the data byte. Values: ■ Disabled (0) ■ Enabled (1) Default Value: Disabled Enabled: This parameter is enabled if DWC-APB-Advanced-Source license is detected. Parameter Name: UART_9BIT_DATA_EN

Table 3-2 UART Configuration Parameters (Continued)

Label	Description
IrDA SIR Mode Support	Configures the peripheral to have IrDA 1.0 SIR infrared mode. For more details, refer to the "IrDA 1.0 SIR Protocol" section of data book. Values: ■ Disabled (0x0) ■ Enabled (0x1) Default Value: Disabled Enabled: Always Parameter Name: SIR_MODE
Low Power IrDA SIR Mode Support	Configures the peripheral to operate in a low-power IrDA SIR mode. As the DW_apb_uart does not support a low-power mode with a counter system to maintain a 1.63us infrared pulse, Asynchronous Serial Clock Support will be automatically enabled, and the sclk must be fixed to 1.8432Mhz. This provides a 1.63us sir_out_n pulse at 115.2kbaud. Values: ■ Disabled (0x0) ■ Enabled (0x1) Default Value: Disabled Enabled: SIR_MODE==1 Parameter Name: SIR_LP_MODE
Support for IrDA SIR Low Power Reception Capabilities	Configures the peripheral to have SIR low power pulse reception capabilities. Two additional Low power Divisor Registers are implemented and must be written with a divisor that will give a baud rate of 115.2k for the low power pulse detection functionality to operate correctly. Asynchronous Serial Clock support is automatically enabled in this mode. Values: ■ Disabled (0) ■ Enabled (1) Default Value: Disabled Enabled: SIR_MODE==1 Parameter Name: SIR_LP_RX
Auto Flow Control	Configures the peripheral to have the D16750-compatible auto flow control mode. For more details, refer to "Auto Flow Control" section of the data book. Values: ■ Disabled (0x0) ■ Enabled (0x1) Default Value: Disabled Enabled: FIFO_MODE!=0 Parameter Name: AFCE_MODE

Table 3-2 UART Configuration Parameters (Continued)

Label	Description
RTC Flow Control Trigger	When set to 0, the DW_apb_uart uses the same receiver trigger level described in FCR.RCVR register both for generating a DMA request and a handshake signal (rts_n). When set to 1, the DW_apb_uart uses two separate trigger levels for a DMA request and handshake signal (rts_n) in order to maximize throughput on the interface. NOTE: Almost-Full Trigger refers to two available slots in the FIFO. Values: ■ RX FIFO Threshold Trigger (0x0) ■ RX FIFO Almost-Full Trigger (0x1) Default Value: RX FIFO Threshold Trigger Enabled: AFCE_MODE!=0 Parameter Name: RTC_FCT
Include Baud Clock Reference Output Signal (baudout_n) on Interface?	Configures the peripheral to have a baud clock reference output (baudout_n) pin on the interface. Values: ■ false (0) ■ true (1) Default Value: true Enabled: Always Parameter Name: BAUD_CLK
Remove Busy Functionality?	Configures the peripheral to be fully 16550 compatible. This is achieved by not having the busy functionality implemented. Values: ■ false (0) ■ true (1) Default Value: false Enabled: Always Parameter Name: UART_16550_COMPATIBLE
Fractional Baud Rate Divisor Support	Configures the peripheral to have Fractional Baud Rate Divisor. If enabled, new Fractional divisor latch register (DLF) is included to program the fractional divisor values. For more information about this feature, see "Fractional Baud Rate Support" section in the data book. Values: ■ Disabled (0) ■ Enabled (1) Default Value: Disabled Enabled: This parameter is enabled if DWC-APB-Advanced-Source license is detected. Parameter Name: FRACTIONAL_BAUD_DIVISOR_EN

Table 3-2 UART Configuration Parameters (Continued)

Label	Description
Fractional Divisor Width	Specifies the width of the fractional divisor. A high value means more precision but long averaging period. For more information about this feature, see "Fractional Baud Rate Support" section in the data book. Values: 4, ..., 6 Default Value: 4 Enabled: FRACTIONAL_BAUD_DIVISOR_EN==1 Parameter Name: DLF_SIZE

3.3 Storage Configuration Parameters

Table 3-3 Storage Configuration Parameters

Label	Description
Storage Configuration	
UART FIFO Depth	Receiver and Transmitter FIFO depth in bytes. A setting of NONE means no FIFOs, which implies the 16450-compatible mode of operation. Most enhanced features are unavailable in the 16450 mode such as the Auto Flow Control and Programmable THRE interrupt modes. Setting a FIFO depth greater than 256 restricts the FIFO Memory to External only. For more details, refer to the "FIFO Support" section of the databook. Values: 0, 16, 32, 64, 128, 256, 512, 1024, 2048 Default Value: 16 Enabled: Always Parameter Name: FIFO_MODE
FIFO Memory Type	Selects between external, user-supplied memory or internal DesignWare memory (DW_ram_r_w_s_dff) for the receiver and transmitter FIFOs. FIFO depths greater than 256 restrict FIFO Memory selection to external. In addition, selection of internal memory restricts the Memory Read Port Type to Dflip-flop-based, synchronous read port RAMs. Values: ■ External (0) ■ Internal (1) Default Value: External Enabled: FIFO_MODE!=0 && FIFO_MODE<=256 Parameter Name: MEM_SELECT_USER
Include FIFO Access Mode?	Configures the peripheral to have a programmable FIFO access mode. This is used for test purposes to allow the receiver FIFO to be written and the transmit FIFO to be read when FIFO's are implemented and enabled. When FIFO's are not implemented or not enabled it allows the RBR to be written and the THR to be read. For more details, refer to "FIFO Support" section in the data book. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: FIFO_ACCESS

3.4 Synchronization Configuration Parameters

Table 3-4 Synchronization Configuration Parameters

Label	Description
Synchronization Configuration	
Asynchronous Serial Clock Support	When set to Disabled, the DW_apb_uart is implemented with one system clock (pclk). When set to Enabled, two system clocks (pclk and sclk) are implemented in order to accommodate accurate serial baud rate settings, as well as APB bus interface requirements. Selecting Disabled, or a one-system clock, greatly restricts system clock settings available for accurate baud rates. For more details, refer to "Synchronization" section of the databook. Values: ■ Disabled (1) ■ Enabled (2) Default Value: Disabled Enabled: SIR_LP_MODE!=1 && SIR_LP_RX!=1 Parameter Name: CLOCK_MODE
Clock Domain Crossing Synchronization Depth?	Sets the number of synchronization stages to be placed on clock domain crossing signals. ■ 2: 2-stage synchronization with positive-edge capturing at both the stages ■ 3: 3-stage synchronization with positive-edge capturing at all stages ■ 4: 4-stage synchronization with positive-edge capturing at all stages Values: 2, 3, 4 Default Value: 2 Enabled: Always Parameter Name: SYNC_DEPTH

3.5 Additional Features Parameters

Table 3-5 Additional Features Parameters

Label	Description
DMA Configuration	
Include Additional DMA Signals on Interface?	Configures the peripheral to have four additional DMA signals on the interface so that the device is compatible with the DesignWare DMA controller interface requirements. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: DMA_EXTRA
Active Low DMA Signals?	Selects the polarity of the DMA interface signals. Values: ■ false (0) ■ true (1) Default Value: true Enabled: Always Parameter Name: DMA_POL
Assert Tx Req On Reset?	Selects the DMA Tx Request assertion logic. When set to 1, DMA Tx Request will be asserted upon reset. When set to 0, DMA Tx Request will not be asserted upon reset. It will be asserted only after LCR register is written. Values: ■ false (0) ■ true (1) Default Value: true Enabled: Always Parameter Name: DMA_HS_REQ_ON_RESET
Additional Features	
Programmable THRE Interrupt Mode	Configures the peripheral to have a programmable Transmitter Hold Register Empty (THRE) interrupt mode. For more information, refer to "Programmable THRE Interrupt" section of the data book. Values: ■ Disabled (0x0) ■ Enabled (0x1) Default Value: Disabled Enabled: FIFO_MODE!=0 Parameter Name: THRE_MODE_USER

Table 3-5 Additional Features Parameters (Continued)

Label	Description
Include Clock Gate Enable Output on Interface?	Configures the peripheral to have a clock gate enable output signal on the interface that indicates that the device is inactive, so clocks may be gated. This parameter is the reset value of register bit LCR_EXT.CGEN. Values: ■ false (0) ■ true (1) Default Value: false Enabled: Always Parameter Name: CLK_GATE_EN
Include On-chip Debug Output Signals on Interface?	Configures the peripheral to have on-chip debug pins on the interface. Values: ■ false (0) ■ true (1) Default Value: false Enabled: Always Parameter Name: DEBUG
Add Version and ID Registers, Enable FIFO Status, Shadow and Encoded Parameters Register Options ?	Configures the peripheral to have the option to include the FIFO status registers, shadow registers and encoded parameter register. Also configures the peripheral to have the UART component version and the peripheral ID registers. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: Always Parameter Name: ADDITIONAL_FEATURES
Include Software Accessible FIFO Status Registers?	Configures the peripheral to have three additional FIFO status registers. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: FIFO_MODE!=0 && ADDITIONAL_FEATURES==1 Parameter Name: FIFO_STAT

Table 3-5 Additional Features Parameters (Continued)

Label	Description
Include Additional Shadow Registers for Reducing Software Overhead?	Configures the peripheral to have nine additional registers that shadow some of the existing register bits that are regularly modified by software. These can be used to reduce the software overhead that is introduced by having to perform read-modify writes. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: ADDITIONAL_FEATURES==1 Parameter Name: SHADOW
Include Configuration Parameter Register?	Configures the peripheral to have a component parameter register. Values: ■ false (0x0) ■ true (0x1) Default Value: false Enabled: ADDITIONAL_FEATURES==1 Parameter Name: UART_ADD_ENCODED_PARAMS
LSR clear Trigger?	Adds a register field IER.ELCOLR and selects the method for clearing the status in the LSR register. This is applicable only for Overrun Error (OE), Parity Error (PE), Framing Error (FE), and Break Interrupt (BI) status bits. For more information, refer to Table 2-6 of the DW_apb_uart databook. When set to 0: ■ LSR status bits (PE, FE and BI) are cleared either on reading Rx FIFO (RBR Read) or On reading LSR register. ■ The Overrun Error Interrupt status is cleared only on reading LSR register. When set to 1: ■ If register field IER.ELCOLR = 0: LSR status bits (OE, PE, FE and BI) are cleared either on reading Rx FIFO (RBR Read) or On reading LSR register. ■ If register field IER.ELCOLR = 1: LSR status bits (OE, PE, FE and BI) are cleared only on reading LSR register. Values: ■ RBR Read or LSR Read (0) ■ LSR Read (1) Default Value: RBR Read or LSR Read Enabled: FIFO_MODE!=0 Parameter Name: LSR_STATUS_CLEAR

Signal Descriptions

This chapter details all possible I/O signals in the controller. For configurable IP titles, your actual configuration might not contain all of these signals.

Inputs are on the left of the signal diagrams; outputs are on the right.

Attention: For configurable IP titles, do not use this document to determine the exact I/O footprint of the controller. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the I/O signals for your actual configuration at workspace/report/IO.html or workspace/report/IO.xml after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the I/O signals that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the widths might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

In addition to describing the function of each signal, the signal descriptions in this chapter include the following information:

Active State: Indicates whether the signal is active high or active low. When a signal is not intended to be used in a particular application, then this signal needs to be tied or driven to the inactive state (opposite of the active state).

Registered: Indicates whether or not the signal is registered directly inside the IP boundary without intervening logic (excluding simple buffers). A value of No does not imply that the signal is not synchronous, only that there is some combinatorial logic between the signal's origin or destination register and the boundary of the controller. A value of N/A indicates that this information is not provided for this IP title.

Synchronous to: Indicates which clocks in the IP sample this input (drive for an output) when considering all possible configurations. A particular configuration might not have all of the clocks listed. This clock might not be the same as the clock that your application logic should use to clock (sample/drive) this pin. For more details, consult the clock section in the databook. The presence of the postfix SuperList indicates a list of all possible clocks over all possible configs. Consult coreConsultant report for which clock applies to your specific configuration.

Exists: Names of configuration parameters that populate this signal in your configuration.

Validated by: Assertion or de-assertion of signals that validates the signal being described.

Attributes used with Synchronous To

- Clock name - The name of the clock that samples an input or drives and output.
- None - This attribute may be used for clock inputs, hard-coded outputs, feed-through (direct or combinatorial), dangling inputs, unused inputs and asynchronous outputs.
- Asynchronous - This attribute is used for asynchronous inputs and asynchronous resets.

The I/O signals are grouped as follows:

- APB Completer Interface on [page 99](#)
- Application Interface on [page 102](#)
- Miscellaneous on [page 103](#)
- FIFO Interface on [page 104](#)
- Modem Interface on [page 107](#)
- DMA Interface on [page 109](#)
- Serial Interface on [page 113](#)
- Infrared Interface on [page 114](#)
- Clock Control Interface on [page 115](#)
- Debug Interface on [page 116](#)
- RS485 Interface on [page 117](#)
- Interrupt Interface on [page 119](#)

4.1 APB Completer Interface Signals

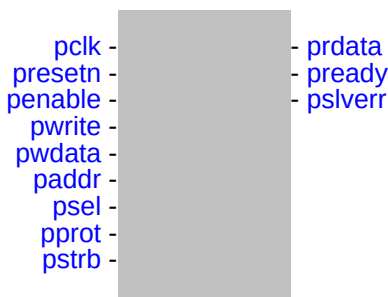


Table 4-1 APB Completer Interface Signals

Port Name	I/O	Description
pclk	I	APB clock used in the APB interface to program registers. Exists: Always Synchronous To: None Registered: N/A Power Domain: SINGLE_DOMAIN Active State: N/A
presetn	I	APB clock-domain reset. Asynchronous assertion, synchronous de-assertion. The reset must be synchronously de-asserted after the rising edge of pclk. DW_apb_uart does not contain logic to perform this synchronization, so it must be provided externally. Exists: Always Synchronous To: Asynchronous Registered: N/A Power Domain: SINGLE_DOMAIN Active State: Low
penable	I	APB enable control used for timing read/write operations. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
pwrite	I	APB Write control. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High

Table 4-1 APB Completer Interface Signals (Continued)

Port Name	I/O	Description
pwdata[(APB_DATA_WIDTH-1):0]	I	APB write data bus. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
paddr[(UART_ADDR_SLICE_LHS-1):0]	I	APB address bus. Uses the lower bits of the APB address bus for register decode. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
psel	I	APB peripheral select. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
pprot[2:0]	I	APB4 Protection type. The input bits should be of equal or greater privilege than the corresponding protection activated level bit of the accessed register to gain access to the protected registers. Else the DW_apb_uart generates an error. If protection level is turned off, any value on the corresponding bit is acceptable. This signal is ignored if PSLVERR_RESP_EN==0. Exists: SLAVE_INTERFACE_TYPE>1 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
pstrb[((APB_DATA_WIDTH/8)-1):0]	I	APB4 Write strobe bus. A high on individual bits in the pstrb bus indicate that the corresponding incoming write data byte on APB bus is to be updated in the addressed register. Exists: SLAVE_INTERFACE_TYPE>1 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High

Table 4-1 APB Completer Interface Signals (Continued)

Port Name	I/O	Description
prdata[(APB_DATA_WIDTH-1):0]	O	APB read data bus. Exists: Always Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
pready	O	The ready signal, used to extend the APB transfer and it is also used to indicate the end of a transaction when there is a high in the access phase of a transaction. Exists: SLAVE_INTERFACE_TYPE>0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
pslverr	O	APB3 Completer error response signal. The signal issues an error when some error condition occurs, as specified in Completer error response section. Exists: SLAVE_INTERFACE_TYPE>0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

4.2 Application Interface Signals

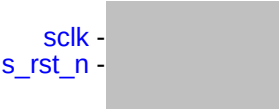


Table 4-2 Application Interface Signals

Port Name	I/O	Description
sclk	I	Serial Interface Clock. Exists: CLOCK_MODE==2 Synchronous To: None Registered: N/A Power Domain: SINGLE_DOMAIN Active State: N/A
s_rst_n	I	Serial Interface Reset. Exists: CLOCK_MODE==2 Synchronous To: Asynchronous Registered: N/A Power Domain: SINGLE_DOMAIN Active State: Low

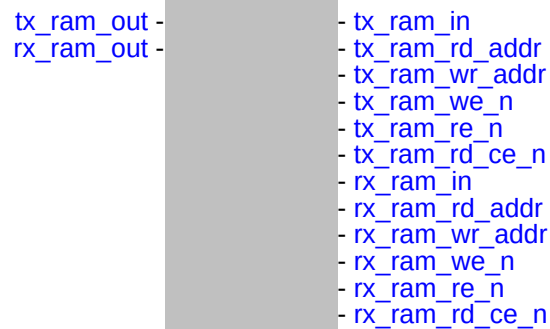
4.3 Miscellaneous Signals

scan_mode - 

Table 4-3 Miscellaneous Signals

Port Name	I/O	Description
scan_mode	I	Scan mode. Used to ensure that test automation tools can control all asynchronous flop signals. This signal is used only when design in test mode. During functional mode, this must be set to 0. This signal is not synchronized inside the IP Exists: Always Synchronous To: None Registered: No Power Domain: SINGLE_DOMAIN Active State: High

4.4 FIFO Interface Signals



tx_ram_out -
rx_ram_out -

- tx_ram_in
- tx_ram_rd_addr
- tx_ram_wr_addr
- tx_ram_we_n
- tx_ram_re_n
- tx_ram_rd_ce_n
- rx_ram_in
- rx_ram_rd_addr
- rx_ram_wr_addr
- rx_ram_we_n
- rx_ram_re_n
- rx_ram_rd_ce_n

Table 4-4 FIFO Interface Signals

Port Name	I/O	Description
tx_ram_out[(TX_RAM_DATA_WIDTH-1):0]	I	Data to the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
rx_ram_out[(RX_RAM_DATA_WIDTH-1):0]	I	Data to the receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
tx_ram_in[(TX_RAM_DATA_WIDTH-1):0]	O	Data from the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
tx_ram_rd_addr[(FIFO_ADDR_WIDTH-1):0]	O	Read address pointer for the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A

Table 4-4 FIFO Interface Signals (Continued)

Port Name	I/O	Description
tx_ram_wr_addr[(FIFO_ADDR_WIDTH-1):0]	O	Write address pointer for the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
tx_ram_we_n	O	Write enable for the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
tx_ram_re_n	O	Read enable for the transmit FIFO RAM wake-up. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
tx_ram_rd_ce_n	O	Read port chip enable for the transmit FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
rx_ram_in[(RX_RAM_DATA_WIDTH-1):0]	O	Data from the receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
rx_ram_rd_addr[(FIFO_ADDR_WIDTH-1):0]	O	Read address pointer for the receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A

Table 4-4 FIFO Interface Signals (Continued)

Port Name	I/O	Description
rx_ram_wr_addr[(FIFO_ADDR_WIDTH-1):0]	O	Write address pointer for the receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A
rx_ram_we_n	O	Write enable for the receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
rx_ram_re_n	O	Read enable for the receive FIFO RAM wake-up. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
rx_ram_rd_ce_n	O	Read port chip enable for receive FIFO RAM. Exists: FIFO_MODE!=0 && MEM_SELECT==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

4.5 Modem Interface Signals



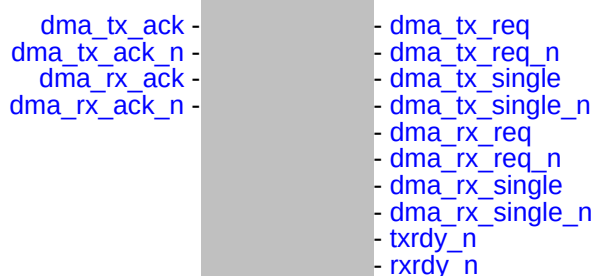
Table 4-5 Modem Interface Signals

Port Name	I/O	Description
cts_n	I	Clear To Send Modem Status. Exists: Always Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
dsr_n	I	Data Set Ready Modem Status input. Exists: Always Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
dcd_n	I	Data Carrier Detect Modem Status input. Exists: Always Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
ri_n	I	Ring Indicator Status input. Exists: Always Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
dtr_n	O	Modem Control Data Terminal Ready Output. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low

Table 4-5 Modem Interface Signals (Continued)

Port Name	I/O	Description
rts_n	O	Modem Control Request To Send output. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
out2_n	O	Modem Control Programmable output 2. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
out1_n	O	Modem Control Programmable output 1. Exists: Always Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low

4.6 DMA Interface Signals



dma_tx_ack -
 dma_tx_ack_n -
 dma_rx_ack -
 dma_rx_ack_n -

dma_tx_req
 dma_tx_req_n
 dma_tx_single
 dma_tx_single_n
 dma_rx_req
 dma_rx_req_n
 dma_rx_single
 dma_rx_single_n
 txrdy_n
 rxrdy_n

Table 4-6 DMA Interface Signals

Port Name	I/O	Description
dma_tx_ack	I	DMA Transmit Acknowledge (Active High) indicates that the DMA Controller has transmitted the block of data to the DW_apb_uart for transmission. Exists: DMA_EXTRA==1 && DMA_POL==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High
dma_tx_ack_n	I	DMA Transmit Acknowledge (Active Low) indicates that the DMA Controller has transmitted the block of data to the DW_apb_uart for transmission. Exists: DMA_EXTRA==1 && DMA_POL==1 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
dma_rx_ack	I	DMA Receive Acknowledge (Active High) indicates that the DMA Controller has transmitted the block of data from the DW_apb_uart. Exists: DMA_EXTRA==1 && DMA_POL==0 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High

Table 4-6 DMA Interface Signals (Continued)

Port Name	I/O	Description
dma_rx_ack_n	I	DMA Receive Acknowledge (Active Low) indicates that the DMA Controller has transmitted the block of data from the DW_apb_uart. Exists: DMA_EXTRA==1 && DMA_POL==1 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: Low
dma_tx_req	O	Transmit Buffer Ready (Active High) indicates that the Transmit buffer requires service from the DMA controller. Exists: DMA_POL==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_tx_req_n	O	Transmit Buffer Ready (Active Low) indicates that the Transmit buffer requires service from the DMA controller. Exists: DMA_POL==1 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
dma_tx_single	O	DMA Transmit FIFO Single (Active High) informs the DMA Controller that there is at least one free entry in the Transmit buffer/FIFO. This output does not request a DMA transfer. Exists: DMA_EXTRA==1 && DMA_POL==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_tx_single_n	O	DMA Transmit FIFO Single (Active Low) informs the DMA Controller that there is at least one free entry in the Transmit buffer/FIFO. This output does not request a DMA transfer. Exists: DMA_EXTRA==1 && DMA_POL==1 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Table 4-6 DMA Interface Signals (Continued)

Port Name	I/O	Description
dma_rx_req	O	Receive Buffer Ready (Active High) indicates that the Receive buffer requires service from the DMA controller. Exists: DMA_POL==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_rx_req_n	O	Receive Buffer Ready (Active Low) indicates that the Receive buffer requires service from the DMA controller. Exists: DMA_POL==1 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
dma_rx_single	O	DMA Receive FIFO Single (Active High) informs the DMA controller that there is at least one free entry in the Receive buffer/FIFO. This output does not request a DMA transfer. Exists: DMA_EXTRA==1 && DMA_POL==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
dma_rx_single_n	O	DMA Receive FIFO Single (Active Low) informs the DMA controller that there is at least one free entry in the Receive buffer/FIFO. This output does not request a DMA transfer. Exists: DMA_EXTRA==1 && DMA_POL==1 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low
txrdy_n	O	This transmit buffer read signal is used for backward compatibility of older DW_apb_uart components to indicate that the Transmit buffer requires service from the DMA controller. Exists: DMA_EXTRA==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

Table 4-6 DMA Interface Signals (Continued)

Port Name	I/O	Description
rxrdy_n	O	This receive buffer read signal is used for backward compatibility of older DW_apb_uart components to indicate that the Receive buffer requires service from the DMA controller. Exists: DMA_EXTRA==0 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

4.7 Serial Interface Signals



Table 4-7 Serial Interface Signals

Port Name	I/O	Description
sin	I	Serial Input. Exists: Always Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
sout	O	Serial Output. Exists: Always Synchronous To: CLOCK_MODE==2 ? "sclk" : "pclk" Registered: Yes Power Domain: SINGLE_DOMAIN Active State: N/A

4.8 Infrared Interface Signals

sir_in -  - sir_out_n

Table 4-8 Infrared Interface Signals

Port Name	I/O	Description
sir_in	I	IrDA SIR Input. Exists: SIR_MODE==1 Synchronous To: Asynchronous Registered: No Power Domain: SINGLE_DOMAIN Active State: N/A
sir_out_n	O	IrDA SIR Output. Exists: SIR_MODE==1 Synchronous To: CLOCK_MODE==2 ? "sclk" : "pclk" Registered: Yes Power Domain: SINGLE_DOMAIN Active State: Low

4.9 Clock Control Interface Signals

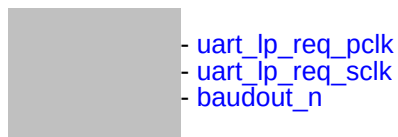


Table 4-9 Clock Control Interface Signals

Port Name	I/O	Description
uart_lp_req_pclk	O	pclk domain clock gate signal indicates that the UART is inactive, so clocks may be gated to put the device in a low-power (lp) mode. Exists: CLK_GATE_EN==1 Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
uart_lp_req_sclk	O	sclk domain clock gate signal indicates that the UART is inactive, so clocks may be gated to put the device in a low-power (lp) mode. Exists: CLK_GATE_EN==1 && CLOCK_MODE==2 Synchronous To: sclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High
baudout_n	O	Transmit clock output. Exists: BAUD_CLK==1 Synchronous To: CLOCK_MODE==2 ? "sclk" : "pclk" Registered: No Power Domain: SINGLE_DOMAIN Active State: Low

4.10 Debug Interface Signals



Table 4-10 Debug Interface Signals

Port Name	I/O	Description
debug[31:0]	O	On-chip debug signals are as follows: ▪ debug[31:14] = RAZ ▪ debug[13] = RX push indication (RBR or RX FIFO) ▪ debug[12] = TX pop indication (THR or TX FIFO) ▪ debug[11:10] = Receive Trigger (FCR[7:6]) ▪ debug[9:8] = TX Empty Trigger (FCR[5:4]) ▪ debug[7] = DMA Mode (FCR[3]) ▪ debug[6:1] = Individual interrupt sources: ▪ debug[6] = Line status Interrupt ▪ debug[5] = Data available Interrupt ▪ debug[4] = character Timeout Interrupt ▪ debug[3] = THRE interrupt ▪ debug[2] = modem status interrupt ▪ debug[1] = busy detect interrupt ▪ debug[0] = FIFO enable (FCR[0]) Note: The debug[1] signal (busy detect interrupt) is never asserted if UART_16550_COMPATIBLE = YES in coreConsultant. Exists: DEBUG==1 Synchronous To: pclk Registered: No Power Domain: SINGLE_DOMAIN Active State: High

4.11 RS485 Interface Signals

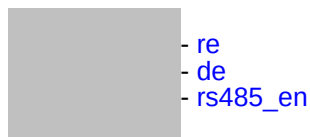


Table 4-11 RS485 Interface Signals

Port Name	I/O	Description
re	O	Receiver Enable Signal. This signal is used to activate and de-activate the Receiver driver. This signal is asserted before the start of receiving serial transfer from DW_apb_uart. This signal is controlled through: - Writing into the RE_EN register; this serves as software override option. - If RE_EN is programmed to 1, then based on the data available in the TX FIFO DW_apb_uart controller automatically controls the 're' signal. Polarity of this signal is set by RE_POL bit in TCR register. Exists: (UART_RS485_INTERFACE_EN==1) Synchronous To: CLOCK_MODE==2 ? "sclk" : "pclk" Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High, if TCR[1]=1; Low, if TCR[1]=0
de	O	Driver Enable Signal. This signal is used to activate and de-activate the transmitter driver. This signal is asserted before the start of transmitting serial transfer from DW_apb_uart. This signal is controlled through: - Writing into the DE_EN register; this serves as software override option - If DE_EN is programmed to 1, then based on the data available in the TX FIFO DW_apb_uart controller automatically controls the 'de' signal. Polarity of this signal is set by DE_POL bit in TCR register. Exists: (UART_RS485_INTERFACE_EN==1) Synchronous To: CLOCK_MODE==2 ? "sclk" : "pclk" Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High, if TCR[2]=1; Low, if TCR[2]=0

Table 4-11 RS485 Interface Signals (Continued)

Port Name	I/O	Description
rs485_en	O	RS485 Enable Signal. This signal indicates whether the DW_apb_uart is enabled for RS485 Mode or RS232 Mode. ■ 0 - RS232 Mode. ■ 1 - RS485 Mode. Exists: (UART_RS485_INTERFACE_EN==1) Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

4.12 Interrupt Interface Signals



Table 4-12 Interrupt Interface Signals

Port Name	I/O	Description
intr	O	Interrupt. Exists: Always Synchronous To: pclk Registered: Yes Power Domain: SINGLE_DOMAIN Active State: High

Register Descriptions

This chapter details all possible registers in the IP. They are arranged hierarchically into maps and blocks (banks). Your actual configuration might not contain all of these registers.

Attention: For configurable IP titles, do not use this document to determine the exact attributes of your register map. It is for reference purposes only.

When you configure the controller in coreConsultant, you must access the register attributes for your actual configuration at `workspace/report/ComponentRegisters.html` or `workspace/report/ComponentRegisters.xml` after you have completed the report creation activity. That report comes from the exact same source as this chapter but removes all the registers that are not in your actual configuration. This does not apply to non-configurable IP titles. In addition, all parameter expressions are evaluated to actual values. Therefore, the Offset and Memory Access values might change depending on your actual configuration.

Some expressions might refer to TCL functions or procedures (sometimes identified as **<functionof>**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the controller in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Exists Expressions

These expressions indicate the combination of configuration parameters required for a register, field, or block to exist in the memory map. The expression is only valid in the local context and does not indicate the conditions for existence of the parent. For example, the expression for a bit field in a register assumes that the register exists and does not include the conditions for existence of the register.

Offset

The term *Offset* is synonymous with *Address*.

Memory Access Attributes

The Memory Access attribute is defined as `<ReadBehavior>/<WriteBehavior>` which are defined in the following table.

Table 5-1 Possible Read and Write Behaviors

Read (or Write) Behavior	Description
RC	A read clears this register field.
RS	A read sets this register field.
RM	A read modifies this register field in a manner not covered above.
Wo	You can only write once to this register field.
W1C	A write of 1 clears this register field.
W1S	A write of 1 sets this register field.
W1T	A write of 1 toggles this register field.
W0C	A write of 0 clears this register field.
W0S	A write of 0 sets this register field.
W0T	A write of 0 toggles this register field.
WC	Any write clears this register field.
WS	Any write sets this register field.
WM	A write modifies this register field in a manner not covered above.
no Read Behavior attribute	You cannot read this register. It is Write-Only.
no Write Behavior attribute	You cannot write to this register. It is Read-Only.

Table 5-2 Memory Access Examples

Memory Access	Description
R	Read-only register field.
W	Write-only register field.
R/W	Read/write register field.
R/W1C	You can read this register field. Writing 1 clears it.
RC/W1C	Reading this register field clears it. Writing 1 clears it.
R/Wo	You can read this register field. You can only write to it once.

Special Optional Attributes

Some register fields might use the following optional attributes.

Table 5-3 Optional Attributes

Attribute	Description
Volatile	As defined by the IP-XACT specification. If true, indicates in the case of a write followed by read, or in the case of two consecutive reads, there is no guarantee as to what is returned by the read on the second transaction or that this return value is consistent with the write or read of the first transaction. The element implies there is some additional mechanism by which this field can acquire new values other than by reads/writes/resets and other access methods known to IP-XACT. For example, when the core updates the register field contents.
Testable	As defined by the IP-XACT specification. Possible values are unconstrained, untestable, readOnly, writeAsRead, restore. Untestable means that this field is untestable by a simple automated register test. For example, the read-write access of the register is controlled by a pin or another register. readOnly means that you should not write to this register; only read from it. This might apply for a register that modifies the contents of another register.
Reset Mask	As defined by the IP-XACT specification. Indicates that this register field has an unknown reset value. For example, the reset value is set by another register or an input pin; or the register is implemented using RAM.
* Varies	Indicates that the memory access (or reset) attribute (read, write behavior) is not fixed. For example, the read-write access of the register is controlled by a pin or another register. Or when the access depends on some configuration parameter; in this case the post-configuration report in coreConsultant gives the actual access value.

Component Banks/Blocks

The following table shows the address blocks for each memory map. Follow the link for an address block to see a table of its registers.

Table 5-4 Address Banks/Blocks for Memory Map: uart_memory_map

Address Block	Description
uart_address_block on page 124	Address block of DW_apb_uart Exists: Always

5.1 uart_memory_map/uart_address_block Registers

Address block of DW_apb_uart. Follow the link for the register to see a detailed description of the register.

Table 5-5 Registers for Address Block: uart_memory_map/uart_address_block

Register	Offset	Description
RBR on page 126	0x0	Receive Buffer Register
THR on page 128	0x0	Transmit Holding Register
DLL on page 130	0x0	Divisor Latch (Low)
IER on page 132	0x4	Interrupt Enable Register
DLH on page 135	0x4	Divisor Latch High
IIR on page 136	0x8	Interrupt Identification Register
FCR on page 138	0x8	FIFO Control Register
LCR on page 141	0xc	Line Control Register
MCR on page 145	0x10	Modem Control Register
LSR on page 149	0x14	Line Status Register
MSR on page 156	0x18	Modem Status Register
SCR on page 161	0x1c	Scratchpad Register
LPDLL on page 162	0x20	Low Power Divisor Latch Low
LPDLH on page 164	0x24	Low Power Divisor Latch High
SRBRn (for n = 0; n <= 15) on page 166	0x30 + n*0x4	Shadow Receive Buffer Register
STHRn (for n = 0; n <= 15) on page 168	0x30 + n*0x4	Shadow Transmit Holding Register
FAR on page 170	0x70	FIFO Access Register
TFR on page 172	0x74	Transmit FIFO Read
RFW on page 173	0x78	Receive FIFO Write
USR on page 175	0x7c	UART Status register
TFL on page 178	0x80	Transmit FIFO Level
RFL on page 179	0x84	Receive FIFO Level
SRR on page 180	0x88	Software Reset Register
SRTS on page 182	0x8c	Shadow Request to Send

Table 5-5 Registers for Address Block: uart_memory_map/uart_address_block (Continued)

Register	Offset	Description
SBCR on page 184	0x90	Shadow Break Control Register
SDMAM on page 186	0x94	Shadow DMA Mode Register
SFE on page 188	0x98	Shadow FIFO Enable Register
SRT on page 189	0x9c	Shadow RCVR Trigger Register
STET on page 191	0xa0	Shadow TX Empty Trigger Register
HTX on page 193	0xa4	Halt TX
DMASA on page 194	0xa8	DMA Software Acknowledge Register
TCR on page 195	0xac	Transceiver Control Register
DE_EN on page 198	0xb0	Driver Output Enable Register
RE_EN on page 199	0xb4	Receiver Output Enable Register
DET on page 200	0xb8	Driver Output Enable Timing Register
TAT on page 202	0xbc	Turn Around Timing Register
DLF on page 204	0xc0	Divisor Latch Fraction Register
RAR on page 205	0xc4	Receive Address Register
TAR on page 207	0xc8	Transmit Address Register
LCR_EXT on page 208	0xcc	Line Extended Control Register
UART_PROT_LEVEL on page 212	0xd0	UART Protection level
REG_TIMEOUT_RST on page 213	0xd4	Register timeout counter reset value
CPR on page 215	0xf4	Component Parameter Register
UCV on page 219	0xf8	UART Component Version
CTR on page 220	0xfc	Component Type Register

5.1.1 RBR

- **Name:** Receive Buffer Register
- **Description:** Receive Buffer Register.
This register can be accessed only when the DLAB bit (LCR[7]) is cleared.
- **Size:** 32 bits
- **Offset:** 0x0
- **Exists:** Always

x:y	x:0
RSVD_RBR	RBR

Table 5-6 Fields for Register: RBR

Bits	Name	Memory Access	Description
x:y	RSVD_RBR	R	RBR 31to9or8 Reserved bits and read as zero (0). Value After Reset: 0x0 Exists: Always Range Variable[x]: "(UART_9BIT_DATA_EN==1) ? \"23\" : \"24\" + \"(UART_9BIT_DATA_EN==1) ? \"0x9\" : \"0x8\" - 1 Range Variable[y]: \"(UART_9BIT_DATA_EN==1) ? \"0x9\" : \"0x8\""

Table 5-6 Fields for Register: RBR (Continued)

Bits	Name	Memory Access	Description
x:0	RBR	R	Receive Buffer Register. This register contains the data byte received on the serial input port (sin) in UART mode or the serial infrared input (sir_in) in infrared mode. The data in this register is valid only if the Data Ready (DR) bit in the Line status Register (LSR) is set. If in non-FIFO mode (FIFO_MODE == NONE) or FIFOs are disabled (FCR[0] set to 0), the data in the RBR must be read before the next data arrives, otherwise it will be overwritten, resulting in an over-run error. If in FIFO mode (FIFO_MODE != NONE) and FIFOs are enabled (FCR[0] set to 1), this register accesses the head of the receive FIFO. If the receive FIFO is full and this register is not read before the next data character arrives, then the data already in the FIFO will be preserved but any incoming data will be lost and an over-run error occurs. Note: ■ When UART_9BIT_DATA_EN=0, this field width is 8. ■ When UART_9BIT_DATA_EN=1, this field width is 9. Value After Reset: 0x0 Exists: Always Range Variable[x]: "(UART_9BIT_DATA_EN==1) ? \"9\" : \"8\" - 1

5.1.2 THR

- **Name:** Transmit Holding Register
- **Description:** Transmit Holding Register.
This register can be accessed only when the DLAB bit (LCR[7]) is cleared.
- **Size:** 32 bits
- **Offset:** 0x0
- **Exists:** Always

x:y	x:0
RSVD_THR	THR

Table 5-7 Fields for Register: THR

Bits	Name	Memory Access	Description
x:y	RSVD_THR	R	THR 31to9or8 Reserved bits and read as zero (0). Value After Reset: 0x0 Exists: Always Range Variable[x]: "(UART_9BIT_DATA_EN==1) ? \"23\" : \"24\"" + "(UART_9BIT_DATA_EN==1) ? \"0x9\" : \"0x8\"" - 1 Range Variable[y]: "(UART_9BIT_DATA_EN==1) ? \"0x9\" : \"0x8\""

Table 5-7 Fields for Register: THR (Continued)

Bits	Name	Memory Access	Description
x:0	THR	W	Transmit Holding Register. This register contains data to be transmitted on the serial output port (sout) in UART mode or the serial infrared output (sir_out_n) in infrared mode. Data should only be written to the THR when the THR Empty (THRE) bit (LSR[5]) is set. If in non-FIFO mode or FIFO's are disabled (FCR[0] set to zero) and THRE is set, writing a single character to the THR clears the THRE. Any additional writes to the THR before the THRE is set again causes the THR data to be overwritten. If in FIFO mode and FIFO's are enabled (FCR[0] set to one) and THRE is set, x number of characters of data may be written to the THR before the FIFO is full. The number x (default=16) is determined by the value of FIFO Depth that is set during configuration. Any attempt to write data when the FIFO is full results in the write data being lost. Note: 1. When UART_9BIT_DATA_EN=0, this field width is 8. 2. When UART_9BIT_DATA_EN=1, this field width is 9. The 9th bit is applicable only when LCR_EXT[3]=1. Value After Reset: 0x0 Exists: Always Range Variable[x]: "(UART_9BIT_DATA_EN==1) ? \"9\" : \"8\" - 1

5.1.3 DLL

- **Name:** Divisor Latch (Low)
- **Description:** Divisor Latch (Low).

If UART_16550_COMPATIBLE = No, then this register can be accessed only when the DLAB bit (LCR[7]) is set and the UART is not busy - that is, USR[0] is 0; otherwise this register can be accessed only when the DLAB bit (LCR[7]) is set.

- **Size:** 32 bits
- **Offset:** 0x0
- **Exists:** Always

31:8	RSVD_DLL_31to8
7:0	DLL

Table 5-8 Fields for Register: DLL

Bits	Name	Memory Access	Description
31:8	RSVD_DLL_31to8	R	DLL 31to8 Reserved bits and read as zero (0). Value After Reset: 0x0 Exists: Always

Table 5-8 Fields for Register: DLL (Continued)

Bits	Name	Memory Access	Description
7:0	DLL	R/W	Divisor Latch (Low). This register makes up the lower 8-bits of a 16-bit, read/write, Divisor Latch register that contains the baud rate divisor for the UART. The output baud rate is equal to the serial clock (pclk if one clock design, sclk if two clock design (CLOCK_MODE == Enabled)) frequency divided by sixteen times the value of the baud rate divisor, as follows: baud rate = (serial clock freq) / (16 * divisor). Note that with the Divisor Latch Registers (DLL and DLH) set to zero, the baud clock is disabled and no serial communications will occur. Also, once the DLL is set, at least 8 clock cycles of the slowest DW_apb_uart clock should be allowed to pass before transmitting or receiving data. Value After Reset: 0x0 Exists: Always

5.1.4 IER

- **Name:** Interrupt Enable Register
- **Description:** Interrupt Enable Register.

This register can be accessed only when the DLAB bit (LCR[7]) is cleared.

- **Size:** 32 bits
- **Offset:** 0x4
- **Exists:** Always

31:8	RSVD_IER_31to8
7	PTIME
6:5	RSVD_IER_6to5
4	ELCOLR
3	EDSSI
2	ELSI
1	ETBEI
0	ERBFI

Table 5-9 Fields for Register: IER

Bits	Name	Memory Access	Description
31:8	RSVD_IER_31to8	R	IER 31to8 Reserved bits and read as zero (0). Value After Reset: 0x0 Exists: Always
7	PTIME	* Varies	Programmable THRE Interrupt Mode Enable. Writeable only when THRE_MODE_USER == Enabled, always readable. This is used to enable/disable the generation of THRE Interrupt. Values: ■ 0x0 (DISABLED): Disable Programmable THRE Interrupt Mode ■ 0x1 (ENABLED): Enable Programmable THRE Interrupt Mode Value After Reset: 0x0 Exists: Always Memory Access: "(THRE_MODE_USER==1 && FIFO_MODE!=0) ? \"read-write\" : \"read-only\""

Table 5-9 Fields for Register: IER (Continued)

Bits	Name	Memory Access	Description
6:5	RSVD_IER_6to5	R	IER 6to5 Reserved bits read as zero (0). Value After Reset: 0x0 Exists: Always
4	ELCOLR	* Varies	Interrupt Enable Register: ELCOLR, this bit controls the method for clearing the status in the LSR register. This is applicable only for Overrun Error, Parity Error, Framing Error, and Break Interrupt status bits. 0: LSR status bits are cleared either on reading Rx FIFO (RBR Read) or On reading LSR register. 1: LSR status bits are cleared only on reading LSR register. Writeable only when LSR_STATUS_CLEAR = 1, otherwise always readable. Values: 0x0 (DISABLED): Disable ALC 0x1 (ENABLED): Enable ALC Value After Reset: 0x0 Exists: Always Memory Access: "(LSR_STATUS_CLEAR==1) ? \"read-write\" : \"read-only\""
3	EDSSI	R/W	Enable Modem Status Interrupt. This is used to enable/disable the generation of Modem Status Interrupt. This is the fourth highest priority interrupt. Values: 0x0 (DISABLED): Disable Modem Status Interrupt 0x1 (ENABLED): Enable Modem Status Interrupt Value After Reset: 0x0 Exists: Always
2	ELSI	R/W	Enable Receiver Line Status Interrupt. This is used to enable/disable the generation of Receiver Line Status Interrupt. This is the highest priority interrupt. Values: 0x0 (DISABLED): Disable Receiver Line Status Interrupt 0x1 (ENABLED): Enable Receiver Line Status Interrupt Value After Reset: 0x0 Exists: Always

Table 5-9 Fields for Register: IER (Continued)

Bits	Name	Memory Access	Description
1	ETBEI	R/W	Enable Transmit Holding Register Empty Interrupt. This is used to enable/disable the generation of Transmitter Holding Register Empty Interrupt. This is the third highest priority interrupt. Values: ■ 0x0 (DISABLED): Disable Transmit empty interrupt ■ 0x1 (ENABLED): Enable Transmit empty interrupt Value After Reset: 0x0 Exists: Always
0	ERBFI	R/W	Enable Received Data Available Interrupt. This is used to enable/disable the generation of Received Data Available Interrupt and the Character Timeout Interrupt (if in FIFO mode and FIFO's enabled). These are the second highest priority interrupts. Values: ■ 0x0 (DISABLED): Disable Receive data Interrupt ■ 0x1 (ENABLED): Enable Receive data Interrupt Value After Reset: 0x0 Exists: Always

5.1.5 DLH

- **Name:** Divisor Latch High
- **Description:** Divisor Latch High (DLH) Register.

If UART_16550_COMPATIBLE = No, then this register can be accessed only when the DLAB bit (LCR[7]) is set and the UART is not busy, that is, USR[0] is 0; otherwise this register can be accessed only when the DLAB bit (LCR[7]) is set.

- **Size:** 32 bits
- **Offset:** 0x4
- **Exists:** Always

31:8	7:0
RSVD_DLH	dlh

Table 5-10 Fields for Register: DLH

Bits	Name	Memory Access	Description
31:8	RSVD_DLH	R	DLH 31to8 Reserved bits and read as zero (0). Value After Reset: 0x0 Exists: Always
7:0	dlh	R/W	Upper 8-bits of a 16-bit, read/write, Divisor Latch register that contains the baud rate divisor for the UART. The output baud rate is equal to the serial clock (pclk if one clock design, sclk if two clock design (CLOCK_MODE == Enabled)) frequency divided by sixteen times the value of the baud rate divisor, as follows: baud rate = (serial clock freq) / (16 * divisor). Note that with the Divisor Latch Registers (DLL and DLH) set to zero, the baud clock is disabled and no serial communications will occur. Also, once the DLH is set, at least 8 clock cycles of the slowest DW_apb_uart clock should be allowed to pass before transmitting or receiving data. Value After Reset: 0x0 Exists: Always

5.1.6 IIR

- **Name:** Interrupt Identification Register
- **Description:** Interrupt Identification Register
- **Size:** 32 bits
- **Offset:** 0x8
- **Exists:** Always

31:8	7:6	5:4	3:0
RSVD_IIR_31to8	FIFOSE	RSVD_IIR_5to4	IID

Table 5-11 Fields for Register: IIR

Bits	Name	Memory Access	Description
31:8	RSVD_IIR_31to8	R	IIR 31to8 Reserved bits and read as 0. Value After Reset: 0x0 Exists: Always
7:6	FIFOSE	R	FIFOs Enabled. This is used to indicate whether the FIFOs are enabled or disabled. Values: ■ 0x0 (DISABLED): FIFOs are disabled ■ 0x3 (ENABLED): FIFOs are enabled Value After Reset: 0x0 Exists: Always
5:4	RSVD_IIR_5to4	R	IIR 5to4 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-11 Fields for Register: IIR (Continued)

Bits	Name	Memory Access	Description
3:0	IID	R	Interrupt ID (or IID). This indicates the highest priority pending interrupt which can be one of the following types specified in Values. For information on several levels into which the interrupt priorities are split into, see the 'Interrupts' section in the DW_apb_uart Databook. Note: An interrupt of type 0111 (busy detect) will never get indicated if UART_16550_COMPATIBLE == YES in coreConsultant. Bit 3 indicates an interrupt can only occur when the FIFOs are enabled and used to distinguish a Character Timeout condition interrupt. Values: ■ 0x0 (MODEM_STATUS): modem status ■ 0x1 (NO_INTERRUPT_PENDING): no interrupt pending ■ 0x2 (THR_EMPTY): THR empty ■ 0x4 (RECEIVED_DATA_AVAILABLE): received data available ■ 0x6 (RECEIVER_LINE_STATUS): receiver line status ■ 0x7 (BUSY_DETECT): busy detect ■ 0xc (CHARACTER_TIMEOUT): character timeout Value After Reset: 0x1 Exists: Always

5.1.7 FCR

- **Name:** FIFO Control Register
- **Description:** This register is only valid when the DW_apb_uart is configured to have FIFO's implemented (FIFO_MODE != NONE). If FIFO's are not implemented, this register does not exist and writing to this register address will have no effect.
- **Size:** 32 bits
- **Offset:** 0x8
- **Exists:** FIFO_MODE != 0

31:8	7:6	5:4	3	2	1	0
RSVD_FCR_31to8	RT	TET	DMAM	XFIFOR	RFIFOR	FIFOE

Table 5-12 Fields for Register: FCR

Bits	Name	Memory Access	Description
31:8	RSVD_FCR_31to8	R	FCR 31to8 Reserved bits and read as 0. Value After Reset: 0x0 Exists: Always
7:6	RT	W	RCVR Trigger (or RT). This is used to select the trigger level in the receiver FIFO at which the Received Data Available Interrupt will be generated. In auto flow control mode, it is used to determine when the rts_n signal will be de-asserted only when RTC_FCT is disabled. It also determines when the dma_rx_req_n signal will be asserted when in certain modes of operation. For details on DMA support, refer to 'DMA Support' section of data book. Values: ■ 0x0 (FIFO_CHAR_1): 1 character in FIFO ■ 0x1 (FIFO_QUARTER_FULL): FIFO 1/4 full ■ 0x2 (FIFO_HALF_FULL): FIFO 1/2 full ■ 0x3 (FIFO_FULL_2): FIFO 2 less than full Value After Reset: 0x0 Exists: Always

Table 5-12 Fields for Register: FCR (Continued)

Bits	Name	Memory Access	Description
5:4	TET	* Varies	TX Empty Trigger (or TET). Writes will have no effect when THRE_MODE_USER == Disabled. This is used to select the empty threshold level at which the THRE Interrupts will be generated when the mode is active. It also determines when the dma_tx_req_n signal will be asserted when in certain modes of operation. For details on DMA support, refer to 'DMA Support' section of data book. Values: 0x0 (FIFO_EMPTY): FIFO Empty 0x1 (FIFO_CHAR_2): 2 characters in FIFO 0x2 (FIFO_QUARTER_FULL): FIFO 1/4 full 0x3 (FIFO_HALF_FULL): FIFO 1/2 full Value After Reset: 0x0 Exists: Always Memory Access: "(THRE_MODE_USER==1) ? \"write-only\" : \"read-only\""
3	DMAM	W	DMA Mode (or DMAM). This determines the DMA signalling mode used for the dma_tx_req_n and dma_rx_req_n output signals when additional DMA handshaking signals are not selected (DMA_EXTRA == NO). For details on DMA support, refer to 'DMA Support' section of data book. Values: 0x0 (MODE0): Mode 0 0x1 (MODE1): Mode 1 Value After Reset: 0x0 Exists: Always
2	XFIFOR	W	XMIT FIFO Reset (or XFIFOR). This resets the control portion of the transmit FIFO and treats the FIFO as empty. This will also de-assert the DMA TX request and single signals when additional DMA handshaking signals are selected (DMA_EXTRA == YES). Note that this bit is 'self-clearing' and it is not necessary to clear this bit. Values: 0x1 (RESET): Transmit FIFO reset Value After Reset: 0x0 Exists: Always

Table 5-12 Fields for Register: FCR (Continued)

Bits	Name	Memory Access	Description
1	RFIFOR	W	RCVR FIFO Reset (or RFIFOR). This resets the control portion of the receive FIFO and treats the FIFO as empty. This will also de-assert the DMA RX request and single signals when additional DMA handshaking signals are selected (DMA_EXTRA == YES). Note that this bit is 'self-clearing' and it is not necessary to clear this bit. Values: 0x1 (RESET): Receive FIFO reset Value After Reset: 0x0 Exists: Always
0	FIFOE	W	FIFO Enable (or FIFOE). This enables/disables the transmit (XMIT) and receive (RCVR) FIFOs. Whenever the value of this bit is changed both the XMIT and RCVR controller portion of FIFOs is reset. Values: 0x0 (DISABLED): FIFO disabled 0x1 (ENABLED): FIFO enabled Value After Reset: 0x0 Exists: Always

5.1.8 LCR

- **Name:** Line Control Register
- **Description:** Line Control Register
- **Size:** 32 bits
- **Offset:** 0xc
- **Exists:** Always

31:8	7	6	5	4	3	2	1:0
RSVD_LCR_31to8	DLAB	BC	SP	EPS	PEN	STOP	DLS

Table 5-13 Fields for Register: LCR

Bits	Name	Memory Access	Description
31:8	RSVD_LCR_31to8	R	LCR 31to8 Reserved bits and read as 0. Value After Reset: 0x0 Exists: Always
7	DLAB	R/W	Divisor Latch Access Bit. If UART_16550_COMPATIBLE == NO then, writeable only when UART is not busy (USR[0] is zero), otherwise always writable and always readable. This bit is used to enable reading and writing of the Divisor Latch register (DLL and DLH/LPDLL and LPDLH) to set the baud rate of the UART. This bit must be cleared after initial baud rate setup in order to access other registers. Values: ■ 0x0 (DISABLED): Divisor Latch register is writable only when UART Not BUSY ■ 0x1 (ENABLED): Divisor Latch register is always readable and writable Value After Reset: 0x0 Exists: Always

Table 5-13 Fields for Register: LCR (Continued)

Bits	Name	Memory Access	Description
6	BC	R/W	Break Control Bit. This is used to cause a break condition to be transmitted to the receiving device. If set to one the serial output is forced to the spacing (logic 0) state. When not in Loopback Mode, as determined by MCR[4], the sout line is forced low until the Break bit is cleared. If SIR_MODE == Enabled and active (MCR[6] set to one) the sir_out_n line is continuously pulsed. When in Loopback Mode, the break condition is internally looped back to the receiver and the sir_out_n line is forced low. Values: 0x0 (DISABLED): Serial output is released for data transmission 0x1 (ENABLED): Serial output is forced to spacing state Value After Reset: 0x0 Exists: Always
5	SP	R/W	Stick Parity. If UART_16550_COMPATIBLE = NO, then writable only when UART is not busy (USR[0] is 0); otherwise always writable and always readable. This bit is used to force parity value. When PEN, EPS and Stick Parity are set to 1, the parity bit is transmitted and checked as logic 0. If PEN and Stick Parity are set to 1 and EPS is a logic 0, then parity bit is transmitted and checked as a logic 1. If this bit is set to 0, Stick Parity is disabled. Values: 0x0 (DISABLED): Stick parity disabled 0x1 (ENABLED): Stick parity enabled Value After Reset: 0x0 Exists: Always

Table 5-13 Fields for Register: LCR (Continued)

Bits	Name	Memory Access	Description
4	EPS	R/W	Even Parity Select. If UART_16550_COMPATIBLE == NO then, writeable only when UART is not busy (USR[0] is zero), otherwise always writable and always readable. This is used to select between even and odd parity, when parity is enabled (PEN set to one). If set to one, an even number of logic '1's is transmitted or checked. If set to zero, an odd number of logic '1's is transmitted or checked. Values: ■ 0x0 (ODD_PARITY): an odd parity is transmitted or checked ■ 0x1 (EVEN_PARITY): an even parity is transmitted or checked Value After Reset: 0x0 Exists: Always
3	PEN	R/W	Parity Enable. If UART_16550_COMPATIBLE == NO then, writeable only when UART is not busy (USR[0] is zero), otherwise always writable and always readable. This bit is used to enable and disable parity generation and detection in transmitted and received serial character respectively. Values: ■ 0x0 (DISABLED): disable parity ■ 0x1 (ENABLED): enable parity Value After Reset: 0x0 Exists: Always

Table 5-13 Fields for Register: LCR (Continued)

Bits	Name	Memory Access	Description
2	STOP	R/W	Number of stop bits. If UART_16550_COMPATIBLE == NO then, writeable only when UART is not busy (USR[0] is zero), otherwise always writable and always readable. This is used to select the number of stop bits per character that the peripheral will transmit and receive. If set to zero, one stop bit is transmitted in the serial data. If set to one and the data bits are set to 5 (LCR[1:0] set to zero) one and a half stop bits is transmitted. Otherwise, two stop bits are transmitted. Note that regardless of the number of stop bits selected the receiver will only check the first stop bit. Note: The STOP bit duration implemented by DW_apb_uart may appear longer due to idle time inserted between characters for some configurations and baud clock divisor values in the transmit direction; for details on idle time between transmitted transfers, refer to 'Back-to-Back Character Stream Transmission' section in data book. Values: ■ 0x0 (STOP_1BIT): 1 stop bit ■ 0x1 (STOP_1_5BIT_OR_2BIT): 1.5 stop bits when DLS (LCR[1:0]) is zero, else 2 stop bit Value After Reset: 0x0 Exists: Always
1:0	DLS	R/W	Data Length Select (or CLS as used in legacy). If UART_16550_COMPATIBLE == NO then, writeable only when UART is not busy (USR[0] is zero), otherwise always writable and always readable. When DLS_E in LCR_EXT is set to 0, this register is used to select the number of data bits per character that the peripheral will transmit and receive. Values: ■ 0x0 (CHAR_5BITS): 5 data bits per character ■ 0x1 (CHAR_6BITS): 6 data bits per character ■ 0x2 (CHAR_7BITS): 7 data bits per character ■ 0x3 (CHAR_8BITS): 8 data bits per character Value After Reset: 0x0 Exists: Always

5.1.9 MCR

- **Name:** Modem Control Register
- **Description:** Modem Control Register
- **Size:** 32 bits
- **Offset:** 0x10
- **Exists:** Always

31:7	6	5	4	3	2	1	0
RSVD_MCR_31to7	SIRE	AFCE	LoopBack	OUT2	OUT1	RTS	DTR

Table 5-14 Fields for Register: MCR

Bits	Name	Memory Access	Description
31:7	RSVD_MCR_31to7	R	MCR 31to7 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
6	SIRE	* Varies	SIR Mode Enable Writeable only when SIR_MODE == Enabled, always readable. This is used to enable/ disable the IrDA SIR Mode features as described in section 'IrDA 1.0 SIR Protocol' in the databook. Note: To enable SIR mode, write the appropriate value to the MCR register before writing to the LCR register. For details of the recommended programming sequence, refer to 'Programing Examples' section of data book. Values: ■ 0x0 (DISABLED): IrDA SIR Mode disabled ■ 0x1 (ENABLED): IrDA SIR Mode enabled Value After Reset: 0x0 Exists: Always Memory Access: "(SIR_MODE==1) ? \"read-write\" : \"read-only\""

Table 5-14 Fields for Register: MCR (Continued)

Bits	Name	Memory Access	Description
5	AFCE	* Varies	Auto Flow Control Enable Writeable only when AFCE_MODE == Enabled, always readable. When FIFOs are enabled and the Auto Flow Control Enable (AFCE) bit is set, Auto Flow Control features are enabled as described in section 'Auto Flow Control' in data book. Values: 0x0 (DISABLED): Auto Flow Control Mode disabled 0x1 (ENABLED): Auto Flow Control Mode enabled Value After Reset: 0x0 Exists: Always Memory Access: "(AFCE_MODE==1) ? \"read-write\" : \"read-only\""
4	LoopBack	R/W	LoopBack Bit This is used to put the UART into a diagnostic mode for test purposes. If operating in UART mode (SIR_MODE != Enabled OR NOT active, MCR[6] set to zero), data on the sout line is held high, while serial data output is looped back to the sin line, internally. In this mode all the interrupts are fully functional. Also, in loopback mode, the modem control inputs (dsr_n, cts_n, ri_n, dcd_n) are disconnected and the modem control outputs (dtr_n, rts_n, out1_n, out2_n) are looped back to the inputs, internally. If operating in infrared mode (SIR_MODE == Enabled AND active, MCR[6] set to one), data on the sir_out_n line is held low, while serial data output is inverted and looped back to the sir_in line. Values: 0x0 (DISABLED): Loopback mode disabled 0x1 (ENABLED): Loopback mode enabled Value After Reset: 0x0 Exists: Always

Table 5-14 Fields for Register: MCR (Continued)

Bits	Name	Memory Access	Description
3	OUT2	R/W	OUT2 This is used to directly control the user-designated Output2 (out2_n) output. The value written to this location is inverted and driven out on out2_n. Note that in Loopback mode (MCR[4] set to one), the out2_n output is held inactive high while the value of this location is internally looped back to an input. Values: 0x0 (OUT2_0): out2_n de-asserted (logic 1) 0x1 (OUT2_1): out2_n asserted (logic 0) Value After Reset: 0x0 Exists: Always
2	OUT1	R/W	OUT1 This is used to directly control the user-designated Output1 (out1_n) output. The value written to this location is inverted and driven out on out1_n. Note that in Loopback mode (MCR[4] set to one), the out1_n output is held inactive high while the value of this location is internally looped back to an input. Values: 0x0 (OUT1_0): out1_n de-asserted (logic 1) 0x1 (OUT1_1): out1_n asserted (logic 0) Value After Reset: 0x0 Exists: Always

Table 5-14 Fields for Register: MCR (Continued)

Bits	Name	Memory Access	Description
1	RTS	R/W	Request to Send. This is used to directly control the Request to Send (rts_n) output. The Request To Send (rts_n) output is used to inform the modem or data set that the UART is ready to exchange data. When Auto RTS Flow Control is not enabled (MCR[5] set to zero), the rts_n signal is set low by programming MCR[1] (RTS) to a high. In Auto Flow Control, AFCE_MODE == Enabled and active (MCR[5] set to one) and FIFO's enable (FCR[0] set to one), the rts_n output is controlled in the same way, but is also gated with the receiver FIFO threshold trigger (rts_n is inactive high when above the threshold). The rts_n signal will be de-asserted when MCR[1] is set low. Note that in Loopback mode (MCR[4] set to one), the rts_n output is held inactive high while the value of this location is internally looped back to an input. Values: 0x0 (INACTIVE): Request to Send rts_n de-asserted (logic 1) 0x1 (ACTIVE): Request to Send rts_n asserted (logic 0) Value After Reset: 0x0 Exists: Always
0	DTR	R/W	Data Terminal Ready. This is used to directly control the Data Terminal Ready (dtr_n) output. The value written to this location is inverted and driven out on dtr_n. The Data Terminal Ready output is used to inform the modem or data set that the UART is ready to establish communications. Note that in Loopback mode (MCR[4] set to one), the dtr_n output is held inactive high while the value of this location is internally looped back to an input. Values: 0x0 (INACTIVE): dtr_n de-asserted (logic1) 0x1 (ACTIVE): dtr_n asserted (logic 0) Value After Reset: 0x0 Exists: Always

5.1.10 LSR

- **Name:** Line Status Register
- **Description:** Line Status Register
- **Size:** 32 bits
- **Offset:** 0x14
- **Exists:** Always

31:9	RSVD_LSR_31to9
8	ADDR_RCVD
7	RFE
6	TEMT
5	THRE
4	BI
3	FE
2	PE
1	OE
0	DR

Table 5-15 Fields for Register: LSR

Bits	Name	Memory Access	Description
31:9	RSVD_LSR_31to9	R	LSR 31to9 Reserved bits read as zero. Value After Reset: 0x0 Exists: Always

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
8	ADDR_RCVD	R	Address Received Bit. If 9Bit data mode (LCR_EXT[0]=1) is enabled, this bit is used to indicate the 9th bit of the receive data is set to 1. This bit can also be used to indicate whether the incoming character is address or data. ■ 1 = Indicates the character is address. ■ 0 = Indicates the character is data. In the FIFO mode, since the 9th bit is associated with a character received, it is revealed when the character with the 9th bit set to 1 is at the top of the FIFO. Reading the LSR clears the 9BIT. Note: User needs to ensure that interrupt gets cleared (reading LSR register) before the next address byte arrives. If there is a delay in clearing the interrupt, then Software will not be able to distinguish between multiple address related interrupt. Values: ■ 0x1 (RCVD_1): Incoming character is address ■ 0x0 (RCVD_0): Incoming character is data Value After Reset: 0x0 Exists: UART_9BIT_DATA_EN == 1
7	RFE	R	Receiver FIFO Error bit. This bit is only relevant when FIFO_MODE != NONE AND FIFO's are enabled (FCR[0] set to one). This is used to indicate if there is at least one parity error, framing error, or break indication in the FIFO. This bit is cleared when the LSR is read and the character with the error is at the top of the receiver FIFO and there are no subsequent errors in the FIFO. Values: ■ 0x0 (NO_RX_FIFO_ERROR): No error in RX FIFO ■ 0x1 (RX_FIFO_ERROR): Error in RX FIFO Value After Reset: 0x0 Exists: FIFO_MODE != 0

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
6	TEMT	R	Transmitter Empty bit. If in FIFO mode (FIFO_MODE != NONE) and FIFO's enabled (FCR[0] set to one), this bit is set whenever the Transmitter Shift Register and the FIFO are both empty. If in the non-FIFO mode or FIFO's are disabled, this bit is set whenever the Transmitter Holding Register and the Transmitter Shift Register are both empty. Values: 0x0 (DISABLED): Transmitter not empty 0x1 (ENABLED): Transmitter empty Value After Reset: 0x1 Exists: Always
5	THRE	R	Transmit Holding Register Empty bit. If THRE_MODE_USER = Disabled or THRE mode is disabled (IER[7] set to zero) and regardless of FIFO's being implemented/enabled or not, this bit indicates that the THR or TX FIFO is empty. This bit is set whenever data is transferred from the THR or TX FIFO to the transmitter shift register and no new data has been written to the THR or TX FIFO. This also causes a THRE Interrupt to occur, if the THRE Interrupt is enabled. If THRE_MODE_USER == Enabled AND FIFO_MODE != NONE and both modes are active (IER[7] set to one and FCR[0] set to one respectively), the functionality is switched to indicate the transmitter FIFO is full, and no longer controls THRE interrupts, which are then controlled by the FCR[5:4] threshold setting. Programmable THRE interrupt mode operation is described in detail in section 'Programmable THRE Interrupt' in data book. Values: 0x0 (DISABLED): THRE interrupt control is disabled 0x1 (ENABLED): THRE interrupt control is enabled Value After Reset: 0x1 Exists: Always

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
4	BI	R	Break Interrupt bit. This is used to indicate the detection of a break sequence on the serial input data. If in UART mode it is set whenever the serial input, <i>sin</i>, is held in a logic '0' state for longer than the sum of start time + data bits + parity + stop bits. If in infrared mode it is set whenever the serial input, <i>sir_in</i>, is continuously pulsed to logic '0' for longer than the sum of start time + data bits + parity + stop bits. A break condition on serial input causes one and only one character, consisting of all zeros, to be received by the UART. In the FIFO mode, the character associated with the break condition is carried through the FIFO and is revealed when the character is at the top of the FIFO. If configuration parameter <code>LSR_STATUS_CLEAR = 0</code>: - Reading the LSR or RBR register clears the BI bit. If configuration parameter <code>LSR_STATUS_CLEAR = 1</code>: - If register field <code>IER.ELCOLR = 0</code>: Reading the LSR or RBR register clears the BI bit. - If register field <code>IER.ELCOLR = 1</code>: Reading the LSR register clears the BI bit. In the non-FIFO mode, the BI indication occurs immediately and persists until the LSR is read. Note: If a FIFO is full when a break condition is received, a FIFO overrun occurs. The break condition and all the information associated with it-parity and framing errors-is discarded; any information that a break character was received is lost. Values: 0x0 (NO_BREAK): No break sequence detected 0x1 (BREAK): Break sequence detected Value After Reset: 0x0 Exists: Always

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
3	FE	R	Framing Error bit. This is used to indicate the occurrence of a framing error in the receiver. A framing error occurs when the receiver does not detect a valid STOP bit in the received data. In the FIFO mode, since the framing error is associated with a character received, it is revealed when the character with the framing error is at the top of the FIFO. When a framing error occurs the UART will try resynchronize. It does this by assuming that the error was due to the start bit of the next character and then continues receiving the other bit i.e. data, and/or parity and stop. It should be noted that the Framing Error (FE) bit (LSR[3]) will be set if a break interrupt has occurred, as indicated by Break Interrupt (BI) bit (LSR[4]). This happens because the break character implicitly generates a framing error by holding the sin input to logic 0 for longer than the duration of a character. If configuration parameter LSR_STATUS_CLEAR = 0: ■ Reading the LSR or RBR register clears the FE bit. If configuration parameter LSR_STATUS_CLEAR = 1: ■ If register field IER.ELCOLR = 0: Reading the LSR or RBR register clears the FE bit. ■ If register field IER.ELCOLR = 1: Reading the LSR register clears the FE bit. Values: ■ 0x0 (NO_FRAMING_ERROR): no framing error ■ 0x1 (FRAMING_ERROR): framing error Value After Reset: 0x0 Exists: Always

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
2	PE	R	Parity Error bit. This is used to indicate the occurrence of a parity error in the receiver if the Parity Enable (PEN) bit (LCR[3]) is set. In the FIFO mode, since the parity error is associated with a character received, it is revealed when the character with the parity error arrives at the top of the FIFO. It should be noted that the Parity Error (PE) bit (LSR[2]) will be set if a break interrupt has occurred, as indicated by Break Interrupt (BI) bit (LSR[4]). In this situation, the Parity Error bit is set if parity generation and detection is enabled (LCR[3]=1) and the parity is set to odd (LCR[4]=0). If configuration parameter LSR_STATUS_CLEAR = 0: ■ Reading the LSR or RBR register clears the PE bit. If configuration parameter LSR_STATUS_CLEAR = 1: ■ If register field IER.ELCOLR = 0: Reading the LSR or RBR register clears the PE bit. ■ If register field IER.ELCOLR = 1: Reading the LSR register clears the PE bit. Values: ■ 0x0 (NO_PARITY_ERROR): no parity error ■ 0x1 (PARITY_ERROR): parity error Value After Reset: 0x0 Exists: Always

Table 5-15 Fields for Register: LSR (Continued)

Bits	Name	Memory Access	Description
1	OE	R	Overrun error bit. This is used to indicate the occurrence of an overrun error. This occurs if a new data character was received before the previous data was read. In the non-FIFO mode, the OE bit is set when a new character arrives in the receiver before the previous character was read from the RBR. When this happens, the data in the RBR is overwritten. In the FIFO mode, an overrun error occurs when the FIFO is full and a new character arrives at the receiver. The data in the FIFO is retained and the data in the receive shift register is lost. If configuration parameter LSR_STATUS_CLEAR = 0: ■ Reading the LSR register clears the OE bit. ■ Note: In this case the OE bit is only cleared on LSR register read to avoid loss of OE interrupt status information by DMA operation (RBR read). However, with respect to PE, FE and BI interrupt status bits, these information are available with the received data in RX FIFO. the read of RBR register. If configuration parameter LSR_STATUS_CLEAR = 1: ■ If register field IER.ELCOLR = 0: Reading the LSR or RBR register clears the OE bit. ■ If register field IER.ELCOLR = 1: Reading the LSR register clears the OE bit. Values: ■ 0x0 (NO_OVER_RUN_ERROR): no overrun error ■ 0x1 (OVER_RUN_ERROR): overrun error Value After Reset: 0x0 Exists: Always
0	DR	R	Data Ready bit. This is used to indicate that the receiver contains at least one character in the RBR or the receiver FIFO. This bit is cleared when the RBR is read in the non-FIFO mode, or when the receiver FIFO is empty, in the FIFO mode. Values: ■ 0x0 (NOT_READY): data not ready ■ 0x1 (READY): data ready Value After Reset: 0x0 Exists: Always

5.1.11 MSR

- **Name:** Modem Status Register
- **Description:** Whenever bits 0, 1, 2 or 3 is set to logic one, to indicate a change on the modem control inputs, a modem status interrupt will be generated if enabled via the IER regardless of when the change occurred. The bits (bits 0, 1, 3) can be set after a reset-even though their respective modem signals are inactive-because the synchronized version of the modem signals have a reset value of 0 and change to value 1 after reset. To prevent unwanted interrupts due to this change, a read of the MSR register can be performed after reset.
- **Size:** 32 bits
- **Offset:** 0x18
- **Exists:** Always

31:8	7	6	5	4	3	2	1	0
RSVD_MSR_31to8	DCD	RI	DSR	CTS	DDCD	TERI	DDSR	DCTS

Table 5-16 Fields for Register: MSR

Bits	Name	Memory Access	Description
31:8	RSVD_MSR_31to8	R	MSR 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-16 Fields for Register: MSR (Continued)

Bits	Name	Memory Access	Description
7	DCD	R	Data Carrier Detect. This is used to indicate the current state of the modem control line dcd_n. That is this bit is the complement dcd_n. When the Data Carrier Detect input (dcd_n) is asserted it is an indication that the carrier has been detected by the modem or data set. In Loopback Mode (MCR[4] set to one), DCD is the same as MCR[3] (Out2). Values: 0x0 (DEASSERTED): dcd_n input is de-asserted (logic 1) 0x1 (ASSERTED): dcd_n input is asserted (logic 0) Value After Reset: 0x0 Exists: Always
6	RI	R	Ring Indicator. This is used to indicate the current state of the modem control line ri_n. That is this bit is the complement ri_n. When the Ring Indicator input (ri_n) is asserted it is an indication that a telephone ringing signal has been received by the modem or data set. In Loopback Mode (MCR[4] set to one), RI is the same as MCR[2] (Out1). Values: 0x0 (DEASSERTED): ri_n input is de-asserted (logic 1) 0x1 (ASSERTED): ri_n input is asserted (logic 0) Value After Reset: 0x0 Exists: Always
5	DSR	R	Data Set Ready. This is used to indicate the current state of the modem control line dsr_n. That is this bit is the complement dsr_n. When the Data Set Ready input (dsr_n) is asserted it is an indication that the modem or data set is ready to establish communications with the DW_apb_uart. In Loopback Mode (MCR[4] set to one), DSR is the same as MCR[0] (DTR). Values: 0x0 (DEASSERTED): dsr_n input is de-asserted (logic 1) 0x1 (ASSERTED): dsr_n input is asserted (logic 0) Value After Reset: 0x0 Exists: Always

Table 5-16 Fields for Register: MSR (Continued)

Bits	Name	Memory Access	Description
4	CTS	R	Clear to Send. This is used to indicate the current state of the modem control line cts_n. That is, this bit is the complement cts_n. When the Clear to Send input (cts_n) is asserted it is an indication that the modem or data set is ready to exchange data with the DW_apb_uart. In Loopback Mode (MCR[4] set to one), CTS is the same as MCR[1] (RTS). Values: 0x0 (DEASSERTED): cts_n input is de-asserted (logic 1) 0x1 (ASSERTED): cts_n input is asserted (logic 0) Value After Reset: 0x0 Exists: Always
3	DDCD	R	Delta Data Carrier Detect. This is used to indicate that the modem control line dcd_n has changed since the last time the MSR was read. Reading the MSR clears the DDCD bit. In Loopback Mode (MCR[4] set to one), DDCD reflects changes on MCR[3] (Out2). Note, if the DDCD bit is not set and the dcd_n signal is asserted (low) and a reset occurs (software or otherwise), then the DDCD bit will get set when the reset is removed if the dcd_n signal remains asserted. Values: 0x0 (NO_CHANGE): No change on dcd_n since last read of MSR 0x1 (CHANGE): change on dcd_n since last read of MSR Value After Reset: 0x0 Exists: Always

Table 5-16 Fields for Register: MSR (Continued)

Bits	Name	Memory Access	Description
2	TERI	R	Trailing Edge of Ring Indicator. This is used to indicate that a change on the input ri_n (from an active low, to an inactive high state) has occurred since the last time the MSR was read. Reading the MSR clears the TERI bit. In Loopback Mode (MCR[4] set to one), TERI reflects when MCR[2] (Out1) has changed state from a high to a low. Values: 0x0 (NO_CHANGE): no change on ri_n since last read of MSR 0x1 (CHANGE): change on ri_n since last read of MSR Value After Reset: 0x0 Exists: Always
1	DDSR	R	Delta Data Set Ready. This is used to indicate that the modem control line dsr_n has changed since the last time the MSR was read. Reading the MSR clears the DDSR bit. In Loopback Mode (MCR[4] set to one), DDSR reflects changes on MCR[0] (DTR). Note, if the DDSR bit is not set and the dsr_n signal is asserted (low) and a reset occurs (software or otherwise), then the DDSR bit will get set when the reset is removed if the dsr_n signal remains asserted. Values: 0x0 (NO_CHANGE): no change on dsr_n since last read of MSR 0x1 (CHANGE): change on dsr_n since last read of MSR Value After Reset: 0x0 Exists: Always

Table 5-16 Fields for Register: MSR (Continued)

Bits	Name	Memory Access	Description
0	DCTS	R	Delta Clear to Send. This is used to indicate that the modem control line <code>cts_n</code> has changed since the last time the MSR was read. Reading the MSR clears the DCTS bit. In Loopback Mode (MCR[4] set to one), DCTS reflects changes on MCR[1] (RTS). Note, if the DCTS bit is not set and the <code>cts_n</code> signal is asserted (low) and a reset occurs (software or otherwise), then the DCTS bit will get set when the reset is removed if the <code>cts_n</code> signal remains asserted. Values: ■ 0x0 (NO_CHANGE): no change on <code>cts_n</code> since last read of MSR ■ 0x1 (CHANGE): change on <code>cts_n</code> since last read of MSR Value After Reset: 0x0 Exists: Always

5.1.12 SCR

- **Name:** Scratchpad Register
- **Description:** Scratchpad Register
- **Size:** 32 bits
- **Offset:** 0x1c
- **Exists:** Always

31:8	7:0
RSVD_SCR_31to8	SCR

Table 5-17 Fields for Register: SCR

Bits	Name	Memory Access	Description
31:8	RSVD_SCR_31to8	R	SCR 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
7:0	SCR	R/W	This register is for programmers to use as a temporary storage space. It has no defined purpose in the DW_apb_uart. Value After Reset: 0x0 Exists: Always

5.1.13 LPDLL

- **Name:** Low Power Divisor Latch Low
- **Description:** Low Power Divisor Latch Low Register.

This register is only valid when the DW_apb_uart is configured to have SIR low-power reception capabilities implemented (SIR_LP_RX = Yes). If SIR low-power reception capabilities are not implemented, this register does not exist and reading from this register address returns 0.

If UART_16550_COMPATIBLE = No, then this register can be accessed only when the DLAB bit (LCR[7]) is set and the UART is not busy, that is, USR[0] is 0; otherwise this register can be accessed only when the DLAB bit (LCR[7]) is set.

- **Size:** 32 bits
- **Offset:** 0x20
- **Exists:** (SIR_LP_RX == 1) && (SIR_MODE == 1)

31:8	RSVD_LPDLL_31to8
7:0	LPDLL

Table 5-18 Fields for Register: LPDLL

Bits	Name	Memory Access	Description
31:8	RSVD_LPDLL_31to8	R	LPDLL 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-18 Fields for Register: LPDLL (Continued)

Bits	Name	Memory Access	Description
7:0	LPDLL	R/W	This register makes up the lower 8-bits of a 16-bit, read/write, Low Power Divisor Latch register that contains the baud rate divisor for the UART which must give a baud rate of 115.2K. This is required for SIR Low Power (minimum pulse width) detection at the receiver. The output low power baud rate is equal to the serial clock (sclk) frequency divided by sixteen times the value of the baud rate divisor, as follows: $\text{Low power baud rate} = (\text{serial clock freq}) / (16 * \text{divisor})$ Therefore a divisor must be selected to give a baud rate of 115.2K. Note: When the Low Power Divisor Latch Registers (LPDLL and LPDLH) are set to zero, the low power baud clock is disabled and no low power pulse detection (or any pulse detection for that matter) will occur at the receiver. Also, once the LPDLL is set at least 8 clock cycles of the slowest DW_apb_uart clock should be allowed to pass before transmitting or receiving data. Value After Reset: 0x0 Exists: Always

5.1.14 LPDLH

- **Name:** Low Power Divisor Latch High
- **Description:** Low Power Divisor Latch High Register.

This register is valid only when the DW_apb_uart is configured to have SIR low-power reception capabilities implemented (SIR_LP_RX = Yes). If SIR low-power reception capabilities are not implemented, this register does not exist and reading from this register address returns 0.

If UART_16550_COMPATIBLE = No, then this register can be accessed only when the DLAB bit (LCR[7]) is set and the UART is not busy that is, USR[0] is 0; otherwise this register can be accessed only when the DLAB bit (LCR[7]) is set.

- **Size:** 32 bits
- **Offset:** 0x24
- **Exists:** (SIR_LP_RX == 1) && (SIR_MODE == 1)

31:8	RSVD_LPDLH_31to8
7:0	LPDLH

Table 5-19 Fields for Register: LPDLH

Bits	Name	Memory Access	Description
31:8	RSVD_LPDLH_31to8	R	LPDLH 31to8 Reserved and read as 0. Value After Reset: 0x0 Exists: Always

Table 5-19 Fields for Register: LPDLH (Continued)

Bits	Name	Memory Access	Description
7:0	LPDLH	R/W	This register makes up the upper 8-bits of a 16-bit, read/write, Low Power Divisor Latch register that contains the baud rate divisor for the UART which must give a baud rate of 115.2K. This is required for SIR Low Power (minimum pulse width) detection at the receiver. The output low power baud rate is equal to the serial clock (sclk) frequency divided by sixteen times the value of the baud rate divisor, as follows: $\text{Low power baud rate} = (\text{serial clock freq}) / (16 * \text{divisor})$ Therefore a divisor must be selected to give a baud rate of 115.2K. Note: When the Low Power Divisor Latch Registers (LPDLL and LPDLH) are set to zero, the low power baud clock is disabled and no low power pulse detection (or any pulse detection for that matter) will occur at the receiver. Also, once the LPDLH is set, at least 8 clock cycles of the slowest DW_apb_uart clock should be allowed to pass before transmitting or receiving data. Value After Reset: 0x0 Exists: Always

5.1.15 SRBRn (for n = 0; n <= 15)

- **Name:** Shadow Receive Buffer Register
- **Description:** This register is valid only when the DW_apb_uart is configured to have additional shadow registers implemented (SHADOW = YES). If shadow registers are not implemented, this register does not exist and reading from this register address returns 0.

This register can be accessed only when the DLAB bit (LCR[7]) is cleared.

- **Size:** 32 bits
- **Offset:** 0x30 + n*0x4
- **Exists:** SHADOW == 1

31:y	RSVD_SRBRn
x:0	SRBRn

Table 5-20 Fields for Register: SRBRn (for n = 0; n <= 15)

Bits	Name	Memory Access	Description
31:y	RSVD_SRBRn	R	SRBR0 31 to SRBRN_REG_SIZE Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Range Variable[y]: SRBRN_REG_SIZE

Table 5-20 Fields for Register: SRBRn (for n = 0; n <= 15) (Continued)

Bits	Name	Memory Access	Description
x:0	SRBRn	R	Shadow Receive Buffer Register n. This is a shadow register for the RBR and has been allocated sixteen 32-bit locations so as to accommodate burst accesses from the requester. This register contains the data byte received on the serial input port (sin) in UART mode or the serial infrared input (sir_in) in infrared mode. The data in this register is valid only if the Data Ready (DR) bit in the Line status Register (LSR) is set. If in non-FIFO mode (FIFO_MODE == NONE) or FIFOs are disabled (FCR[0] set to zero), the data in the RBR must be read before the next data arrives, otherwise it will be overwritten, resulting in an overrun error. If in FIFO mode (FIFO_MODE != NONE) and FIFOs are enabled (FCR[0] set to one), this register accesses the head of the receive FIFO. If the receive FIFO is full and this register is not read before the next data character arrives, then the data already in the FIFO will be preserved but any incoming data will be lost. An overrun error will also occur. Note: ■ When UART_9BIT_DATA_EN=0, this field width is 8. ■ When UART_9BIT_DATA_EN=1, this field width is 9. Value After Reset: 0x0 Exists: Always Range Variable[x]: SRBRN_REG_SIZE - 1

5.1.16 STHRn (for n = 0; n <= 15)

- **Name:** Shadow Transmit Holding Register
- **Description:** Shadow Transmit Holding Register. This register is valid only when the DW_apb_uart is configured to have additional shadow registers implemented (SHADOW = YES). If shadow registers are not implemented, this register does not exist, and reading from this register address returns 0.

This register can be accessed only when the DLAB bit (LCR[7]) is cleared.
- **Size:** 32 bits
- **Offset:** 0x30 + n*0x4
- **Exists:** SHADOW == 1

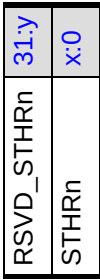


Table 5-21 Fields for Register: STHRn (for n = 0; n <= 15)

Bits	Name	Memory Access	Description
31:y	RSVD_STHRn	R	STHRn 31 to STHRN_REG_SIZE Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Range Variable[y]: STHRN_REG_SIZE

Table 5-21 Fields for Register: STHRn (for n = 0; n <= 15) (Continued)

Bits	Name	Memory Access	Description
x:0	STHRn	W	Shadow Transmit Holding Register n. This is a shadow register for the THR and has been allocated sixteen 32-bit locations so as to accommodate burst accesses from the requester. This register contains data to be transmitted on the serial output port (sout) in UART mode or the serial infrared output (sir_out_n) in infrared mode. Data should only be written to the THR when the THR Empty (THRE) bit (LSR[5]) is set. If in non-FIFO mode or FIFO's are disabled (FCR[0] set to zero) and THRE is set, writing a single character to the THR clears the THRE. Any additional writes to the THR before the THRE is set again causes the THR data to be overwritten. If in FIFO mode and FIFO's are enabled (FCR[0] set to one) and THRE is set, x number of characters of data may be written to the THR before the FIFO is full. The number x (default=16) is determined by the value of FIFO Depth that you set during configuration. Any attempt to write data when the FIFO is full results in the write data being lost. Note: ■ When UART_9BIT_DATA_EN=0, this field width is 8. ■ When UART_9BIT_DATA_EN=1, this field width is 9. The 9th bit is applicable only when LCR_EXT[3]=1. Value After Reset: 0x0 Exists: Always Range Variable[x]: STHRN_REG_SIZE - 1

5.1.17 FAR

- **Name:** FIFO Access Register
- **Description:** FIFO Access Register
- **Size:** 32 bits
- **Offset:** 0x70
- **Exists:** Always



Table 5-22 Fields for Register: FAR

Bits	Name	Memory Access	Description
31:1	RSVD_FAR_31to1	R	FAR 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-22 Fields for Register: FAR (Continued)

Bits	Name	Memory Access	Description
0	FAR	* Varies	Writes will have no effect when FIFO_ACCESS == No, always readable. This register is use to enable a FIFO access mode for testing, so that the receive FIFO can be written by the requester and the transmit FIFO can be read by the requester when FIFO's are implemented and enabled. When FIFOs are not implemented or not enabled it allows the RBR to be written by the requester and the THR to be read by the requester. Note, that when the FIFO access mode is enabled/disabled, the control portion of the receive FIFO and transmit FIFO is reset and the FIFO's are treated as empty. Values: ■ 0x0 (DISABLED): FIFO access mode disabled ■ 0x1 (ENABLED): FIFO access mode enabled Value After Reset: 0x0 Exists: Always Memory Access: "(FIFO_ACCESS==1) ? \"read-write\" : \"read-only\""

5.1.18 TFR

- **Name:** Transmit FIFO Read
- **Description:** This register is valid only when the DW_apb_uart is configured to have the FIFO access test mode available (FIFO_ACCESS = YES). If not configured, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x74
- **Exists:** FIFO_ACCESS == 1

31:8	RSVD_TFR_31to8
7:0	TFR

Table 5-23 Fields for Register: TFR

Bits	Name	Memory Access	Description
31:8	RSVD_TFR_31to8	R	TFR 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
7:0	TFR	R	Transmit FIFO Read. These bits are only valid when FIFO access mode is enabled (FAR[0] is set to one). When FIFO's are implemented and enabled, reading this register gives the data at the top of the transmit FIFO. Each consecutive read pops the transmit FIFO and gives the next data value that is currently at the top of the FIFO. When FIFO's are not implemented or not enabled, reading this register gives the data in the THR. Value After Reset: 0x0 Exists: Always

5.1.19 RFW

- **Name:** Receive FIFO Write
- **Description:** This register is valid only when the DW_apb_uart is configured to have the FIFO access test mode available (FIFO_ACCESS = YES). If not configured, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x78
- **Exists:** FIFO_ACCESS == 1

31:10	9	8	7:0
RSVD_RFW_31to10	RFFE	RFPE	RFWD

Table 5-24 Fields for Register: RFW

Bits	Name	Memory Access	Description
31:10	RSVD_RFW_31to10	R	RFW 31to10 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
9	RFFE	W	Receive FIFO Framing Error. These bits are only valid when FIFO access mode is enabled (FAR[0] is set to one). When FIFO's are implemented and enabled, this bit is used to write framing error detection information to the receive FIFO. When FIFO's are not implemented or not enabled, this bit is used to write framing error detection information to the RBR. Values: ■ 0x0 (DISABLED): Frame error disabled ■ 0x1 (ENABLED): Frame error enabled Value After Reset: 0x0 Exists: Always

Table 5-24 Fields for Register: RFW (Continued)

Bits	Name	Memory Access	Description
8	RFPE	W	Receive FIFO Parity Error. These bits are only valid when FIFO access mode is enabled (FAR[0] is set to one). When FIFO's are implemented and enabled, this bit is used to write parity error detection information to the receive FIFO. When FIFO's are not implemented or not enabled, this bit is used to write parity error detection information to the RBR. Values: ■ 0x0 (DISABLED): Parity error disabled ■ 0x1 (ENABLED): Parity error enabled Value After Reset: 0x0 Exists: Always
7:0	RFWD	W	Receive FIFO Write Data. These bits are only valid when FIFO access mode is enabled (FAR[0] is set to one). When FIFO's are implemented and enabled, the data that is written to the RFWD is pushed into the receive FIFO. Each consecutive write pushes the new data to the next write location in the receive FIFO. When FIFO's are not implemented or not enabled, the data that is written to the RFWD is pushed into the RBR. Value After Reset: 0x0 Exists: Always

5.1.20 USR

- **Name:** UART Status register
- **Description:** UART Status register.
- **Size:** 32 bits
- **Offset:** 0x7c
- **Exists:** Always

31:5	4	3	2	1	0
RSVD_USR_31to5	RFF	RFNE	TFE	TFNF	BUSY

Table 5-25 Fields for Register: USR

Bits	Name	Memory Access	Description
31:5	RSVD_USR_31to5	R	USR 31to5 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Volatile: true
4	RFF	R	Receive FIFO Full. This bit is only valid when FIFO_STAT == YES. This is used to indicate that the receive FIFO is completely full. That is: This bit is cleared when the RX FIFO is no longer full. Values: ■ 0x0 (NOT_FULL): Receive FIFO not full ■ 0x1 (FULL): Receive FIFO full Value After Reset: 0x0 Exists: (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1) Volatile: true

Table 5-25 Fields for Register: USR (Continued)

Bits	Name	Memory Access	Description
3	RFNE	R	Receive FIFO Not Empty. This bit is only valid when FIFO_STAT == YES. This is used to indicate that the receive FIFO contains one or more entries. This bit is cleared when the RX FIFO is empty. Values: 0x0 (EMPTY): Receive FIFO is empty 0x1 (NOT_EMPTY): Receive FIFO is not empty Value After Reset: 0x0 Exists: (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1) Volatile: true
2	TFE	R	Transmit FIFO Empty. This bit is only valid when FIFO_STAT == YES. This is used to indicate that the transmit FIFO is completely empty. This bit is cleared when the TX FIFO is no longer empty. Values: 0x0 (NOT_EMPTY): Transmit FIFO is not empty 0x1 (EMPTY): Transmit FIFO is empty Value After Reset: "((FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1)) ? 0x1 : 0x0" Exists: (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1) Volatile: true
1	TFNF	R	Transmit FIFO Not Full. This bit is only valid when FIFO_STAT == YES. This is used to indicate that the transmit FIFO is not full. This bit is cleared when the TX FIFO is full. Values: 0x0 (FULL): Transmit FIFO is full 0x1 (NOT_FULL): Transmit FIFO is not full Value After Reset: "((FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1)) ? 0x1 : 0x0" Exists: (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1) Volatile: true

Table 5-25 Fields for Register: USR (Continued)

Bits	Name	Memory Access	Description
0	BUSY	R	UART Busy. This bit is only valid when UART_16550_COMPATIBLE == NO. This indicates that a serial transfer is in progress, when cleared indicates that the DW_apb_uart is idle or inactive. This bit will be set to 1 (busy) under any of the following conditions: - Transmission in progress on serial interface - Transmit data present in THR, when FIFO access mode is not being used (FAR = 0) and the baud divisor is non-zero ({DLH,DLL} does not equal 0) when the divisor latch access bit is 0 (LCR.DLAB = 0) - Reception in progress on the interface - Receive data present in RBR, when FIFO access mode is not being used (FAR = 0) Note: It is possible for the UART Busy bit to be cleared even though a new character may have been sent from another device. That is, if the DW_apb_uart has no data in the THR and RBR and there is no transmission in progress and a start bit of a new character has just reached the DW_apb_uart. This is due to the fact that a valid start is not seen until the middle of the bit period and this duration is dependent on the baud divisor that has been programmed. If a second system clock has been implemented (CLOCK_MODE == Enabled), the assertion of this bit will also be delayed by several cycles of the slower clock. Values: ■ 0x0 (IDLE): DW_apb_uart is idle or inactive ■ 0x1 (BUSY): DW_apb_uart is busy (actively transferring data) Value After Reset: 0x0 Exists: UART_16550_COMPATIBLE == 0 Volatile: true

5.1.21 TFL

- **Name:** Transmit FIFO Level
- **Description:** TFL register is valid only when the DW_apb_uart is configured to have additional FIFO status registers implemented (FIFO_STAT = YES). If status registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x80
- **Exists:** (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1)

31:y	RSVD_TFL_31toADDR_WIDTH
x:0	tfl

Table 5-26 Fields for Register: TFL

Bits	Name	Memory Access	Description
31:y	RSVD_TFL_31toADDR_WIDTH	R	TFL 31 to ADDR_WIDTH Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Range Variable[y]: FIFO_ADDR_WIDTH + 1
x:0	tfl	R	Transmit FIFO Level. This indicates the number of data entries in the transmit FIFO. Value After Reset: 0x0 Exists: Always Range Variable[x]: FIFO_ADDR_WIDTH

5.1.22 RFL

- **Name:** Receive FIFO Level
- **Description:** RFL register is valid only when the DW_apb_uart is configured to have additional FIFO status registers implemented (FIFO_STAT = YES). If status registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x84
- **Exists:** (FIFO_STAT == 1) && (FIFO_MODE != 0) && (ADDITIONAL_FEATURES == 1)

31:y	RSVD_RFL_31toADDR_WIDTH
x:0	rfl

Table 5-27 Fields for Register: RFL

Bits	Name	Memory Access	Description
31:y	RSVD_RFL_31toADDR_WIDTH	R	RFL 31 to ADDR_WIDTH Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Range Variable[y]: FIFO_ADDR_WIDTH + 1
x:0	rfl	R	Receive FIFO Level. This indicates the number of data entries in the receive FIFO. Value After Reset: 0x0 Exists: Always Range Variable[x]: FIFO_ADDR_WIDTH

5.1.23 SRR

- **Name:** Software Reset Register
- **Description:** This register is valid only when the DW_apb_uart is configured to have additional shadow registers implemented (SHADOW = YES). If shadow registers are not implemented, this register does not exist and reading from this register address returns 0.

For more information on the amount of time that serial clock modules need in order to see new register values and reset their respective state machines, refer to the 'Clock Support' subsection in the data book.

- **Size:** 32 bits
- **Offset:** 0x88
- **Exists:** SHADOW == 1

31:3	2	1	0
RSVD_SRR_31to3	XFR	RFR	UR

Table 5-28 Fields for Register: SRR

Bits	Name	Memory Access	Description
31:3	RSVD_SRR_31to3	R	SRR 31to3 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-28 Fields for Register: SRR (Continued)

Bits	Name	Memory Access	Description
2	XFR	* Varies	XMIT FIFO Reset. Writes will have no effect when FIFO_MODE == NONE. This is a shadow register for the XMIT FIFO Reset bit (FCR[2]). This can be used to remove the burden on software having to store previously written FCR values (which are pretty static) just to reset the transmit FIFO. This resets the control portion of the transmit FIFO and treats the FIFO as empty. This will also de-assert the DMA TX request and single signals when additional DMA handshaking signals are selected (DMA_EXTRA = YES). Note that this bit is 'self-clearing'. It is not necessary to clear this bit. Value After Reset: 0x0 Exists: Always Memory Access: "(FIFO_MODE !=0) ? \"write-only\" : \"read-only\""
1	RFR	* Varies	RCVR FIFO Reset. Writes will have no effect when FIFO_MODE == NONE. This is a shadow register for the RCVR FIFO Reset bit (FCR[1]). This can be used to remove the burden on software having to store previously written FCR values (which are pretty static) just to reset the receive FIFO. This resets the control portion of the receive FIFO and treats the FIFO as empty. This will also de-assert the DMA RX request and single signals when additional DMA handshaking signals are selected (DMA_EXTRA == YES). Note that this bit is 'self-clearing' and it is not necessary to clear this bit. Value After Reset: 0x0 Exists: Always Memory Access: "(FIFO_MODE !=0) ? \"write-only\" : \"read-only\""
0	UR	W	UART Reset. This asynchronously resets the DW_apb_uart and synchronously removes the reset assertion. For a two clock implementation both pclk and sclk domains will be reset. Values: ■ 0x0 (NO_RESET): No Uart Reset ■ 0x1 (RESET): Uart reset Value After Reset: 0x0 Exists: Always

5.1.24 SRTS

- **Name:** Shadow Request to Send
- **Description:** SRTS register is valid only when the DW_apb_uart is configured to have additional shadow registers implemented (SHADOW = YES). If shadow registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x8c
- **Exists:** SHADOW == 1

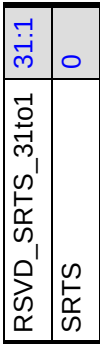


Table 5-29 Fields for Register: SRTS

Bits	Name	Memory Access	Description
31:1	RSVD_SRTS_31to1	R	SRTS 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-29 Fields for Register: SRTS (Continued)

Bits	Name	Memory Access	Description
0	SRTS	R/W	Shadow Request to Send. This is a shadow register for the RTS bit (MCR[1]), this can be used to remove the burden of having to performing a read modify write on the MCR. This is used to directly control the Request to Send (rts_n) output. The Request To Send (rts_n) output is used to inform the modem or data set that the UART is ready to exchange data. When Auto RTS Flow Control is not enabled (MCR[5] set to zero), the rts_n signal is set low by programming MCR[1] (RTS) to a high. In Auto Flow Control, AFCE_MODE == Enabled and active (MCR[5] set to one) and FIFO's enable (FCR[0] set to one), the rts_n output is controlled in the same way, but is also gated with the receiver FIFO threshold trigger (rts_n is inactive high when above the threshold) only when RTC Flow Trigger is disabled; otherwise it is gated by the receiver FIFO almost-full trigger, where 'almost full' refers to two available slots in the FIFO (rts_n is inactive high when above the threshold). Note that in Loopback mode (MCR[4] set to one), the rts_n output is held inactive high while the value of this location is internally looped back to an input. Values: ■ 0x0 (DEASSERTED): Shadow Request to Send uart_rts_n logic1 ■ 0x1 (ASSERTED): Shadow Request to Send uart_rts_n logic0 Value After Reset: 0x0 Exists: Always

5.1.25 SBCR

- **Name:** Shadow Break Control Register
- **Description:** SBCR register is valid only when the DW_apb_uart is configured to have additional shadow registers implemented (SHADOW = YES). If shadow registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x90
- **Exists:** SHADOW == 1

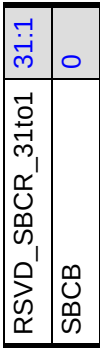


Table 5-30 Fields for Register: SBCR

Bits	Name	Memory Access	Description
31:1	RSVD_SBCR_31to1	R	SBCR 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-30 Fields for Register: SBCR (Continued)

Bits	Name	Memory Access	Description
0	SBCB	R/W	Shadow Break Control Bit. This is a shadow register for the Break bit (LCR[6]), this can be used to remove the burden of having to performing a read modify write on the LCR. This is used to cause a break condition to be transmitted to the receiving device. If set to one the serial output is forced to the spacing (logic 0) state. When not in Loopback Mode, as determined by MCR[4], the sout line is forced low until the Break bit is cleared. If SIR_MODE == Enabled and active (MCR[6] set to one) the sir_out_n line is continuously pulsed. When in Loopback Mode, the break condition is internally looped back to the receiver. Values: ■ 0x0 (NO_BREAK): No spacing on serial output ■ 0x1 (BREAK): Serial output forced to the spacing Value After Reset: 0x0 Exists: Always

5.1.26 SDMAM

- **Name:** Shadow DMA Mode Register
- **Description:** This register is valid only when the DW_apb_uart is configured to have additional FIFO registers implemented (FIFO_MODE != None) and additional shadow registers implemented (SHADOW = YES). If these registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x94
- **Exists:** (FIFO_MODE != 0) && (SHADOW == 1)

31:1	0
RSVD_SDMAM_31to1	SDMAM

Table 5-31 Fields for Register: SDMAM

Bits	Name	Memory Access	Description
31:1	RSVD_SDMAM_31to1	R	SDMAM 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-31 Fields for Register: SDMAM (Continued)

Bits	Name	Memory Access	Description
0	SDMAM	R/W	Shadow DMA Mode. This is a shadow register for the DMA mode bit (FCR[3]). This can be used to remove the burden of having to store the previously written value to the FCR in memory and having to mask this value so that only the DMA Mode bit gets updated. This determines the DMA signalling mode used for the dma_tx_req_n and dma_rx_req_n output signals when additional DMA handshaking signals are not selected (DMA_EXTRA == NO). See section 5.9 on page 54 for details on DMA support. Values: ■ 0x0 (MODE_0): Mode 0 ■ 0x1 (MODE_1): Mode 1 Value After Reset: 0x0 Exists: Always

5.1.27 SFE

- **Name:** Shadow FIFO Enable Register
- **Description:** SFE register is valid only when the DW_apb_uart is configured to have additional FIFO registers implemented (FIFO_MODE != None) and additional shadow registers implemented (SHADOW = YES). If these registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x98
- **Exists:** (FIFO_MODE != 0) && (SHADOW == 1)

31:1	
RSVD_SFE_31to1	
SFE	0

Table 5-32 Fields for Register: SFE

Bits	Name	Memory Access	Description
31:1	RSVD_SFE_31to1	R	SFE 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
0	SFE	R/W	Shadow FIFO Enable. This is a shadow register for the FIFO enable bit (FCR[0]). This can be used to remove the burden of having to store the previously written value to the FCR in memory and having to mask this value so that only the FIFO enable bit gets updated. This enables/disables the transmit (XMIT) and receive (RCVR) FIFO's. If this bit is set to zero (disabled) after being enabled then both the XMIT and RCVR controller portion of FIFO's will be reset. Values: ■ 0x0 (DISABLED): FIFOs are disabled ■ 0x1 (ENABLED): FIFOs are enabled Value After Reset: 0x0 Exists: Always

5.1.28 SRT

- **Name:** Shadow RCVR Trigger Register
- **Description:** SRT register is valid only when the DW_apb_uart is configured to have additional FIFO registers implemented (FIFO_MODE != None) and additional shadow registers implemented (SHADOW = YES). If these registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0x9c
- **Exists:** (FIFO_MODE != 0) && (SHADOW == 1)

31:2	RSVD_SRT_31to2
1:0	SRT

Table 5-33 Fields for Register: SRT

Bits	Name	Memory Access	Description
31:2	RSVD_SRT_31to2	R	SRT 31to2 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-33 Fields for Register: SRT (Continued)

Bits	Name	Memory Access	Description
1:0	SRT	R/W	Shadow RCVR Trigger. This is a shadow register for the RCVR trigger bits (FCR[7:6]). This can be used to remove the burden of having to store the previously written value to the FCR in memory and having to mask this value so that only the RCVR trigger bit gets updated. This is used to select the trigger level in the receiver FIFO at which the Received Data Available Interrupt will be generated. It also determines when the dma_rx_req_n signal will be asserted when DMA Mode (FCR[3]) is set to one. Values: ■ 0x0 (FIFO_CHAR_1): 1 character in FIFO ■ 0x1 (FIFO_QUARTER_FULL): FIFO 1/4 full ■ 0x2 (FIFO_HALF_FULL): FIFO 1/2 full ■ 0x3 (FIFO_FULL_2): FIFO 2 less than full Value After Reset: 0x0 Exists: Always

5.1.29 STET

- **Name:** Shadow TX Empty Trigger Register
- **Description:** This register is valid only when the DW_apb_uart is configured to have FIFOs implemented (FIFO_MODE != NONE) and THRE interrupt support implemented (THRE_MODE_USER = Enabled) and additional shadow registers implemented (SHADOW = YES). If FIFOs are not implemented or THRE interrupt support is not implemented or shadow registers are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0xa0
- **Exists:** (FIFO_MODE != 0) && (THRE_MODE_USER == 1) && (SHADOW == 1)

31:2	RSVD_STET_31to2
1:0	STET

Table 5-34 Fields for Register: STET

Bits	Name	Memory Access	Description
31:2	RSVD_STET_31to2	R	STET 31to2 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-34 Fields for Register: STET (Continued)

Bits	Name	Memory Access	Description
1:0	STET	R/W	Shadow TX Empty Trigger. This is a shadow register for the TX empty trigger bits (FCR[5:4]). This can be used to remove the burden of having to store the previously written value to the FCR in memory and having to mask this value so that only the TX empty trigger bit gets updated. Writes will have no effect when THRE_MODE_USER == Disabled. This is used to select the empty threshold level at which the THRE Interrupts will be generated when the mode is active. Values: ■ 0x0 (FIFO_EMPTY): FIFO empty ■ 0x1 (FIFO_CHAR_2): 2 characters in FIFO ■ 0x2 (FIFO_QUARTER_FULL): FIFO 1/4 full ■ 0x3 (FIFO_HALF_FULL): FIFO 1/2 full Value After Reset: 0x0 Exists: Always

5.1.30 HTX

- **Name:** Halt TX
- **Description:** Halt TX
- **Size:** 32 bits
- **Offset:** 0xa4
- **Exists:** Always

31:1	0
RSVD_HTX_31to1	HTX

Table 5-35 Fields for Register: HTX

Bits	Name	Memory Access	Description
31:1	RSVD_HTX_31to1	R	HTX 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
0	HTX	* Varies	Halt TX. Writes will have no effect when FIFO_MODE == NONE, always readable. This register is use to halt transmissions for testing, so that the transmit FIFO can be filled by the requester when FIFO's are implemented and enabled. Note, if FIFO's are implemented and not enabled the setting of the halt TX register will have no effect on operation. Values: ■ 0x0 (DISABLED): Halt Transmission disabled ■ 0x1 (ENABLED): Halt Transmission enabled Value After Reset: 0x0 Exists: Always Memory Access: "(FIFO_MODE==0) ? \"read-only\" : \"read-write\""

5.1.31 DMASA

- **Name:** DMA Software Acknowledge Register
- **Description:** DMA Software Acknowledge Register
- **Size:** 32 bits
- **Offset:** 0xa8
- **Exists:** Always

31:1	0
RSVD_DMASA_31to1	DMASA

Table 5-36 Fields for Register: DMASA

Bits	Name	Memory Access	Description
31:1	RSVD_DMASA_31to1	R	DMASA 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
0	DMASA	* Varies	DMA Software Acknowledge. Writes will have no effect when DMA_EXTRA == No. This register is use to perform DMA software acknowledge if a transfer needs to be terminated due to an error condition. For example, if the DMA disables the channel, then the DW_apb_uart should clear its request. This will cause the TX request, TX single, RX request and RX single signals to de-assert. Note that this bit is 'self-clearing' and it is not necessary to clear this bit. Values: ■ 0x1 (SOFT_ACK): DMA software acknowledge Value After Reset: 0x0 Exists: Always Memory Access: "(DMA_EXTRA==1) ? \"write-only\" : \"read-only\""

5.1.32 TCR

- **Name:** Transceiver Control Register
- **Description:** This register is used to enable or disable RS485 mode and also control the polarity values for Driven enable (de) and Receiver Enable (re) signals.

This register is only valid when the DW_apb_uart is configured to have RS485 interface implemented (UART_RS485_INTERFACE_EN = ENABLED). If RS485 interface is not implemented, this register does not exist and reading from this register address returns zero.

- **Size:** 32 bits
- **Offset:** 0xac
- **Exists:** UART_RS485_INTERFACE_EN == 1

31:5	4:3	2	1	0
RSVD_TCR_31to5	XFER_MODE	DE_POL	RE_POL	RS485_EN

Table 5-37 Fields for Register: TCR

Bits	Name	Memory Access	Description
31:5	RSVD_TCR_31to5	R	TCR 31to5 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-37 Fields for Register: TCR (Continued)

Bits	Name	Memory Access	Description
4:3	XFER_MODE	* Varies	Transfer Mode. Values: 0x0 (XFER_MODE_0): In this mode, transmit and receive can happen simultaneously. The user can enable DE_EN, RE_EN at any point of time. Turn around timing as programmed in the TAT register is not applicable in this mode. 0x1 (XFER_MODE_1): In this mode, DE and RE are mutually exclusive. Either DE or RE only one of them is expected to be enabled through programming. Hardware will consider the Turn Around timings which are programmed in the TAT register while switching from RE to DE or DE to RE. For transmission Hardware will wait if it is in middle of receiving any transfer, before it starts transmitting. 0x2 (XFER_MODE_2): In this mode, DE and RE are mutually exclusive. Once DE_EN/RE_EN is programmed - by default 're' will be enabled and DW_apb_uart controller will be ready to receive. If the user programs the TX FIFO with the data then DW_apb_uart, after ensuring no receive is in progress, disable 're' and enable 'de' signal. Once the TX FIFO becomes empty, 're' signal gets enabled and 'de' signal will be disabled. In this mode of operation hardware will consider the Turn Around timings which are programmed in the TAT register while switching from RE to DE or DE to RE. In this mode, 'de' and 're' signals are strictly complementary to each other. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""
2	DE_POL	* Varies	Driver Enable Polarity. Values: 0x0 (LOW): DE signal is active low 0x1 (HIGH): DE signal is active high Value After Reset: UART_DE_POL Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

Table 5-37 Fields for Register: TCR (Continued)

Bits	Name	Memory Access	Description
1	RE_POL	* Varies	Receiver Enable Polarity. Values: 0x0 (LOW): RE signal is active low 0x1 (HIGH): RE signal is active high Value After Reset: UART_RE_POL Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""
0	RS485_EN	* Varies	RS485 Transfer Enable. 0 : In this mode, the transfers are still in the RS232 mode. All other fields in this register are reserved and register DE_EN/RE_EN/TAT are also reserved. 1 : In this mode, the transfers will happen in RS485 mode. All other fields of this register are applicable. Values: 0x0 (DISABLE): RS232 Mode 0x1 (ENABLE): RS485 Mode Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

5.1.33 DE_EN

- **Name:** Driver Output Enable Register
- **Description:** The Driver Output Enable Register (DE_EN) is used to control the assertion and de-assertion of the DE signal.

This register is only valid when the DW_apb_uart is configured to have RS485 interface implemented (UART_RS485_INTERFACE_EN = ENABLED). If RS485 interface is not implemented, this register does not exist and reading from this register address will return zero.

- **Size:** 32 bits
- **Offset:** 0xb0
- **Exists:** UART_RS485_INTERFACE_EN == 1

31:1	
RSVD_DE_EN_31to1	
0	DE_Enable

Table 5-38 Fields for Register: DE_EN

Bits	Name	Memory Access	Description
31:1	RSVD_DE_EN_31to1	R	DE_EN 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
0	DE_Enable	* Varies	DE Enable control. The 'DE Enable' register bit is used to control assertion and de-assertion of 'de' signal. Values: ■ 0x0 (DE_ASSERT): De-assert 'de' signal ■ 0x1 (ASSERT): Assert 'de' signal Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

5.1.34 RE_EN

- **Name:** Receiver Output Enable Register
- **Description:** The Receiver Output Enable Register (RE_EN) is used to control the assertion and de-assertion of the RE signal.

This register is only valid when the DW_apb_uart is configured to have RS485 interface implemented (UART_RS485_INTERFACE_EN = ENABLED). If the RS485 interface is not implemented, this register does not exist and reading from this register address will return zero.

- **Size:** 32 bits
- **Offset:** 0xb4
- **Exists:** UART_RS485_INTERFACE_EN == 1

31:1	RSVD_RE_EN_31to1
0	RE_Enable

Table 5-39 Fields for Register: RE_EN

Bits	Name	Memory Access	Description
31:1	RSVD_RE_EN_31to1	R	RE_EN 31to1 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
0	RE_Enable	* Varies	RE Enable control. The 'RE Enable' register bit is used to control assertion and de-assertion of 're' signal. Values: ■ 0x0 (DE_ASSERT): De-assert 're' signal ■ 0x1 (ASSERT): Assert 're' signal Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

5.1.35 DET

- **Name:** Driver Output Enable Timing Register
- **Description:** The Driver Output Enable Timing Register (DET) is used to control the DE assertion and de-assertion timings of 'de' signal.

This register is only valid when the DW_apb_uart is configured to have RS485 interface implemented (UART_RS485_INTERFACE = ENABLED). If RS485 interface is not implemented, this register does not exist and reading from this register address will return zero.

- **Size:** 32 bits
- **Offset:** 0xb8
- **Exists:** UART_RS485_INTERFACE_EN == 1

RSVD_DE_DEAT_31to24	31:24
DE_De-assertion_Time	23:16
RSVD_DE_AT_15to8	15:8
DE_Assertion_Time	7:0

Table 5-40 Fields for Register: DET

Bits	Name	Memory Access	Description
31:24	RSVD_DE_DEAT_31to24	R	DET 31to24 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
23:16	DE_De-assertion_Time	* Varies	Driver Enable de-assertion time. This field controls the amount of time (in terms of number of serial clock periods) between the end of stop bit on the sout to the falling edge of Driver output enable signal. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

Table 5-40 Fields for Register: DET (Continued)

Bits	Name	Memory Access	Description
15:8	RSVD_DE_AT_15to8	R	DET 15to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
7:0	DE_Assertion_Time	* Varies	Driver Enable assertion time. This field controls the amount of time (in terms of number of serial clock periods) between the assertion of rising edge of Driver output enable signal to serial transmit enable. Any data in transmit buffer, will start on serial output (sout) after the transmit enable. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

5.1.36 TAT

- **Name:** Turn Around Timing Register
- **Description:** The Turn Around Timing Register (TAT) is used to hold the turnaround time between switching of 're' and 'de' signals.

This register is only valid when the DW_apb_uart is configured to have the RS485 interface implemented (UART_RS485_INTERFACE_EN = ENABLED). If RS485 interface is not implemented, this register does not exist and reading from this register address will return zero.
- **Size:** 32 bits
- **Offset:** 0xbc
- **Exists:** UART_RS485_INTERFACE_EN == 1

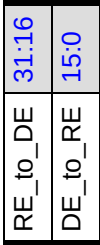


Table 5-41 Fields for Register: TAT

Bits	Name	Memory Access	Description
31:16	RE_to_DE	* Varies	Receiver Enable to Driver Enable TurnAround time. Turnaround time (in terms of serial clock) for RE De-assertion to DE assertion. Note: - If the DE assertion time in the DET register is 0, then the actual value is the programmed value + 3. - If the DE assertion time in the DET register is 1, then the actual value is the programmed value + 2. - If the DE assertion time in the DET register is greater than 1, then the actual value is the programmed value + 1. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

Table 5-41 Fields for Register: TAT (Continued)

Bits	Name	Memory Access	Description
15:0	DE_to_RE	* Varies	Driver Enable to Receiver Enable TurnAround time. Turnaround time (in terms of serial clock) for DE De-assertion to RE assertion. Note: The actual time is the programmed value + 1. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_RS485_INTERFACE_EN==1) ? \"read-write\" : \"read-only\""

5.1.37 DLF

- **Name:** Divisor Latch Fraction Register
- **Description:** This register is only valid when the DW_apb_uart is configured to have Fractional Baud rate Divisor implemented (FRACTIONAL_BAUD_DIVISOR_EN = ENABLED). If Fractional Baud rate divisor is not implemented, this register does not exist and reading from this register address will return zero.
- **Size:** 32 bits
- **Offset:** 0xc0
- **Exists:** FRACTIONAL_BAUD_DIVISOR_EN == 1

31:y	x:0
RSVD_DLF	DLF

Table 5-42 Fields for Register: DLF

Bits	Name	Memory Access	Description
31:y	RSVD_DLF	R	DLF 31 to DLF_SIZE Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Range Variable[y]: DLF_SIZE
x:0	DLF	* Varies	Fractional part of divisor. The fractional value is added to integer value set by DLH, DLL. Fractional value is determined by (Divisor Fraction value)/(2 ^{DLF_SIZE}). For information on DLF values to be programmed for DLF_SIZE=4, see the 'Fractional Baud Rate Support' section in the DW_apb_uart Databook. Value After Reset: 0x0 Exists: Always Range Variable[x]: DLF_SIZE - 1 Memory Access: "(FRACTIONAL_BAUD_DIVISOR_EN==1) ? \"read-write\" : \"read-only\""

5.1.38 RAR

- **Name:** Receive Address Register
- **Description:** Receive Address Register
- **Size:** 32 bits
- **Offset:** 0xc4
- **Exists:** UART_9BIT_DATA_EN == 1

31:8	7:0
RSVD_RAR_31to8	RAR

Table 5-43 Fields for Register: RAR

Bits	Name	Memory Access	Description
31:8	RSVD_RAR_31to8	R	RAR 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-43 Fields for Register: RAR (Continued)

Bits	Name	Memory Access	Description
7:0	RAR	* Varies	This is an address matching register during receive mode. If the 9-th bit is set in the incoming character then the remaining 8-bits will be checked against this register value. If the match happens then sub-sequent characters with 9-th bit set to 0 will be treated as data byte until the next address byte is received. Note: - This register is applicable only when 'ADDR_MATCH'(LCR_EXT[1] and 'DLS_E' (LCR_EXT[0]) bits are set to 1. - If UART_16550_COMPATIBLE is configured to 0, then RAR should be programmed only when UART is not busy. - If UART_16550_COMPATIBLE is configured to 0, then RAR can be programmed at any point of the time. However, user must not change this register value when any receive is in progress. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_9BIT_DATA_EN==1) ? \"read-write\" : \"read-only\""

5.1.39 TAR

- **Name:** Transmit Address Register
- **Description:** Transmit Address Register
- **Size:** 32 bits
- **Offset:** 0xc8
- **Exists:** UART_9BIT_DATA_EN == 1

31:8	7:0
RSVD_TAR_31to8	TAR

Table 5-44 Fields for Register: TAR

Bits	Name	Memory Access	Description
31:8	RSVD_TAR_31to8	R	TAR 31to8 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
7:0	TAR	* Varies	This is an address matching register during transmit mode. If DLS_E (LCR_EXT[0]) bit is enabled, then DW_apb_uart will send the 9-bit character with 9-th bit set to 1 and remaining 8-bit address will be sent from this register provided 'SEND_ADDR' (LCR_EXT[2]) bit is set to 1. Note: - This register is used only to send the address. The normal data should be sent by programming THR register. - Once the address is started to send on the DW_apb_uart serial lane, then 'SEND_ADDR' bit will be auto-cleared by the hardware. Value After Reset: 0x0 Exists: Always Memory Access: "(UART_9BIT_DATA_EN==1) ? \"read-write\" : \"read-only\""

5.1.40 LCR_EXT

- **Name:** Line Extended Control Register
- **Description:** Line Extended Control Register
- **Size:** 32 bits
- **Offset:** 0xcc
- **Exists:** (UART_9BIT_DATA_EN == 1) || (CLK_GATE_EN == 1)

31:5	RSVD_LCR_EXT
4	CGEN
3	TRANSMIT_MODE
2	SEND_ADDR
1	ADDR_MATCH
0	DLS_E

Table 5-45 Fields for Register: LCR_EXT

Bits	Name	Memory Access	Description
31:5	RSVD_LCR_EXT	R	LCR_EXT 31to5 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always Volatile: true
4	CGEN	R/W	Clock Gate Enable Signal. This bit is used to gate the clock cycles and enable the low power mode when the device is inactive. Value After Reset: CLK_GATE_EN Exists: CLK_GATE_EN == 1 Volatile: true

Table 5-45 Fields for Register: LCR_EXT (Continued)

Bits	Name	Memory Access	Description
3	TRANSMIT_MODE	R/W	Transmit mode control bit. This bit is used to control the type of transmit mode during 9-bit data transfers. Values: 0x1 (TRANSMIT_MODE_1): Divisor Latch register is always readable and writable. In this mode of operation, Transmit Holding Register (THR) and Shadow Transmit Holding Register (STHR) are 9-bit wide. The user needs to ensure that the THR/STHR register is written correctly for address/data. Address: 9th bit is set to 1 Data : 9th bit is set to 0 Note: Transmit address register (TAR) is not applicable in this mode of operation. 0x0 (TRANSMIT_MODE_0): In this mode of operation, Transmit Holding Register (THR) and Shadow Transmit Holding register (STHR) are 8-bit wide. The user needs to program the address into Transmit Address Register (TAR) and data into the THR/STHR register. SEND_ADDR bit is used as a control knob to indicate the DW_apb_uart on when to send the address. Value After Reset: 0x0 Exists: UART_9BIT_DATA_EN == 1 Volatile: true

Table 5-45 Fields for Register: LCR_EXT (Continued)

Bits	Name	Memory Access	Description
2	SEND_ADDR	R/W	Send address control bit. This bit is used as a control knob for the user to determine when to send the address during transmit mode.Note: 1. This bit is auto-cleared by the hardware, after sending out the address character. User is not expected to program this bit to 0. 2. This field is applicable only when DLS_E bit is set to 1 and TRANSMIT_MODE is set to 0. Values: ■ 0x1 (SEND_ADDR_1): 9-bit character will be transmitted with 9-th bit set to 1 and the remaining 8-bits will match to what is being programmed in 'Transmit Address Register'. ■ 0x0 (SEND_ADDR_0): 9-bit character will be transmitted with 9-th bit set to 0 and the remaining 8-bits will be taken from the TXFIFO which is programmed through 8-bit wide THR/STHR register. Value After Reset: 0x0 Exists: UART_9BIT_DATA_EN == 1 Volatile: true
1	ADDR_MATCH	R/W	Address Match Mode. This bit is used to enable the address match feature during receive. ■ In Address match mode, DW_apb_uart will wait until the incoming character with 9-th bit set to 1. And further checks to see if the address matches with what is programmed in 'Receive Address Match Register'. If match is found, then sub-sequent characters will be treated as valid data and DW_apb_uart starts receiving data. ■ In Normal mode, DW_apb_uart will start to receive the data and 9-bit character will be formed and written into the receive RXFIFO. User is responsible to read the data and differentiate b/n address and data. Note: This field is applicable only when DLS_E is set to 1. Values: ■ 0x0 (DISABLE): Normal Mode ■ 0x1 (ENABLE): Address Match Mode Value After Reset: 0x0 Exists: UART_9BIT_DATA_EN == 1 Volatile: true

Table 5-45 Fields for Register: LCR_EXT (Continued)

Bits	Name	Memory Access	Description
0	DLS_E	R/W	Extension for DLS. This bit is used to enable 9-bit data for transmit and receive transfers. Value After Reset: 0x0 Exists: UART_9BIT_DATA_EN == 1 Volatile: true

5.1.41 **UART_PROT_LEVEL**

- **Name:** UART Protection level
- **Description:** UART Protection level register
- **Size:** 32 bits
- **Offset:** 0xd0
- **Exists:** (SLAVE_INTERFACE_TYPE > 1 && PSLVERR_RESP_EN==1 && HC_PROT_LEVEL==0) ? 1 : 0

31:3	RSVD_UART_PROT_LEVEL
2:0	UART_PROT_LEVEL

Table 5-46 Fields for Register: UART_PROT_LEVEL

Bits	Name	Memory Access	Description
31:3	RSVD_UART_PROT_LEVEL	R	UART_PROT_LEVEL[31:29] Reserved field-read-only Value After Reset: 0x0 Exists: Always
2:0	UART_PROT_LEVEL	* Varies	Protection level register. Enabling protection on any of its three bits would require a greater or equal privilege on the PPROT signal to gain access to protected registers. Value After Reset: PROT_LEVEL_RST Exists: Always Memory Access: {(HC_PROT_LEVEL==0) ? "read-write" : "read-only"}

5.1.42 REG_TIMEOUT_RST

- **Name:** Register timeout counter reset value
- **Description:** Name: Register timeout counter reset register This register keeps the reset value of reg_timer counter register. The reset value of the register is REG_TIMEOUT_DEFAULT The default reset value can be further modified if HC_REG_TIMEOUT_VALUE = 0. The final programmed value (or the default reset value if not programmed) determines what value the reg_timeout counter register starts counting down from. A zero on the counter will break the hung transaction with PSLVERR high
- **Size:** 32 bits
- **Offset:** 0xd4
- **Exists:** (((SLAVE_INTERFACE_TYPE>0 && PSLVERR_RESP_EN==1 && REG_TIMEOUT_WIDTH>0) ? 1 : 0)==1) ? 1 : 0

31:y	RSVD_REG_TIMEOUT_RST
x:0	REG_TIMEOUT_RST

Table 5-47 Fields for Register: REG_TIMEOUT_RST

Bits	Name	Memory Access	Description
31:y	RSVD_REG_TIMEOUT_RST	R	Reserved bits - Read Only Value After Reset: 0x0 Exists: Always Volatile: true Range Variable[y]: REG_TIMEOUT_WIDTH

Table 5-47 Fields for Register: REG_TIMEOUT_RST (Continued)

Bits	Name	Memory Access	Description
x:0	REG_TIMEOUT_RST	R/W	This field holds reset value of REG_TIMEOUT counter register. Value After Reset: REG_TIMEOUT_VALUE Exists: [<functionof> "(HC_REG_TIMEOUT_VALUE==0) ? 1 : 0"] Volatile: true Range Variable[x]: REG_TIMEOUT_WIDTH - 1

5.1.43 CPR

- **Name:** Component Parameter Register
- **Description:** Component Parameter Register. This register is valid only when UART_ADD_ENCODED_PARAMS = 1. If the UART_ADD_ENCODED_PARAMS parameter is not set, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0xf4
- **Exists:** UART_ADD_ENCODED_PARAMS == 1

31:24	RSVD_CPR_31to24
23:16	FIFO_MODE
15:14	RSVD_CPR_15to14
13	DMA_EXTRA
12	UART_ADD_ENCODED_PARAMS
11	SHADOW
10	FIFO_STAT
9	FIFO_ACCESS
8	ADDITIONAL_FEAT
7	SIR_LP_MODE
6	SIR_MODE
5	THRE_MODE
4	AFCE_MODE
3:2	RSVD_CPR_3to2
1:0	APB_DATA_WIDTH

Table 5-48 Fields for Register: CPR

Bits	Name	Memory Access	Description
31:24	RSVD_CPR_31to24	R	CPR 31to24 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always

Table 5-48 Fields for Register: CPR (Continued)

Bits	Name	Memory Access	Description
23:16	FIFO_MODE	R	Encoding of FIFO_MODE configuration parameter value. Values: 0x0 (FIFO_MODE_0): FIFO mode is 0 0x1 (FIFO_MODE_16): FIFO mode is 16 0x2 (FIFO_MODE_32): FIFO mode is 32 0x4 (FIFO_MODE_64): FIFO mode is 64 0x8 (FIFO_MODE_128): FIFO mode is 128 0x10 (FIFO_MODE_256): FIFO mode is 256 0x20 (FIFO_MODE_512): FIFO mode is 512 0x40 (FIFO_MODE_1024): FIFO mode is 1024 0x80 (FIFO_MODE_2048): FIFO mode is 2048 Value After Reset: UART_ENCODED_FIFO_MODE Exists: Always
15:14	RSVD_CPR_15to14	R	CPR 15to14 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
13	DMA_EXTRA	R	Encoding of DMA_EXTRA configuration parameter value. Values: 0x0 (DISABLED): DMA_EXTRA disabled 0x1 (ENABLED): DMA_EXTRA enabled Value After Reset: DMA_EXTRA Exists: Always
12	UART_ADD_ENCODED_PARAMS	R	Encoding of UART_ADD_ENCODED_PARAMS configuration parameter value. Values: 0x0 (DISABLED): UART_ADD_ENCODED_PARAMS disabled 0x1 (ENABLED): UART_ADD_ENCODED_PARAMS enabled Value After Reset: UART_ADD_ENCODED_PARAMS Exists: Always
11	SHADOW	R	Encoding of SHADOW configuration parameter value. Values: 0x0 (DISABLED): SHADOW disabled 0x1 (ENABLED): SHADOW enabled Value After Reset: SHADOW Exists: Always

Table 5-48 Fields for Register: CPR (Continued)

Bits	Name	Memory Access	Description
10	FIFO_STAT	R	Encoding of FIFO_STAT configuration parameter value. Values: 0x0 (DISABLED): FIFO_STAT disabled 0x1 (ENABLED): FIFO_STAT enabled Value After Reset: FIFO_STAT Exists: Always
9	FIFO_ACCESS	R	Encoding of FIFO_ACCESS configuration parameter value. Values: 0x0 (DISABLED): FIFO_ACCESS disabled 0x1 (ENABLED): FIFO ACCESS enabled Value After Reset: FIFO_ACCESS Exists: Always
8	ADDITIONAL_FEAT	R	Encoding of ADDITIONAL_FEATURES configuration parameter value. Values: 0x0 (DISABLED): Additional features disabled 0x1 (ENABLED): Additional features enabled Value After Reset: ADDITIONAL_FEATURES Exists: Always
7	SIR_LP_MODE	R	Encoding of SIR_LP_MODE configuration parameter value. Values: 0x0 (DISABLED): SIR_LP mode disabled 0x1 (ENABLED): SIR_LP mode enabled Value After Reset: SIR_LP_MODE Exists: Always
6	SIR_MODE	R	Encoding of SIR_MODE configuration parameter value. Values: 0x0 (DISABLED): SIR mode disabled 0x1 (ENABLED): SIR mode enabled Value After Reset: SIR_MODE Exists: Always

Table 5-48 Fields for Register: CPR (Continued)

Bits	Name	Memory Access	Description
5	THRE_MODE	R	Encoding of THRE_MODE configuration parameter value. Values: 0x0 (DISABLED): THRE mode disabled 0x1 (ENABLED): THRE mode enabled Value After Reset: THRE_MODE_RST Exists: Always
4	AFCE_MODE	R	Encoding of AFCE_MODE configuration parameter value. Values: 0x0 (DISABLED): AFCE mode disabled 0x1 (ENABLED): AFCE mode enabled Value After Reset: AFCE_MODE Exists: Always
3:2	RSVD_CPR_3to2	R	CPR 3to2 Reserved bits read as 0. Value After Reset: 0x0 Exists: Always
1:0	APB_DATA_WIDTH	R	Encoding of APB_DATA_WIDTH configuration parameter value. Values: 0x0 (APB_8BITS): APB data width is 8 bits 0x1 (APB_16BITS): APB data width is 16 bits 0x2 (APB_32BITS): APB data width is 32 bits Value After Reset: UART_ENCODED_APB_WIDTH Exists: Always

5.1.44 UCV

- **Name:** UART Component Version
- **Description:** UCV register is valid only when the DW_apb_uart is configured to have additional features implemented (ADDITIONAL_FEATURES = YES). If additional features are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0xf8
- **Exists:** ADDITIONAL_FEATURES == 1



Table 5-49 Fields for Register: UCV

Bits	Name	Memory Access	Description
31:0	UART_Component_Version	R	ASCII value for each number in the version, followed by *. For example 32_30_31_2A represents the version 2.01* Value After Reset: UART_COMP_VERSION Exists: Always

5.1.45 CTR

- **Name:** Component Type Register
- **Description:** CTR is register is valid only when the DW_apb_uart is configured to have additional features implemented (ADDITIONAL_FEATURES = YES). If additional features are not implemented, this register does not exist and reading from this register address returns 0.
- **Size:** 32 bits
- **Offset:** 0xfc
- **Exists:** ADDITIONAL_FEATURES == 1

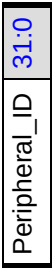


Table 5-50 Fields for Register: CTR

Bits	Name	Memory Access	Description
31:0	Peripheral_ID	R	This register contains the peripherals identification code. Value After Reset: UART_COMP_TYPE Exists: Always

6

Programming the DW_apb_uart

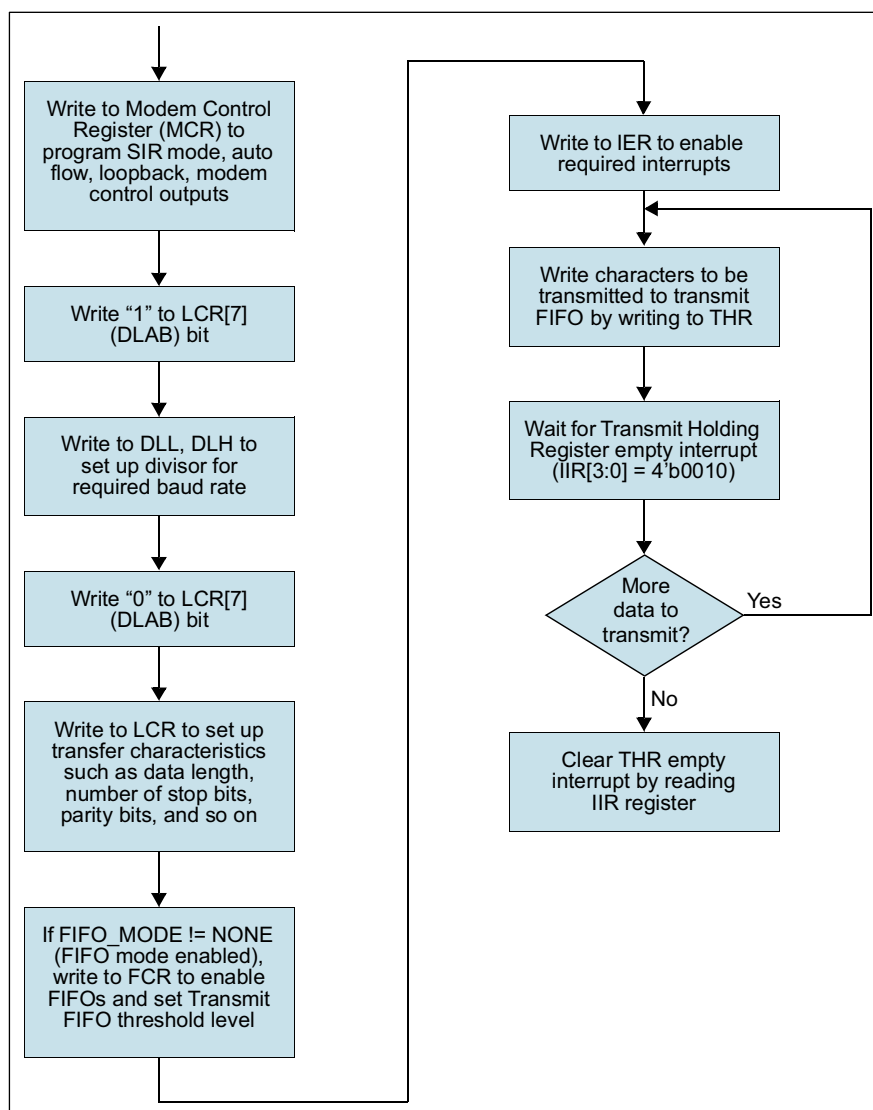
The following topics provide information necessary to program the DW_apb_uart.

- [“Programming Examples”](#) on page [222](#)
- [“Programming Flow in RS485 Mode”](#) on page [223](#)
- [“Programming Flow in 9-bit Data Mode”](#) on page [226](#)
- [“Programming Flow for Fractional Baud Rate”](#) on page [229](#)

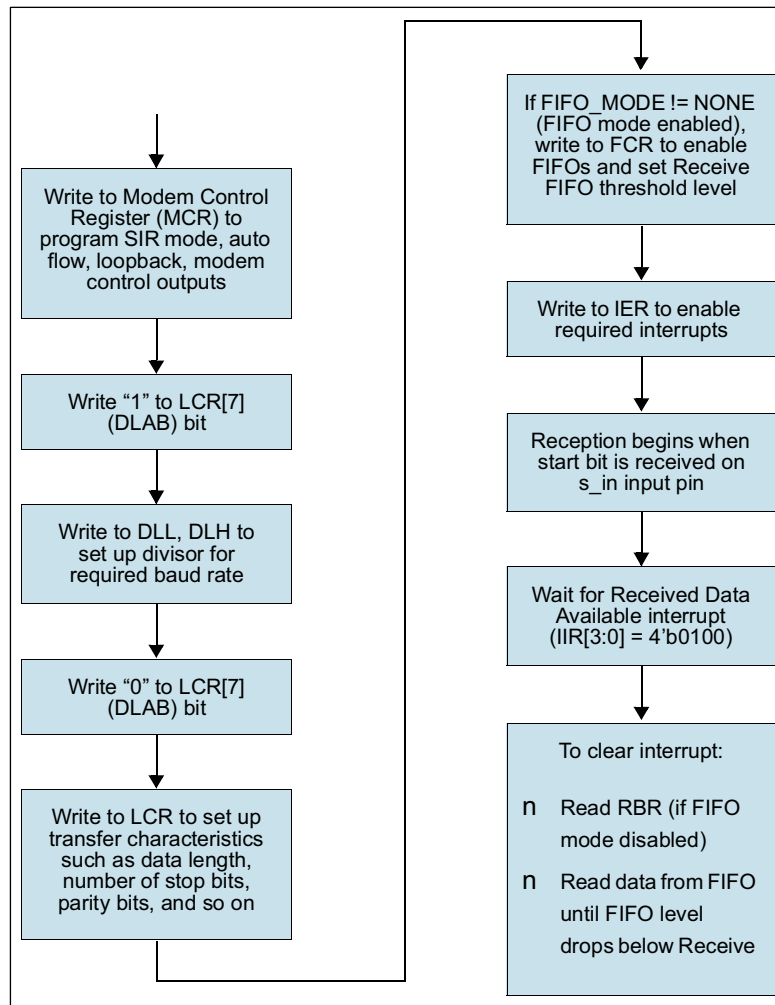
6.1 Programming Examples

The flow diagram in [Figure 6-1](#) shows the programming sequence for setting up the DW_apb_uart for transmission.

Figure 6-1 Flowchart for DW_apb_uart Transmit Programming Example



The flow diagram in [Figure 6-2](#) shows the programming sequence for setting up the DW_apb_uart for reception.

Figure 6-2 Flowchart for DW_apb_uart Receive Programming Example

6.2 Programming Flow in RS485 Mode

6.2.1 Full Duplex Mode (XFER_MODE=0)

- Program the TCR register to set the XFER_MODE(0), DE_POL (polarity of 'de' signal) and RE_POL (polarity of 're' signals).
- Program the DE assertion and de-assertion timing in the DET register.
- Program RE_EN and DE_EN register to assert 're' signal and 'de' signal, respectively.
- Perform the data transmission and reception.
- Program RE_EN to de-assert 're' signal.
- 'The de' signal gets de-asserted based on TxFIFO empty. Program DE_EN to '0' if you do not want to transmit further.

6.2.2 Software-Enabled Half Duplex Mode (XFER_MODE=1)

- Program the TCR register to set the XFER_MODE (1), DE_POL (polarity of 'de' signal) and RE_POL (polarity of 're' signals).
- Program the DE assertion and de-assertion timing in the DET register.
- Program the turnaround times in TAT register.
- Program RE_EN and DE_EN registers to assert 're' signal and 'de' signal, respectively.
- Perform the data transmission/receive.
- Program the RE_EN register to de-assert 're' signal.
- Program the DE_EN register to '0', before programming RE_EN to '1'.
 - 'de' signal de-assertion is based on TxFIFO empty condition and is taken care by the Hardware. You need to program DE_EN to '0' only in the situation where you want to change the mode.

6.2.3 Hardware enabled Half Duplex mode (XFER_MODE=2)

- Program the TCR register to set the XFER_MODE (2), DE_POL (polarity of 'de' signal) and RE_POL (polarity of 're' signals).
- Program the DE assertion and de-assertion timing in the DET register.
- Program the turnaround times in TAT register.
- Program RE_EN and DE_EN register to enable the transmit and receive paths.
- Perform the Data transmission / receive.
- Once the 'RE_EN' and 'DE_EN' is programmed to '1', then by default 're' signal is asserted and 'de' is de-asserted. When the software pushes the data into the TX FIFO and if there is no ongoing receive transfer, then the 're' signal is de-asserted and then the 'de' signal gets asserted until the TX FIFO has data to be transmitted.
- RE_EN and DE_EN still serves as the software overrides to decide when to shutdown transmit and receive paths.

**Note**

- Irrespective of other configurations (parameters enabled), RS485 mode is not applicable when loopback mode is enabled. However, external signal rs485_en is still asserted when TCR[0] (RS485 EN) bit is asserted even in loopback mode.
- When Clock Gating is enabled (CLK_GATE_EN=1), the clock gate enable signals uart_lp_req_pclk for single clock implementations or uart_lp_req_pclk and uart_lp_req_sclk for two clock implementations is used to indicate the following:
 - Transmit and receive pipelines are clear (no data).
 - No activity has occurred.
 - Modem control input signals have not changed in more than one character time—the time taken to TX/RX a character—so that clocks can be gated.

A character is made up of:

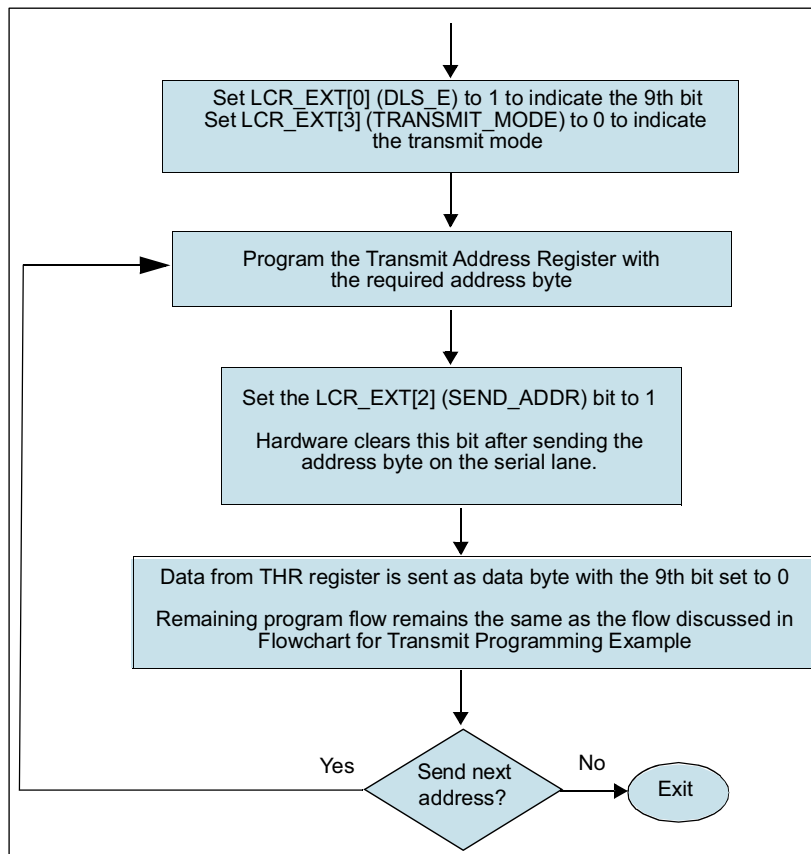
start_bit + data_bits + parity (optional) + stop_bits

If UART_RS485_INTERFCAE_EN=1 and rs485_en (TCR[0]) =1, then DW_apb_uart also ensures that counters related to DET/TAT are also taken care of before entering into low power mode.

6.3 Programming Flow in 9-bit Data Mode

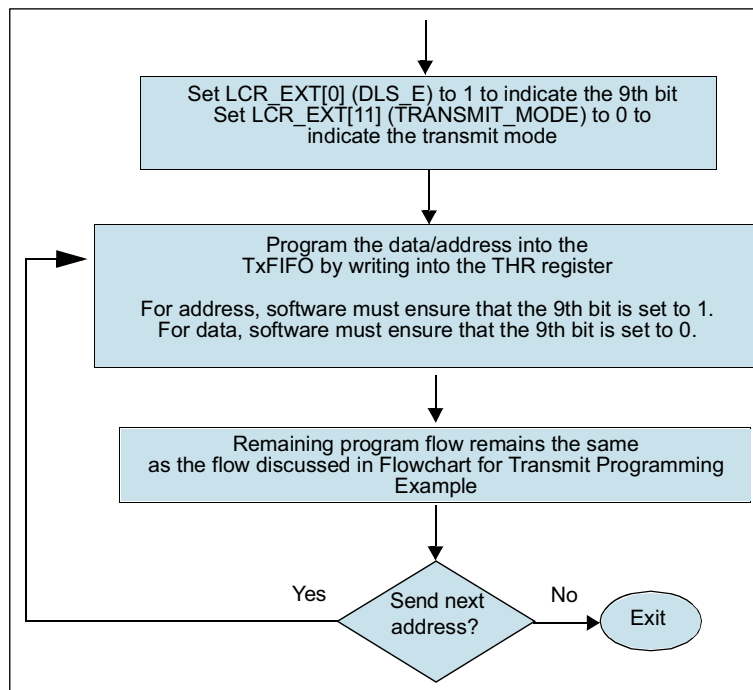
6.3.1 Transmit Mode 0

Figure 6-3 Auto Address Transmit Mode



6.3.2 Transmit Mode 1

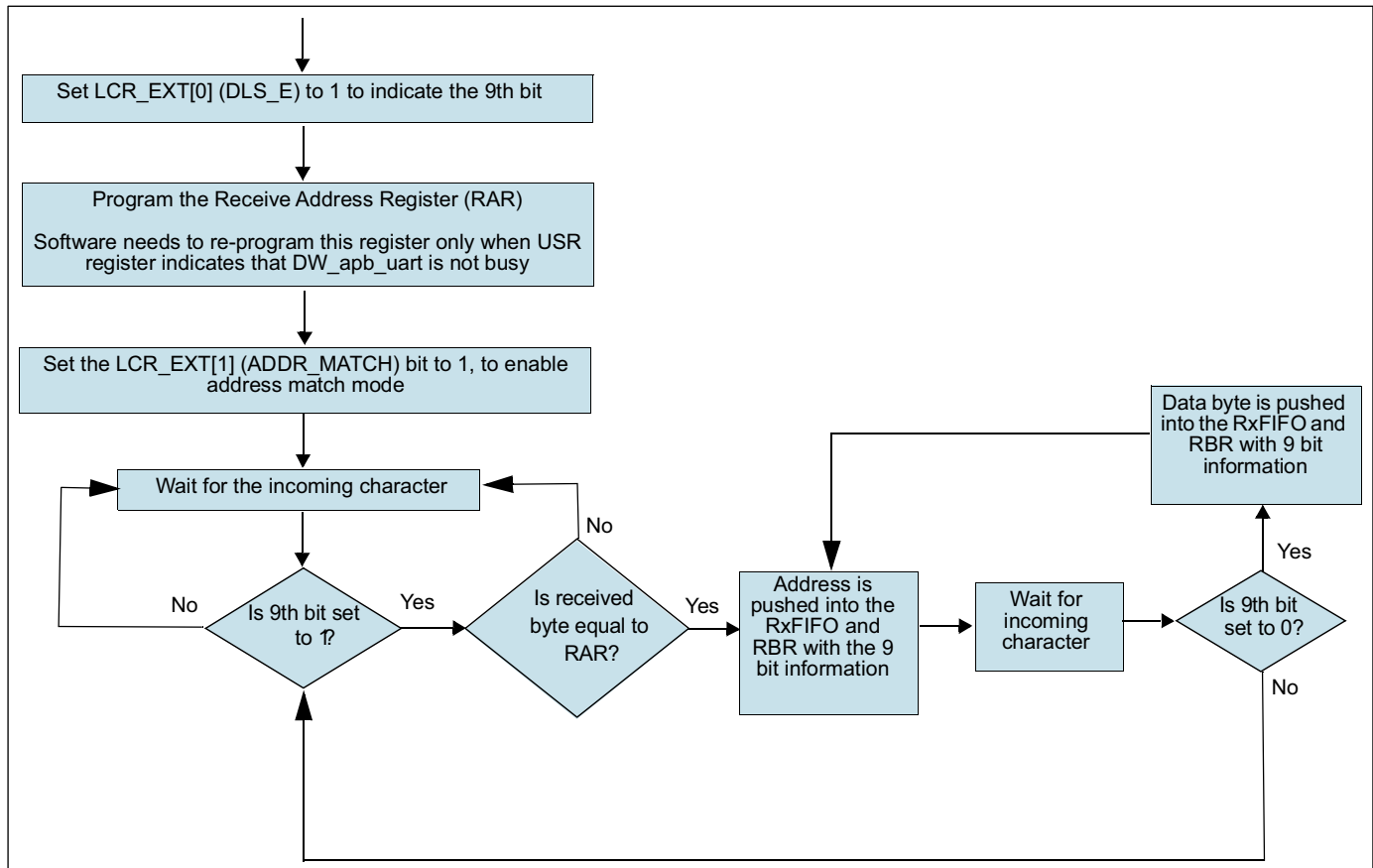
Figure 6-4 Normal Transmit Mode Programming Flow



6.3.3 Hardware Address Match Receive mode

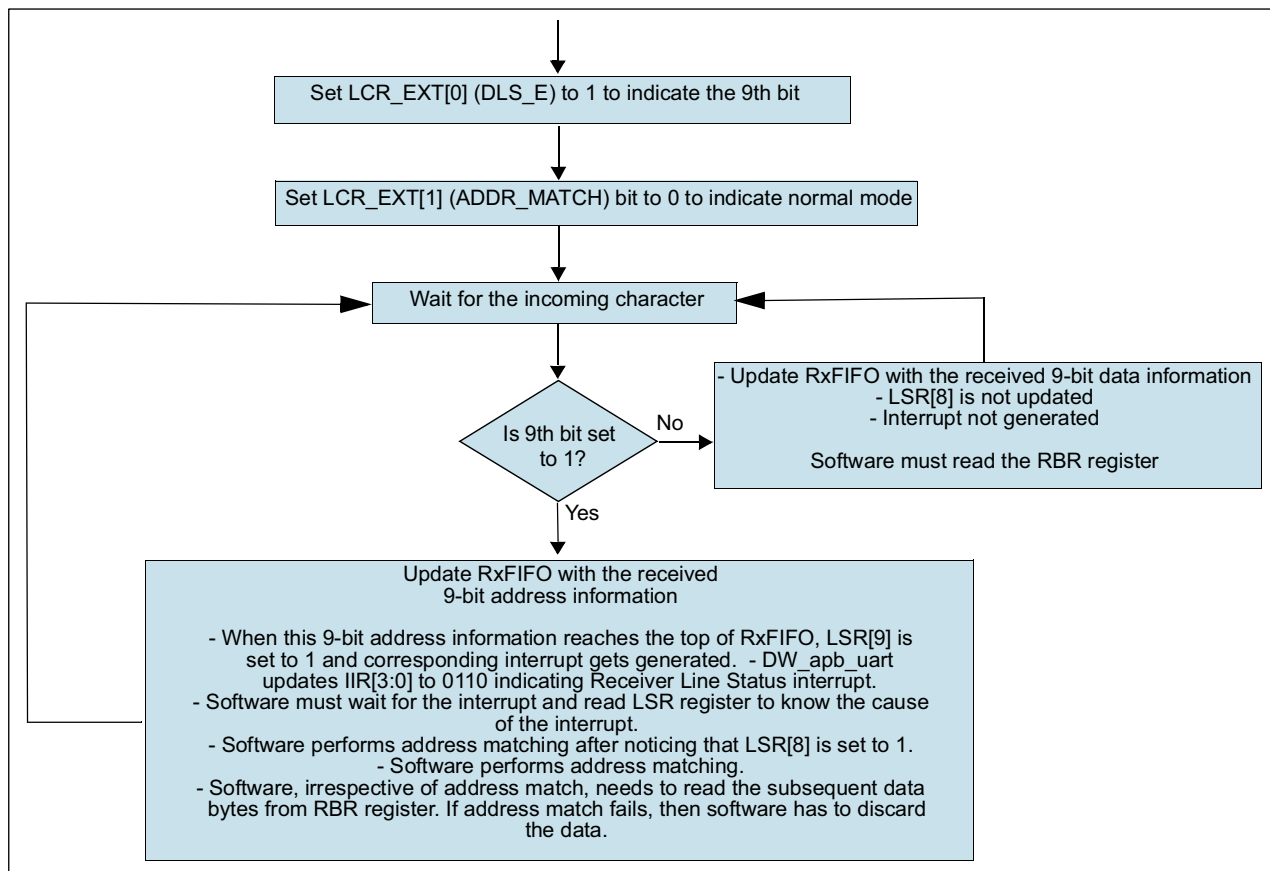
For setting up DW_apb_uart in hardware address match receive mode, follow the programming sequence mentioned in [Figure 6-2](#) and additional steps as mentioned in [Figure 6-5](#).

Figure 6-5 Hardware Address Match Receive Programming Flow



6.3.4 Software Address Match Receive Mode

Figure 6-6 Software Address Match Receive Programming Mode



Note

In the FIFO mode, if UART_9BIT_DATA_EN=1, the character timeout also considers the 9th bit.

6.4 Programming Flow for Fractional Baud Rate

The programming flow for fractional baud rate is the same as explained in “[Programming Examples](#)” on page [222](#) except that you must configure the DLF register before programming the DLH and DLL registers if the FRACTIONAL_BAUD_DIVISOR_EN parameter is enabled.

7

Verification

This chapter provides an overview of the verification environment, testbench and tests available for DW_apb_uart verification. Once the DW_apb_uart has been configured and the verification environment set up, simulations can be automatically ran.

**Note**

The DW_apb_uart verification testbench is built with DesignWare Verification IP (VIP). Make sure you have the supported version of the VIP components for this release, otherwise, you may experience some tool compatibility problems. For more information about supported tools in this release, see the *DesignWare Synthesizable Components for AMBA 2, AMBA 3 AXI, and AMBA 4 AXI Installation Guide*.

**Note**

The packaged test benches are only for validating the IP configuration in coreConsultant GUI. It is not for system level validation.
IPs that have the Vera test bench packaged, these test benches are encrypted.

This chapter discusses the following sections:

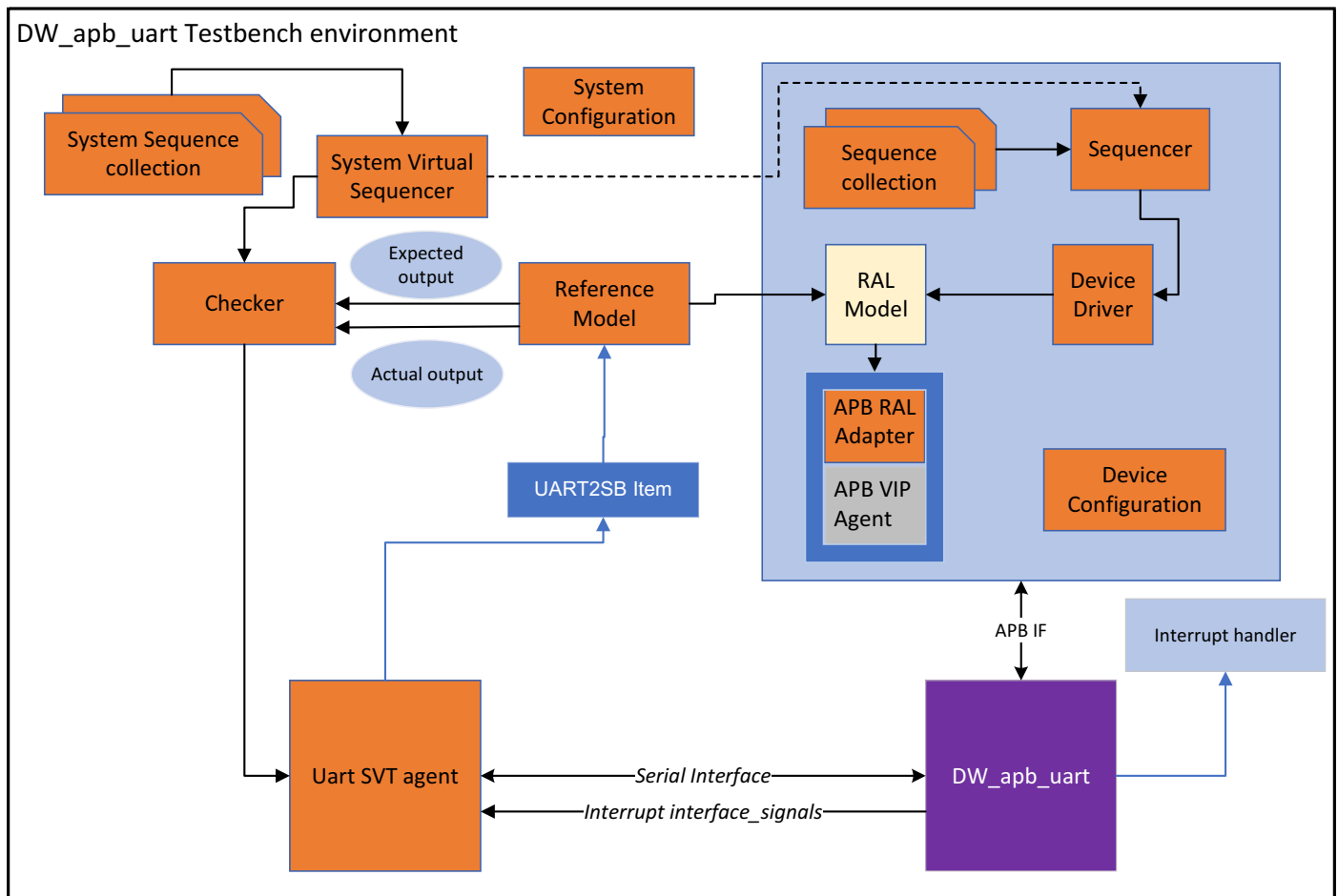
- [“Verification Environment Overview”](#) on page 232
- [“Testbench Directories and Files”](#) on page 233

7.1 Verification Environment Overview

DW_apb_uart is verified using a UVM-methodology-based constrained random verification environment. The environment is capable of generating random scenarios and the test case has hooks to control the scenarios to be generated.

Figure 7-1, shows the verification environment of the DW_apb_uart testbench:

Figure 7-1 DW_apb_uart UVM Verification Environment



The testbench consists of the following elements:

- Testbench uses the standard SVT VIP for the protocol interfaces:
 - APB SVT VIP Interface for connecting requester and completer ports of DUT.
 - APB VIP system environment (svt_apb_system_env): APB requester agent and APB completer agent provide VIP supported randomized transactions with bus instantiation on either side of the DUT.
 - UART SVT VIP is used on serial interface.
- Testbench uses the custom components:

- ❑ **Device Agent**
This component is responsible for creating traffic in the application bus interface (APB) which can get converted on to the UART transfers on the serial interface.
- ❑ **Device sequencer**
This component is a standard UVM sequencer, it fetches the top level sequence items from the scenario sequences and feeds the device driver.
- ❑ **Device driver**
This block is responsible for generating traffic on the Transmit path and completing traffic on the Receive path of the design. Device driver is designed in a layered structure, where the Uart transactions are handled in the upper layer.
The bus protocol driver along with the RAL forms the lower layer, which directly interacts with the design bus interface.
The DMA requests from the design are also being taken care by the device driver.
- ❑ **Reference Model or Checker**
The Reference Model is responsible for updating the RAL content for the activity done outside of main register interface.

7.2 Testbench Directories and Files

The DW_apb_uart verification environment contains the following directories and associated files.

[Table 7-1](#) shows the various directories and associated files:

Table 7-1 DW_apb_uart Testbench Directory Structure

Directory	Description
<configured workspace>/sim/testbench	Contains the top level testbench module (test_top.sv) and the DUT to the testbench wrapper (dut_sv_wrapper.sv).
<configured workspace>/sim/testbench/env	Contains the testbench files. For example scoreboard, sequences, VIP, environment, sequencers, and agents.
<configured workspace>/sim/	Contains the supporting files to compile and run the simulation. After the completion of the simulation, the log files are stored here.
<configured workspace>/sim/test_*	Contains individual test cases. After the completion of the simulation, the test specific log files and if applicable the waveform files are stored here.

7.3 Packaged Testcases

The simulation environment that comes as a package file includes some demonstrative tests. Some or all of the packaged demonstrative tests, depending upon their applicability to the chosen configuration, are displayed in **Setup and Run Simulations > Testcases** in the coreConsultant GUI.

The associated test cases shipped and their description is explained in [Table 7-2](#).

Table 7-2 DW_apb_uart Test Description

Test Name	Test Description
test_uart_basic_tx_fifo	This test builds the <code>dw_apb_uart_tx_sequence</code> . - This test initiates a write transfer with <code>FIFO_DEPTH > 0</code> - This test validates the basic transmission serial line functionality of UART across all available combinations
test_fractional_baud_rate	This test builds the <code>dw_apb_uart_tx_rx_sequence</code> . - This test initiates a read and write transfer. - This test validates the transmit and receive serial line functionalities of UART with fractional baud rate.

Integration Considerations

After you have configured, tested, and synthesized your component with the coreTools flow, you can integrate the component into your own design environment.

8.1 Accessing Top-level Constraints

To get SDC constraints out of coreConsultant, you need to first complete the synthesis activity and then use the “write_sdc” command to write out the results:

1. This cC command sets synthesis to write out scripts only, without running DC:

```
set_activity_parameter Synthesize ScriptsOnly 1
```

2. This cC command autocompletes the activity:

```
autocomplete_activity Synthesize
```

3. Finally, this cC command writes out SDC constraints:

```
write_sdc <filename>
```

8.2 Coherency

Coherency is where bits within a register are logically connected. For instance, part of a register is read at time 1 and another part is read at time 2. Being coherent means that the part read at time 2 is at the same value it was when the register was read at time 1. The unread part is stored into a shadow register and this is read at time 2. When there is no coherency, no shadow registers are involved.

A bus requester may need to be able to read the contents of a register, regardless of the data bus width, and be guaranteed of the coherency of the value read. A bus requester may need to be able to write a register coherently regardless of the data bus width and use that register only when it has been fully programmed. This may need to be the case regardless of the relationship between the clocks.

Coherency enables a value to be read that is an accurate reflection of the state of the counter, independent of the data bus width, the counter width, and even the relationship between the clocks. Additionally, a value written in one domain is transferred to another domain in a seamless and coherent fashion.

Throughout this appendix the following terms are used:

- **Writing.** A bus requester programs a configuration register. An example is programming the load value of a counter into a register.

- **Transferring.** The programmed register is in a different clock domain to where it is used, therefore, it needs to be transferred to the other clock domain.
- **Loading.** Once the programmed register is transferred into the correct clock domain, it needs to be loaded or used to perform its function. For example, once the load value is transferred into the counter domain, it gets loaded into the counter.

8.2.1 Writing Coherently

Writing coherently means that all the bits of a register can be written at the same time. A peripheral may have programmable registers that are wider than the width of the connected APB data bus, which prevents all the bits being programmed at the same time unless additional coherency circuitry is provided.

The programmable register could be the load value for a counter that may exist in a different clock domain. Not only does the value to be programmed need to be coherent, it also needs to be transferred to a different clock domain and then loaded into the counter. Depending on the function of the programmable register, a qualifier may need to be generated with the data so that it knows when the new value is currently transferred and when it should be loaded into the counter.

Depending on the system and on the register being programmed, there may be no need for any special coherency circuitry. One example that requires coherency circuitry is a 32-bit timer within an 8-bit APB system. The value is entirely programmed only after four 8-bit wide write transfers. It is safe to transfer or use the register when the last byte is currently written. An example where no coherency is required is a 16-bit wide timer within a 16-bit APB system. The value is entirely programmed after a single 16-bit wide write transfer.

Coherency circuitry enables the value to be loaded into the counter only when fully programmed and crossed over clock domains if the peripheral clock is not synchronous to the processor clock. While the load register is being programmed, the counter has access to the previous load value in case it needs to reload the counter.

Coherency circuitry is only added in cores where it is needed. The coherency circuitry incorporates an upper byte method that requires users to program the load register in LSB to MSB order when the peripheral width is smaller than the register width. When the upper byte is programmed, the value can be transferred and loaded into the load register. When the lower bytes are being programmed, they need to be stored in shadow registers so that the previous load register is available to the counter if it needs to reload. When the upper byte is programmed, the contents of the shadow registers and the upper byte are loaded into the load register.

The upper byte is the top byte of a register. A register can be transferred and loaded into the counter only when it has been fully programmed. A new value is available to the counter once this upper byte is written into the register. The following table shows the relationship between the register width and the peripheral bus width for the generation of the correct upper byte. The numbers in the table represent bytes, Byte 0 is the LSB and Byte 3 is the MSB. NCR means that no coherency circuitry is required, as the entire register is written with one access.

Table 8-1 Upper Byte Generation

	Upper Byte Bus Width		
Load Register Width	8	16	32
1 - 8	NCR	NCR	NCR
9 - 16	1	NCR	NCR
17 - 24	2	2	NCR
25 - 32	3	2 (or 3)	NCR

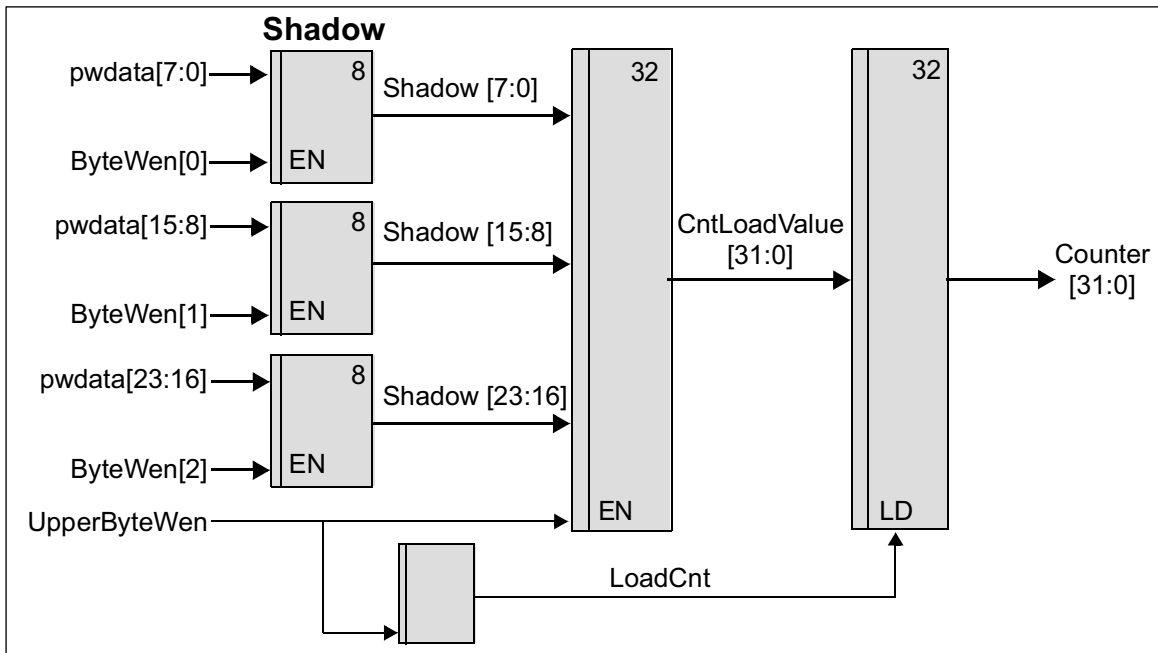
There are three relationship cases to be considered for the processor and peripheral clocks:

- Identical
- Synchronous (phase coherent but of an integer fraction)
- Asynchronous

8.2.1.1 Identical Clocks

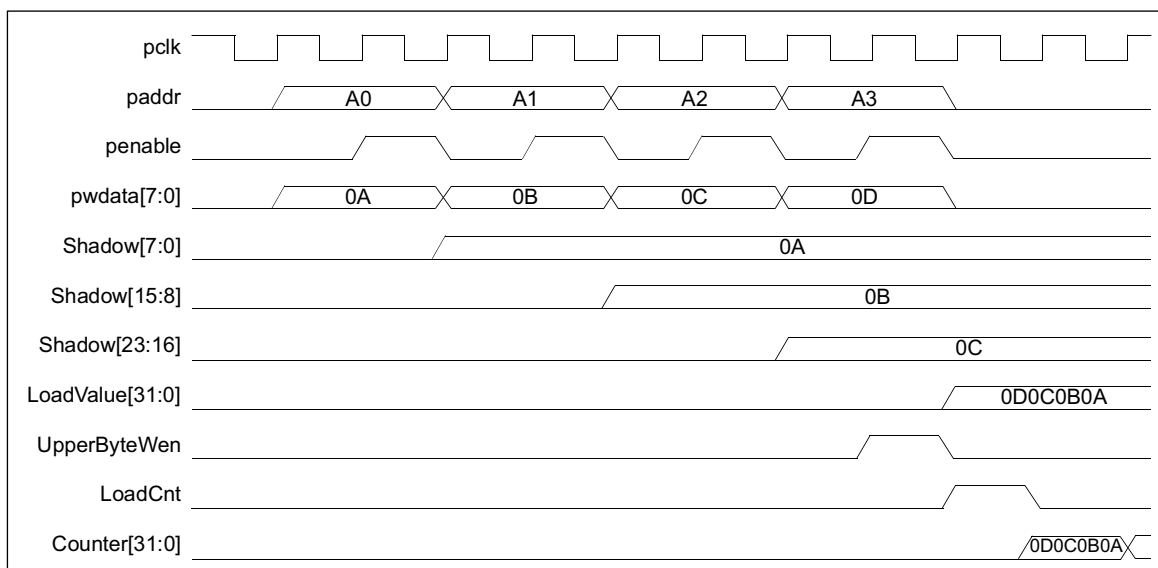
The following figure illustrates an RTL diagram for the circuitry required to implement the coherent write transaction when the APB bus clock and peripheral clocks are identical.

Figure 8-1 Coherent Loading – Identical Synchronous Clocks



The following figure shows a 32-bit register that is written over an 8-bit data bus, as well as the shadow registers being loaded and then loaded into the counter when fully programmed. The LoadCnt signal lasts for one cycle and is used to load the counter with CntLoadValue.

Figure 8-2 Coherent Loading – Identical Synchronous Clocks



Each of the bytes that make up the load register are stored into shadow registers until the final byte is written. The shadow register is up to three bytes wide. The contents of the shadow registers and the final byte are transferred into the CntLoadValue register when the final byte is written. The counter uses this register to load/initialize itself. If the counter is operating in a periodic mode, it reloads from this register each time the count expires.

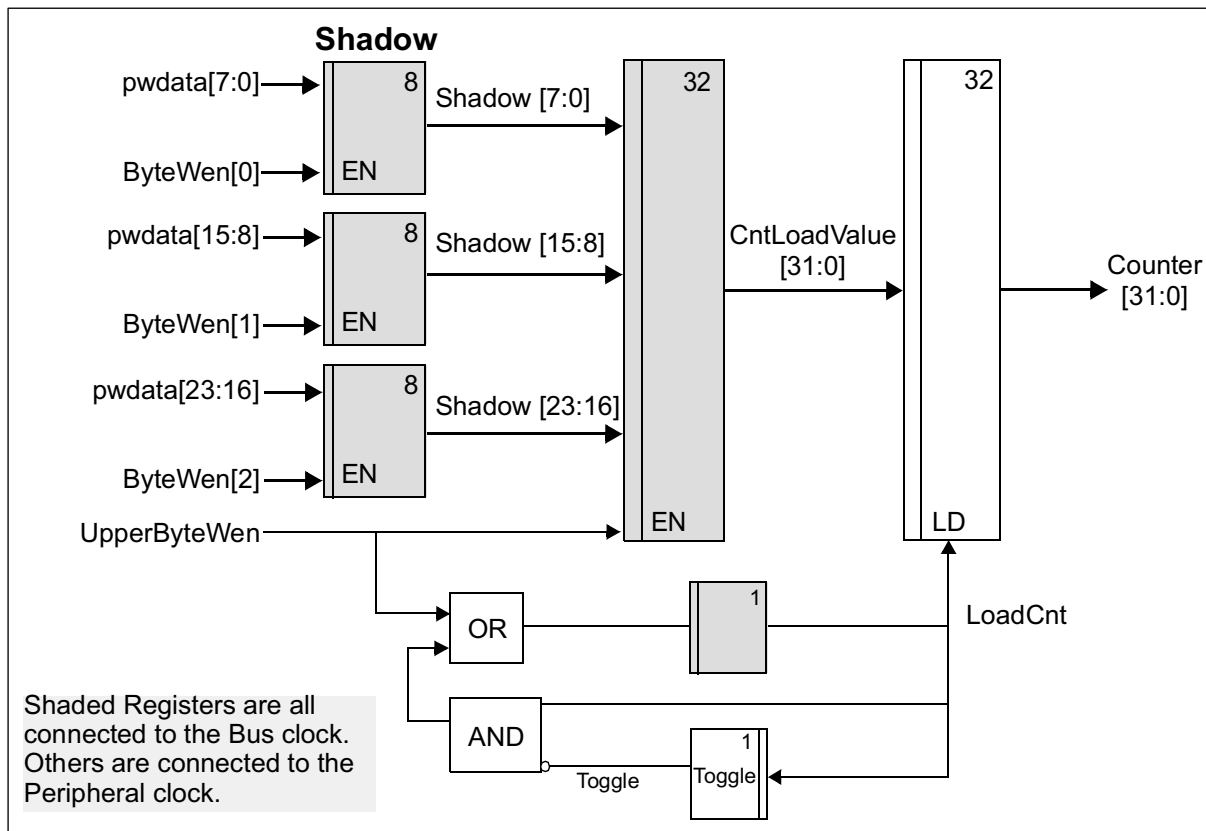
By using the shadow registers, the CntLoadValue is kept stable until it can be changed in one cycle. This allows the counter to be loaded in one access and the state of the counter is not affected by the latency in programming it. When there is a new value to be loaded into the counter initially, this is signaled by LoadCnt = 1. After the upper byte is written, the LoadCnt goes to zero.

8.2.1.2 Synchronous Clocks

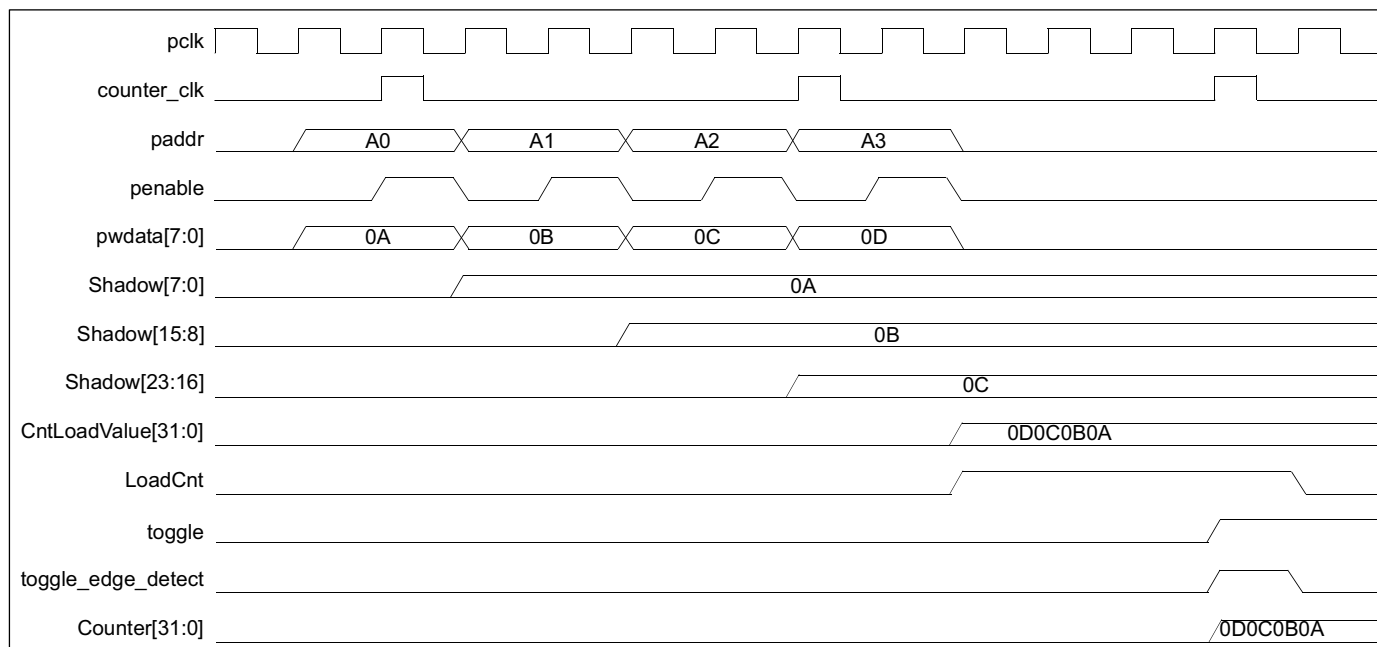
When the clocks are synchronous but do not have identical periods, the circuitry needs to be extended so that the LoadCnt signal is kept high until a rising edge of the counter clock occurs. This extension is necessary so that the value can be loaded, using LoadCnt, into the counter on the first counter clock edge. At the rising edge of the counter clock if LoadCnt is high, then a register clocked with the counter clock toggles, otherwise it keeps its current value. A circuit detecting the toggling is used to clear the original LoadCnt by looking for edge changes. The value is loaded into the counter when a toggle has been detected. Once it is loaded, the counter should be free to increment or decrement by normal rules.

The following figure shows an RTL diagram for the circuitry required to implement the coherent write when the bus and peripheral clocks are synchronous.

Figure 8-3 Coherent Loading – Synchronous Clocks

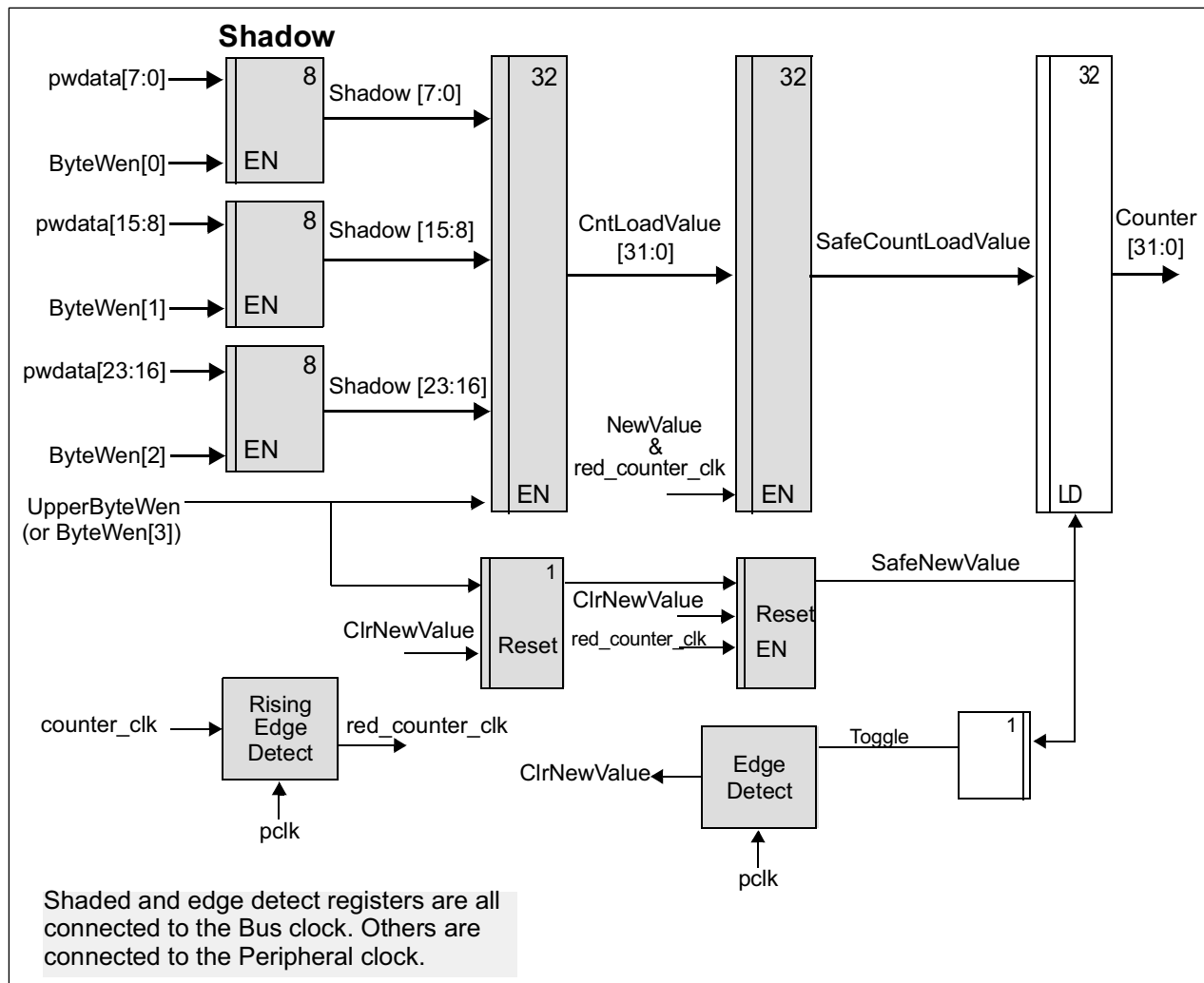


The following figure shows a 32-bit register being written over an 8-bit data bus, as well as the shadow registers being loaded and then loaded into the counter when fully programmed. The LoadCnt signal is extended until a change in the toggle is detected and is used to load the counter.

Figure 8-4 Coherent Loading – Synchronous Clocks

8.2.1.3 Asynchronous Clocks

When the clocks are asynchronous, the processor clock needs to be three-times the speed of the peripheral clock for the re-timing to operate correctly. The high pulse time of the peripheral clock needs to be greater than the period of the processor clock. [Figure 8-5](#) shows an RTL diagram for the circuitry required to implement the coherent write when the bus and peripheral clocks are asynchronous.

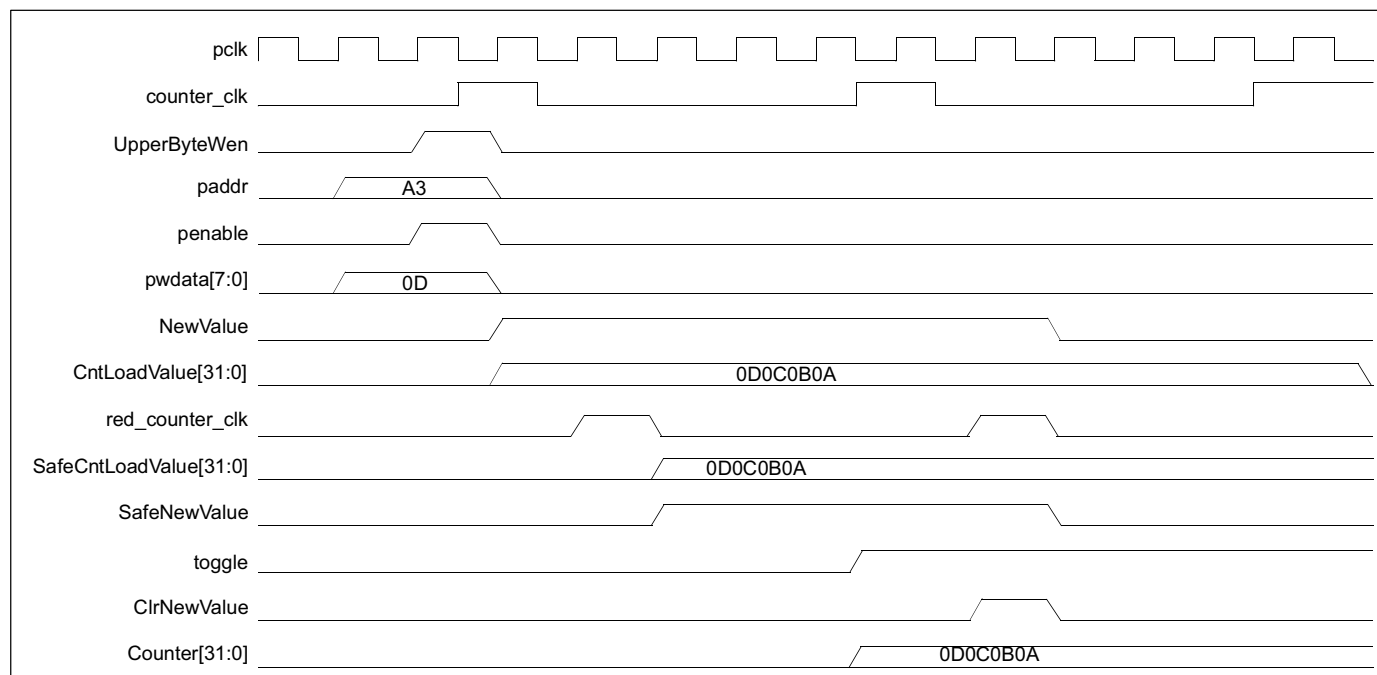
Figure 8-5 Coherent Loading – Asynchronous Clocks

When the clocks are asynchronous, you need to transfer the contents of the register from one clock domain to another. It is not desirable to transfer the entire register through meta-stability registers, as coherency is not guaranteed with this method. The circuitry needed requires the processor clock to be used to re-time the peripheral clock. Upon a rising edge of the re-timed clock, the new value signal, `NewValue`, is transferred into a safe new value signal, `SafeNewValue`, which happens after the edge of the peripheral clock has occurred.

Every time there is a rising edge of the peripheral clock detected, the `CntLoadValue` is transferred into a `SafeCntLoadValue`. This value is used to transfer the load value across the clock domains. The `SafeCntLoadValue` only changes a number of bus clock cycles after the peripheral clock edge changes. A counter running on the peripheral clock is able to use this value safely. It could be up to two peripheral clock periods before the value is loaded into the counter. Along with this loaded value, there is also a single bit transferred that is used to qualify the loading of the value into the counter.

The timing diagram depicted in the following figure does not show the shadow registers being loaded. This is identical to the loading for the other clock modes.

Figure 8-6 Coherent Loading – Asynchronous Clocks



The NewValue signal is extended until a change in the toggle is detected and is used to update the safe value. The SafeNewValue is used to load the counter at the rising edge of the peripheral clock. Each time a new value is written the toggle bit is flipped and the edge detection of the toggle is used to remove both the NewValue and the SafeNewValue.

8.2.2 Reading Coherently

For writing to registers, an upper-byte concept is proposed for solving coherency issues. For read transactions, a lower-byte concept is required. The following table provides the relationship between the register width and the bus width for the generation of the correct lower byte.

Table 8-2 Lower Byte Generation

	Lower Byte Bus Width		
Counter Register Width	8	16	32
1 - 8	NCR	NCR	NCR
9 - 16	0	NCR	NCR
17 - 24	0	0	NCR
25 - 32	0	0	NCR

Depending on the bus width and the register width, there may be no need to save the upper bits because the entire register is read in one access, in which case there is no problem with coherency. When the lower byte is read, the remaining upper bytes within the counter register are transferred into a holding register. The holding register is the source for the remaining upper bytes. Users must read LSB to MSB for this solution to operate correctly. NCR means that no coherency circuitry is required, as the entire register is read with one access.

There are two cases regarding the relationship between the processor and peripheral clocks to be considered as follows:

- Identical and/or synchronous
- Asynchronous

8.2.2.1 Synchronous Clocks

When the clocks are identical and/or synchronous, the remaining unread bits (if any) need to be saved into a holding register once a read is started. The first read byte must be the lower byte provided in the previous table, which causes the other bits to be moved into the holding register, `SafeCntVal`, provided that the

register cannot be read in one access. The upper bytes of the register are read from the holding register rather than the actual register so that the value read is coherent. This is illustrated in the following figure and in the timing diagram after it.

Figure 8-7 Coherent Registering – Synchronous Clocks

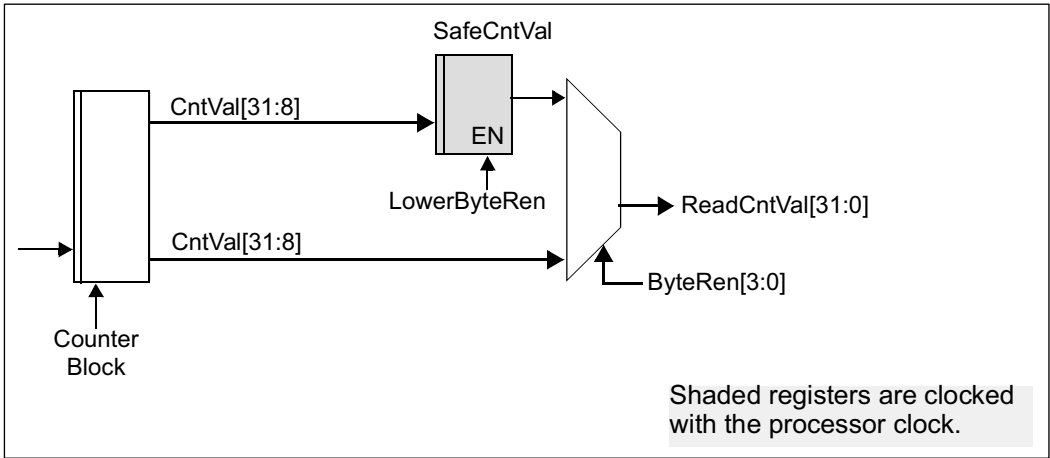
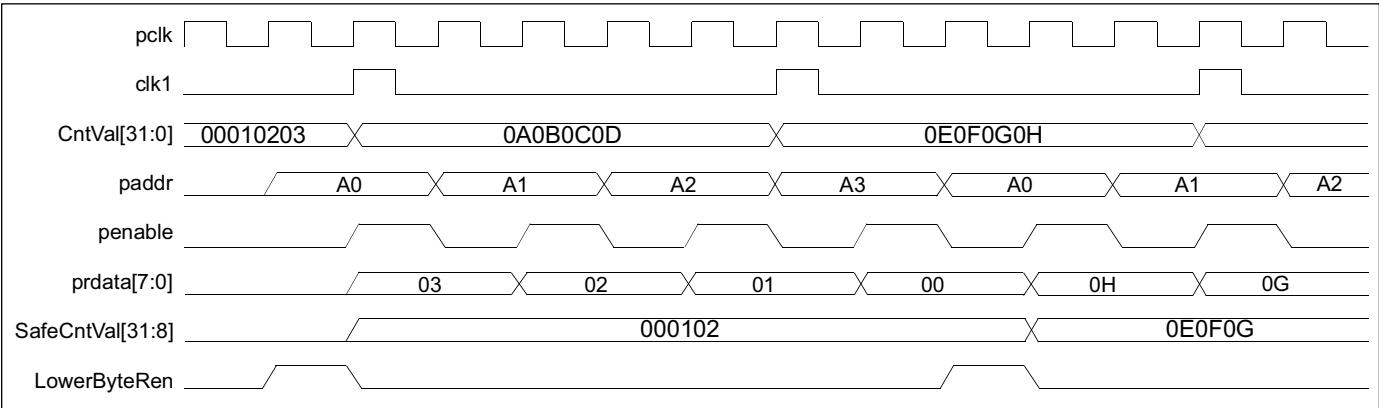


Figure 8-8 Coherent Registering – Synchronous Clocks



8.2.2.2 Asynchronous Clocks

When the clocks are asynchronous, the processor clock needs to be three times the speed of the peripheral clock for the re-timing to operate correctly. The high pulse time of the peripheral clock needs to be greater than the period of the processor clock.

To safely transfer a counter value from the counter clock domain to the bus clock domain, the counter clock signal should be transferred to the bus clock domain. When the rising edge detect of this re-timed counter clock signal is detected, it is safe to use the counter value to update a shadow register that holds the current value of the counter.

While reading the counter contents it may take multiple APB transfers to read the value.



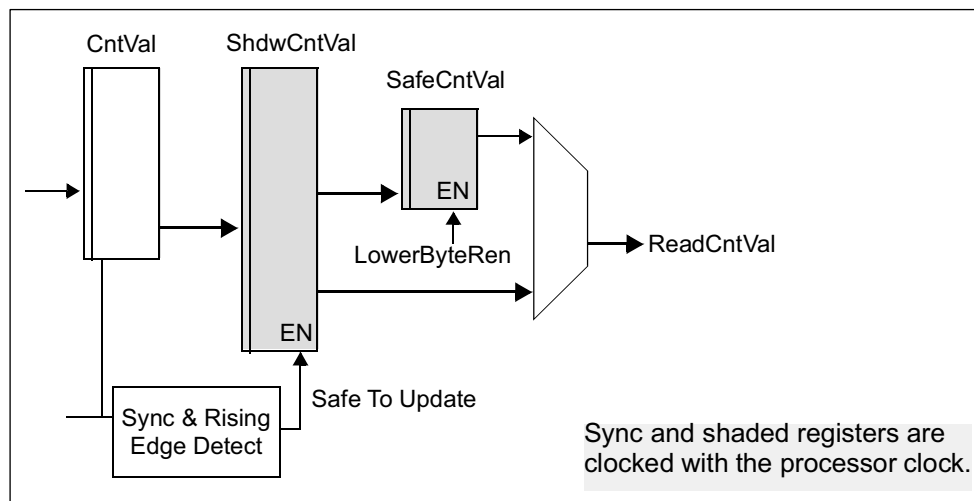
You must read LSB to MSB when the bus width is narrower than the counter width.

Once a read transaction has started, the value of the upper register bits need to be stored into a shadow register so that they can be read with subsequent read accesses. Storing these upper bits preserves the coherency of the value that is being read. When the processor reads the current value it actually reads the contents of the shadow register instead of the actual counter value. The holding register is read when the bus width is narrower than the counter width. When the LSB is read, the value comes from the shadow register; when the remaining bytes are read they come from the holding register. If the data bus width is wide enough to read the counter in one access, then the holding registers do not exist.

The counter clock is registered and successively pipelined to sense a rising edge on the counter clock. Having detected the rising edge, the value from the counter is known to be stable and can be transferred into the shadow register. The coherency of the counter value is maintained before it is transferred, because the value is stable.

The following figure illustrates the synchronization of the counter clock and the update of the shadow register.

Figure 8-9 Coherency and Shadow Registering – Asynchronous Clocks



8.3 Timing Exceptions

For details on quasi-static signals in the design, refer to manual.sgdc report generated by the Run SpyGlass RTL Checker activity of the coreConsultant.

8.4 Performance

This section discusses performance and the hardware configuration parameters that affect the performance of the DW_apb_uart.

8.4.1 Power Consumption, Frequency, Area and DFT Coverage

[Table 8-3](#) provides information about the synthesis results (power consumption, frequency and area) and DFT coverage of the DW_apb_uart using the industry standard 7nm technology library.

Table 8-3 Synthesis Results for DW_apb_uart

Configuration	Operating Frequency	Gate Count	Power Consumption		TetraMax Coverage (%)		SpyGlass StuckAtCov (%)
			Static Power	Dynamic Power	StuckAtTest	Transition	
Default Configuration	100 MHz	3202	8nW	0.017 mW	99.95	95.62	99.7
Typical Configuration - 1 ^a	100 MHz	13528	30nW	0.083 mW	99.84	95.32	99.7
Typical Configuration - 2 ^b	100 MHz	51752	100 nW	0.278 mW	99.97	95.07	99.9

a. For the list of configuration parameters and their values for the Typical Configuration 1, see [Table 8-4](#).

b. For the list of configuration parameters and their values for the Typical Configuration 2, see [Table 8-5](#) on page 248.

Table 8-4 Typical Configuration - 1

Parameter	Value
ADDITIONAL_FEATURES	0x1
AFCE_MODE	0x1
APB_DATA_WIDTH	32
BAUD_CLK	1
CLK_GATE_EN	1
CLOCK_MODE	2
DEBUG	1
DLF_SIZE	4
DMA_EXTRA	0x1
DMA_HS_REQ_ON_RESET	1
DMA_POL	1
FIFO_ACCESS	0x1
FIFO_MODE	2048

Table 8-4 Typical Configuration - 1

Parameter	Value
FIFO_STAT	0x1
FRACTIONAL_BAUD_DIVISOR_EN	1
LSR_STATUS_CLEAR	0
MEM_SELECT_USER	0
RTC_FCT	0x0
SHADOW	0x1
SIR_LP_MODE	0x1
SIR_LP_RX	1
SIR_MODE	0x1
THRE_MODE_USER	0x1
UART_16550_COMPATIBLE	1
UART_9BIT_DATA_EN	1
UART_ADD_ENCODED_PARAMS	0x1
UART_DE_POL	0x1
UART_RE_POL	0x1
UART_RS485_INTERFACE_EN	1
SLAVE_INTERFACE_TYPE	2
PSLVERR_RESP_EN	1

Table 8-5 Typical Configuration - 2

Parameter	Value
ADDITIONAL_FEATURES 1	1
AFCE_MODE 1	1
APB_DATA_WIDTH 8	8
CLK_GATE_EN 1	1
CLOCK_MODE 1	1
DEBUG 1	1
DMA_EXTRA 1	1

Table 8-5 Typical Configuration - 2

Parameter	Value
FIFO_ACCESS 1	1
FIFO_MODE 256	256
FIFO_STAT 0	0
MEM_SELECT_USER 1	1
RTC_FCT 0	0
SHADOW 0	0
THRE_MODE_USER 1	1
SLAVE_INTERFACE_TYPE 2	2
PSLVERR_RESP_EN 1	1

A

Basic Core Module (BCM) Library

The Basic Core Module (BCM) Library is a library of commonly used blocks for the Synopsys DesignWare IP development. These BCMs are configurable on an instance-by-instance basis and, for the majority of BCM designs, there is an equivalent (or nearly equivalent) DesignWare Building Block (DWBB) component.

This appendix contains the following sections:

- [“BCM Library Components”](#) on page 251
- [“Synchronizer Methods”](#) on page 252

A.1 BCM Library Components

[Table A-1](#) describes the list of BCM library components used in DW_apb_uart.

Table A-1 BCM Library Components

BCM Module Name	BCM Description	DWBB Equivalent
DW_apb_uart_bcm21	Single clock data bus synchronizer	DW_sync
DW_apb_uart_bcm06	Synchronous (Single Clock) FIFO Controller with Dynamic Flags	DW_fifoctrl_s1_df
DW_apb_uart_bcm23	Pulse Synchronizer with Acknowledge	DW_pulseack_sync
DW_apb_uart_bcm25	Data Bus Synchronizer with Acknowledge	DW_data_sync
DW_apb_uart_bcm41	Single Clock Data Bus Synchronizer with Configurable Polarity Reset Note: Same as DWbb_bcm21 but with additional RST_VAL parameter	--
DW_apb_uart_bcm57	Synchronous Write-Port, Asynchronous Read-Port RAM (Flip-Flop Based)	DW_ram_r_w_s_dff

A.2 Synchronizer Methods

This section describes the synchronizer methods (blocks of synchronizer functionality) that are used in the DW_apb_uart to cross clock boundaries.

This section contains the following sections:

- “Synchronizers Used in DW_apb_uart” on page 252
- “Synchronizer 1: Simple Double Register Synchronizer” on page 252
- “Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset” on page 253
- “Synchronizer 3: Simple Double Register Synchronizer with Acknowledge” on page 253



Note

The DesignWare Building Blocks (DWBB) contains several synchronizer components with functionality similar to methods documented in this appendix. For more information about the DWBB synchronizer components go to:

<https://www.synopsys.com/dw/buildingblock.php>

A.2.1 Synchronizers Used in DW_apb_uart

Each of the synchronizers and synchronizer sub-modules are comprised of verified DesignWare Basic Core (BCM) RTL designs. The BCM synchronizer designs are identified by the synchronizer type. The corresponding RTL files comprising the BCM synchronizers used in the DW_apb_uart are listed and cross referenced to the synchronizer type in Table A-2. Note that certain BCM modules are contained in other BCM modules, as they are used in a building block fashion.

Table A-2 Synchronizers Used in DW_apb_uart

Synchronizer Module File	Sub Module File	Synchronizer Type and Number
DW_apb_uart_bcm21.v		Synchronizer 1: Simple Multiple Register Synchronizer
DW_apb_uart_bcm41.v	DW_apb_uart_bcm21.v	Synchronizer 2: Simple Multiple Register Synchronizer with Configurable Polarity Reset
DW_apb_uart_bcm25.v	DW_apb_uart_bcm21.v DW_apb_uart_bcm23.v	Synchronizer 3: Simple Multiple register Synchronizer with Acknowledge



Note

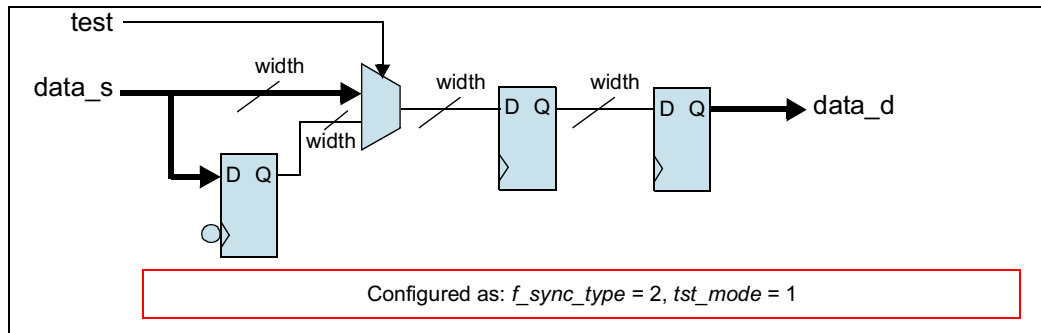
The BCM21 is a basic multiple register based synchronizer module used in the design. It can be replaced with equivalent technology specific synchronizer cell.

A.2.2 Synchronizer 1: Simple Double Register Synchronizer

This is a single clock data bus synchronizer for synchronizing data that crosses asynchronous clock boundaries. This synchronization scheme is always present for synchronizing the Modem control signals. If pclk and sclk are asynchronous (CLOCK_MODE =Enabled) then DW_apb_uart_bcm21 is instantiated

inside the core for synchronization. This uses two stage synchronization process (both using positive edge of clock).

Figure A-1 Block Diagram of Synchronizer 1 With Two Stage Synchronization (Both Positive Edge)



A.2.3 Synchronizer 2: Simple Double Register Synchronizer with Configurable Polarity Reset

This is a single clock data bus synchronizer for synchronizing data that crosses asynchronous clock boundaries with configurable polarity reset. The synchronization scheme (DW_apb_uart_bcm41.v) is always present in core for synchronization of sin input signal. This DW_apb_uart_bcm41 synchronizer is similar to the DW_apb_uart_bcm21 synchronizer and the polarity of the output of this synchronizer can be configured.

A.2.4 Synchronizer 3: Simple Double Register Synchronizer with Acknowledge

This synchronizer (DW_apb_uart_bcm25.v) passes data values from the source domain to the destination domain through a hand-shake protocol which includes an acknowledge back to the source domain. This module uses clock-domain-crossing techniques to safely transfer pulses between logic operating on different clocks and acknowledge guaranteeing the pulse has arrived in the destination domain. This synchronizer is present based on the core configuration. If pclk and sclk are synchronous (CLOCK_MODE=Enabled), then DW_apb_uart_bcm25.v is used to synchronize the data signals from one domain to the other domain.

This synchronizer uses the same DW_apb_uart_bcm21.v module to synchronize the data from source clock domain to destination clock domain.

B

Internal Parameter Descriptions

Provides a description of the internal parameters that might be indirectly referenced in expressions in the Signals, Parameters, or Registers chapters. These parameters are not visible in the coreConsultant GUI and most of them are derived automatically from visible parameters. **You must not set any of these parameters directly.**

Some expressions might refer to TCL functions or procedures (sometimes identified as **function_of**) that coreConsultant uses to make calculations. The exact formula used by these TCL functions is not provided in this chapter. However, when you configure the core in coreConsultant, all TCL functions and parameters are evaluated completely; and the resulting values are displayed where appropriate in the coreConsultant GUI reports.

Table B-1 Internal Parameters

Parameter Name	Equals To
FIFO_ADDR_WIDTH	[function_of: FIFO_MODE]
MEM_SELECT	(((FIFO_MODE!=0)&&(FIFO_MODE<=256))?MEM_SELECT_USER:0)
POW_2_REG_TIMEOUT_WIDTH	[function_of: REG_TIMEOUT_WIDTH]
RXFIFO_RW	((UART_9BIT_DATA_EN==1)?11:10)
RX_RAM_DATA_WIDTH	RXFIFO_RW
SRBRN_REG_SIZE	((UART_9BIT_DATA_EN==1)?9:8)
STHRN_REG_SIZE	((UART_9BIT_DATA_EN==1)?9:8)
THRE_MODE	((FIFO_MODE != 0) ? ((THRE_MODE_USER == 1) ? 1 : 0) : 0)
THRE_MODE_RST	(FIFO_MODE != 0 && THRE_MODE_USER==1) ? 1 : 0
TXFIFO_RW	((UART_9BIT_DATA_EN==1)?9:8)
TX_RAM_DATA_WIDTH	TXFIFO_RW

Table B-1 Internal Parameters (Continued)

Parameter Name	Equals To
UART_ADDR_SLICE_LHS	8
UART_COMP_TYPE	0x44570110
UART_COMP_VERSION	0x3430342a
UART_ENCODED_APB_WIDTH	[function_of: APB_DATA_WIDTH]
UART_ENCODED_FIFO_MODE	[function_of: FIFO_MODE]

C

Application Notes

This appendix provides application notes for DW_apb_uart.

Q. The DesignWare DW_apb_uart Databook states that the timeout detection hardware block is optional, but I do not see any configuration parameter available for this feature. How do I set this option?

A. If FIFO_MODE!=NONE or CLK_GATE_EN=1, you get this timeout detector block instantiated at the top level. If CLK_GATE_EN=1, clock gating circuitry is also included in the Timeout Detector block.

Q. What is the difference between bits PTIME (bit 7) and ETBEI (bit 1) of the IER register?

A. PTIME is used to enable the Programmable THRE Interrupt Mode, when DW_apb_uart is configured to support this mode (THRE_MODE_USER = Enabled). The PTIME bit field is writable only when the Programmable THRE Interrupt Mode Enable configuration parameter (THRE_MODE_USER) is set to True. It is always readable. This is used to enable or disable the generation of THRE Interrupt.

ETBEI is used to enable the interrupt regardless of the setting of the Programmable THRE Interrupt Mode. The interrupt provides the following information depending on whether the Programmable THRE Interrupt Mode is enabled or disabled. The interrupt indicates either the transmitter holding register empty (THRE_MODE_USER is disabled) or the transmit FIFO at or below threshold (THRE_MODE_USER is enabled).

Thus, DW_apb_uart has been configured to have Programmable THRE Interrupt Mode, the PTIME bit is used to switch between the two modes of operation.

Q. When I read the Component Parameter register (CPR), it always returns a value of 0. Why does this occur?

A. The CPR is only valid when DW_apb_uart is configured to have the Component Parameter register implemented (UART_ADD_ENCODED_PARAMS is set to Yes). If the Component Parameter register is not implemented, this register does not exist and reading from this register address returns 0.

Q. Why is there IIR[3:0]=0x7, an additional busy detect interrupt in comparison to the 16550 National specification?

A. Busy functionality helps to safe guard against errors if the LCR, DLL, and/or DLH registers are changed during a transaction even though they should only be set during initialization (as stated in the National specification for DLL/DLH, section 8.3 p.16).

Q. *Why are there two resets in DW_apb_uart?*

- A. When operating in asynchronous serial clock mode, dedicated reset signals for the different clock domains are required. All the logic operating on pclk is reset by presetn, while the logic operating on sclk is reset by s_rst_n.



Note

- The presetn and s_rst_n signals must be synchronous to the pclk and sclk clock signals, respectively.
- For correct operation, the logic on both clock domains should be reset simultaneously; resetting only one clock domain results in undetermined behavior. When de-asserting the reset signals, s_rst_n should be de-asserted first.

The software reset (when this feature is enabled) resets both pclk and sclk logic; although this signal is generated in the pclk domain, internal synchronization ensures it can be safely used in the sclk domain without the risk of metastability.

When operating in synchronous serial clock mode all logic is reset by the presetn signal.

Q. *Is it possible to do burst (back-to-back) FIFO reads/writes? For example, can the write and enable lines be held for two consecutive clocks to do back-to-back transfers?*

- A. DW_apb_uart does accommodate burst FIFO reads and writes using the SRBR (Shadow Receive Buffer Register) and STHR (Shadow Transmit Holding Register).

Q. *What activity occurs on the sout and sir_out_n signals in UART and IR mode?*

- A. The serial data out signal sout is driven high if DW_apb_uart is in loopback mode or serial infrared mode. Otherwise, it is assigned to the current bit of the character that is being transmitted.

Q. *Is there a way to find out whether DW_apb_uart is operating in either normal serial data mode or in Infrared mode?*

- A. The only way to find out whether DW_apb_uart is operating in either normal serial data mode or in Infrared mode is to check the Modem Control Register (MCR) bit 6. If MCR[6]=0, IrDA SIR Mode disabled. If MCR[6]=1, IrDA SIR Mode is enabled. This bit is writable only when SIR_MODE is enabled.

Q. *Is DW_apb_uart completely compliant with the D16750 specification from Texas Instruments?*

- A. No, DW_apb_uart is not completely compliant with the D16750 TI specification. DW_apb_uart does not support features such as sleep mode (has an enable bit in the IER) and low-power mode (has an enable bit in the IER), which seem to be for a uart/clock oscillator control within their chip.

Q. *What is the safest way to hold DW_apb_uart in reset or non-response state when it is being initialized so that no characters during this time period are received/transmitted?*

- A. When you are in the initialization stage, there are two ways to ensure that no characters during this time period are received/transmitted:
- a. Set DLL and DLH to 0; configure the control registers for transfer, for instance, set LCR register (data size, stop bits, and so on); set the MCR register; set the FCR register to enable FIFOs; and set IER register to enable interrupts. Once this is completed, write to the divisor registers DLL and DLH to set the bit rate and then write the data to be transmitted into the THR register.
 - b. There is a loopback mode (MCR[4]) that provides a local loopback feature. When this bit is set to logic 1, the transmitter Serial Output (SOUT) is set to a logic 1 state, the receiver Serial Input (SIN)

is disconnected, and the output of the Transmitter Shift Register is “looped back” into the Receiver Shift Register input. So DW_apb_uart does not receive anything because the sin signal is disconnected.

Put DW_apb_uart in loopback mode; setup the control registers for transfer (for instance, write the divisor registers to set the bit rate); set LCR register (data size, stop bits, and so on); set MCR register; set FCR register to enable FIFOs; and set IER register to enable interrupts. Once this is completed, write 0 in MCR[4] (no loopback mode) and then write the data to be transmitted into the THR register.

- Q. After initialization of DW_apb_uart, if you want to program the baud rate, how can you make sure you do not receive/send any characters during this configuration time?
- A. After initialization of DW_apb_uart, once TX/RX has started, if you want to reprogram the baud rate, make sure that the serial transfer has been completed. The safest way to accomplish this is to poll USR[0] (Busy) bit, and when DW_apb_uart is not busy, change the register values.
- Q. What is the functionality of the SIR_MODE and SIR_LP_MODE configuration parameters?
- A. IrDA 1.0 SIR mode specifies a maximum baud rate of 115.2 Kbaud. But if you want to operate using the low-power pulse duration (SIR_LP_MODE=1) of 1.63us, you must run at 115.2K baud. DW_apb_uart automatically handles this by being configured with asynchronous clock support.

If SIR_MODE is set to 1, you can have a baud rate anywhere from 9.6K to 115.2K with 3/16 nominal pulse width support. However, if SIR_LP_MODE is set to 1, you must run at 115.2K.

If you set CLOCK_MODE to 2, SIR_MODE to 1, and run at 115.2K, you are getting functionality for SIR_LP_MODE set to 1. If you set a baud rate higher or lower than 115.2K in SIR_LP_MODE, it still generates 3/16 pulse width but does not work properly because it violates the requirement of 1.63us for low-power IrDA SIR mode.

D

Glossary

active command queue	Command queue from which a model is currently taking commands; see also command queue.
activity	A set of functions in coreConsultant that step you through configuration, verification, and synthesis of a selected core.
application design	Overall chip-level design into which a subsystem or subsystems are integrated.
BFM	Bus-Functional Model — A simulation model used for early hardware debug. A BFM simulates the bus cycles of a device and models device pins, as well as certain on-chip functions. See also Full-Functional Model.
big-endian	Data format in which most significant byte comes first; normal order of bytes in a word.
blocked command stream	A command stream that is blocked due to a blocking command issued to that stream; see also command stream, blocking command, and non-blocking command.
blocking command	A command that prevents a testbench from advancing to next testbench statement until this command executes in model. Blocking commands typically return data to the testbench from the model.
command channel	Manages command streams. Models with multiple command channels execute command streams independently of each other to provide full-duplex mode function.
command stream	The communication channel between the testbench and the model.
completer	Device or model that is controlled by and responds to a requester.
component	A generic term that can refer to any synthesizable IP or verification IP in the DesignWare Library. In the context of synthesizable IP, this is a configurable block that can be instantiated as a single entity (VHDL) or module (Verilog) in a design.
configuration	The act of specifying parameters for a core prior to synthesis; can also be used in the context of VIP.

configuration intent	Range of values allowed for each parameter associated with a reusable core.
core	Any configurable block of synthesizable IP that can be instantiated as a single entity (VHDL) or module (Verilog) in a design. Core is the preferred term for a big piece of IIP. Anything that requires coreConsultant for configuration, as well as anything in the DesignWare Cores library, is a core.
core developer	Person or company who creates or packages a reusable core. All the cores in the DesignWare Library are developed by Synopsys.
core integrator	Person who uses coreConsultant or coreAssembler to incorporate reusable cores into a system-level design.
coreAssembler	Synopsys product that enables automatic connection of a group of cores into a subsystem. Generates RTL and gate-level views of the entire subsystem.
coreConsultant	A Synopsys product that lets you configure a core and generate the design views and synthesis views you need to integrate the core into your design. Can also synthesize the core and run the unit-level testbench supplied with the core.
coreKit	An unconfigured core and associated files, including the core itself, a specified synthesis methodology, interfaces definitions, and optional items such as verification environment files and core-specific documentation.
cycle command	A command that executes and causes HDL simulation time to advance.
decoder	Software or hardware subsystem that translates from and “encoded” format back to standard format.
design context	Aspects of a component or subsystem target environment that affect the synthesis of the component or subsystem.
design creation	The process of capturing a design as parameterized RTL.
Design View	A simulation model for a core generated by coreConsultant.
DesignWare cores	A specific collection of synthesizable cores that are licensed individually. For more information, refer to www.synopsys.com/designware .
DesignWare Library	A collection of synthesizable IP and verification IP components that is authorized by a single DesignWare license. Products include SmartModels, VMT model suites, DesignWare Memory Models, Building Block IP, and the DesignWare Synthesizable Components.
dual role device	Device having the capabilities of function and host (limited).
endian	Ordering of bytes in a multi-byte word; see also little-endian and big-endian.
Full-Functional Mode	A simulation model that describes the complete range of device behavior, including code execution. See also BFM.
GPIO	General Purpose Input Output.
GTECH	A generic technology view used for RTL simulation of encrypted source code by non-Synopsys simulators.

hard IP	Non-synthesizable implementation IP.
HDL	Hardware Description Language – examples include Verilog and VHDL.
IIP	Implementation Intellectual Property — A generic term for synthesizable HDL and non-synthesizable “hard” IP in all of its forms (coreKit, component, core, MacroCell, and so on).
implementation view	The RTL for a core. You can simulate, synthesize, and implement this view of a core in a real chip.
instantiate	The act of placing a core or model into a design.
interface	Set of ports and parameters that defines a connection point to a component.
IP	Intellectual property — A term that encompasses simulation models and synthesizable blocks of HDL code.
little-endian	Data format in which the least-significant byte comes first.
MacroCell	Bigger IP blocks (6811, 8051, memory controller) available in the DesignWare Library and delivered with coreConsultant.
model	A Verification IP component or a Design View of a core.
monitor	A device or model that gathers performance statistics of a system.
non-blocking command	A testbench command that advances to the next testbench statement without waiting for the command to complete.
peripheral	Generally refers to a small core that has a bus connection, specifically an APB interface.
requester	Device or model that initiates and controls another device or peripheral.
RTL	Register Transfer Level. A higher level of abstraction that implies a certain gate-level structure. Synthesis of RTL code yields a gate-level design.
SDRAM	Synchronous Dynamic Random Access Memory; high-speed DRAM adds a separate clock signal to control signals.
SDRAM controller	A memory controller with specific connections for SDRAMs.
SoC	System on a chip.
soft IP	Any implementation IP that is configurable. Generally referred to as synthesizable IP.
static controller	Memory controller with specific connections for Static memories such as asynchronous SRAMs, Flash memory, and ROMs.
subsystem	In relation to coreAssembler, highest level of RTL that is automatically generated.
synthesis intent	Attributes that a core developer applies to a top-level design, ports, and core.

synthesizable IP	A type of Implementation IP that can be mapped to a target technology through synthesis. Sometimes referred to as Soft IP.
technology-independent	Design that allows the technology (that is, the library that implements the gate and via widths for gates) to be specified later during synthesis.
Testsuite Regression Environment (TRE)	A collection of files for stand-alone verification of the configured component. The files, tests, and functionality vary from component to component.
VIP	Verification Intellectual Property — A generic term for a simulation model in any form, including a Design View.
workspace	A network location that contains a personal copy of a component or subsystem. After you configure the component or subsystem (using coreConsultant or coreAssembler), the workspace contains the configured component/subsystem and generated views needed for integration of the component/subsystem at the top level.
wrap, wrapper	Code, usually VHDL or Verilog, that surrounds a design or model, allowing easier interfacing. Usually requires an extra, sometimes automated, step to create the wrapper.
zero-cycle command	A command that executes without HDL simulation time advancing.